FEDB L849
B8040
B-R4.2

COURS DE PROGRAMMATION GAMMA 60

COURS DE PROGRAMMATION
GAMMA 60

S O M M A I R E

Pages

INTRODUCTION GENERALE

Idée directrice du Gamma 60 1

PREMIERE PARTIE

Description et logique générale de fonctionnement 3

Chapitre 1 : Structure organique générale 3

I - L'Unité Centrale 3
II - Les Eléments 7
III - La Codification des Informations dans le Gamma 60 9
IV - Le Contrôle Interne du Gamma 60 10

Chapitre 2 : Fonctionnement du Gamma 60 13

I - Introduction et définition préalables 13
II - Exploitation d'un programme filiforme 14
III - Exploitation d'un programme maillé 17
IV - Symbolisme fonctionnel 19
V - Illustration des paragraphes précédents par un exemple concret 21
VI - Le dispositif de "Chaîne de reprise" 24
VII - Les éléments virtuels 26

Chapitre 3 : Structure générale et exploitation logique des différentes sortes d'instructions canoniques 33

I - Instructions canoniques et instruction complète 33
II - Structure générale de l'instruction canonique . 33
III - L'instruction de coupure 34
IV - L'instruction directive 40
V - Les branchements 42
VI - L'instruction d'adresse 43
VII - La fonction d'adresse 44

DEUXIEME PARTIE

Description détaillée des éléments et du code symbolique de programmation 46

Chapitre 1 : Généralités 46

I - Code A et code B 46
II - Dessin d'une feuille de programmation 47
III - Code des adresses 49
IV - Code des branchements 50
V - Code des coupures 51
VI - Illustration 52

Chapitre 2 : Fonctions centrales 53

I - Définition 53
II - Les registres de programmation de l'unité centrale 53
III - Les directives centrales 54

Pages

IV - Remarques générales sur la substitution implicite et exemples 58
V - Tableau de code des directives centrales 60

Chapitre 3 : Les Eléments Centraux de Traitement 61
**Le Calculateur arithmétique* 61

I - Structure des nombres 61
II - Virgule normalisée, virgule programmée 63
III - Description et fonctionnement du Calculateur arithmétique 65
IV - Les opérations et les directives correspondantes 67
**Le Comparateur général* 73
I - Fonction de comparateur général 73
II - Description et fonctionnement général 74
III - Les Directives 75
IV - Temps opératoires 78
V - Incidents 78
**Le Traducteur* 79
I - Fonctions du Traducteur 79
II - Structure et fonctionnement du traducteur 85
III - Les directives du Traducteur 96
IV - Les codes clichés 100
V - Compléments sur la traduction des nombres 103
VI - Le datum de reprise 105
VII - Profil du catène qualitatif 109
**Le Calculateur logique* 110
I - Fonctions du calculateur logique 110
II - Structure du calculateur logique 110
III - Directives du calculateur logique 111
IV - Temps opératoires 120

Chapitre 4 : Les éléments périphériques 122
I - Organes d'entrée-sortie 122
**Le Lecteur perforateur de carte* 122
I - Fonctions du lecteur perforateur 122
II - Constitution et fonctionnement 123
III - Les Directives du lecteur perforateur 126
IV - Performances 128
**L'imprimante Gamma 60* 130
I - Description et fonctionnement 130
II - Différents types d'états 132
III - Programmation 133
IV - Profil du catène qualitatif 135
II - Mémoires auxiliaires 136
**Le ruban magnétique* 136
I - Structure du ruban magnétique 136
II - Le dérouleur de ruban: fonctionnement et performances 138
III - Programmation du dérouleur en marche discontinue 144
IV - Programmation du dérouleur en marche continue.. 148
V - Le catène qualitatif 150
**Le tambour magnétique* 152
I - Description du tambour 152
II - Programmation 153

IDÉE DIRECTRICE DU GAMMA 60

1°/ La structure générale de toute machine à traiter l'information est la suivante :

. des *organes d'entrée* prennent en charge les informations à traiter (données),

. des *organes de traitement* élaborent, à partir de ces données, les résultats que l'on désire obtenir,

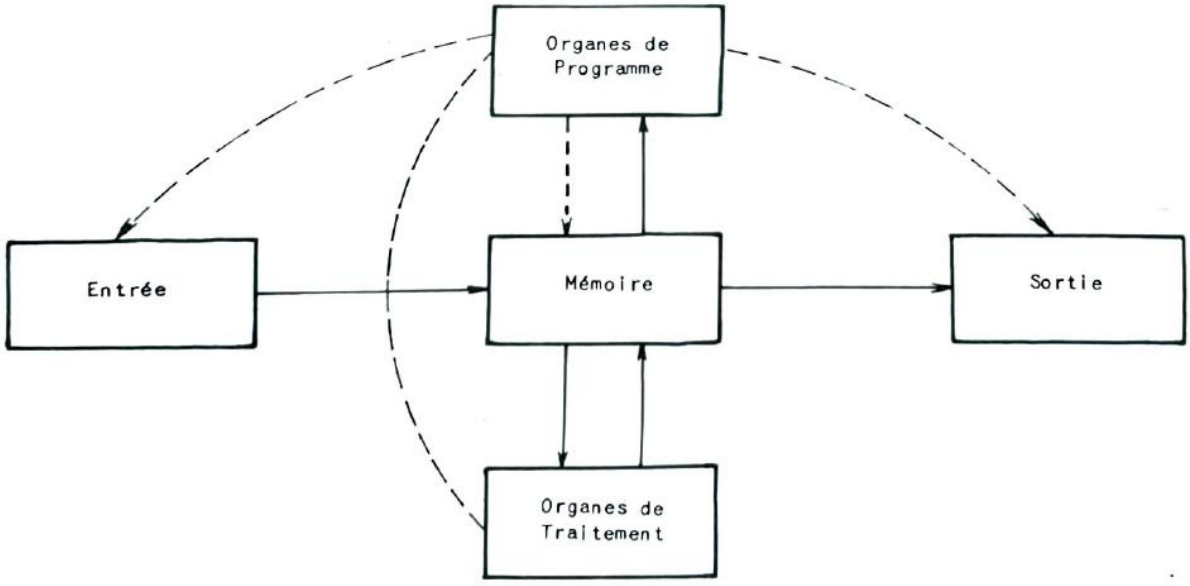
. des *organes de sortie* restituent ces résultats sous la forme voulue (par exemple sous forme d'imprimés présentés selon tel ou tel cliché d'édition).

Afin de permettre de conjuguer de la meilleure façon possible la multiplicité des informations qui interviennent dans l'obtention d'un résultat, ces machines sont dotées de "mémoires" internes de grande capacité. Ces mémoires sont des organes capables d'enregistrer n'importe quelle information, de la conserver et de la restituer à volonté. Les informations que l'on y trouve sont de deux sortes :

. *les informations à traiter* : données initiales et résultats intermédiaires ou définitifs,

. les *"instructions"* dont l'ensemble constitue le "programme", c'est-à-dire la façon dont les données doivent être traitées pour aboutir aux résultats. Ces instructions sont prises en charge depuis leur stockage en mémoire par des organes dits "de programme" qui commandent alors la façon dont fonctionnent les organes d'entrée, de traitement et de sortie.

D'où le schéma suivant très général d'une machine à traiter l'information :



2°/ Comme toute machine à traiter l'information, le GAMMA 60 comprend :

. des organes d'entrée, de sortie et de traitement que nous appellerons *"éléments"* en attendant de donner une définition plus précise à ce mot,

. des organes de mémoire et de programme qui constituent *l'"Unité Centrale"* (à noter que certains organes de programme sont propres aux éléments et en font partie contrairement à ceux qui, étant communs à tous les éléments, sont dans l'Unité Centrale (cf. 1ère partie - chapitre 1 - § 1).

La spécialisation de chaque élément, dans l'accomplissement de fonctions bien particulières, ne prend, dans le GAMMA 60, tout son sens et toute son efficacité que grâce à leur *autonomie de fonctionnement*.

Cette autonomie des éléments est la caractéristique fondamentale du GAMMA 60 : nous en définirons plus tard la portée exacte et les limites, mais, dès maintenant, il faut bien préciser qu'elle n'en fait pas des machines indépendantes car ils sont tous tributaires de l'Unité Centrale qui est le coeur du GAMMA 60.

Cette caractéristique organique fondamentale entraîne pour l'utilisateur la conséquence capitale suivante : elle rend possible *le fonctionnement simultané de plusieurs éléments*. Ceci signifie que dans le GAMMA 60 *plusieurs séquences de programme peuvent se dérouler simultanément*, chacune d'elles le faisant dans le même temps que si elle était seule. Ces séquences simultanées peuvent, soit faire partie d'un même programme "maillé", soit faire partie de problèmes totalement indépendants ; concrètement, cela veut dire qu'il est possible de traiter en même temps, sur le GAMMA 60, des problèmes aussi différents qu'un calcul de trajectoire de fusée et un problème de paye. Plus concrètement encore, cela signifie que sur le même GAMMA 60 il est possible de faire en même temps une mise à jour de fichier sur ruban magnétique, une ou plusieurs conversions ruban magnétique-imprimante, et une ou plusieurs introductions de cartes, ces trois opérations pouvant éventuellement faire partie de trois problèmes indépendants.

1ère partie

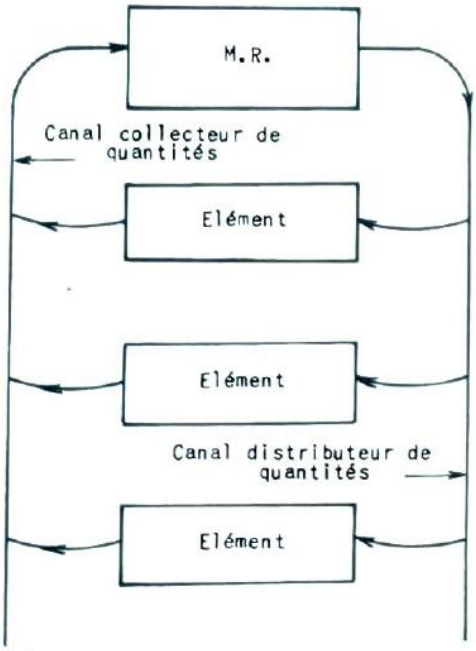
DESCRIPTION ET LOGIQUE GÉNÉRALE DE FONCTIONNEMENT

chapitre 1

STRUCTURE ORGANIQUE GÉNÉRALE (1)

Le GAMMA 60 ne comporte pas de liaison directe des éléments entre eux. Considérés sous cet aspect, ils sont absolument indépendants les uns des autres. L'unité de la machine est assurée par la liaison de tous les éléments avec un "carrefour" auquel chaque élément accède dans les deux sens par deux "canaux" *(le canal collecteur de quantités* et le *canal distributeur de quantités) communs à tous les éléments.* Le "carrefour" est la Mémoire Rapide (M.R.), organe principal de l'Unité Centrale.

Un premier schéma du GAMMA 60 est alors le schéma ci-contre :



1 - L'UNITE CENTRALE

L'Unité Centrale comprend essentiellement :

- La Mémoire Rapide centrale (M.R.) et le Distributeur de Transferts (D.T.) qui lui est étroitement associé,
- Les organes centraux de programme : le Distributeur de Programme (D.P.) et ses organes annexes.

1°/ Mémoire Rapide et Distributeur de Transferts

1.1.- La Mémoire Rapide contient toutes les informations dont on peut avoir besoin au cours du déroulement d'une séquence de programme, c'est-à-dire à la fois les instructions du programme lui-même et les informations sur lesquelles se fait le travail et que l'on désigne sous le vocable général d'opérandes. C'est par elle que transitent obligatoirement toutes les informations lorsqu'on les transfère d'un élément à un autre.

(1) - pour tout ce chapitre, on se reportera à la planche "Organisation Générale du GAMMA 60".

Physiquement, c'est une mémoire à tores magnétiques en ferrite à cycle d'hysteresis rectangulaire. Les deux états possibles de magnétisation de ces tores représentent la présence ou l'absence d'information comme dans tous les systèmes bistables utilisés dans les calculateurs numériques dont le principe de codage ne peut être que binaire.

Elle se compose d'un certain nombre de blocs de 24 plans de 64 x 64 - 4.096 tores. On peut donc aussi considérer qu'elle est composée de 4.096 "colonnes" de 24 tores superposés. Or, l'ensemble des 24 informations élémentaires (que nous appellerons "*monades*") contenues dans ces 24 tores constitue le quantum d'information utilisé dans les transferts Mémoire Rapide-Eléments. Ce quantum d'information porte le nom de *"catène"*.

1.2.- A un instant donné, on ne peut introduire ou extraire de la Mémoire Rapide qu'un seul catène à la fois. Le temps de cette introduction ou extraction est de 11 µs ; on exprime encore ceci en disant que la durée du *cycle M.R.* est de 11 µs ou que le temps d'accès de la Mémoire Rapide est de 11 µs puisque, tous les 11 µs, on peut introduire ou extraire un catène dans n'importe quelle "colonne" de la Mémoire Rapide. Chacune de ces "colonnes" porte un numéro appelé "*adresse*".

La Mémoire Rapide constitue donc une mémoire *adressable* par catène (4.096 adresses par bloc). Une Mémoire Rapide peut être constituée de 1 à 8 blocs ainsi définis et peut donc contenir jusqu'à 32.768 adresses numérotées de 0 à 32.767. Toutes ces adresses sont *banalisées* quant aux instructions du programme et aux opérandes, c'est-à-dire que l'adresse 3.500, par exemple, peut contenir une instruction du programme dans une phase du travail et un opérande dans une autre phase.

Il convient de noter que, lorsqu'un catène est extrait d'une "colonne" de Mémoire Rapide, celle-ci est remise aussitôt (c'est-à-dire au cours du même cycle M.R.) dans le même état magnétique. On peut donc dire que c'est un "double" du catène que l'on extrait, l'original demeurant en Mémoire Rapide. Il est ainsi possible d'extraire autant de doubles que l'on veut et ce, à des cycles différents de Mémoire Rapide. D'autre part, quand un catène est introduit dans une certaine "colonne", il efface celui qui s'y trouvait précédemment pour le remplacer purement et simplement.

1.3.- Les transferts Mémoire Rapide vers éléments se font par l'intermédiaire d'un "*canal distributeur de quantités*" unique qui dessert tous les éléments. Symétriquement, les transferts éléments vers Mémoire Rapide se font au moyen d'un "*canal collecteur de quantités*" également unique pour amener en Mémoire Rapide les informations en provenance des divers éléments.

Les catènes sont transmis sur les canaux à la suite les uns des autres à destination ou en provenance de tel ou tel élément. Du fait que, d'une part, *les éléments sont autonomes*, du fait, d'autre part, que ce sont eux qui *ont l'initiative des transferts*, en faisant, quand ils en ont besoin, des demandes de transfert, il est fréquent que plusieurs éléments demandent un transfert en même temps.

1.4.- Ces dernières considérations font apparaître la nécessité de cet organe étroitement associé à la Mémoire Rapide qu'est le *Distributeur de Transferts* (D.T.). Il a pour but de répartir la disponibilité des canaux entre les divers éléments ; pour cela il est essentiellement constitué par des *circuits de priorité* vers lesquels sont dirigées toutes les demandes de transferts des éléments. Grâce à une chaîne de priorité préétablie, ils déterminent l'élément qui, parmi les demandeurs, sera le bénéficiaire du prochain transfert.

Supposons, par exemple, qu'un tambour magnétique et le calculateur arithmétique demandent en même temps un transfert : le premier servi sera le premier des deux dans la chaîne de priorité du Distributeur de Transferts, c'est-à-dire le tambour magnétique. En effet, sans que l'on ait encore décrit cet élément, le lecteur doit savoir qu'un transfert tambour-Mémoire Rapide ne peut s'effectuer qu'au moment où la quantité inscrite en un endroit précis du tambour (qui est une mémoire adressable) passe sous la tête de lecture. Il s'ensuit que refuser la priorité au tambour

quand il demande un transfert risquerait de reculer celui-ci d'un tour complet de tambour, soit 20 ms. Au contraire, en la lui accordant, le transfert s'effectue aussitôt et le cycle M.R. suivant, soit 11 µs plus tard seulement, la demande du calculateur arithmétique sera à son tour satisfaite.

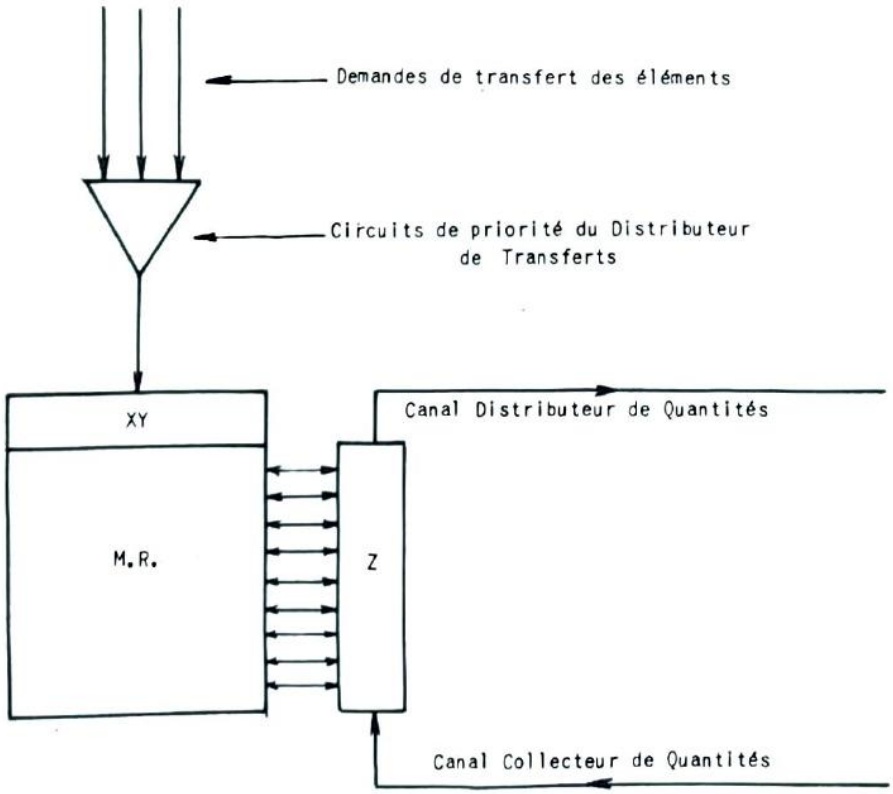
1.5.- Une fois choisi l'élément bénéficiaire du transfert, ce sont les *circuits de sélection de la Mémoire Rapide* (ou circuits XY) qui choisissent, selon le programme, l'adresse où sera transféré le catène venant de l'élément ou bien celle où sera pris le catène à transférer vers l'élément.

Dans le cas d'un transfert Mémoire Rapide vers élément, les circuits XY sélectionnent la "colonne" correspondant à l'adresse voulue. Celle-ci est alors explorée et les 24 monades du catène qu'elle contient sont envoyées en parallèle dans un certain registre Z lui-même relié aux canaux distributeur et collecteur. C'est de ce registre Z que le catène est finalement envoyé vers l'élément par l'intermédiaire du canal distributeur tandis que la "colonne" explorée est remise dans son état initial pour pouvoir éventuellement extraire un nouveau double du catène.

Dans le cas d'un transfert de sens inverse, le catène arrive dans le registre Z par l'intermédiaire du canal collecteur. C'est de là qu'il est ensuite envoyé en parallèle dans la "colonne" sélectionnée par les circuits XY.

Le registre Z apparaît ainsi avoir un rôle analogue à celui du "commun" d'un sélecteur mécanique rotatif.

On peut finalement donner de la Mémoire Rapide et de ses organes annexes le schéma suivant :



2°/ *Les Organes Centraux de programme :*
Le Distributeur de Programme (D.P.)

2.1.- Le Distributeur de Programme est l'organe central qui reçoit les demandes d'instructions des éléments en activité exactement comme le Distributeur de Transferts en reçoit les demandes de transferts : comme pour ce dernier, la possibilité de travail simultané de plusieurs éléments entraîne la possibilité de voir le Distributeur de Programme recevoir au même moment plusieurs demandes d'instructions. De même que le Distributeur de Transferts, il possède donc des circuits de priorité qui sélectionnent, parmi toutes les demandes en instance, la plus prioritaire.

Le rôle du Distributeur de Programme consiste alors essentiellement à "lire"en Mémoire Rapide les instructions destinées à l'élément sélectionné, à les interpréter partiellement et à les envoyer dans cet état vers le "programme local" (cf. § sur les éléments) de l'élément par l'intermédiaire du *canal distributeur d'instructions.*

2.2.- Il faut bien noter que pour "lire" une instruction en Mémoire Rapide, le Distributeur de Programme doit, comme les éléments, faire une demande de transfert au Distributeur de Transferts. Il se trouve également des cas où le Distributeur de Programme doit renvoyer en Mémoire Rapide soit des instructions modifiées, soit certaines quantités comme on le verra plus loin ; là encore il devra faire une demande de transfert au Distributeur de Transferts. Ainsi il existe des échanges Distributeur de Programme-Mémoire Rapide qui, comme les échanges Eléments-Mémoire Rapide, sont tributaires du Distributeur de Transferts.

Le Distributeur de Transferts peut donc être amené à régler les conflits qui peuvent se présenter entre les demandes de transferts des éléments et celles du Distributeur de Programme. D'une façon générale, les demandes de quantités d'un élément sont prioritaires par rapport aux demandes d'instructions d'un élément de même nature. Par exemple, si un dérouleur de ruban magnétique demande un transfert tandis que le Distributeur de Programme demande le transfert d'une instruction pour le compte d'un autre dérouleur, la demande de transfert du premier dérouleur sera satisfaite la première.

En revanche, s'il s'agit d'éléments de nature différentes, il pourra se faire que les demandes du Distributeur de Programme soient prioritaires par rapport à certaines demandes d'éléments. Par exemple, si le Distributeur de Programme s'alimente en instructions pour le compte d'un lecteur de cartes, il sera prioritaire par rapport au comparateur général demandant des transferts de quantités ; ceci de façon à ne pas risquer de provoquer l'arrêt mécanique du lecteur de cartes par manque d'ins tructions.

Il sort du cadre de cet exposé d'entrer dans le détail de ces mécanismes dont la connaissance n'est pas utile au programmeur puisqu'il n'a pas à s'en préoccuper : c'est la machine elle-même qui règle les conflits que la simultanéité engendre au niveau des canaux, faisant ainsi l'auto-ordonnancement des travaux en cours. Néanmoins il était nécessaire, à partir de quelques exemples, d'exposer les principes d'après lesquels sont résolus ces conflits.

2.3.- L'interprétation des instructions par le Distributeur de Programme se fait, entre autres, en fonction de ce que l'on appelle le "*catène qualitatif*" de l'élément qui regroupe toutes les informations caractéristiques de l'état de cet élément. Chaque élément a ainsi son catène qualitatif qui est constitué par la réunion des informations matérialisées par l'état de 24 mémoires unitaires, liées à l'élément et indiquant l'état des comparateurs, des mémoires signe, les incidents, etc...

Chaque fois que le Distributeur de Programme prend en charge un élément pour lui donner ses instructions, le catène qualitatif de cet élément est transmis en unité centrale. Il y est conservé dans un registre annexe du Distributeur de Programme, le registre A (ou registre d'analyse) tant que celui-ci travaille pour le compte de cet

élément. En effet, les instructions que ce dernier doit recevoir peuvent être fonction du contenu de son catène qualitatif.

2.4.- Au cours de son exécution, il est souvent nécessaire de faire évoluer le programme dans tel ou tel sens selon les résultats précédemment obtenus.

Il faut faire des choix, reconnaître certaines éventualités, déceler la conjonction de certaines conditions : or, tout cela, c'est du calcul logique (réunions et intersections logiques) et, de ce point de vue, l'organe qui en est normalement chargé le *calculateur logique*, apparaît comme un organe central de programme, auxiliaire indispensable du Distributeur de Programme dans certains cas.

Si, dans le titre de ce paragraphe, il ne figure pas parmi les organes centraux de programme, c'est simplement parce que, organiquement, il ne fait pas partie de l'Unité Centrale. C'est un élément dont la logique générale de fonctionnement ne diffère pas de celle des autres éléments : comme eux, il reçoit des instructions du Distributeur de Programme, prend ses opérandes en Mémoire Rapide et y renvoie le résultat de ses calculs, fonctionne de façon autonome et a son catène qualitatif propre (pour plus de détails, voir le § suivant sur les éléments ; on y remarquera que le calculateur logique est classé dans les éléments de traitement à côté du calculateur arithmétique).

2.5.- Or, parmi les opérations logiques qui font évoluer le programme dans un sens ou dans un autre et dont il a été question dans le § précédent, il en est que sait faire le Distributeur de Programme associé à ses organes annexes de l'Unité Centrale. Par exemple, il existe une instruction qui a pour but de poursuivre le programme à tel ou tel numéro d'instruction selon l'état de telle position du catène qualitatif d'un élément ; cette opération de choix est faite par le Distributeur de Programme.

Ainsi, dans le GAMMA 60, on a divisé les opérations logiques en deux catégories :

. les unes, simples ou fréquentes, comme celle dont il vient d'être question, sont faites en Unité Centrale,

. les autres, compliquées et longues ou simples mais rarement rencontrées, sont le fait du calculateur logique élément autonome (calculs sur polynômes logiques, calculs d'adresses par l'opérateur binaire, etc...).

Les opérations effectuées en Unité Centrale portent le nom de "fonctions centrales". Elles sont exécutées sur l'ordre de certaines instructions dites "directives centrales" qui seront étudiées dans la 2ème partie du volume, chapitre 2.

II - LES ELEMENTS

1°/ On appelle "*élément*" dans le GAMMA 60 tout organe ou ensemble d'organes capable de fonctionner de façon autonome dès lors qu'il a reçu :

. des instructions lui disant ce qu'il doit faire,
. des instructions lui donnant l'adresse des opérandes en Mémoire Rapide.

Toutes ces instructions lui viennent de l'Unité Centrale. Organiquement, les éléments sont donc rigoureusement indépendants les uns des autres ; il est évident qu'à partir du moment où des éléments ont envoyé des quantités en Mémoire Rapide, ces mêmes quantités peuvent être utilisées par n'importe quel autre élément. Si l'on se place sur le plan fonctionnel, il est non moins évident que si un élément 2 a besoin, pour travailler, des résultats élaborés par un élément 1, il ne pourra se mettre au travail tant que l'élément 1 n'aura pas élaboré ces résultats : en ce sens, l'élément 2 dépend évidemment de l'élément 1, mais cette dépendance est fonctionnelle et non organique, il s'agit d'une question de programme, non de machine.

2°/ D'après ce que nous savons déjà de l'Unité Centrale, tous les éléments sont capables :

. d'émettre des demandes d'instructions vers le Distributeur de Programme lorsqu'ils en ont besoin pour continuer à travailler. Ces demandes se font par l'intermédiaire d'un *"canal demande d'instructions"* distinct pour chaque élément.

. d'émettre vers le Distributeur de Transferts des demandes de transferts de quantités avec la Mémoire Rapide. Ces demandes se font par l'intermédiaire d'un *"canal demande de quantités"* distinct pour chaque élément.

. de recevoir des instructions depuis le Distributeur de Programme par un *"canal distributeur d'instructions"* unique sur lequel sont greffés tous les éléments.

. de recevoir des quantités (opérandes) depuis la Mémoire Rapide par l'intermédiaire d'un *"canal distributeur de quantités"* unique pour tous les éléments.

. de traiter les opérandes conformément aux instructions reçues.

. d'envoyer des quantités vers la Mémoire Rapide par l'intermédiaire d'un *"canal collecteur de quantités"* unique pour tous les éléments.

3°/ D'une façon générale, on peut distinguer dans tout élément deux parties :

. les *circuits de positions* où s'effectue le traitement des informations (circuits d'addition du calculateur arithmétique par exemple),

. les *circuits de commande* ou *programme local*. C'est à eux qu'aboutit le canal distributeur d'instructions. Dès qu'ils ont reçu du Distributeur de Programme les instructions nécessaires, ils sont capables de diriger le travail de l'élément de façon autonome.

Le programme local constitue, après le Distributeur de Programme et le calculateur logique, le dernier chaînon des organes de programme. C'est à lui qu'incombe la phase finale de l'interprétation du programme, puisque c'est lui qui commande le fonctionnement de l'élément devenu autonome.

4°/ A chaque élément sont attachés, en outre, trois types d'organes essentiels à leur fonctionnement :

. Le *Registre d'Adresse Instruction* (R.A.I.) qui contient, lorsque l'élément est en "activité" (cf. Chapitre 2) l'adresse de la prochaine instruction dont il aura besoin.

. Les *Registres d'Adresse Courante* (R.A.C.) en nombre variable (de 1 à 4) selon les éléments. Ce sont des registres qui contiennent l'adresse des opérandes sur lesquels doit travailler l'élément. Avant que l'élément ne se mette au travail de façon autonome, ils sont "chargés" de ces adresses par des instructions spéciales comme on le verra plus loin. Physiquement (comme le R.A.I.) ce sont des mémoires à 15 positions binaires qui peuvent donc contenir, exprimée en binaire pur, une adresse quelconque de 0 à 32.767.

. Le *Registre de Liaison* (R.L.H.). C'est un organe indispensable pour tous les transferts Mémoire Rapide-éléments. En effet, le débit de la Mémoire Rapide est de un catène toutes les 11 µs ; le débit propre des éléments est beaucoup plus faible. Par exemple, sur le tambour magnétique, un catène est lu ou écrit en 98 µs ; le débit du tambour est donc 9 fois plus petit que celui de la Mémoire Rapide. Il est donc nécessaire d'avoir entre l'élément et la Mémoire Rapide un organe qui joue un rôle de "tampon", de "changeur de débit" ; cet organe, c'est le registre de liaison. D'un côté, il met 11 µs à échanger un catène avec la Mémoire Rapide, de l'autre il échange ce catène avec l'élément proprement dit à la vitesse caractéristique du débit de l'élément.

Par exemple, dans un transfert Mémoire Rapide-Tambour, le registre de liaison du tambour sera alimenté au cours d'un cycle M.R., mais son contenu mettra 98 µs, soit le temps de 9 cycles M.R., à se déverser sur le tambour. Autrement dit, lorsqu'un transfert tambour d'un certain nombre de catènes sera en cours, le canal distributeur de quantités ne sera occupé que pendant 1 cycle sur 9 par ce transfert, les 8 autres cycles restant disponibles pour desservir les autres éléments qui pourraient être au travail en même temps que le Tambour.

5°/ Dans la deuxième partie du volume , nous donnerons une description détaillée des différents éléments. Il importe toutefois d'en donner dès maintenant une classification :

a) Eléments centraux de traitement

Calculateur arithmétique
Calculateur logique
Comparateur général
Traducteur

b) Eléments périphériques

. d'entrée-sortie :

Lecteurs de cartes
Lecteurs de bande perforée
Lecteurs-perforateurs de cartes
Imprimantes

. de stockage :

Tambours magnétiques
Dérouleurs de rubans magnétiques.

Il est à noter que les éléments de stockage peuvent également être considérés comme des éléments d'entrée ou de sortie. En effet, si l'on stocke des informations sur tambour ou sur ruban, ce n'est pas pour le plaisir de les stocker, mais pour les utiliser en les réintroduisant en Unité Centrale au moment voulu et avec un débit plus homogène à la rapidité des organes de traitement que ne l'est celui d'un lecteur de cartes par exemple.

III - LA CODIFICATION DES INFORMATIONS DANS LE GAMMA 60

On distingue deux types d'informations codées dans le GAMMA 60 :

1°/ Les instructions du programme

Tout programme est constitué d'une suite d'instructions élémentaires dites *"instructions canoniques"*, chacune d'elles occupant un catène. L'état des 24 positions de ce catène est ce que l'on appelle la structure ou le profil de l'instruction canonique. Nous reviendrons plus tard sur la structure de ces instructions canoniques lorsque nous les aurons davantage décrites.

Toutefois, dès maintenant, nous pouvons indiquer que, dans un très grand nombre d'instructions canoniques, les 15 dernières positions indiquent soit une adresse en Mémoire Rapide, soit une longueur exprimée en catène (longueur d'un transfert Mémoire Rapide-Tambour par exemple). Ces adresses sont codées en *binaire pur* et peuvent ainsi, éventuellement, faire directement l'objet de calculs d'adresses dans le calculateur logique.

On a vu au § sur l'Unité Centrale que toute adresse en Mémoire Rapide était un nombre de 0 à 32.767 maximum. Rappelant brièvement les principes de la numération binaire, nous allons voir que, pour exprimer 32.767 en binaire, il suffit de 15 positions binaires.

En numération décimale, on dispose de 10 chiffres 0,1,2 .....9, et dans un nombre chaque chiffre est affecté selon son rang d'un "poids" qui est une puissance de 10. Ainsi 4.832 signifie :

$$\begin{aligned} &4 \times 1.000 \quad \text{ou} \quad 4 \times 10^3 \\ +\ &8 \times 100 \quad\quad + 8 \times 10^2 \\ +\ &3 \times 10 \quad\quad + 3 \times 10^1 \\ +\ &2 \times 1 \quad\quad + 2 \times 10^0 \end{aligned}$$

On voit ainsi que le premier chiffre à droite est affecté du poids $10^0 = 1$,

le deuxième du poids 10^1 - 10, le troisième du poids 10^2 - 100, etc ... le nième du poids 10^{n-1}.

En numération binaire, on ne dispose que des deux chiffres 0 et 1 et dans un nombre exprimé en binaire chaque chiffre est affecté d'un poids qui est une puissance de 2 (2^{n-1} pour le nième chiffre à partir de la droite). Ainsi 1.011 signifie :

1×2^3 soit 8
$+ 0 \times 2^2$
$+ 1 \times 2^1$ + 2
$+ 1 \times 2^0$ + 1

soit 11 exprimé en décimal.

On voit donc qu'un nombre de 2 chiffres binaires est au maximum égal à $2^2 - 1 = 3$, un nombre de 3 chiffres binaires au maximum égal à $2^3 - 1 = 7$, et d'une façon générale, un nombre de n chiffres binaires au maximum égal à $2^n - 1$.

Ainsi 32.767, qui est égal à $2^{15} - 1$, s'exprimera en binaire par un nombre de 15 chiffres, tous des 1.

2°/ Les informations proprement dites qui font l'objet des traitements

Ces informations sont soit de caractère strictement numérique, soit de caractère alphanumérique.

Dans le premier cas, on conserve aux nombres leur caractère de représentation décimale en employant le système de codification appelé *"décimal codé binaire"*. Ainsi, le GAMMA 60 effectuera directement ses calculs dans le système décimal. En "décimal codé binaire", on représente chaque chiffre décimal par un ensemble de 4 monades affectées chacune des poids binaires 1,2,4 et 8. Les 16 possibilités offertes ainsi par ces 4 monades permettront de représenter les 10 chiffres décimaux, 6 restant inutilisées. Un catène pourra être ainsi divisé en 24/4 = 6 tétrades dont chacune contiendra un chiffre décimal.

Dans le second cas, on doit pouvoir représenter les 26 lettres de l'alphabet, les chiffres décimaux et différents signes de ponctuation. On utilise alors pour représenter les caractères alphanumériques 6 monades. Les 64 combinaisons qu'elles permettent sont largement suffisantes. Un catène pourra alors être divisé en 24/6 = 4 *hexades* dont chacune contiendra un caractère alphanumérique.

En résumé, un catène peut contenir :
. soit une instruction canonique,
. soit 6 chiffres décimaux,
. soit 4 caractères alphanumériques.

IV - LE CONTROLE INTERNE DU GAMMA 60

Le contrôle interne du GAMMA 60 s'étend à toutes les phases du traitement de l'information, depuis l'entrée en machine jusqu'à l'extraction des résultats. Cela pose 3 sortes de problèmes quelque peu différents :

1.- le contrôle des transferts d'informations
2.- le contrôle du traitement proprement dit (calcul, traduction, interpretation des instructions par les organes de programme)
3.- le contrôle des organes d'entrée-sortie.

D'un autre point de vue, il faut également considérer :
. le contrôle des circuits de commande
. le contrôle des circuits de position.

1°/ *Contrôle des Transferts*

1.1.- *Transfert d'informations codées.* C'est le cas du transfert des informations catène par catène des éléments vers l'Unité Centrale ou de l'Unité Centrale vers les éléments. Ces catènes peuvent contenir soit des informations alphanumériques (6 monades par caractères, soit des informations numériques (4 monades par chiffre), soit même des instructions canoniques (au moment de l'alimentation du programme en Mémoire Rapide à partir d'un tambour par exemple). Dans tous les cas, chaque catène est accompagnée de sa clef par 7.

Cette clef s'obtient de la façon suivante : chaque catène, quelle que soit la nature des informations qu'il contient, est considéré comme un nombre binaire de 24 positions et sa clef est le reste obtenu lorsqu'on le divise, en binaire pur bien entendu, par 7. Cette clef par 7 (qui varie de 0 à 6 compris) nécessite 3 monades.

Ainsi chaque catène se compose de 24 monades d'information proprement dite, plus 3 monades de clef ; c'est sous cette forme qu'ils sont stockés dans les diverses mémoires du GAMMA 60 : Mémoire Rapide centrale, tambours ou rubans magnétiques (il en résulte en particulier que chaque bloc de Mémoire Rapide sera formé technologiquement, non de 24, mais de 27 plans superposés de 4.096 tores).

Le contrôle s'effectue alors comme suit : lorsqu'on lit un catène en Mémoire Rapide pour le transférer en tambour par exemple, on le lit clef comprise et un générateur de clef forme de son côté la clef des 24 monades d'information lues ; la vérification de l'identité entre la clef lue et la clef formée garantit l'exactitude de la lecture. A la réception sur le tambour, on vérifie que le catène reçu a été transferé sans altération par le fait que la clef reçue (qui a accompagné le catène dans tout son transfert) est identique à la clef qu'un générateur de clef local forme à l'arrivée du catène. Tous les transferts d'information codée sont ainsi contrôlés par la vérification de la concordance information-clef. Ceci est encore vrai lorsque les informations transmises le sont par quantités de moins de 1 catène (cas de certains transferts à l'intérieur de l'Unité Centrale).
La quantité transférée (15 ou 9 monades en général) est encore considérée comme un nombre binaire (de 15 ou 9 positions) et l'on forme aussi sa clef par 7.

Ce type de contrôle est d'une très grande sécurité que l'on peut presque qualifier d'absolue : il ne peut pas laisser passer la perte de 2 monades alors qu'un contrôle par double clef de parité a 1 chance sur 8 de laisser passer ce genre d'erreur ; pour ne pas détecter la perte de 3 monades, celles-çi doivent être consécutives (condition pour que le reste de la division par 7 ne soit pas altéré). Des calculs de probabilité ont montré que dans le cas de 3 défaillances simultanées, il y avait environ 7 fois moins de chances de laisser passer ce genre d'erreur avec la clef par 7 qu'on en aurait avec une double clef de parité et que cet écart en faveur de la clef par 7 allait en s'accroissant avec le nombre de défaillances simultanées.

1.2.- *Transfert d'informations non codées.* C'est le cas de signaux transmis sur un seul fil : on double alors le transfert de l'information sur deux circuits différents.

A ce type d'informations non codées se rattachent les signaux qui font fonctionner les *circuits de commande.* Pour en contrôler le bon fonctionnement, on ne les double pas, mais on opère de la façon suivante : dans toute la machine, les informations codées elles-mêmes et leurs clefs sont traitées sur des circuits de position distincts ; ces circuits de position distincts ont eux-mêmes leurs circuits de commande distincts avec leurs enchaînements propres ; la concordance information-clef vérifie alors à la fois le bon fonctionnement des circuits de commande et des circuits de position.

2°/ Contrôle du traitement proprement dit

Il s'appuie sur le fait que l'on fait subir le même traitement aux informations et aux clefs. On applique alors le théorème suivant : la clef du résultat obtenu sur les informations elles-mêmes est identique au résultat obtenu en faisant le même traitement sur les clefs des opérandes. Ceci s'exprime de façon plus lapidaire en disant : la clef du résultat est égale au résultat des clefs.

Deux cas se présentent :

a) L'organe de traitement conserve à l'information son caractère binaire : tout ce qui a été sur les transferts demeure valable en remplaçant le mot transfert par le mot traitement (cas du calculateur logique et des traitements effectués sur les instructions en Unité Central).

b) L'organe de traitement ne conserve pas à l'information son caractère binaire cas de l'unité arithmétique et du traducteur. Il faut envisager chaque cas séparément. On en parlera dans la seconde partie du volume , lorsqu'on décrira les éléments en question.

3°/ Contrôle des éléments d'entrée-sortie

Les problèmes soulevés sont spéciaux pour chaque organe. On les étudiera également dans la seconde partie au moment de leur description.

FONCTIONNEMENT DU GAMMA 60

I - INTRODUCTION ET DEFINITIONS PREALABLES

1°/ Supposons la machine en fonctionnement. Plusieurs éléments travaillent en même temps, chacun d'eux effectuant une opération intégrée dans des séquences de programme différentes.

Dans une zone de Mémoire Rapide figurent les instructions relatives aux différentes séquences; par exemple, dans les adresses 1.000 à 1.125, les instructions de la séquence 1, de 2.130 à 2.175, les instructions de la séquence 2, etc ...

Dans une autre zone de Mémoire Rapide se trouvent les quantités sur lesquelles sont faites les opérations.

Chaque séquence de programme empruntera successivement un certain nombre d'éléments. Certains d'entre eux pourront d'ailleurs être utilisés par plusieurs séquences simultanées, un même élément pouvant travailler successivement pour chacune de ces séquences. *L'auto-ordonnancement des opérations est réalisé grâce :*

. aux circuits de priorité du Distributeur de Programme et du Distributeur de Transferts,

. au mécanisme dit *"de chaîne de reprise"* qui est mis en oeuvre lorsqu'un même élément est appelé en même temps par plusieurs séquences simultanées (cf. § VI).

2°/ *Etat des éléments*

A un instant donné, les éléments qui ne sont utilisés par aucune séquence sont dits au *"repos"*. Les autres sont dits en *"activité"* : ils ont alors connaissance d'une adresse d'instruction courante conservée dans leur *Registre d'Adresse Instruction* (R.A.I.).

Cet état d'activité présente deux aspects :

. *Attente*
C'est l'état de l'élément qui a terminé sa dernière opération en fonctionnement autonome et qui n'a pas encore reçu les instructions lui permettant de fonctionner à nouveau.

. *Opération*
C'est l'état de l'élément qui vient de recevoir ses instructions et qui fonctionne alors de façon autonome sur son programme local.

Ces définitions préalables étant données, nous commencerons par expliquer le fonctionnement du GAMMA 60 traitant un programme filiforme.

II - EXPLOITATION D'UN PROGRAMME FILIFORME

1°/ Le Distributeur de Programme au service d'un élément

En expliquant comment le Distributeur de Programme donne à un élément les instructions dont il a besoin pour entrer en phase opératoire, nous serons amenés

à en décrire les principaux registres et à donner un schéma fonctionnel de ses relations avec les éléments et les autres organes de l'Unité Centrale. On trouvera le schéma à la fin de ce § et l'on pourra s'y reporter pour toutes les explications qui vont suivre.

1.1.- Supposons la machine en fonctionnement. Plusieurs éléments sont enétat d'attente et envoient au Distributeur de Programme des demandes d'instructions par l'intermédiaire de leurs canaux *"demandes d'instructions"* respectifs.

Ces demandes d'instructions (D.I.) arrivent sur les circuits de priorité du Distributeur de Programme qui sélectionnent la demande de l'élément le plus prioritaire ; ceci a pour effet de faire apparaître le numéro de l'élément ainsi sélectionné dans un registre spécialisé du Distributeur de Programme, le *Registre Numéro d'Elément* (R.N.E.) qui, comme les R.A.C. et les R.A.I. des éléments, est un registre de 15 positions binaires (on verra, en effet, que dans certaines instructions canoniques, les 15 premières positions qui, dans d'autres instructions, contiennent une adresse ou une longueur comme on l'a déja vu, contiennent cette fois-ci un numéro d'élément exprimé en binaire pur).

1.2.- L'apparition du Numéro de l'élément sélectionné dans le R.N.E. fait apparaître à son tour le contenu du R.A.I. de cet élément dans un autre registre spécialisé du Distributeur de Programme : le *Registre Adresse de Programme Actuel* (R.A.P.A). Ceci se fait de la façon suivante : les R.A.I. de tous les éléments sont technologiquement groupés en une même matrice à tores, la Matrice Adresse Programme (M.A.P.), physiquement semblable à la Mémoire Rapide et située géographiquement dans l'Unité Centrale. Chaque "colonne" de 15 tores de cette matrice est le R.A.I. d'un élément, le numéro de cet élément étant présisément l'adresse de cette colonne dans la matrice.

Le mécanisme de sélection dans cette matrice est analogue au mécanisme de sélection dans la Mémoire Rapide : des circuits X sélectionnent la "colonne" dont l'adresse est contenue dans le R.N.E.; son contenu est alors envoyé dans un registre Z d'où il est à son tour envoyé dans le R.A.P.A. (registre de 15 positions lui aussi).

1.3.- Dès que le Numéro d'Instruction à alimenter apparaît dans le R.A.P.A., il apparaît également, mais progressé de 1, dans un autre registre de 15 positions, le *Registre Adresse de Programme Progressé* (R.A.P.P.). Le Distributeur de Programme est alors prêt à lire en Mémoire Rapide les Instructions destinées à mettre l'élément sélectionné (qui est également dit : élément *demandeur* d'instructions) au travail.

Pour cela, il fait une demande de transfert au Distributeur de Transferts qui l'agrée à un moment donné. Le catène dont l'adresse figure dans le R.A.P.A. est alors sélectionné en Mémoire Rapide et envoyé dans les circuits d'analyse du canal distributeur de quantités et du *Registre d'Entrée du Distributeur de Programme* (R.E.D.P.). Les circuits d'analyse du Distributeur de Programme interprétent le contenu de ce catène et, selon le type d'instructions reconnu,

. ou bien ils chargent les R.A.C. de l'élément demandeur d'une adresse d'opérande. Ce chargement s'effectue sans passer par les canaux (cf. schéma) parce que, comme les R.A.I., tous les R.A.C. de tous les éléments sont géographiquement groupés dans l'Unité Centrale.

. ou bien ils envoient des "directives" au programme local de l'élément par l'intermédiaire du *Registre de Sortie du Distributeur de Programme* (R.S.D.P.) et du canal distributeur d'instructions.

. ou bien ils renvoient des informations en Mémoire Rapide par l'intermédiaire du R.S.D.P. et du canal collecteur de quantités.

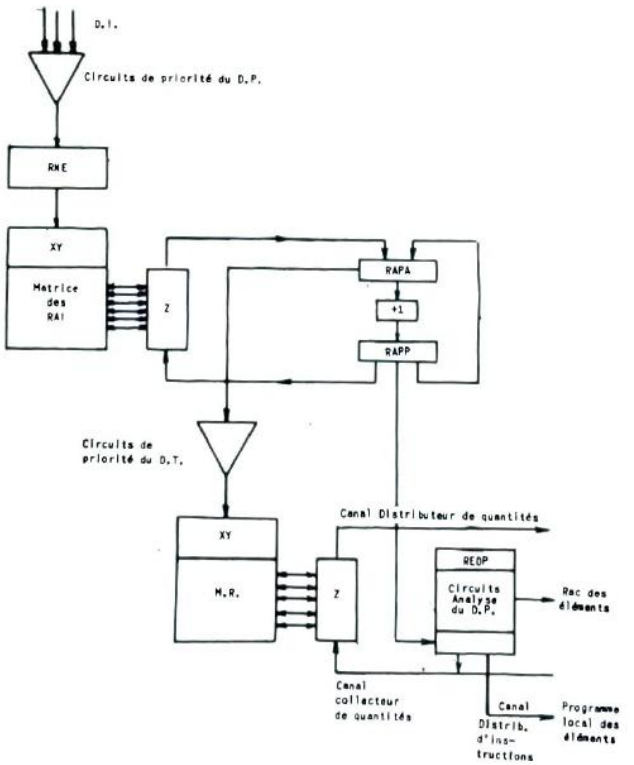
. ou bien encore, dans des cas que nous verrons plus loin (coupures et branchements), ils agissent d'une façon spéciale qui n'entre pas dans les catégories précédentes.

1.4.- Le contenu du R.A.P.P. est alors envoyé (d'une façon générale) dans le R.A.P.A. et le même mécanisme recommence avec l'instruction suivante : le Distributeur de Programme fait une demande de transfert au Distributeur de Transferts etc ..., puis le contenu du R.A.P.P. est envoyé dans le R.A.P.A. et ainsi de suite jusqu'à ce que l'on ait alimenté toutes les instructions nécessaires à mettre l'élément en phase opératoire. A ce moment là, le contenu du R.A.P.P. (qui est l'adresse de la prochaine instruction dont aura besoin l'élément à la fin de sa phase opératoire) est envoyé, non pas dans le R.A.P.A., mais dans le R.A.I. de l'élément dont le Numéro figure toujours dans le R.N.E. ; ceci par l'intermédiaire du registre Z de la matrice des R.A.I., et le numéro d'élément est effacé dans le R.N.E.

A ce moment là (en faisant abstraction du cas où le mécanisme de chaîne de reprise est mis en oeuvre, cas examiné en détail au chapitre 3), le Distributeur de Programme est libéré de la séquence de programme dont il a été jusqu'ici question. Il peut alors prendre en considération l'une des Demandes d'Instructions qui peuvent être en attente à ses circuits de priorité pour se mettre au service d'un nouvel élément travaillant sur une autre séquence de programme qui pourra ainsi s'exécuter simultanément à la première.

1.5.- Bien entendu, comme on l'a dit au chapitre 1, pendant tout le temps que le Distributeur de Programme est au service d'un élément, le catène qualitatif de cet élément est dans un registre A (registre d'Analyse) auxiliaire du Distributeur du Programme. Ce registre, à ne pas confondre avec les circuits d'analyse du Distributeur de Programme, n'est pas figuré sur le schéma ci-dessous pour ne pas le charger. Néanmoins, comme on le verra plus loin, certaines instructions exploitent son contenu. Le catène qualitatif de l'élément y est envoyé au moment où son numéro apparaît dans le R.N.E.

Enfin, sur le schéma figure une liaison dont l'explication ne se trouve pas dans les alinéas précédents. C'est la liaison qui va du R.A.P.P. au R.S.D.P. En effet, comme on le verra plus loin, il est des instructions après lesquelles le contenu du R.A.P.P. est envoyé dans le R.S.D.P.



2°/ *Les différents types d'Instructions canoniques*

On vient de voir comment le Distributeur de Programme donne à un élément tous les renseignements nécessaires pour le mettre au travail. Quels sont ces renseignements ?

. Une condition préalable à remplir est que l'élément soit en activité, c'est-à-dire qu'il ait connaissance dans son R.A.I. d'une adresse d'instruction.

. Il faut ensuite désigner à l'élément ses opérandes et lui dire ce qu'il doit en faire.

. Enfin, il faut éventuellement lui donner la possibilité d'orienter son travail dans tel ou tel sens suivant les résultats obtenus antérieurement.

Or il est impossible de faire tenir tous ces renseignements dans une seule instruction canonique ; d'où la nécessité de les répartir dans plusieurs instructions canoniques de types différents, chaque type d'instruction étant spécialisé dans un type de renseignement déterminé.

Ces différents types d'instructions canoniques sont :

Instruction C (coupure)

C'est l'instruction qui met l'élément en activité en le désignant. Son effet est de charger son R.A.I. du Numéro d'instruction qui suit celui de la coupure. Ce nom de "coupure" sera justifié dans le chapitre 3.

Instruction A (Adresse)

Toute instruction A désigne à l'élément l'un des opérandes sur lesquels il devra travailler en chargeant l'un de ses R.A.C. de l'adresse de cet opérande en Mémoire Rapide.

Instruction D (Directive)

C'est l'instruction qui dit à l'élément ce qu'il doit faire des opérandes que l'on vient de lui désigner. C'est lorsque l'élément a reçu une telle instruction qu'il a tous les renseignements utiles pour commencer sa phase de travail autonome. C'est donc à la suite d'une telle instruction que le contenu du R.A.P.P. sera renvoyé dans le R.A.I. de l'élément et non dans le R.A.P.A. (on verra par la suite qu'il existe certaines directives, directives préparatoires élément et directives centrales qui agissent de façon différente).

Instruction B (Branchement)

C'est une instruction qui modifie la progression naturelle du Numéro d'instruction du R.A.P.A. en le forçant (sous certaines conditions ou non) à une adresse désignée par l'instruction elle-même. Le contenu du R.A.P.P. est, dans ce cas, envoyé dans le R.S.D.P.

L' ensemble des instructions canoniques nécessaires pour mettre un élément au travail constitue ce que l'on appelle une *instruction complète*. Les instructions qui, dans un programme, jalonnent les instructions complètes sont donc les *Directives* (sauf les directives préparatoires et les directives centrales).

On voit que rien, dans les instructions canoniques A, B ou D, ne désigne l'élément auquel elles sont destinées. On ne le sait que parce qu'elles ont été appelées à la demande de tel élément dont le numéro figure alors dans le R.N.E. du Distributeur de Programme.

En d'autres termes, toutes les instructions qui, dans une séquence de Programme, suivent une instruction de coupure, sont destinées à l'élément désigné par la coupure. Une telle suite d'instructions canoniques (qui peuvent constituer plusieurs instructions complètes) est une *séquence élémentaire*, ainsi dénommée parce que toutes les instructions canoniques d'une telle séquence sont destinées au même élément. Les instructions qui, dans un programme, jalonnent les séquences élémentaires sont les instructions de coupure (sur élément réel ; cette précision étant donnée pour les distinguer d'autres instructions de type C qui ne sont pas à considérer dans le découpage d'un programme en séquences élémentaires).

3°/ Fonctionnement autonome d'un élément

Dans le cas le plus général, on distingue 3 phases dans le travail autonome d'un élément :

a) Une phase d'alimentation des opérandes : l'élément fait au Distributeur de Transferts des demandes de transferts pour les alimenter. Chaque fois qu'une demande de transfert est prise en considération, un catène de Mémoire Rapide (dont l'adresse est contenue dans un R.A.C. de l'élément) est sélectionné et envoyé à l'élément ; le R.A.C. qui a désigné le catène à transférer progresse de 1 pour préparer le transfert du prochain catène. Ainsi les opérandes dont la longueur est supérieure à 1 catène sont alimentés catène par catène, le R.A.C. de l'opérande correspondant progressant chaque fois de 1. L'élément fait une demande de transfert pour chaque catène.

b) Une phase de travail proprement dit pendant laquelle les canaux sont complètement disponibles pour d'autres éléments.

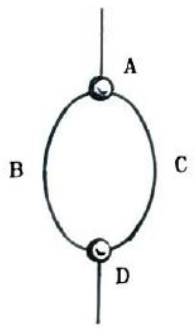
c) Une phase de rangement en Mémoire Rapide du résultat obtenu à l'adresse désignée par un autre R.A.C. (ceci se faisant aussi catène par catène avec progression du R.A.C. de 1 à chaque fois).

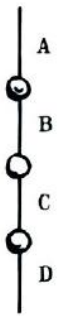
Il est, bien entendu, des cas où l'on ne peut distinguer ces 3 phases : on verra plus loin la description d'un transfert Tambour. Le cas idéal où ces 3 phases sont bien distinctes est celui du Calculateur Arithmétique par exemple.

III - EXPLOITATION D'UN PROGRAMME MAILLE

Lorsqu'on étudie un problème, ou même simplement une partie de problème, et qu'on en est arrivé à la phase finale de programmation, il faut commencer par établir un organigramme d'enchaînement des opérations à effectuer avant de passer à la codification des instructions. Cet organigramme, du type de celui présenté au § V, se présentera presque toujours et de façon tout à fait naturelle sous la forme d'un organigramme maillé, c'est-à-dire qu'il y aura des moments où, à la fin d'une opération donnée, on devra effectuer à la fois 2 ou plusieurs opérations indépendantes avant de reprendre une autre partie du programme.

A la fin de la séquence A par exemple, on devra effectuer les séquences indépendantes B et C avant de traiter la séquence D.





Sur une machine qui ne sait traiter que des programmes filiformes, on est obligé de transformer un tel organigramme maillé en organigramme filiforme, c'est-à-dire de décider, par programme, d'un ordre dans lequel on exécutera successivement B et C, ceci pour aboutir à un organigramme filiforme.

Dans une machine comme le GAMMA 60 qui, comme nous l'avons vu, peut traiter simultanément plusieurs séquences indépendantes filiformes, il suffit pour pouvoir traiter l'organigramme tel qu'il se présente, d'avoir des instructions qui permettent :

1°) de dédoubler une séquence A en 2 séquences B et C qui seront ensuite traitées simultanément et indépendamment l'une de l'autre.

2°) de regrouper les séquences B et C en une séquence D, c'est-à-dire de n'autoriser le début de D que lorsque les séquences B et C seront toutes deux terminées, étant bien entendu que l'on ne sait pas laquelle de B ou de C se terminera la première.

Le dédoublement de séquence s'obtient au moyen d'une instruction appelée instruction SIMU (simultanéité) qui permet de lancer à côté de la branche ordinaire dont l'adresse est celle de l'instruction SIMU progressée de 1, une branche extraordinaire dont l'adresse est indiquée par l'instruction SIMU elle-même. Cette instruction est une instruction de type C, nous verrons pourquoi dans le Chapitre 3.

Le Distributeur de Programme exploite cette instruction SIMU de la façon suivante : soit en adresse 100 une instruction SIMU indiquant 200 comme adresse de branche extraordinaire. Le Distributeur de Programme exploitera cette instruction en envoyant l'adresse extraordinaire 200 dans le R.A.P.A. et en rangeant le contenu du R.A.P.P. (soit 101) dans un registre spécial de mise en réserve (non figuré sur schéma du § II.1). Le Distributeur de Programme traite alors la branche extraordinaire de façon tout à fait normale jusqu'à trouver une instruction qui le libère de cette séquence (c'est-à-dire soit une coupure qui, mettant un élément en activité, lui confie en quelque sorte la gestion de cette séquence en le chargeant de faire ultérieurement des demandes d'instructions au Distributeur de Programme, soit une directive).

L'adresse de la branche ordinaire 101 est alors ramenée dans le R.A.P.A. et le Distributeur de Programme la traite jusqu'à une instruction (coupure ou directive) le libérant de cette séquence. A partir de ce moment, les deux branches vont s'exécuter simultanément et indépendamment l'une de l'autre ; il n'y a plus de branche ordinaire et de branche extraordinaire.

Dans le cas où l'on aurait une séquence qui, à un moment donné, devrait lancer plus de deux branches simultanées, on le ferait par une cascade de SIMU et l'on verra au chapitre 3 comment le Distributeur de Programme exploite une telle cascade.

Pour le regroupement de plusieurs séquences en une seule, l'organisation du programme est fondé sur la remarque suivante : il est impossible que les séquences à regrouper se terminent sur des séquences élémentaires ayant trait au même élément. En effet, un élément ne peut travailler que pour le compte d'une seule séquence à la fois et s'il est appelé simultanément par plusieurs séquences, le mécanisme de chaîne de reprise sera mis en jeu (cf. § VI). Ce mécanisme a simplement pour effet de faire que les séquences appelantes seront desservies par l'élément, l'une après l'autre, dans l'ordre même où elles se sont présentées. Si donc toutes les séquences simultanées à regrouper se terminaient par une séquence élémentaire se rapportant au même élément, le programme pourrait quand même fonctionner, mais, à ce dernier stade, la simultanéité ne pourrait plus jouer. Autant faire alors le regroupement plus haut alors que les diverses séquences sont prises en charge par des éléments différents.

Dès lors, comme on ne sait pas quelle sera la dernière séquence qui se terminera, on est obligé de commencer la séquence de regroupement par une coupure désignant l'élément travaillant au début de cette séquence, même si cet élément travaillait déjà à la fin d'une des séquences à regrouper. *L'instruction de regroupement de séquences sera donc l'instruction de coupure.*

A cet effet, l'instruction de coupure n'est pas obligatoirement effective chaque fois qu'elle est appelée par le Distributeur de Programme. Cela n'est vrai que lorsqu'elle se trouve en cours d'une séquence filiforme, c'est-à-dire lorsqu'il n'y a qu'une condition attendue.

Par contre, lorsqu'elle servira à regrouper des séquences (3 par exemple), il faudra qu'elle ne devienne effective que lorsque toutes les séquences à regrouper auront été terminées (c'est-à-dire dans le cas présent, lorsque la coupure aura été appelée pour la 3ème fois). Pour cela, on indique dans l'instruction de coupure elle-même le *nombre de conditions supplémentaires attendues* (nombre de séquences à regrouper moins une). Chaque fois qu'elle est appelée, le Distributeur de Programme

le compte et l'instruction ne devient effective que lorsque le nombre d'appels est égal au nombre de conditions attendues. On peut ainsi regrouper sur une instruction de coupure jusqu'à 4 séquences. Si ce nombre est insuffisant, on utilise une cascade de coupures, comme on utilise une cascade de SIMU pour plus de 2 séquences à lancer simultanément.

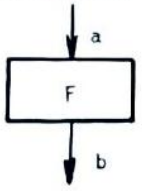
Bien entendu, pour que la même instruction de coupure soit appelée à la fin des diverses séquences à regrouper, il faut que toutes ces séquences (sauf une peut-être) se terminent par un "Branchement toujours" qui renvoie systématiquement le programme à l'adresse de cette coupure.

En résumé, en dehors de l'amélioration très notable des performances qui en résulte par rapport aux machines qui doivent traiter l'une après l'autre les différentes branches d'un même programme, la simultanéité offre, au moyen de l'instruction SIMU, une très grande facilité de programmation : elle permet, en effet, de traiter l'organigramme sous la forme même où il se présente naturellement à l'analyse ; elle permet ainsi d'économiser au programmeur le travail de transformation d'un réseau maillé en programme filiforme.

IV - SYMBOLISME FONCTIONNEL

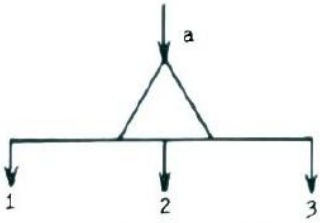
Comme on l'a vu au § précédent, il faut toujours commencer, avant toute programmation instruction par instruction, par établir un organigramme de l'enchaînement logique des opérations à effectuer. Pour établir cet organigramme, on utilise un symbolisme spécial, dit symbolisme fonctionnel. Ce paragraphe a pour objet la description de ce symbolisme.

1°/ Opération

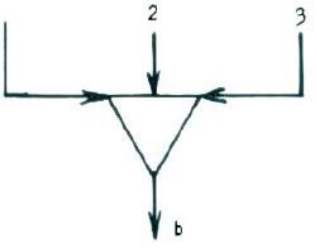


Ce rectangle précédé et suivi d'une flèche représente une opération ou une fonction plus complexe initiée par le conditionnement "a" et lançant le conditionnement "b". A l'intérieur du rectangle, on désigne la nature de l'opération ou de la fonction en question.

2°/ Condition "ou" exclusif



ventilation "ou"



regroupement "ou"

Le premier symbole indique que le conditionnement "a" ne lance que l'un des conditionnements 1,2 ou 3 selon certaines observations faites au cours des opérations qui précèdent.

Le second symbole indique que le conditionnement "b" est lancé par l'un quelconque des conditionnements 1, 2 ou 3. Il symbolise en quelque sorte la somme

logique de plusieurs conditionnements (le chiffre de 3 conditionnements n'est pas limitatif).

3°/ Condition "et"

Le symbole de gauche indique que le conditionnement "a" lance les 3 conditionnements 1, 2 et 3. C'est une répartition logique de conditionnements (programmation par instructions SIMU).

Le symbole de droite indique qu'il faut attendre la réalisation des trois conditionnements 1,2,3, pour réaliser le lancement du conditionnement "b". C'est un produit logique de conditionnements.

4°/ Positionnement d'aiguillage

La ventilation logique K est positionnée en 2.

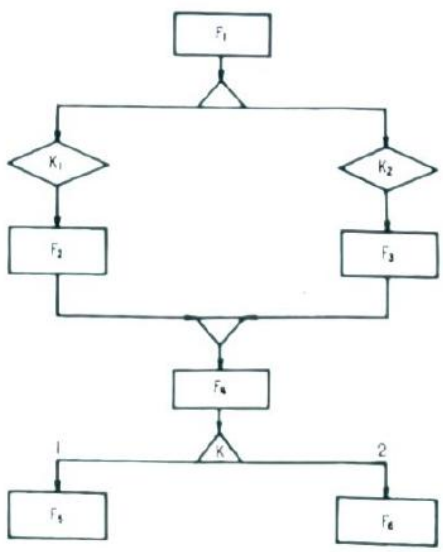
Ce symbole n'est pas utilisé dans l'organigramme traité au § suivant.

Pour en préciser l'utilisation, un exemple vaut mieux que de longs discours : la fonction F_1 (qui peut être une comparaison) lance la fonction F_2 ou F_3 qui (l'une ou l'autre) lancent à leur tour la fonction F_4. Cette fonction doit elle-même lancer la fonction F_5 ou F_6 selon que l'on aura précédemment exécuté la fonction F_2 ou F_3. Sur la branche où se trouve F_2, on mettra donc le positionnement K_1, ce qui veut dire que, lorsque l'on arrive à la ventilation K, on lancera le conditionnement 1. Même chose pour K_2 (cf. organigramme page suivante).

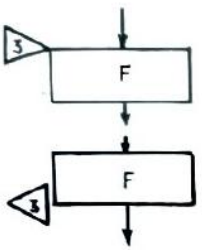
Ainsi, si F_2 ----------> F_5

si F_3 ----------> F_6

On reviendra sur cette question dans la deuxième partie du cours pour voir comment un tel positionnement peut se programmer.



5°/ Eléments virtuels

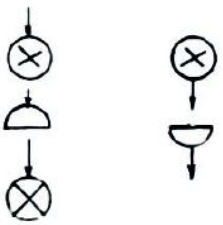


Occupation de l'élément virtuel 3 au début de la fonction F.

Libération de l'élément virtuel 3 à la fin de la fonction F.

Nous expliquerons plus loin ce que sont ces éléments virtuels. Signalons simplement dès à présent qu'ils servent à interdire l'exécution simultanée de certaines séquences de programme qui utilisent, par exemple, une même zone de Mémoire Rapide.

6°/ Symboles divers



Transfert graphique d'un conditionnement sur un même organigramme (pour éviter de trop longues flèches rendant difficile à comprendre l'organigramme).

Transfert graphique d'un conditionnement sur un organigramme différent.

Fin de séquence (suppression logique d'un conditionnement).

V - ILLUSTRATION DES PARAGRAPHES PRECEDENTS PAR UN EXEMPLE CONCRET

Nous nous proposons d'illustrer le fonctionnement du GAMMA 60 sur l'organigramme de la planche 1 considérée comme un exemple.

La planche 2 est un diagramme qui montre l'évolution des différents registres des éléments et du Distributeur de Programme pour les opérations situées aux niveaux 1, 2 et 3 de l'organigramme.

1°/ Les transferts Tambour

Sur l'organigramme, nous avons fait précéder le Transfert Tambour (T.T.) 1 d'une flèche, ce qui signifie qu'il n'est qu'une partie d'un organigramme plus vaste. Ceci veut dire que nous prenons la machine en marche.

Le R.A.I. du tambour contient l'adresse de la prochaine instruction à utiliser ; c'est-à-dire que cet élément est "en activité". Le Distributeur de Programme va, à un moment donné, travailler pour son compte, c'est-à-dire lui donner toutes les instructions utiles pour le mettre au travail. Or, quelles sont les instructions?

Le tambour doit savoir :

. l'adresse en Mémoire Rapide où l'on doit prendre ou envoyer le premier catène objet du transfert,
. l'adresse en Tambour où doit être pris ou envoyé ce catène,
. le sens du transfert ainsi que sa longueur exprimée en nombre de catènes

Ces trois sortes de renseignements sont contenues en Mémoire Rapide dans 3 catènes d'adresses 100, 101 et 102.

Lorsque le Distributeur de Programme prend en considération la Demande d'Instructions du Tambour (cf. planche 2) le Numéro d'élément du Tambour, soit 102, apparaît dans le R.N.E., puis le contenu du R.A.I. du Tambour, soit 100, est envoyé dans le R.A.P.A.

Le Distributeur de Programme peut alors alimenter les instructions pour le compte du Tambour en faisant à chaque fois une demande de transfert au Distributeur de Transferts.

A l'adresse 100, il y a une instruction A qui indique l'adresse en Mémoire Rapide où l'on devra prendre le premier catène à transférer. Au reçu de cette instruction, le Distributeur de Programme charge le R.A.C. du Tambour (le tambour n'a qu'un seul R.A.C.) de cette adresse, soit 2.545.

Le R.A.P.A. progresse alors de 1 en recevant le contenu du R.A.P.P. (non figuré sur la planche 2). Le Distributeur de Programme fait une nouvelle demande de transfert : il reçoit alors l'instruction qui se trouve en 101. C'est une instruction Da (Directive Adresse) dont le rôle est d'indiquer l'adresse en Tambour où doit être envoyé ou pris le premier catène objet du transfert ; elle charge le Registre Elément (R.A.E.) de cette adresse, soit 310.

Comme toute directive élément, cette instruction libère le Distributeur de Programme de la séquence en cours en renvoyant le contenu du R.A.P.P. (soit celui du R.A.P.A. + 1) dans le R.A.I. du Tambour. Mais celui-ci n'a pas tous les renseignements nécessaires pour pouvoir se mettre en phase de travail autonome (cette directive est du type "directive préparatoire") ; il doit donc faire une nouvelle demande d'instructions au Distributeur de Programme. Lorsqu'elle est prise en considération, le contenu du R.A.I. (soit alors 102) est envoyé dans le R.A.P.A. et l'instruction se trouvant à l'adresse 102 est alimentée. C'est une instruction D (Directive Opératoire) qui indique au programme local de l'élément à la fois :

. le sens du transfert (ici Mémoire Rapide vers Tambour),

. sa longueur. Cette longueur, ici 5, ira s'enregistrer dans un registre particulier du programme local, le Registre Longueur (R.L.).

Après cette directive, le Distributeur de Programme est à nouveau libéré (le contenu du R.A.P.P. 103 va dans le R.A.I. du tambour) mais cette fois-ci le tambour peut entrer en phase de travail autonome.

Pendant ce temps, le Distributeur de Programme pourra éventuellement donner des instructions à un autre élément (si un autre problème que celui représenté sur l'organigramme de la planche 1 est en cours en même temps dans la machine).

Quant au transfert lui-même, il s'exécute catène par catène comme il a été expliqué au premier chapitre (§ II). Pour chaque catène transféré, le tambour doit faire une demande de transfert au Distributeur de Transferts ; d'autre part, son exécution entraîne une progression de 1 du R.A.C. et du R.A.E. et un décomptage de 1 du R.L. Ainsi, à chaque transfert, le catène dont l'adresse en Mémoire Rapide se trouve dans le R.A.C. est envoyé sur le tambour à l'adresse contenue dans le R.A.E. le processus s'arrête lorsque le R.L. est arrive à 0.

Finalement ce transfert aura amené les 5 catènes dont les adresses Mémoire Rapide vont de 2.545 à 2.549 compris, sur le tambour aux adresses 310 à 314 compris. En fin d'opération, les contenus des R.A.C., R.A.E. et R.L. du tambour sont respectivement 2.550, 315 et 0 tandis que le contenu de son R.A.I. est toujours 103.

A ce moment là, le tambour, ayant terminé sa phase de travail autonome, fait une nouvelle demande d'instruction au Distributeur de Programme. Pour le 2ème transfert, référencé 2 sur l'organigramme, tout se passe de la même façon que pour le premier ; le détail des opérations peut se suivre sur la planche 2.

2°/ Changement d'élément

A la fin de ce 2ème transfert, il y a au niveau 3 de l'organigramme une comparaison à faire à l'aide du comparateur général. C'est donc cet élément qui doit être mis en activité.

Pour cela, il est nécessaire que la première instruction qui sera appelée à la fin de transfert tambour soit une coupure sur le comparateur général. Sur la planche 2, on voit en effet qu'à la fin du transfert tambour, l'instruction 106 est une coupure sur le comparateur général dont le numéro d'élément est 2. Une telle instruction a pour effet de mettre en activité le Comparateur Général en envoyant dans son R.A.I. l'adresse d'instruction suivante, soit 107.

Ce faisant, le Distributeur de Programme est libéré et il faut que le Comparateur Général fasse une demande d'instruction pour que son numéro d'élément 2 aille dans le R.N.E., que son R.A.I. soit selectionné et que son contenu, soit 107, revienne dans le R.A.P.A. et que le Distributeur de Programme lui donne les renseignements utiles à sa mise en travail autonome.

Le catène 107 est un catène vierge (on verra à propos de la chaîne de reprise pourquoi il faut toujours laisser un catène vierge après une instruction de coupure). Son effet est nul et le Distributeur de Programme passe aussitôt à l'exploitation des instructions canoniques suivantes. On trouve deux instructions d'adresse pour charger 2 R.A.C. du Comparateur Général des adresses en Mémoire Rapide des deux termes de la comparaison, puis une directive opératoire qui donne la longueur de la comparaison (nombre de catènes des termes à comparer).

Dans la phase de travail autonome du Comparateur Général, les opérandes sont alimentés alternativement dans le Comparateur Général, catène par catène, et chaque fois qu'un catène de chaque opérande est alimenté, les deux catènes sont comparés. A chaque comparaison, le R.L. est décompté de 1 et son passage à 0 provoque la fin de la comparaison : toutefois, comme les premiers catènes contiennent les poids forts des nombres et qu'eux-mêmes sont comparés poids forts en tête, la comparaison s'arrête dès que deux catènes sont inégaux. Elle ne va jusqu'au bout que si les deux nombres sont égaux ou si l'inégalité ne se trouve que dans le dernier catène.

3°/ Branchement conditionné

Au niveau 4 de l'organigramme, on voit que l'on doit exécuter la séquence S^1 ou la séquence S^2 selon que le résultat de la comparaison aura été - ou $\neq$, l'exécution d'une séquence excluant bien entendu celle de l'autre.

Au cours de la comparaison, son résultat s'est inscrit dans le catène qualitatif du Comparateur Général. Lorsqu'après la comparaison, le Comparateur Général fait à nouveau une demande d'instruction, son catène qualitatif est transmis dans le registre A de l'Unité Centrale et l'instruction appelée à ce moment là (instruction 111) est un "branchement conditionné" qui exploite le contenu du registre A (catène qualitatif).

Si la condition n'est pas réalisée, son effet est simplement de laisser venir le contenu du R.A.P.P. dans le R.A.P.A. (soit 112) et donc d'entreprendre la séquence S^1. Si la condition est réalisée, l'effet de cette instruction B est de forcer le contenu du R.A.P.A. à la valeur portée par cette instruction elle-même (adresse extraordinaire) et d'envoyer le contenu du R.A.P.P. dans le R.S.D.P. pour son utilisation éventuelle par une directive centrale (cf. § VII 3). Le R.A.P.A. contenant alors 600, on entreprendra la séquence S^2.

4°/ Branchement systématique

Au niveau 6 de l'organigramme, on voit que, quelle que soit la séquence traitée précédemment (S^1 ou S^2), on doit passer à la séquence S^3 dont la première instruction se trouve en adresse 121.

Si l'on est passé par S^1, il n'y a pas de problème : après l'instruction 120, le Distributeur de Programme traitera naturellement l'instruction 121.

Si l'on est passé par S^2, il faut pour revenir en 121 que la dernière instruction de S^2 (d'adresse 615) soit un "branchement toujours" dont l'effet est de renvoyer systématiquement à l'adresse extraordinaire qu'il contient (ici 121).

5°/ Dédoublement et regroupement de séquences

Après la séquence S^3, on doit lancer simultanément deux séquences, l'une débutant par un transfert tambour, l'autre par un calcul arithmétique. Pour cela, la dernière instruction de S^3 sera en 130 une SIMU indiquant 500 en adresse extraordinaire. Le Distributeur de Programme exploitera d'abord la branche extraordinaire puis la branche ordinaire comme il a été expliqué au § III.

Au niveau 11 de l'organigramme, on doit regrouper ces deux séquences sur une séquence de traduction. Ceci se fera sur la coupure désignant le traducteur (instruction 146). Cette coupure indiquera 1 condition supplémentaire attendue puisqu'il y a 2 branches à regrouper. Elle deviendra effective lorsqu'elle aura été appelée :

. d'une part à la fin de la première branche et de façon naturelle à la suite de l'instruction 145,
. d'autre part, à la fin de la seconde branche grâce à un "branchement toujours" que l'on mettra en 560 et qui renverra en 146.

Il faut bien noter que ceci ne préjuge en rien de l'ordre dans lequel se terminent les deux branches.

VI - LE DISPOSITIF DE "CHAINE DE REPRISE"

1°/ Introduction de la notion de chaîne de reprise

Aux niveaux 8, 9 et 10 de l'organigramme peuvent se présenter toutes les éventualités entraînées par le traitement simultané de deux séquences : conflits de demandes de transferts ou d'instructions réglés par les circuits de priorité du Distributeur de Programme et du Distributeur de Transferts, on a vu comment au Chapitre 1.

Mais, aux niveaux 9 et 10, une nouvelle éventualité peut encore se produire. En effet, le Comparateur Général figure dans les deux branches, dans un cas au niveau 9, dans l'autre au niveau 10 : n'oublions pas toutefois que ces niveaux ne sont que des références et n'ont aucune prétention à figurer une échelle des temps ; ils représentent tout au plus un ordre de succession des opérations à l'intérieur d'une même branche de programme. Il n'est donc pas impossible que le Comparateur Général travaillant pour le compte de l'une des branches soit appelé par l'autre alors qu'il n'est pas libéré de la première.

Supposons par exemple qu'appelé par la coupure figurant en adresse 141, il ait été mis au travail et qu'à ce moment là, à la demande du calculateur arithmétique, le Distributeur de Programme appelle la coupure 510 demandant également le Comparateur Général. Normalement, l'action de cette coupure serait de libérer le calculateur arithmétique et de mettre le Comparateur Général en activité en envoyant 511 dans son R.A.I. Or ce R.A.I. contient déjà 146, adresse de l'instruction qui doit être appelée à la demande du Comparateur Général quand il aura fini sa phase opératoire.

Pour résoudre ce conflit, on pourrait évidemment penser que cette seconde coupure ne doit devenir effective que lorsque le Comparateur Général aura été libéré de la première séquence, le calculateur arithmétique conservant en attendant l'adresse 510 de la coupure dans son R.A.I. Cette façon de procéder ne pouvait être viable pour les deux raisons suivantes :

. le Calculateur Arithmétique aurait continué à être immobilisé au delà de sa

phase de travail propre, alors qu'il pourrait lui-même se trouver demandé par une autre séquence du travail en cours.

. mais surtout, si l'élément mis au travail à la suite du Comparateur Général sur la première branche était précisément le calculateur arithmétique, il se reproduirait un interblocage des deux séquences.

En conséquence, la solution suivante a été adoptée : l'adresse 511 qui ne peut être envoyée dans le R.A.I. du Comparateur Général est envoyée dans un registre auxiliaire également propre au Comparateur Général, qu'on appelle registre B et qui est un registre de mise en réserve. Ceci permet donc de libérer l'élément demandeur de la coupure (le Calculateur arithmétique) dès la fin de son travail. Lorsque, par l'effet d'une coupure sur un autre élément, le R.A.I. du Comparateur Général sera à nouveau disponible, le contenu de B lui sera transféré et le Comparateur Général pourra alors travailler pour la seconde séquence.

Mais, dans le cas où un même élément pouvait être appelé par plus de deux séquences à la fois, on ne pouvait multiplier les registres de mise en réserve. Aussi a-t-on prévu pour ce cas un processus dit de "chaîne de reprise" qui est un mécanisme très spécifique du GAMMA 60.

Nous allons donc examiner ce qui se passe lorsque plus de deux séquences peuvent appeler le même élément. Supposons que 4 séquences (cf. planche 3) travaillent sur le calculateur arithmétique, l'une, des instructions 250 à 260, la deuxième de 540 à 578, la troisième de 112 à 125, la quatrième de 5.647 à 5.682. Cela veut dire que les instructions 250, 540, 112 et 5.647 sont des coupures sur le Calculateur arithmétique et que les instructions 260, 578, 125 et 5.682 sont des coupures sur un élément quelconque autre que le calculateur arithmétique.

2°/ Constitution de la chaîne de reprise

Sur la planche 3, les différentes colonnes verticales représentent l'évolution des différents registres du calculateur arithmétique et des séquences en cours; les lignes horizontales représentent l'état de tous ces registres et de ces séquences aux instants successifs considérés.

Le Calculateur Arithmétique étant au repos, nous supposerons que la première instruction l'appelant est la coupure d'adresse 250. Comme on l'a déjà expliqué, son R.A.I. se charge de l'adresse 251 et la séquence 1 commence à se dérouler. Pendant que le Calculateur Arithmétique est en travail autonome et que son R.A.I. contient 256, le Distributeur de Programme sort l'instruction 540 qui est encore une coupure sur le Calculateur Arithmétique. Le R.A.I. étant déjà occupé, on range l'adresse 541 non seulement dans le registre de mise en réserve B dont on a déjà parlé, mais encore dans un autre registre C également propre à l'élément. Plus précisément, et comme l'indique la flèche sur la planche 3, on va consulter le registre C, et, le trouvant vide, on range 541 à la fois dans B et C.

Un peu plus tard, la première séquence en étant arrivée à 258, mais la deuxième en étant toujours à 541, le Distributeur de Programme sort à nouveau une instruction d'adresse 112 qui est encore une coupure sur le Calculateur Arithmétique. Comme dans le cas précédent, le R.A.I. n'étant pas vide, on consulte le registre C et, voyant qu'il contient 541, on range l'adresse progressée de 1 de la coupure 112, c'est-à-dire 113, à cette adresse 541 (qui est l'adresse d'un catène qui suit une instruction de coupure, c'est-à-dire un catène vierge), tandis qu'en même temps, on substitue 113 à 541 dans le registre C. On voit ainsi la raison pour laquelle on doit laisser un catène vierge à la suite d'une instruction de coupure : c'est grâce à lui que l'on peut constituer de proche en proche une chaîne de reprise non bornée supérieurement.

Lorsque la quatrième coupure d'adresse 5.647 se présente, le même mécanisme se reproduit, on consulte C et, voyant qu'il contient 113, on range 5.648 à

l'adresse 113 tandis que dans C lui-même, on substitue 5.648 à 113. Pendant tout ce temps, le contenu de B est resté 541.

On voit donc apparaître les rôles des différents registres de chaque élément :

. Le R.A.I. contient l'adresse courante de la séquence en cours,

. Le registre B contient l'adresse de la prochaine séquence à exécuter après la libération de la séquence en cours. C'est le premier maillon de la chaîne de reprise

. Le registre C contient l'adresse où l'on doit ranger l'adresse de la prochaine séquence utilisant le calculateur arithmétique, qui pourrait se présenter. C'est le dernier maillon de la chaîne de reprise.

3°/ Exploitation de la chaîne de reprise

La première séquence est maintenant arrivée à 260. Cette instruction est une coupure sur le tambour qui libère donc le calculateur arithmétique de cette séquence. Il peut alors traiter la première séquence en attente dont l'adresse se trouve dans le registre B. Pour cela, le contenu de B, soit 541, est transmis dans le R.A.I. tandis que le contenu de l'adresse 541, soit 113, est envoyé à son tour dans B et que le catène 541 est lui-même remis à 0.

A la fin de cette deuxième séquence, le mécanisme de reprise de la troisième est absolument identique. A la fin de la troisième, le mécanisme de reprise de la quatrième est encore le même, à ceci près qu'aucune nouvelle séquence ne s'étant présentée dans l'intervalle (ce qui aurait pu se produire), le catène 5.648 est vierge. Transmettre son contenu dans B, c'est le remettre à 0. Dans ce cas, on remet aussi C à 0.

Comme on le voit, le mécanisme de la chaîne de reprise a finalement pour but de *reprendre les différentes séquences dans l'ordre même où elles se sont présentées.*

Tout ce mécanisme évidemment assez compliqué se fait, bien entendu, automatiquement et, à strictement parler, *le programmeur n'a pas besoin de le connaître. Il doit simplement savoir qu'il doit laisser un catène vierge à la suite d'une instruction de coupure.* Toutefois, on ne peut l'ignorer si l'on veut arriver à une parfaite compréhension des circonstances de la simultanéité dans le GAMMA 60.

VII - LES "ELEMENTS VIRTUELS"

1°/ Introduction de la notion d'élément virtuel

Il y a des cas où l'on peut avoir besoin d'interdire à certaines séquences de s'exécuter en même temps. Les raisons pour lesquelles on désire agir ainsi peuvent être très diverses : séquences utilisant une même zone de Mémoire Rapide ou un même sous-programme, organisation d'un système de simultanéité verticale (cf. 2), etc...

Cependant, quelle que soit la raison indiquée, le moyen employé est toujours le même. Excluons tout d'abord le cas de séquences élémentaires utilisant toutes le même élément. Dans ce cas, en effet, qu'on le veuille ou non, elles s'exécuteront l'une après l'autre puisqu'un élément donné ne peut travailler que pour le compte d'une seule séquence à la fois ; la coupure qui, en tête de chaque séquence élémentaire, désigne l'élément est, à elle seule, une protection suffisante contre l'exécution simultanée de ces séquences.

Dans le cas général, il s'agit de séquences faisant chacune appel à plusieurs éléments ; le fait que certains d'entre eux puissent se trouver à la fois dans

plusieurs de ces séquences n'est pas exclu et ne change rien au problème.

On associe à chacune de ces séquences un *"élément virtuel"* qui sera le même pour toutes. Cet élément virtuel sera matérialisé en Mémoire Rapide par deux *"catènes-image"* dont les adresses sont consécutives. Soit, par exemple, les séquences S_1, S_2 et S_3 auxquelles on associe l'élément virtuel 1.000 matérialisé par les catènes-image 1.000 et 1.001.

La protection de chacune de ces séquences contre l'éventualité de l'exécution de l'une quelconque des deux autres en même temps qu'elle, s'obtient en plaçant en tête de chacune d'elles l'instruction *"coupure sur l'élément virtuel 1.000"* (instruction de type C comme celles qui mettent en activité les éléments réels).

Le rôle de cette instruction est, dans le cas où aucune des 3 séquences n'est encore en cours, de placer un indice d'occupation dans la première monade du premier catène-image de l'élément virtuel 1.000.

Dans le cas où l'une des séquences est déjà en cours, l'indice d'occupation se trouve déjà placé dans ce catène-image puisque cette séquence a débuté par une "coupure sur élément virtuel 1.000". La deuxième séquence, qui se présente alors et qui débute aussi par une "coupure sur élément virtuel 1.000", trouvera cet élément occupé et devra attendre qu'il soit libéré de la première séquence avant de pouvoir se poursuivre.

Comme on le voit, tout se passe jusqu'ici comme pour des séquences élémentaires utilisant un même élément réel ; entre autres, l'attente de la libération de l'élément virtuel se fera par l'intermédiaire de la constitution d'une chaîne de reprise sur élément virtuel tout à fait identique à la chaîne de reprise sur élément réel, grâce au catène vierge qui suit la coupure sur élément virtuel comme toute coupure, les deux catènes-image jouant le rôle des registres B et C d'un élément réel.

Mais alors qu'un élément réel se libère lui-même de la séquence en cours lorsqu'à sa demande est appelée une coupure demandant un autre élément réel, il ne peut en être de même pour un élément virtuel qui, comme son nom l'indique, n'a aucune existence propre (il n'existe que dans l'esprit du programmeur qui fait appel à lui).

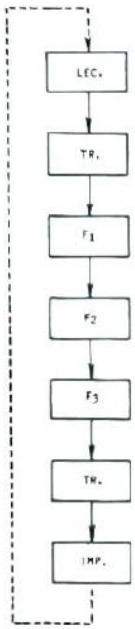
Il est donc nécessaire d'avoir une instruction spéciale de libération d'élément virtuel. Cette instruction est également une instruction de type C : elle a pour rôle soit d'enlever l'indice d'occupation du premier catène-image, soit de permettre l'exécution des séquences en attente de l'élément virtuel par exploitation de sa chaîne de reprise.

Nous allons maintenant examiner 2 cas d'utilisation des éléments virtuels.

2°/ Utilisation des éléments virtuels dans une organisation de simultanéité verticale

Soit l'organigramme filiforme ci-après.. Un lot d'informations, constitué par le contenu d'une carte perforée, est pris en charge par un lecteur de cartes ; est traduit, puis subit un certain nombre de fonctions de traitement. Le résultat final est alors traduit puis imprimé. Ce cycle une fois achevé, on recommence à lire un autre lot d'informations.

Il est bien évident que si l'on procède de la sorte, le lecteur de cartes ne fonctionnera pas pendant que s'accompliront toutes les fonctions autres que la lecture ; il sera donc très mal utilisé. On peut dire la même chose de l'imprimante et de tous les autres éléments mis en oeuvre dans cet organigramme. En résumé, si c'est le seul travail effectué sur la machine, ses possibilités de fonctionnement simultané des éléments ne sont pas utilisées.



Il est facile toutefois de constater que l'information chemine du haut vers le bas de l'organigramme comme un train sur une voie ferrée. De même qu'une telle voie ferrée est divisée en "cantons" et, qu'un train ayant quitté un canton, on autorise le train suivant à pénétrer dans ce canton, de même on peut diviser un tel organigramme en cantons et avoir plusieurs lots d'informations en traitement simultané à des phases différentes de l'organigramme.

Par opposition à la "simultanéité horizontale" envisagée à propos de l'organigramme de la planche 1, on appelle *"simultanéité verticale"* ce mode de traitement où les lots d'informations se suivent ainsi l'un derrière l'autre sur le même organigramme. Il suffit, pour éviter les confusions, de créer sur l'organigramme des "cantons" que nous figurerons par des *éléments virtuels.* Quand un lot d'informations entrera dans un canton, il occupera son élément virtuel et le libérera en sortant. Ainsi, tant qu'une séquence de programme sera protégée par l'occupation d'un élément virtuel, elle ne pourra être utilisée pour un lot d'informations autre que celui qu'elle traite. C'est en somme l'organisation du "block system".

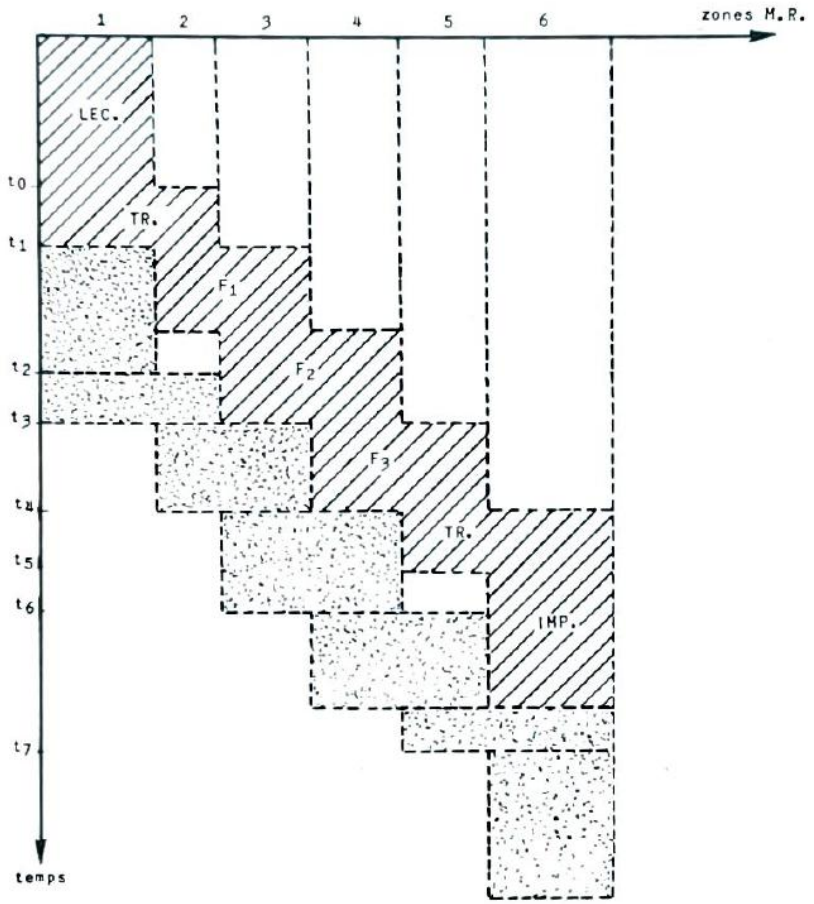
Sur l'organigramme, l'occupation et la libération d'un élément virtuel seront figurées par les symboles exposés dans le § IV. Dans le programme, l'occupation et la libération de cet élément virtuel s'obtiendront par une coupure "occupation" ou "libération" d'élément virtuel.

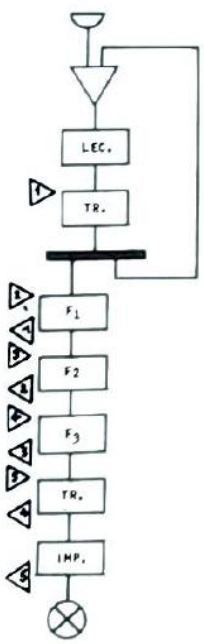
Nous allons voir comment on peut, sur le thème général proposé en début de paragraphe, introduire des simultanéités verticales grâce aux éléments virtuels. Nous allons, pour la simplicité de l'exposition, simplifier un peu les problèmes. Nous supposerons qu'on lit la carte sur une seule brosse pour l'envoyer telle quelle en Mémoire Rapide dans une zone 1. Cette lecture dure de t_0 à t_1. De t_1 à t_2, on traduit le contenu de cette zone 1 pour l'envoyer en zone 2. De t_2 à t_3, on utilise les résultats de cette traduction (en zone 2) pour exécuter la fonction F_1 dont on place les résultats en zone 3 et ainsi de suite.

Dressons le diagramme espace-temps relatif à la Mémoire Rapide et figurant l'occupation des différentes zones au cours du traitement d'un seul lot d'informations (hachures obliques).

On voit sur ce diagramme que l'on pourra relancer le programme sur un deuxième lot d'informations par la lecture d'une deuxième carte, dès que les informations résultant de la lecture brutale de la première carte ne seront plus utiles, c'est-à-dire à la fin de la traduction. Le programme de la traduction devra donc se terminer par une SIMU lançant, d'une part, la fonction F_1 sur le premier lot, d'autre part, relançant la lecture sur le deuxième lot (cf. organigramme final ci-après). Cette fonction de lecture sera donc initiée soit par un conditionnement venant d'un autre organigramme pour le premier lot, soit par un conditionnement "remontant" venant de la SIMU pour les autres ; d'où le symbole de regroupement "OU" avant la lecture sur l'organigramme final.

Sur le diagramme, l'occupation des zones de Mémoire Rapide pour ce deuxième lot est figurée en pointillé. Avec les temps que l'on a supposé pour chaque fonction, on voit que, pour les deux lots d'informations, on n'a aucun chevauchement





de zone. Mais, si l'on est absolument sûr du temps de lecture d'une carte (puisqu'il est déterminé mécaniquement), on l'est moins de la durée d'une fonction de traitement : en effet, si la machine est très chargée, ces fonctions peuvent durer un peu plus longtemps que prévu, par suite de retards possibles apportés à l'exécution de certains transferts, ou d'attentes d'instructions de la part des éléments. Pour cette raison, il est indispensable de se garantir contre un certain chevauchement en divisant l'organigramme en cantons protégés par des éléments virtuels. C'est ainsi qu'il faut interdire l'exécution de la traduction sur le deuxième lot avant que F_1 ne soit terminée sur le premier, l'exécution de F_1 sur le deuxième lot avant que F_2 ne soit terminée sur le premier, etc ... On aura donc un chevauchement de cantons commençant par des coupures sur éléments virtuels et se terminant par des coupures libération d'éléments virtuels. Le véritable organigramme du problème proposé est figuré ci-contre.

3°/ Utilisation des éléments virtuels pour la protection de sous-programmes

Soit 3 programmes différents en cours d'exécution simultanée dans le GAMMA 60 Supposons qu'à un moment donné, il se trouve dans ces 3 programmes une séquence identique : une séquence de calcul par exemple. Il est bien évident que si les 3 programmes arrivent en même temps à cette séquence, ils ne pourront s'exécuter simultanément à partir de ce moment, puisqu'ils font tous appel au même élément : le Calculateur Arithmétique ; le mécanisme de chaine de reprise sera mis en jeu et le calculateur arithmétique traitera successivement ces 3 séquences identiques.

Il est donc parfaitement inutile d'allonger inconsidérément les programmes et d'encombrer inutilement la Mémoire Rapide en répétant 3 fois la même séquence. On la considérera alors comme un sous-programme que l'on placera une seule fois en Mémoire Rapide. Supposons que cette séquence ait normalement dû s'intercaler dans les programmes principaux à partir des adresses 1.000, 1.100 et 1.200 par exemple. Il faudra placer en 1.000, 1.100, et 1.200 un "branchement toujours" renvoyant à l'adresse de la 1ère ligne du sous-programme (100 par exemple). Il faudra ensuite qu'après l'exécution du sous-programme, l'on revienne à l'adresse 1.001, 1.101 ou 1.201 des programmes principaux selon que l'on a exécuté le sous-programme à la suite du branchement 1.000, 1.100, ou 1.200.

On a déjà vu que l'effet d'un branchement effectif était, outre le forçage du R.A.P.A. à l'adresse extraordinaire, l'envoi du contenu du R.A.P.P. dans le R.S.D.P. A l'issue du branchement 1.000 par exemple, on aura 1.001 dans le R.S.D.P. C'est cette adresse dont il faut garder mémoire jusqu'à la fin du sous-programme.

Pour cela, le sous-programme devra débuter par une "directive centrale" dite *"mise en réserve du numéro d'Instruction"*. Cette instruction a pour effet de ranger le contenu du R.S.D.P. à l'adresse Mémoire Rapide qu'elle désigne. Cette adresse sera une "mémoire de manoeuvre" du sous-programme. Comme on le voit, cette instruction n'a de sens qu'après un branchement ; d'autre part, elle doit le suivre immédiatement parce que le R.S.D.P. n'est pas rémanent et qu'en conséquence, il pourrait se trouver des instructions intermédiaires qui le modifient.

Pour revenir au programme principal, le sous-programme devra se terminer par un branchement toujours renvoyant à l'adresse contenue dans la mémoire de manoeuvre (grâce à une fonction d'adresse dans l'instruction de branchement - cf. chapitre 3 § VII). En supposant que le sous-programme se compose de 30 instructions plus une mémoire de manoeuvre à la fin, il aura la structure suivante :

100 Réserve N° Instruction en adresse 130
101 Coupure sur le calculateur arithmétique
.
.
.
.
.
.
129 Branchement toujours à l'adresse contenue dans 130
130 Mémoire de Manoeuvre.

Il faut cependant, lorsque le sous-programme a été appelé par un des programmes principaux, interdire aux autres programmes de l'appeler tant que l'on n'est pas revenu au premier programme. En effet, il se produirait bien un blocage sur la coupure 101 par mise en oeuvre du dispositif de chaîne de reprise, mais auparavant on aurait modifié le contenu de la mémoire de manoeuvre avant de l'avoir utilisé ; si bien que l'on ne retournerait pas, à la fin du sous-programme, au programme d'où l'on était parti.

On interdira l'appel du sous-programme simultanément par plusieurs programme en plaçant, dans chacun de ces programmes, une coupure sur un élément virtuel associé au sous-programme juste avant le branchement renvoyant au sous-programme.

D'autre part, dans chacun de ces programmes, l'instruction qui suit le branchement sera une libération de cet élément virtuel ; c'est, en effet, l'instruction de reprise du programme principal après exécution du sous-programme.

Les programmes principaux auront donc la structure suivante :

```
.                                .
.                                .
.                                .
  998 Coup. El. Virtuel 500      1.098 Coup. El. Virtuel 500
  999 ————————————————————       1.099 ————————————————————
1.000 B. toujours 100            1.100 B. toujours 100
1.001 Libér. El. Virtuel 500     1.101 Libér. El. Virtuel 500
1.002 ————————————————————       1.102 ————————————————————
1.000                            1.103
.                                .
.                                .
.                                .
```

Ainsi, si le sous-programme tend à être appelé par le 2ème programme alors qu'il est déjà mis en oeuvre pour le compte du premier, un mécanisme de chaîne de reprise se mettra en oeuvre sur l'élément virtuel 500.

4°/ Chaîne de reprise sur élément virtuel

Ce mécanisme est tout à fait analogue à celui qui existe pour les éléments réels. Soit donc 3 séquences dont on veut empêcher l'exécution simultanée :

```
100          200          300
 .            .            .
 .            .            .
 .            .            .
 .            .            .
 .            .            .
 .            .            .
120          224          321
```

Les instructions 100, 200 et 300 seront 3 coupures sur le même élément virtuel pour lequel nous choisissons les catènes 1.000 et 1.001 comme "catènes-image".

La première appelée de ces instructions, soit 100, met l'indice d'occupation dans la première position du catène 1.000 (cf. planche 4). Supposons que la coupure 200 arrive avant que la première séquence ne soit terminée, c'est-à-dire l'indice d'occupation étant encore en 1.000. La deuxième séquence ne pourra donc pas s'exécuter et devra être mise en attente. Pour cela, l'adresse 201 sera mise en réserve à l'adresse 1.000 ainsi qu'à l'adresse 1.001 (catènes-image de l'élément virtuel).

Bien noter que l'indice d'occupation n'occupant que la première position du catène 1.000, on peut y placer également l'adresse de la première séquence en attente car, comme toutes les adresses, elle est exprimée en binaire et n'occupe que 15 positions (les 15 dernières du catène 1.000). On voit déjà que le catène 1.000 va jouer le rôle du registre B d'un élément réel et le catène 1.001 le rôle du registre C. Effectivement, lorsque la troisième séquence, débutant sur la coupure 300, arrivera, l'adresse 301 ira se placer d'une part à l'adresse contenue dans le catène 1.001 (soit dans 201), d'autre part dans le catène 1.001 lui-même.

Comme on le voit, le mécanisme de formation de la chaîne de reprise sur élément virtuel est le même que sur élément réel : les deux catènes-image de l'élément virtuel jouant le même rôle que les registres B et C de l'élément réel et le catène vierge qui suit la coupure sur élément virtuel, le même rôle que celui qui suit la coupure sur élément réel.

Lorsque la première séquence arrivera en 120 (libération de l'élément virtuel 1.000), cette instruction provoquera l'exploitation de la chaîne de reprise en permettant l'exécution de la séquence dont l'adresse est dans le catène 1.000 (séquence d'adresse 201) et en envoyant dans le catène 1.000 l'adresse contenue dans le catène 201.

Comme on le voit, tout se passe comme pour un élément réel, à ceci près qu'on n'a pas dit comment était permise l'exécution de la séquence débutant en 201. Ce ne peut être comme avec un élément réel en envoyant 201 dans le R.A.I. puisqu'un élément virtuel n'existant pas n'a pas de R.A.I. Ceci sera vu dans le chapitre 3, en étudiant en détail la logique de l'exploitation de l'instruction C par le Distributeur de Programme.

STRUCTURE GÉNÉRALE ET EXPLOITATION LOGIQUE DES DIFFÉRENTES SORTES D'INSTRUCTIONS CANONIQUES

I - INSTRUCTIONS CANONIQUES ET INSTRUCTION COMPLETE

Dans le chapitre 2 de ce volume, on a défini (§ I) les différents états d'un élément : repos, activité, attente, opération, et l'on a vu comment le Distributeur de Programme lui fournissait les instructions pouvant le faire passer d'un état à un autre (§ II.1).

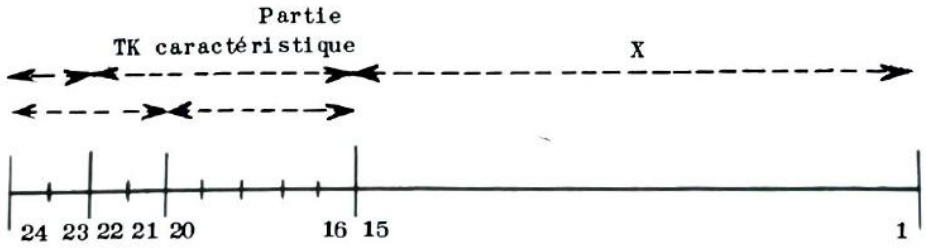
On a vu aussi que la suite des instructions constituant le programme se trouve en Mémoire Rapide dans des adresses successives (sauf cas d'instructions dont le but est précisément de rompre cette succession).

Le contenu de chacun de ces catènes-instructions est une instruction canonique et l'on a vu qu'il existait quatre types d'instructions canoniques, chacun étant spécialisé dans un type de renseignements à fournir aux éléments, et que, pour mettre un élément en phase opératoire, il fallait une série d'instructions canoniques appelée *instruction complète*.

II - STRUCTURE GENERALE DE L'INSTRUCTION CANONIQUE

On appelle *"profil"* de l'instruction canonique l'ensemble des états des 24 positions du catène qui le matérialise. C'est l'analyse et l'interprétation de l'état de ces 24 positions qui permettent au Distributeur de Programme de donner à l'élément les renseignements dont il a besoin pour entrer en opération.

Le profil le plus général d'une instruction canonique est le suivant :



Si l'on numérote les différentes monades du catène de 1 à 24, en allant de la droite à la gauche, on distingue 3 parties :

. Le *TK* (type de catène) qui occupe de 2 à 4 positions qui sont les dernières vers la gauche. Ce sont ces positions qui sont analysées les premières par le Distributeur de Programme et qui lui indiquent de quel type d'instruction il s'agit: A, B, C ou D.

. La *partie caractéristique* qui occupe de 5 à 7 positions selon le TK, ces positions étant celles numérotées de 16 à 20, 21 ou 22. Elle contient, selon les instructions, des renseignements tout à fait variables et définissant complètement l'instruction (Numéros de R.A.C., Fonction d'adresse dont on parlera plus loin,etc..)

. La partie X ou *variable libre* qui occupe les 15 premières positions. Cette variable libre désigne, selon le type d'instruction, soit une adresse, soit une longueur, soit un numéro d'élément (toutes ces quantités sont alors exprimées en binaire pur et, pouvant varier de 0 à 32.767, ont besoin de 15 positions). Toutefois il existe un assez grand nombre d'instructions de type D qui ne comprennent pas de variable libre ; leur TK occupe 4 positions et les 20 autres forment la partie caractéristique.

III - L'INSTRUCTION DE COUPURE

Nous allons maintenant étudier l'exploitation logique des différentes sortes d'instructions canoniques par le Distributeur de Programme. Nous commencerons par la plus compliquée, l'instruction de coupure, en montrant comment c'est elle qui marque les grandes césures du programme et, par là-même, nous justifierons son nom de coupure.

1°/ Introduction

La première fois que nous avons rencontré ce type d'instruction, c'était pour désigner l'élément (réel) auquel étaient destinées les instructions qui suivaient. En effet, rien dans les autres instructions du programme ne dit à quel élément elles sont destinées. Ainsi, les instructions de coupure sur élément réel jalonnent les séquences élémentaires et, déjà, à ce titre, elles indiquent les grandes césures du programme, ce qui justifierait à lui seul leur nom d'instruction de coupure.

Elles marquent aussi les grandes césures du programme parce que c'est l'exploitation de ces instructions par le Distributeur de Programme qui laisse la porte ouverte au plus grand nombre d'éventualités possible quant à la suite de l'alimentation du programme, comme on va le voir.

Nous avons également rencontré les coupures sur éléments virtuels et les coupures "libération d'élément virtuel". Nous avons vu comment elles s'apparentaient aux précédentes par le fait qu'elles pouvaient être à l'origine de la constitution et de l'exploitation de chaînes de reprise sur éléments virtuels. Elles constituent aussi de grandes césures du programme par leur effet d'"antisimultanéité" ou de découpage en cantons.

Une instruction comme la SIMU, par contre, ne saurait s'apparenter aux précédentes que de très loin : dans les précédentes, la variable libre désigne un numéro d'élément (réel ou virtuel), ici elle désigne une adresse de branche extraordinaire. On ne les classe dans ce type d'instructions que parce qu'elles constituent réellement de très importantes césures dans le programme et parce qu'elles peuvent être à l'origine de la constitution d'une chaîne de reprise sur le Distributeur de programme (cf. plus loin) comme les coupures sur éléments peuvent être à l'origine de la constitution de chaînes de reprise sur les éléments.

2°/ *Profil de l'instruction de coupure*
Différentes sortes d'instructions de coupure

Le profil de l'instruction de coupure est le suivant :

. Le TK est 01.

. Les positions 20,21 et 22 indiquent le Type de Coupure (T.C.). Avec ces 3 positions, on a 8 combinaisons possibles, ce qui est suffisant pour indiquer les 7 types de coupure existants.

. Les positions 18 et 19 indiquent le nombre de conditions supplémentaires attendues (m) pour le regroupement (3 conditions maximum).

. En 16 et 17, la fonction d'Adresse (F.A.) (cf. § VII).

On distingue les types de coupure suivants :

. *La coupure normale sur élément réel* qui met en activité l'élément demandé ou, s'il est déjà en activité, met en attente sur sa chaîne de reprise la séquence en cours. En X, se trouve le numéro de l'élément demandé.

. *Deux types de coupures un peu spéciales sur élément réel* également et dont on parlera plus loin.

. *La coupure sur élément virtuel* qui occupe l'élément virtuel ou, s'il est déjà occupé, met en attente sur sa chaîne de reprise la séquence en cours. La partie X indique l'adresse du premier-catène-image de l'élément virtuel.

. *La libération d'élément virtuel* qui permet l'exploitation de sa chaîne de reprise ou le libère s'il n'y a plus rien dans la chaîne de reprise. La partie X indique aussi l'adresse du premier catène-image.

. *La SIMU* qui per met de lancer des simultanéités (cf. § suivant). Bien noter que cette instruction peut aussi servir en même temps à regrouper jusqu'à 4 séquences. La partie X indique l'adresse de la branche extraordinaire.

. *Le STOP* (coupure de fin de séquence) qui libère le dernier élément de la séquence en cours sans en remettre un autre en activité. C'est donc l'instruction qui terminera une séquence de programme. Pour des raisons évidentes, on a exclu la possibilité de regrouper des séquences sur cette coupure. Dans sa partie X, il n'y a rien, pas plus que dans m ou F.A.

Nous allons maintenant expliquer l'exploitation logique par le Distributeur de Programme de chacune de ces instructions de coupure (sauf le STOP dont il n'y a rien d'autre à dire). Ceci n'a d'autre fin que de donner les raisons des *quelques règles de programmation* très simples que nous serons amenés à énoncer. A vrai dire, ce sont elles seules qu'a besoin de connaître le programmeur, mais il les retiendra plus facilement s'il en connaît les raisons que si on les lui donne comme des règles arbitraires.

Nous ne parlerons pas, pour toutes ces instructions de coupure, de la phase préparatoire qui consiste à compter les conditions attendues, phase qui est la même quel que soit le type de coupure et qui a déjà été décrite ; il suffit de savoir que le décompte des conditions attendues se fait dans les positions 18 et 19 du catène vierge suivant l'intention de coupure. Nous supposerons que toutes les conditions sont réalisées et que la coupure est effective.

3°/ SIMU et chaîne de reprise du Distributeur de Programme

Nous avons vu, au chapitre 2 § III, comment, au reçu d'une instruction SIMU, le Distributeur de Programme traite d'abord la branche extraordinaire, puis la branche ordinaire. Nous avons vu que pour cela il rangeait l'adresse de la branche ordinaire dans un registre spécial de mise en réserve, mais nous avons reporté à plus tard l'examen du cas où l'on désire lancer plus de deux branches simultanées par une cascade de SIMU.

On pouvait envisager le type de programmation suivant :

```
100     SIMU    200 ——→ C     CA
101     SIMU    300 ——→ C     Der 1
102     SIMU    400 ——→ C     CG
etc...
```

Dans ce cas, on voit qu'après avoir mis le Calculateur arithmétique en activité par la coupure 200, on reviendrait à 101 qui, étant encore une SIMU, met 102 en réserve pour traiter la coupure 300 après laquelle on reviendrait en 102 etc... Mais ceci entraînait la servitude de programmation suivante : il ne fallait jamais mettre de SIMU sur la branche extraordinaire ou alors cela nécessitait un grand nombre de registres de mise en réserve des adresses des différentes branches ordinaires.

Pour ne pas accepter cette restriction, on a donc été amené à organiser sur le Distributeur de Programme un dispositif de chaîne de reprise analogue à celui des éléments en utilisant :

. deux registres de mise en réserve B et C analogues à ceux des éléments et propres au Distributeur de Programme.

. le catène vierge que le programmeur doit laisser après la SIMU comme après toute coupure, catène qu'il doit placer *sur la branche ordinaire.*

Ce mécanisme de chaîne de reprise du Distributeur de Programme peut encore servir (cf. plus loin) lors de l'exploitation des instructions de libération d'éléments virtuels.

Ainsi, on pourra lancer plusieurs séquences simultanées par une succession de SIMU soit sur les branches ordinaires, soit sur les branches extraordinaires, soit sur les deux sans aucune restriction.

Pour expliquer ce mécanisme, nous prendrons l'exemple suivant dans lequel nous avons choisi de faire la cascade de toutes les SIMU toujours sur la branche extraordinaire (ce qui seul pose un problème) et dans lequel les SIMU ne servent pas à faire de regroupements de séquence (ce qui n'est toutefois pas à exclure).

```
95 C.   CA
.
.
.
.
.
100 SIMU ——→ 200 SIMU ——→ 300 SIMU ——→ 400 C - TM
101 ____     201 ____     301 ____     401 _____
102 A        202 C Der 1  302 C.GC       .
103 A        203 _____    303 ____       .
104 D          .            .            .
 .             .            .            .
 .             .            .            .
 .             .            .            .
```

Sur la planche 5, les colonnes représentent les états successifs des différents registres du Distributeur de Programme, des R.A.I. des éléments et des catènes 101 et 201 de la chaîne de reprise ; les lignes horizontales représentent, à un instant donné, l'état de ces différents registres en fonction du déroulement du programme.

A la demande du Calculateur Arithmétique, l'instruction 100 est appelée ; c'est une SIMU renvoyant en 200 ; le R.A.P.A. est alors forcé à 200 tandis que le contenu de R.A.P.P. est envoyé dans 2 registres du Distributeur de Programme, les registres B et C, après que l'on se soit assuré que le registre C était vide.

L'instruction 200 est alors appelée ; c'est encore une SIMU renvoyant en 300. Comme précédemment, on consultera le contenu de C et, voyant que c'est 101, on enverra le contenu du R.A.P.P. (soit 201) dans 101 (catène vierge suivant la première SIMU sur sa branche ordinaire) tandis que 201 lui-même se substituera à 101 dans C.

L'adresse 300 etant alors venue dans le R.A.P.A., l'instruction qui s'y trouve est alors appelée. C'est encore une SIMU et le même mécanisme se reproduit identiquement.

Cette fois, cependant, l'adresse extraordinaire qu'elle indique (400) est une coupure sur le Tambour Magnétique. Son effet est de le mettre en activité en chargeant son R.A.I. de 401 (ou s'il est déjà en activité, de placer 401 dans sa chaîne de reprise). A ce moment-là, le Tambour Magnétique va pouvoir envoyer une demande d'instruction vers le Distributeur de Programme. Toutefois, avant de la prendre en considération, celui-ci va épuiser la chaîne de reprise qu'on vient de lui constituer : on voit que ses registres B et C ont joué jusqu'ici le même rôle que les registres B et C des éléments ; ils vont continuer.

Le contenu de B (soit 101) va être transféré dans le R.A.P.A. (qui joue ainsi à cet égard le rôle du R.A.I. des éléments) tandis que le contenu de 101 lui-même (soit 201) va, à son tour, venir dans B et qu'il se remettra ensuite à 0. Le Distributeur de Programme va alors appeler, toujours pour le compte du Calculateur Arithmétique, les instructions à partir de 101 et se libérera de cette séquence après l'instruction 104 (Directive) en renvoyant 105 dans le R.A.I. du Calculateur Arithmétique.

Le nouveau contenu de B va, à nouveau, venir dans le R.A.P.A. Les mêmes mécanismes se reproduisent qui, par les coupures 202, mettent finalement le dérouleur 1 en activité en chargeant son R.A.I. de 203. C'est ensuite au tour du Comparateur Général d'être mis en activité par l'instruction 302. A ce moment-là, la chaîne de reprise du Distributeur de Programme est épuisée et il pourra prendre en considération l'une des demandes qui se présentent à ses circuits de priorité.

On voit ainsi que toutes les branches lancées par les SIMU commencent par des coupures ; une seule fait exception dans l'exemple traité : cela n'est possible que parce que les SIMU ne servent pas à regrouper de séquences et que par conséquent on sait parfaitement à la demande de quel élément elles ont été appelées, le Numéro de cet élément figurant dans le R.N.E. D'ailleurs, on aurait parfaitement pu placer cette séquence se rapportant encore au Calculateur Arithmétique ailleurs que sur la branche ordinaire de la première SIMU.

Par contre, si la SIMU 100 avait servi à regrouper avec la séquence venant du Calculateur Arithmétique, une autre séquence venant par exemple du traducteur, il aurait fallu obligatoirement mettre une coupure sur le Calculateur Arithmétique en 102.

Règles de programmation :

1) Après une SIMU, il faut laisser un catène vierge sur la branche ordinaire.

2) Dans une cascade de SIMU, la première instruction effective sur chaque branche doit être une coupure désignant l'élément devant travailler sur cette branche. Exceptionnellement l'une des branches peut échapper à cette règle si cette branche fait travailler précisement l'élément qui est à l'origine de l'appel de la cascade des SIMU et si aucune des SIMU rencontrées pour aboutir à cette branche n'a servi à faire de regroupements.

4°/ Coupures sur éléments virtuels

Pour celle d'occupation d'élément virtuel, c'est très simple :

. si l'élément est libre, le Distributeur de Programme place dans son premier catène-image l'indice d'occupation et poursuit l'alimentation des instructions de la séquence en cours.

. si l'élément est déjà occupé, il met la séquence en attente dans la chaîne de reprise de cet élément virtuel, puis il exploite éventuellement sa chaîne de reprise propre, puis celle de l'élément demandeur de la coupure avant de prendre en considération une des demandes d'instruction d'éléments (réels) en instance à ses circuits de priorité.

Pour la "libération d'élément virtuel", on a déjà vu l'essentiel de l'exploitation de cette instruction à propos de la chaîne de reprise sur élément virtuel. Toutefois, on n'a pas alors dit comment cette instruction permettait l'exploitation de la première séquence en attente : sur la planche 4, une flèche indique simplement que l'adresse contenue dans le premier catène-image en est sortie... Maintenant que l'on a expliqué ce qu'était la chaîne de reprise du Distributeur de Programme, on peut compléter et dire que cette adresse, 201 dans l'exemple de la planche 4, en est sortie pour être envoyée dans la chaîne de reprise du Distributeur de Programme. Le catène d'adresse 201 est en même temps effacé.

Après ceci, le Distributeur de Programme continue l'alimentation des instructions de la séquence en cours jusqu'à trouver une instruction qui lui permettra d'exploiter sa chaîne de reprise propre.

La formation de la chaîne de reprise du Distributeur de Programme est donc provoquée, soit par des instructions libérations d'éléments virtuels , les adresses qui la constituent sont celles des catènes vierges qui suivent, soit des SIMU, soit des coupures d'occupation d'éléments virtuels. Lorsqu'il y a plusieurs séquences protégées par le même élément virtuel, on ne connaît pas l'ordre dans lequel elles se constitueront éventuellement en chaîne de reprise. On ignore donc à la demande, quel élément aura été appelée la "libération d'élément virtuel" qui permettra d'exploiter telle séquence en attente. On appliquera donc les règles suivantes :

En général, dans les sequences protégées par un élément virtuel, *la coupure sur élément virtuel devra être suivie d'une coupure sur élément réel. Si la libération d'élément virtuel a servi à faire un regroupement, elle doit être suivie d'une coupure sur élément réel.*

D'autre part, comme toutes les instructions de coupure, *les coupures d'occupation et de libération d'élément virtuel devront être suivies d'un catène vierge.*
Remarque : Le catène vierge qui suit une coupure de libération d'élément virtuel sert uniquement au décompte des conditions supplémentaires attendues.

L'exploitation de ces coupures est résumée par les schémas de la page suivante .

5°/ Coupure sur élément réel

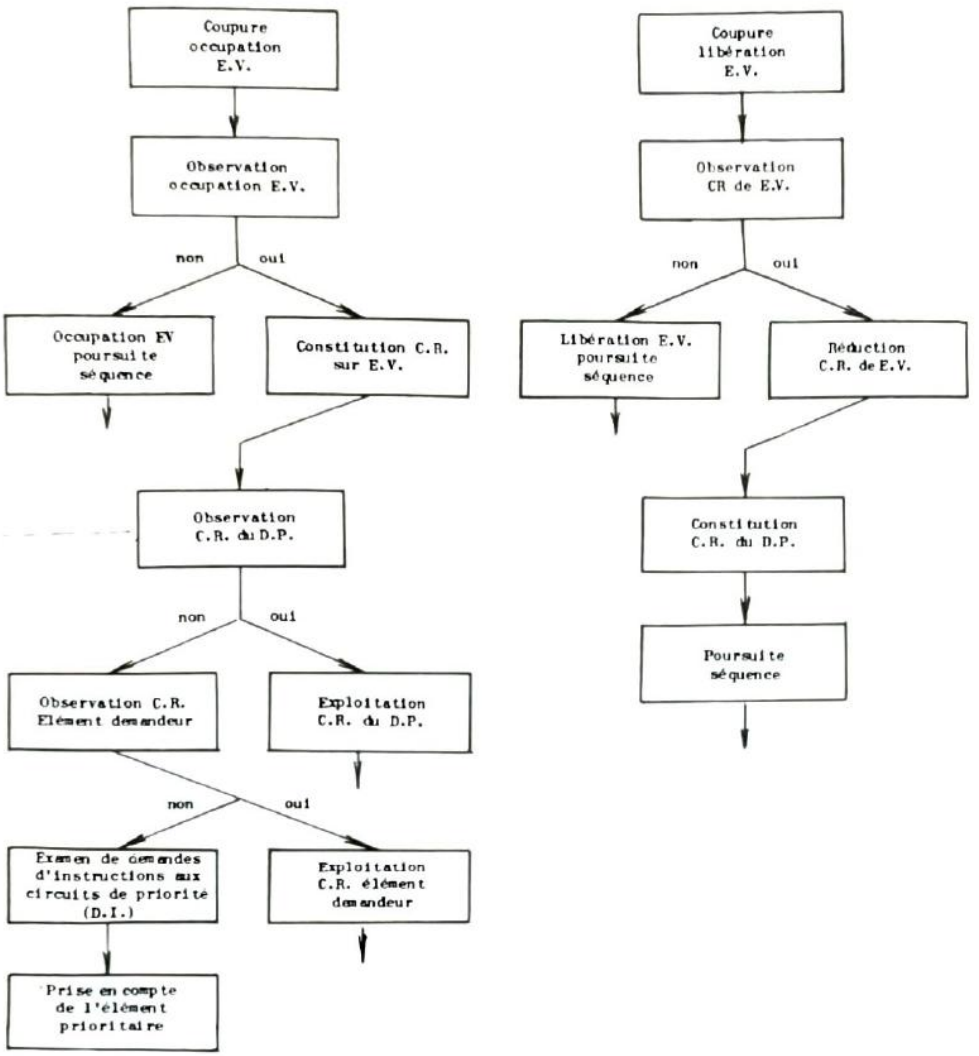
Au reçu d'une coupure *normale* sur élément réel, le Distributeur de Programme:

. met en état d'activité l'élément demandé en chargeant son R.A.I. (ou, s'il est déjà en activité, met la séquence en attente sur sa chaine de reprise),

. exploite éventuellement sa chaîne de reprise propre,

. exploite ensuite la chaîne de reprise de l'élément demandeur ou le libère si aucune séquence n'est en attente de sa disponibilité,

. et seulement alors, prend en considération une des demandes en instance à ses circuits de priorité (bien noter que cette demande n'est pas obligatoirement celle faite par l'élément demandé par la coupure ; il peut y en avoir de plus prioritaires).

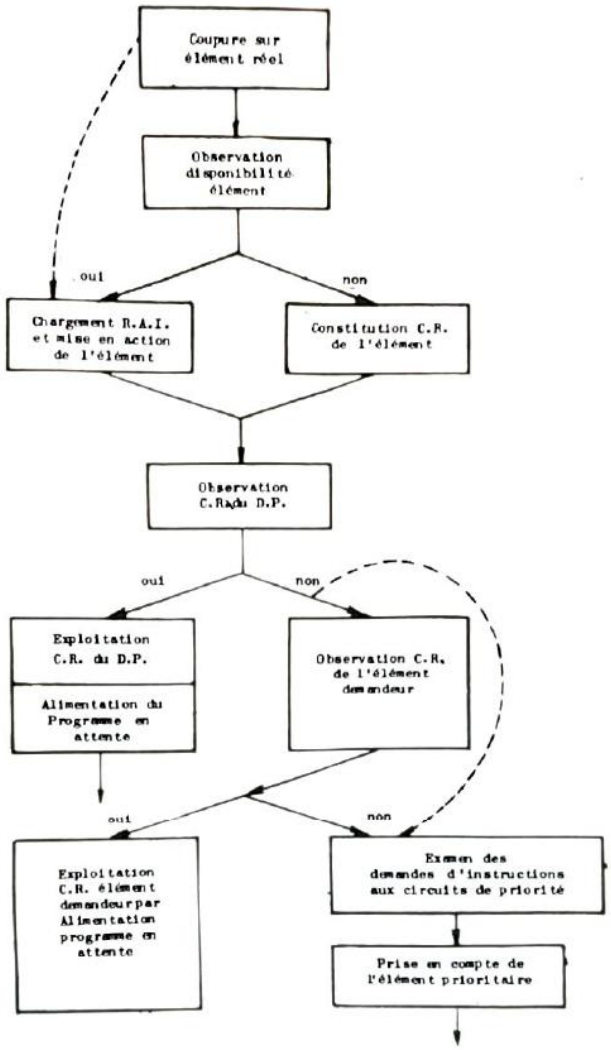


Il existe deux autres sortes de coupures sur élément réel d'un type un peu spécial :

. l'une ne libère pas l'élément demandeur, donc ne permet pas l'exploitation de sa chaîne de reprise.

. l'autre n'observe pas la disponibilité de l'élément demandé et fait passer la séquence qui suit avant celles qui peuvent être en attente sur sa chaîne de reprise. Pour cela, il faut bien entendu que le R.A.I. de cet élément soit libre , c'est-à-dire qu'elle ne peut se trouver que s'il s'est auparavant trouvé une coupure de l'autre type.

Ces deux types de coupure sont rarement utilisés. En fait, les circonstances de leur emploi se réduisent à ceci : après un incident sur un élément, on veut lancer un programme d'incident sans permettre évidemment que l'élément "incidenté" puisse être repris par une autre séquence, et l'on veut, après le programme d'incident, reprendre la séquence que l'on a quittée avant tout autre en attente dans sa chaîne de reprise. Pour cela, le programme d'incident débutera par une coupure du premier type pour mettre en activité l'élément travaillant pour lui. Puis, on reviendra au programme normal par une coupure du second type.



Le schéma précédent représente l'exploitation d'une instruction de coupure ordinaire.

Les pointillés représentent les phases supprimées dans les coupures spéciales:

. le pointillé du bas pour la coupure sans libération de l'élément demandeur,

. le pointillé du haut pour la coupure sans observation de la disponibilité de l'élément demandé.

IV – L'INSTRUCTION DE DIRECTIVE

1°/ Différents types de directives

On distingue deux classes d'instructions D :

a) les *directives Elément* qui, elles-mêmes, se divisent en deux groupes :

. Les Directives préparatoires qui donnent à l'élément certains renseignements que n'ont pu lui donner les instructions A, B ou C. Un exemple type de ces directives est la directive adresse que nous avons rencontrée pour le Tambour dans

le chapitre 2 : elle chargeait un registre Adresse dans l'élément et non un R.A.C. (adresse en Mémoire Rapide) pour lequel on utilise une instruction A.

. Les Directives opératoires : ce sont elles qui disent à l'élément l'opération qu'il doit faire. Ces directives sont les instructions qui mettent les éléments en phase opératoire autonome. Ainsi les *Directives opératoires* qui jalonnent les *instructions complètes* sont-elles aussi d'importantes césures du programme.

b) *les Directives centrales*

Ce sont les instructions qui permettent d'exécuter les opérations logiques simples qui ont été attribuées à l'Unité Centrale (cf. chapitre 1). Elles s'intercalent purement et simplement dans le cours des instructions complètes. Les directives centrales qui permettent de transformer un branchement toujours, en branchement jamais, ou l'inverse, en constituent un bon exemple. C'est grâce à elles que l'on programme un positionnement d'aiguillage (cf. chapitre 2 § IV) : l'aiguillage est un branchement que, grâce à ces instructions, on positionne en toujours ou jamais 20 ou 30 instructions auparavant ou même plus.

La description détaillée de toutes ces directives centrales sera donnée en deuxième partie de ce volume. Si on considère ce type de directive du point de vue "césure" dans le programme, elles sont assimilables aux branchements.

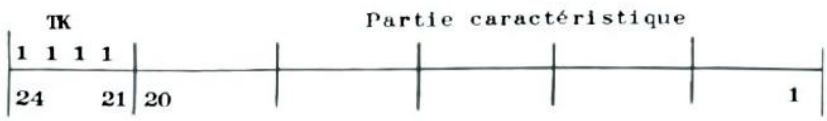
2°/ Structure des instructions D

Du point de vue structure, on distingue parmi les directives éléments : les directives avec variable libre et celles sans variable libre dont la partie caractéristique est tout ce qui n'est pas le TK. Cette classification est totalement différente de celle des directives préparatoires et opératoires ; elle se situe sur un autre plan. Les directives éléments sont caractérisées par un TK de quatre positions.

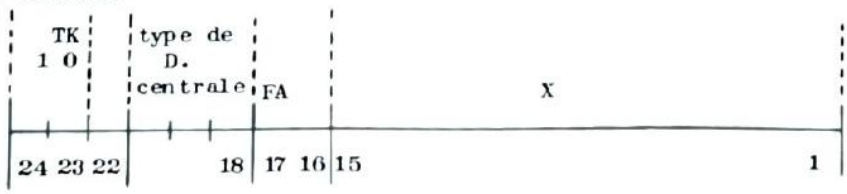
a) Directive Elément avec variable libre

Avec les 3 positions réservées au type de directive, on voit que l'on peut avoir pour un élément un maximum de 8 directives avec variable libre différente.

b) Directive Elément sans variable libre



Les *Directives Centrales*, elles, sont toutes avec variable libre ; elles ont un TK de 2 positions qui est 10. Etant avec variable libre, elles ont une F.A. Il reste donc, dans le catène, 5 positions (soit 32 possibilités) pour indiquer de quelle directive centrale il s'agit. On verra en deuxième partie du volume qu'il n'en existe pas autant.



3°/ Exploitation de l'instruction D

a) Pour une *directive centrale*, le Distributeur de Programme :

. exécute le travail qu'elle indique, puis poursuit l'alimentation des instructions suivantes de la séquence en cours.

b) Pour une *directive préparatoire*, le Distributeur de Programme :

. transmet à l'élément intéressé les renseignements qu'elle comporte,

. exploite ensuite sa chaîne de reprise propre,

. prend alors en considération une des demandes en instance à ses circuits de priorité (cette demande peut être celle qui vient de l'élément qui a reçu la directive préparatoire).

c) Pour une *directive opératoire*, le Distributeur de Programme :

. met en phase de travail autonome l'élément auquel elle s'adresse,

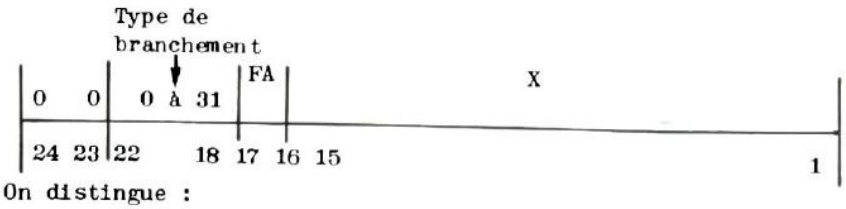
. exploite sa chaîne de reprise propre,

. puis prend en considération une demande en instance à ses circuits de priorité.

V – LES BRANCHEMENTS

Ils ont pour but de modifier la progression naturelle du programme dans des adresses successives de la Mémoire Rapide, en sautant brutalement à l'adresse désignée par sa variable libre. Pour cela, ils forcent le R.A.P.A. à cette adresse. De plus, le contenu du R.A.P.P. est envoyé dans le R.S.D.P. pour une mise en réserve éventuelle dans le cas d'un appel de sous-programme.

Le profil de cette instruction est :



On distingue :

a) les branchements conditionnels

Le branchement n'est effectif que si la condition matérialisée par telle position du catène qualitatif de l'élément demandeur est réalisée. Le Numéro de la position intéressée est spécifié en binaire dans les 5 positions 18 à 22 : type de branchement. Dans le cas contraire, le Numéro d'instruction progresse normalement dans le R.A.P.A. Toute ventilation logique ("ou" exclusif) se réalise par un ou une succession de branchements conditionnels.

On a ainsi les branchements B_1 à B_{24} compris.

b) les branchements systématiques

. Le branchement toujours B_{31} (les 5 monades 18 à 22 sont toutes des 1) : il est toujours effectif.

. Le branchement jamais B_0 : il ne fait rien d'autre que d'envoyer le contenu du R.A.P.P. dans le R.A.P.A. Puisque le TK du branchement est 00, les 7 dernières monades de ce catène instruction sont des 0. Le reste est sans importance. Ainsi la rencontre d'un catène vierge ou ne contenant quelque chose que dans sa partie X (cas d'une mémoire de manoeuvre contenant une adresse ou du catène suivant l'instruction de coupure) se comporte comme un branchement "jamais".

VI - L'INSTRUCTION D'ADRESSE

1°/ Le profil de cette instruction est le suivant :

. Le TK occupe 3 positions.

. Ensuite viennent 2 positions dans lesquelles est indiqué quel est, parmi les 4 types d'instructions d'adresse, celui dont il s'agit (cf. plus loin).

. 2 positions indiquent enfin le Numéro de R.A.C. sur lequel doit porter cette instruction. On sait, en effet, que chaque élément comporte un nombre de R.A.C. égal à 4 tout au plus. Ces R.A.C. sont numérotés de 0 à 3.

. Viennent enfin la F.A. et l'adresse sur 15 positions.

2°/ Les quatre types d'instructions d'adresse sont :

a) Le garnissage

Le R.A.C. indiqué est chargé de l'adresse indiquée dans la partie X de l'instruction (éventuellement par l'intermédiaire de la F.A.).

b) Le Rangement

Le contenu du R.A.C. désigné est rangé en Mémoire Rapide à l'adresse indiquée par l'instruction.

c) L'Addition

On additionne au contenu du R.A.C. la quantité donnée dans l'instruction.

d) La Soustraction

On retranche du contenu du R.A.C. la quantité donnée dans l'instruction. Le résultat se retrouve dans le R.A.C.

3°/ Ces différentes sortes d'instructions permettent de bénéficier du fait qu'au cours d'un transfert d'opérande, le R.A.C. correspondant a évolué du nombre de catènes de l'opérande et qu'il reste à cette valeur tant qu'une autre instruction ne vient pas le modifier.

Ainsi, à la fin d'une phase opératoire, on peut :

. soit ne pas avoir besoin de recharger un R.A.C. avant de relancer une nouvelle phase opératoire en bénéficiant de sa progression naturelle au cours de l'opération précédente.

. soit se livrer (sans avoir recours au calculateur logique) à un calcul d'adresse très simple en ajoutant ou retranchant à son contenu ancien une quantité déterminée.

. soit même mettre en réserve cette valeur dans une mémoire de manoeuvre afin de la réutiliser ultérieurement.

Bien entendu, chaque instruction A n'est relative qu'à un seul R.A.C. ; dans une instruction complète, il y a autant d'instructions A que de R.A.C. à charger, sauf si l'on peut bénéficier de leur progression naturelle pour éviter d'avoir à les recharger.

VII - LA FONCTION D'ADRESSE

1°/ On a déjà fait plusieurs fois allusion à cette fonction d'adresse et, en décrivant les profils des différents catènes instruction, on a vu que chaque fois qu'ils comprenaient une variable libre, les fonctions 16 et 17 servaient à indiquer la fonction d'Adresse (F.A.).

Toutefois, dans l'exemple traité au chapitre 2, on n'en a jamais parlé. Toutes les adresses dont il était question étaient des adresses absolues en Mémoire Rapide. Cela était parfait lorsqu'il s'agissait tout simplement d'expliquer la logique générale de la machine, mais travailler dans ces conditions lorsqu'il s'agit de rédiger vraiment un programme serait absurde : cela imposerait de savoir alors où il serait implanté en Mémoire Rapide au moment de son exploitation.

Or, on n'a pas voulu pareille servitude dans la programmation du GAMMA 60. On a voulu que des personnes différentes puissent programmer séparément les différentes parties d'un problème sans avoir à se soucier de leur implantation réelle en Mémoire Rapide, la coordination et l'implantation de ces différents "sous-programmes" étant le fait d'un "*programme d'assemblage*". Cette condition souhaitable devient absolument nécessaire lorsqu'il s'agit de programmer des sous-programmes "*standards*" qui seront utilisés à plusieurs reprises mais implantés dans des adresses différentes de Mémoire Rapide : il serait, en effet, inadmissible d'avoir à les reprogrammer chaque fois qu'on veut les réutiliser.

La programmation du GAMMA 60 se fera donc en deux phases :

a) la codification ligne à ligne, en forme de sous-programme, des parties originales du problème, les autres ayant été codifiées une fois pour toutes en sous-programmes standards. Ce travail doit se faire sans se soucier de l'implantation en Mémoire Rapide. C'est l'objet de ce volume.

b) l'assemblage de ces différents sous-programmes et leur implantation en Mémoire Rapide pour constituer le programme du problème à traiter.

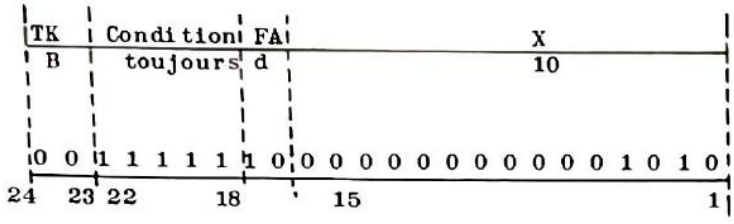
2°/ On est donc bien obligé d'utiliser, au cours de la rédaction d'un sous-programme, un système d'adressage relatif qui revient tout simplement à affecter chaque instruction d'un simple numéro d'ordre de 0 à n. Mais l'on voit tout de suite que lorsqu'on aura à programmer un branchement par exemple, cette instruction ne pourra pas comporter directement l'adresse absolue de renvoi en Mémoire Rapide. Ce sont les possibilités de la Fonction d'Adresse qui permettent de résoudre les problèmes que cela pose.

Cette Fonction d'Adresse est la possibilité qu'a le Distributeur de Programme de faire certains calculs d'adresse simples d'après les indications fournies par 2 positions, 16 et 17, de certaines instructions canoniques qui ne sont pas simplement les instructions d'adresse, mais, d'une façon générale, les instructions qui comportent une variable libre. Le calcul que permet la Fonction d'Adresse porte sur cette variable libre. L'une de ces positions indique la fonction d'Adresse différentielle, l'autre la fonction de Substitution : les 2 fonctions peuvent exister en même temps dans la même instruction.

a) Adresse différentielle (d)

Reprenons l'exemple du branchement. Soit un sous-programme rédigé en adresses relatives de 0 à 38. En 10, on a un branchement renvoyant en adresse relative 30, c'est-à-dire 10 instructions plus loin. C'est cette différence de 10 instructions qu'on appelle adresse différentielle.

L'instruction de branchement sera donc :



Lorsque cette instruction sera exploitée, elle se trouvera par exemple en adresse absolue 526. Le Distributeur de Programme trouvant l'indice d saura que l'adresse extraordinaire à laquelle renvoie le branchement n'est pas 10 mais 526 + 10 - 536.

S'il s'agissait de "remonter" 10 instructions plus haut, on indiquerait dans la partie X non pas 10 mais son complément : 32.768.

b) Substitution (s)

Nous avons dit qu'il existait une instruction d'adresse qui permettait de décharger un R.A.C. dans telle adresse Mémoire Rapide que l'on veut, afin de mettre son contenu en réserve pour une utilisation ultérieure. Cette adresse de Mémoire Rapide est une "mémoire de manoeuvre" : supposons qu'il s'agisse de l'adresse 1.052. Lorsque l'on voudra charger un autre R.A.C. avec cette adresse que l'on aura réservée dans le catène Mémoire Rapide 1.052, on ne pourra rédiger l'instruction correspondante que de la façon suivante : "Charger le R.A.C. n° 2 du contenu de l'adresse 1.052", ce qui se traduira par l'instruction canonique :

T.K.	N° R.A.C.	F.A.	Variable libre
A	2	s	1.052

Au vu de l'indice s, le Distributeur de Programme ne chargera pas le R.A.C. de 1.052 mais, par substitution, du contenu de 1.052.

c) Adresse différentielle et substitution

Il se peut que la mémoire de manoeuvre, dont il vient d'être question, soit une adresse réservée à la fin du sous-programme lui-même.

Si celui-ci va des adresses relatives 0 à 38 et que 39 soit la mémoire de manoeuvre utilisée pour l'instruction A d'adresse relative 10, l'instruction "Charger le R.A.C. n° 2 du contenu de l'adresse relative 39" se traduira par une instruction canonique A d'adresse relative 10 ainsi rédigée :

T.K.	N° R.A.C.	F.A.	Variable libre
A	2	d, s	29

Si, au moment de son exploitation, cette instruction se trouve en adresse absolue 520, son effet sera de charger le R.A.C. par le contenu de l'adresse absolue 520 + 29 - 549.

On voit donc, en définitive, que la Fonction d'Adresse est le moyen qui permet de rédiger les sous-programmes sans s'occuper de leur implantation en Mémoire Rapide, de les stocker en bibliothèque et de les utiliser au moyen du programme d'assemblage.

2ème partie

DESCRIPTION DÉTAILLÉE DES ÉLÉMENTS ET DU CODE SYMBOLIQUE DE PROGRAMMATION

chapitre 1

GÉNÉRALITÉS

I - CODE A ET CODE B

Lorsque le programme se trouve en Mémoire Rapide et qu'il est exploité, chaque instruction canonique tient dans un catène. Nous avons décrit, à la fin de la première partie, le profil des différentes instructions canoniques, du moins dans sa structure générale.

Ce profil constitue le *code interne des instructions canoniques, celui que sait interpréter le Distributeur de Programme. On l'appelle code A.*

Le peu qu'on en a dit suffit à montrer que, si ce code était celui que devrait utiliser le programmeur pour rédiger les programmes, la programmation "ligne à ligne" serait un travail très délicat, nécessitant du programmeur une connaissance parfaite de ce code interne et un effort d'attention énorme. Cette connaissance parfaite du code A serait difficile à acquérir parce qu'il n'est pas du tout mnémotechnique et, d'autre part, un programme rédigé en code A serait pratiquement illisible et donc très difficile à vérifier.

De plus, il faudrait que le programmeur rédige toutes les adresses et longueurs en binaire pur puisque telle est la forme sous laquelle elles doivent figurer dans les 15 premières positions du catène-instruction.

Enfin, dans le cas d'adresses différentielles, le programmeur devrait faire la soustraction entre l'adresse relative à désigner et l'adresse relative de l'instruction elle-même. Et, dans le cas où la différence serait négative, il devrait penser à prendre le complément à 32.768.

On a donc établi à l'usage du programmeur un code symbolique de programmation dit code B, qui évite les défauts que nous venons de signaler pour le code A. *Ce code B est :*

- *mnémotechnique*
- *de rédaction brève*
- *conçu pour faciliter la lecture des programmes.*

De plus, dans ce code, le programmeur rédigera les adresses en décimal par un nombre de 5 chiffres au plus (32.767). Enfin, comme on le verra au § suivant, il n'aura pas à faire la soustraction nécessitée par l'adressage différentiel.

Comme on le verra également au § suivant, chaque instruction canonique sera composée au plus de 13 caractères alphabétiques ou numériques. C'est sous cette forme que les programmes seront introduits directement en machine; on perforera les instructions sur des cartes qu'on lira et traduira comme n'importe quelles cartes. La traduction se fera en code interne alphanumérique (4 caractères par catène) et chaque instruction canonique occupera ainsi 4 catènes, ce qui n'a aucune importance puisqu'aussitôt après un programme interprétatif (qui, lui, aura évidemment dû être rédigé en code A) transformera ces instructions code B en instructions code A occupant 1 catène par instruction canonique.

Cette transposition code B - code A aura ainsi lieu une fois pour toutes au moment de l'introduction du programme en machine. *C'est en code A que les programmes seront stockés en attendant leur utilisation;* ce stockage se fera sur ruban magnétique ou sur tambour.

Il faut bien noter que l'un des rôles du programme de transposition code B - code A sera de transformer en binaire les adresses que le programmeur aura rédigées en décimal et de faire les soustractions nécessitées par l'adressage différentiel.

II. DESSIN D'UNE FEUILLE DE PROGRAMMATION

La planche 6 représente une feuille de programmation. Sur cette feuille, le programme lui-même figure dans le cadre central (en trait gras); encore faut-il en retirer les 4 premières colonnes qui contiennent simplement le numéro de l'instruction. En dehors de ce cadre central, les différentes zones prévues sont là uniquement pour aider le programmeur dans son travail et pour faciliter la relecture du programme.

Nous allons commenter chacune des zones de cette feuille en partant de la gauche.

La première zone "traitement" sert à repérer les différentes phases de l'organigramme d'enchaînement que l'on doit faire préalablement à la rédaction du programme ligne à ligne. En regard de ces phases figurent sur le reste de la feuille les instructions canoniques correspondantes.

La deuxième zone "branchements" aide à repérer au moyen de flèches les adresses extraordinaires des branchements du programme.

La zone "N° de ligne" sert à l'*adressage relatif* des instructions du sous-programme que l'on rédige. Ces adresses relatives sont tout simplement les numéros d'ordre des instructions dans le programme. Dans tout sous-programme, les instructions seront numérotées à partir de 0.

Les quatre zones qui suivent servent de cadre aux instructions canoniques elles-mêmes rédigées en code B. Comme on le voit, elles comportent au total 13 colonnes.

La première de ces zones "type d'opération" (T.O.) comporte 4 colonnes. Dans cette zone, les coupures seront indiquées par un code mnémotechnique à 4 lettres, les directives éléments (préparatoires ou opératoires) par un code à 3 lettres, les directives centrales par un code à 2 lettres, les branchements par un code à 2 chiffres et les adresses par un code à une lettre. Pour les codes à moins de 4 caractères, les colonnes laissées vierges seront celles de gauche.

Ainsi, à la seule vue de cette zone, on aura une idée de la structure générale du programme. Les césures les plus importantes (coupures) y sont marquées par les codes mnémotechniques les plus riches (4 caractères), puis viennent les direc-

tives élément (3 caractères); on trouve ensuite sur le même plan directives centrales et branchements (2 caractères) et enfin les adresses (1 caractère).

Toutefois, il ne faudrait pas confondre cette zone avec le TK des instructions en code A. Elle comporte davantage de renseignements qu'il n'y en a dans le TK. D'ailleurs, d'une façon générale, il ne faut pas chercher de correspondance entre les zones du code B et celles que l'on a distinguées dans la structure du code A, exception faite pour les F.A.

La deuxième zone de l'instruction canonique est de 2 colonnes; c'est la zone "codes complémentaires" (C.C.) qui, comme son nom l'indique, contient dans certains cas des codes précisant le T.O. et différents selon le type d'instruction.

Vient ensuite la zone X de 5 colonnes qui contient la variable libre (5 chiffres maxi) dans les instructions à variable libre et des codes spéciaux que l'on précisera pour chaque cas d'espèce, dans les instructions sans variable libre.

La dernière zone, de deux colonnes, est la *fonction d'adresse* (F.A.). La première colonne sert à noter l'adresse différentielle comme on va le voir tout à l'heure; la seconde sert à noter la substitution d'adresse par un "S" que le programme interprétatif traduira par un 1 en position 16 du catène d'instruction.

Pour l'adresse différentielle, le programmeur aura deux possibilités. Nous avons vu dans la première partie du cours que cette notation était indispensable en code A si l'on voulait que l'instruction canonique soit exploitable quelle que soit l'implantation du programme en Mémoire Rapide. Mais, pour le programmeur qui rédige son programme en adresses relatives de 0 à 38 pour un programme de 39 instructions canoniques (en reprenant l'exemple de la première partie du volume), la notion d'adresse différentielle est seconde par rapport à celle d'adresse relative. Si, à l'adresse relative 10, il a un branchement, sa première réaction sera de chercher à quelle ligne (relative) il doit se renvoyer et c'est seulement alors qu'il calculera la différence entre l'adresse du branchement et son adresse de renvoi. On a voulu éviter au programmeur cette dernière phase de travail pour l'attribuer à la machine.

Dans l'exemple de la première partie, en ligne 10 du programme, il y a un branchement toujours renvoyant en ligne (relative) 20. C'est précisément ce qu'aura à écrire le programmeur en code B. En code B, il écrira 20 dans les deux dernières colonnes de X et, dans la première colonne de F.A., il écrira "R" pour indiquer qu'il s'agit d'une adresse relative. Le programme interprétatif, voyant l'indice R dans cette instruction d'adresse relative 10, fera alors la soustraction :

20 - 10 = 10

transformera ce résultat en binaire, le placera dans les 15 premières positions du catène-instruction en code A, tandis qu'il placera l'indice différentiel en position 17.

S'il s'agissait d'une différence négative (par exemple en 20 un branchement renvoyant en 10), c'est le programme interprétatif qui se chargerait en plus de prendre le complément à 32.768.

Il y a encore une deuxième possibilité de noter l'adresse différentielle : c'est dans le cas de l'utilisation de "mémoires de manoeuvre". Soit à charger un R.A.C. d'une adresse qu'on a précédemment mise en réserve dans une mémoire de manoeuvre. On peut prendre comme mémoire de manoeuvre le catène d'adresse relative 39 dans le cas de notre programme rédigé de 0 à 38. On notera en X et F.A. de l'instruction d'adresse 39 R S et si cette instruction est en ligne 14, le programme interprétatif mettra finalement dans le catène instruction 15 D S et le Distributeur de Programme, lorsqu'il alimentera cette instruction, ira chercher le contenu (indice S) du catène dont l'adresse s'obtient en ajoutant 15 à l'adresse de l'instruction (indice B).

Mais, dans le cas général, au cours d'une rédaction de sous-programme, on a

besoin de plusieurs mémoires de manoeuvre et, d'autre part, quand on a besoin de faire appel à elles, on ne sait pas encore le nombre exact de lignes du programme et donc, il est impossible de donner à ces mémoires une adresse relative dans le programme en supposant qu'on veuille les ranger en fin de programme. Le programmeur fera donc, à côté du programme proprement dit, une liste de mémoires de manoeuvre qu'il numérote de 0 à n. Ainsi, en code B, à tout sous-programme numéroté de 0 à n sera associée une liste de mémoires de manoeuvre de 0 à n.

Lorsque, dans une instruction du programme, on aura à se référer à une mémoire de manoeuvre, on indiquera son numéro dans cette liste et, dans la colonne réservée à l'"adresse différentielle", on utilisera "M" pour bien préciser qu'il s'agit d'un numéro de mémoire de manoeuvre. Au moment de la transposition du code B en code A, c'est la machine elle-même (qui saura alors exactement le nombre de lignes du programme) qui placera les mémoires de manoeuvre en fin de programme, leur donnant ainsi une adresse relative avant de passer à l'adresse différentielle elle-même.

Soit par exemple dans notre programme de 39 lignes numérotées de 0 à 38, une instruction d'adresse relative 20 faisant appel au contenu de la mémoire de manoeuvre n° 3 (ce qui signifie qu'en X et F.A. on a : 3 M S). La machine placera les mémoires de manoeuvre à la fin des 39 lignes de programme, leur donnant à chacune une adresse relative :

39 pour la mémoire de manoeuvre 0
39 + n " " n

Ainsi, 3 M S sera transformé en 42 R S, puis en (42 - 20) D S soit 22 D S.

Enfin, pour terminer l'examen de cette feuille de programmation, le titre de la dernière zone "Observations" indique suffisamment à quoi elle sert. Précisons simplement que les 4 colonnes que l'on y a tracé au début peuvent servir à suivre l'évolution des R.A.C. de l'élément en activité à cette phase du programme. Cela pourra être très utile au programmeur, s'il veut bénéficier de la progression naturelle des R.A.C. pour éviter les instructions d'adresse (spécialement dans une séquence de calcul : voir chapitre sur le Calculateur Arithmétique).

III. CODE DES ADRESSES

	T.O.				C.C.		X	F.A.	
Garnir Ranger Additionner Soustraire				G R A S	N° R.A.C.: 0, 1, 2 ou 3		Adresse Quantité	rien, M ou R	rien ou S

Le tableau ci-dessus fait apparaître la codification des 4 sortes d'instructions adresse. Comme on le voit, il n'y a jamais rien dans la deuxième colonne de code complémentaire. Dans la première, figure le n° du R.A.C. de l'élément bénéficiaire de l'instruction.

.*Garnir* - Le R.A.C. désigné est garni de l'adresse figurant dans X s'il n'y a rien dans F.A. S'il y a quelque chose dans F.A., les opérations indiquées sont effectuées sur X et c'est du résultat ainsi obtenu qu'est garni le R.A.C.

.*Ranger* - Le contenu du R.A.C. désigné est rangé à l'adresse figurant dans X (sous réserve de la F.A.).

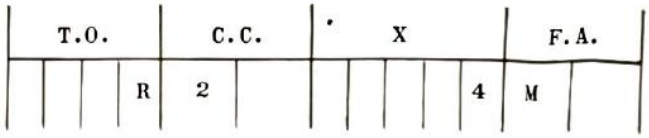
.*Additionner* - On additionne à l'ancien contenu du R.A.C. désigné la quantité

X (sous réserve de la F.A.). Le résultat final se trouve dans le R.A.C.

.*Soustraire* - On retranche la quantité X de l'ancien contenu du R.A.C. Opération analogue à la précédente.

Il faut bien noter qu'entre "garnir" et "ranger", il y a la différence d'une substitution. L'exemple suivant fera comprendre ce que l'on entend par là :

Supposons que l'on veuille profiter de la progression naturelle d'un R.A.C. au cours d'une opération pour éviter un calcul d'adresse. A la fin de l'opération, on range son contenu dans une mémoire de manoeuvre donnée, par exemple la mémoire de manoeuvre n° 4. A cet effet, on écrira l'instruction :



Si un peu plus tard, on veut réutiliser le *contenu* de cette mémoire de manoeuvre pour garnir un R.A.C. (qui peut être celui d'un autre élément ou non), on devra mettre dans l'instruction l'indice substitution. L'instruction sera alors :

IV. CODE DES BRANCHEMENTS

Les Branchements s'indiquent par un T.O. de deux caractères : ce sont 2 chiffres qui sont, en fait, la traduction en décimal du type de branchement figurant dans les 5 positions binaires 18 à 22 du code A. C'est le seul type d'instruction où le T.O. est indiqué par des chiffres.

On aura ainsi :
00 pour le branchement jamais
31 pour le branchement toujours
01 à 24 pour les branchements conditionnels sur la position ainsi désignée du catène qualitatif

Branchements	T.O.	C.C.	X	F.A.
Jamais	0 0		adresse de renvoi *(1)*	rien, M ou R / rien ou S
Toujours	3 1			
Conditionnels	0 1 à 2 4			

(1) ou adresse d'une mémoire si S en F.A.

V – CODE DES COUPURES

Elément	T.O.				C.C.	X	F.A.	
Calculateur arithmétique Calculateur logique Traducteur Comparateur général Tambour magnétique Lecteur de cartes Perforateur de cartes Imprimante Dérouleur de ruban magnétique Lecteur de bande perforée Perforateur de bande	A L T C T L P I D L P	R O R O A E E M E E E	I G A M M C C P R B B	T I D P B T T R U A A	Type de coupure : rien, A ou B	N° d'élément (1)	Rien, M ou R	rien ou S
Coupure sur élément virtuel Libération d'élément virtuel Lancement de séquence simultanée Fin de séquence	P L S S	R I I T	O D M O	T E U P		Adresse (1) catène-image Adresse (1) extraord.	rien M ou R	rien ou S

(1) ou adresse d'une mémoire si S en F.A.

Ce tableau du code B des instructions de coupure est divisé en deux parties.

Celle du haut est relative aux coupures sur élément réel. Dans ce cas, le T.O. indique, par un code mnémotechnique à 4 lettres évident, la nature de l'élément que l'on veut mettre en activité. Quelques-uns de ces codes méritent explication :

LEcteur de CarTes et PErforateur de CarTes sont codés LECT et PECT par opposition à LEBA et PEBA pour LEcteur de BAnde perforée et PErforateur de BAnde. De même, on parle de Bande perforée par opposition à Ruban magnétique. D'où DERU pour DErouleur de RUban.

Le premier code complémentaire indique le type de coupure réelle dont il s'agit :

rien : coupure normale
A : coupure sans libération de l'élément demandeur
B : coupure sans observation de la "disponibilité" de l'élément demandé.

La partie X contient le n° de l'élément (sous réserve de la F.A.). Toutefois, lorsqu'il s'agit d'un élément central de traitement qui est unique, aucune indication n'y est nécessaire : les codes ARIT, LOGI, etc ... suffisent amplement à le désigner sans ambiguité.

Lorsqu'il s'agit d'une sorte d'élément pouvant exister à plusieurs exemplaires, ce qu'on indique dans X, ce n'est pas le n° de l'élément qui figurera dans le R.N.E. du Distributeur de Programme lorsque celui-ci travaillera pour lui (le code mnémotechnique à 4 lettres ne servirait alors à rien) mais tout simplement le n° de l'élément dans la série de ceux de son type.

Par exemple, s'il s'agit du Tambour n° 2, on écrira TAMB 2 et le programme interprétatif traduira ceci en "coupure normale sur l'élément 102" sachant que les tambours portent les n° d'éléments de 100 à....

La deuxième partie du tableau concerne les coupures sur élément virtuel, la SIMU et la fin de séquence. La coupure d'occupation d'élément virtuel se code PROT pour rappeler que cette instruction sert à la PROTection d'un sous-programme, d'une zone de Mémoire Rapide ... Les autres codes sont évidents.

Il n'y a jamais rien dans les colonnes de code complémentaire.

En X, on indique, pour les coupures sur élément virtuel, l'adresse du premier catène-image de l'élément et, pour la SIMU, l'adresse de la branche extraordinaire, avec F.A. éventuelle. Pour STOP, il n'y a rien à indiquer en X ni en F.A.

Le nombre de conditions supplémentaires attendues est indiqué, en code B, dans la partie X du catène vierge qui suit la coupure.

VI - ILLUSTRATION

Pour illustrer les paragraphes précédents, on a écrit sur la planche 6 qui donne le modèle d'une feuille de programmation, le début du programme de l'exemple traité en première partie chapitre 2.

Ce programme très simple est écrit en adresse absolue, ce qui explique qu'il ne comporte aucune fonction d'adresse. On remarquera comment les coupures et les directives se détachent dans la zone des T.O. Parmi les deux directives Tambour, il y en a une préparatoire et une d'exécution.

Pour la D.A.O. (Désignation Adresse Origine), la partie X contient l'adresse tambour dont il faut charger le R.A.E. Pour T.R.T. (Transfert mémoire Rapide vers Tambour), X indique la longueur en catènes de ce transfert.

Ce simple exemple suffit à montrer la "lisibilité" de ce mode d'écriture et combien la connaissance du code A est inutile au programmeur. On voit en particulier combien il lui importe peu que le TK d'une instruction de coupure soit 01 et celui d'une adresse 110.

A titre d'exercice, cependant, l'on pourra, en s'aidant des éléments de code A donnés en première partie et en faisant les suppositions nécessaires pour le reste, essayer d'écrire en code A ces quelques lignes de programme. Cela suffira à montrer combien il est difficile de coder ainsi, combien un programme ainsi rédigé est illisible, en un mot à montrer la supériorité du code B.

FONCTIONS CENTRALES

I - DEFINITION

On a déjà eu l'occasion, au cours de la première partie de ce volume, d'expliquer ce que l'on entendait par fonctions centrales. Rappelons en les caractéristiques.

Ce sont, en général, des opérations de servitude du programme qui s'exécutent au niveau de l'Unité Centrale. Elles n'ont pas leur emploi au niveau du travail autonome des éléments. Les unes sont des fonctions extrêmement simples, d'autres sont plus synthétiques comme le "décomptage itération", mais toutes ont la caractéristique d'être d'un emploi fréquent qui justifie leur exécution en Unité Centrale et non par le Calculateur Logique.

Les instructions qui permettent de les exécuter sont des "directives centrales". S'intercalant au cours des instructions complètes, elles ne libèrent pas le Distributeur de Programme et, du point de vue "césure dans le programme", elles s'assimilent aux branchements et se codent donc par une T.O. de 2 caractères. Ce code mnémotéchnique à deux lettres est évidemment moins riche et donc de signification moins évidente que celui des coupures.

En code B, ces directives centrales ne comportent aucun code complémentaire. Il en existe deux qui n'ont également ni partie X, ni F.A. (cf. tableau en fin de chapitre).

II - LES REGISTRES DE PROGRAMMATION DE L'UNITE CENTRALE

On désigne sous ce nom les registres de l'Unité Centrale auxquels on a accès par les instructions du programme. A l'exception d'un seul, nous avons déjà eu l'occasion d'en parler au cours de la première partie. Nous les rappelerons pour mémoire :

. Les différents registres (R.A.P.A. et R.A.P.P.) qui contiennent les adresses d'instructions du programme en cours. Ce sont des registres de 15 positions binaires.

. Le *R.N.E.* qui contient le n° de l'Elément pour lequel travaille le Distributeur de Programme à un instant donné (7 positions).

. Les registres d'entrée et de sortie du Distributeur de Programme (R.E.D.P. et R.S.D.P.) par lesquels s'effectuent tous les transits entre Mémoire Rapide et Distributeur de Programme (24 positions).

. *Le registre A,* ou registre d'analyse, dans lequel est envoyé le catène qualitatif de l'élément lorsqu'il est pris en considération par le Distributeur de Programme (24 positions).

. *Le registre B* ou registre opérateur. Ce registre B (de 24 positions) ne doit pas être confondu avec le registre B qui, avec le registre C, sert à constituer la chaîne de reprise du Distributeur de Programme. Lorsque, dans la première partie, on parlait de registre B du Distributeur de Programme, c'était de ce dernier dont il s'agissait et on l'associait au Registre C. On les avait nommés ainsi par analogie avec les registres B et C des éléments. Or, on a vu que tous ces mécanismes de chaînes de reprise sont automatiques et que le programmeur n'a pas à s'en occuper. Il faut donc maintenant ne plus s'en soucier jusqu'au point de les oublier.

Lorsque, dans cette seconde partie, *on parlera de registre B de L'Unité Centrale,* il s'agira toujours *de son seul registre opérateur,* celui dans lequelon peut faire des forçages, des additions et soustractions binaires, etc..., registre dont on n'avait pas parlé au cours de la première partie.

III - LES DIRECTIVES CENTRALES

Dans ce paragraphe, on examinera l'une après l'autre les différentes fonctions centrales en décrivant les directives qui permettent de les exécuter et leurs effets exacts.

On peut diviser les directives centrales en quatre grands groupes :

A) Instructions de gestion des différents registres (N° d'instruction, d'élément, d'analyse et opérateur).

B) Instructions de calcul dans le registre opérateur.

C) Instructions de modification de catènes contenus en Mémoire Rapide.

D) Instructions synthétiques :
   Décomptage Itération
   Progression de compteur en Mémoire Rapide
   Substitution d'instruction.

1°/ Instructions de gestion des Registres

1.1.- Rangement de numéro d'instruction (R.I.). On a vu en première partie (chapitre 2, § VII c) les circonstances d'utilisation de cette instruction en tête d'un sous-programme. Rappelons simplement qu'elle permet de ranger à telle adresse de Mémoire Rapide que l'on veut, l'adresse d'instruction à laquelle on devra revenir en fin de sous-programme.

En X, on indique l'adresse Mémoire Rapide où l'on désire ranger le n° d'instruction. Le plus souvent, on aura "M" dans F.A. car la mémoire de rangement sera une mémoire de manoeuvre du sous-programme.

En fin d'opération, le n° d'instruction se trouve dans les 15 premières positions de la mémoire, les 9 dernières ayant été effacées (transfert sur 24 positions).

1.2.- Rangement du n° d'Elément (R.E.). C'est le contenu du R.N.E. que l'on range à l'adresse indiquée en X. Comme pour l'opération précédente, les 17 dernières positions de la mémoire réceptrice sont effacées, le N.E. se trouvant dans les 7 premières.

1.3.- Garnissage du R.N.E. (G.E.)

En X, on indique le n° d'Elément que l'on veut transférer dans le R.N.E. (avec éventuellement F.A.).

1.4.- Transfert dans le registre A (T.A.)

1.5.- Rangement du Registre A (R.A.)

1 6.- Transfert dans le Registre B (T.B.)

1.7.- Rangement du Registre B (R.B.)

Dans ces 4 instructions, on cite en X l'adresse de la Mémoire Rapide émettrice ou réceptrice. Dans les deux cas, transfert ou rangement, le transfert est intégral et a lieu sur les 24 positions. L'organe émetteur n'est pas modifié.

Il faut bien noter que X n'est pas la quantité dont on charge le Registre alors que dans l'instruction G.E. ou dans une instruction de garnissage de R.A.C., est (sauf présence de l'indice S dans la F.A.) la quantité dont on charge le R.N.E. ou le R.A.C. On exprime ce fait en disant que les instructions T.A. et T.B. sont à *substitution implicite*.

2°/ Instructions de calcul dans le Registre opérateur

Il s'agit de calcul binaire sur les 15 premières monades des catènes. Ces instructions peuvent servir à des calculs d'adresse simples.

2.1.- Garnissage du Registre B (G.B.). Cette instruction est tout à fait analogue à l'instruction G.E. ou à un Garnissage de R.A.C. Ce que l'on indique en X est, s'il n'y a pas de F.A., la quantité même dont on désire charger B. Contrairement à T.B., cette instruction ne comporte donc pas de substitution implicite.

Ce qui différencie toutefois cette instruction d'un garnissage de R.A.C., c'est que le registre B a 24 positions tandis que les R.A.C. n'en ont que 15 et cette instruction, ne concerne que les 15 premières positions de B. Les 9 dernières restent invariantes.

L'instruction G.B. sans F.A. signifie donc que l'on envoie la quantité désignée en X dans les 15 premières positions de B, les 9 dernières n'étant pas modifiées. L'instruction G.B. avec "S" signifie que l'on envoie les 15 premières monades du catène d'adresse X dans les 15 premières positions de B, les 9 dernières n'étant pas modifiées.

2.2.- Addition dans B (A.B.)

2.3.- Soustraction dans B (S.B.)

2.4.- Comparaison dans B (C.B.)

Ces 3 opérations sont des opérations à deux opérandes.

Le premier opérande est le contenu des 15 premières positions du registre B.

Le deuxième opérande est le nombre cité en X s'il n'y a pas de F.A., et le nombre binaire contenu dans les 15 premières positions du catène d'adresse X s'il y a "S" dans F.A.

Le résultat se trouve, pour l'addition et la soustraction, dans les 15 premières positions de B. Les 9 dernières positions de B ne sont pas modifiées au cours de cette opération à tel point que, si une addition exigeait un report sur la 16ème position, ce report serait perdu.

Le résultat de la comparaison apparaît dans le registre A par le positionne-

ment d'un indice dans les positions suivantes du registre A :

Positions	:	1	2	3	4	5	6
Signification	:	$<$	$<$	$=$	$>$	$\leqslant$	$\neq$

Le sens de la comparaison est : "contenu de B par rapport à quantité citée dans la C.B.". Bien entendu, la comparaison ne porte également que sur les 15 premières positions des catènes.

2.5.- *Substitution de quantité (S.Q.)*. Cette instruction n'est pas, à proprement parler, une instruction de calcul. On ne l'a placée ici que parce qu'elle termine souvent un calcul d'adresses fait avec les instructions précédentes.

Elle consiste à substituer au contenu *intégral* de B le contenu du catène dont l'adresse figure dans les 15 premières positions de ce registre. Comme on le voit, contrairement aux opérations précédentes, cette opération affecte les 24 positions de B. Elle permet de faire apparaître dans le registre B, au moyen d'une seule instruction, un catène complet dont on vient de calculer l'adresse au moyen d'instructions des types précédents.

Bien entendu, par cette instruction, il n'y a rien à noter dans X, ni dans F.A. et elle se codera toujours simplement S.Q. dans le T.O.

3°/ Instructions de modification de catènes

Il s'agit essentiellement d'instructions permettant de modifier la nature d'un branchement ou de forcer à 1 ou à 0 toutes les positions d'un catène.

3.1.- Forçage à Toujours (F.T.)

3.2.- Forçage à Jamais (F.J.)

3.3.- Forçage Inversion (F.I.)

Pour ces 3 instructions, on cite en X l'adresse d'une instruction de programme qui est un branchement et dont on désire fixer la nature. Pour la première, on force à 1 les positions 18 à 22 du catène branchement (B_{31} - B toujours). Pour la seconde, on force ces mêmes positions à 0 (B_{00} - B jamais). Ces deux instructions servent aux positionnements d'aiguillage (cf. 1ère partie, chapitre 2, § IV). La troisième instruction est utilisée dans les programmes à bascule. Elle transforme un branchement toujours en jamais et vice-versa.

Le registre B est implicite dans ces instructions. Son contenu est modifié. Il faut s'en souvenir pour ne pas être tenté d'utiliser son ancien contenu après de telles instructions.

3.4.- Forçage à 1 (F.U.) (U - initiale de "un")

3.5.- Forçage à 0 (F.Z.) (Z - initiale de "zéro").

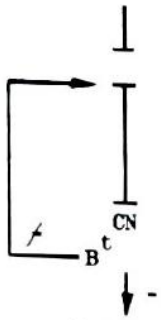
En X, on cite l'adresse d'un catène dont on veut forcer les 24 positions soit à 1, soit à 0 (effacement).

Le registre B n'est pas modifié par ces deux instructions.

4°/ Instructions synthétiques

4.1.- Décomptage Itération (D.I.)

On appelle séquence itérative une séquence de programme que l'on doit effectuer un certain nombre de fois avant de passer à la suite du programme. Toute séquence itérative se terminera donc par des instructions permettant de savoir si on doit la reprendre ou, au contraire, si on doit passer à la suite. Deux cas peuvent se présenter :



a) On ne connaît pas le nombre d'itérations. C'est par exemple le cas où l'on fait un calcul par une méthode d'approximations successives. On s'arrêtera lorsque les résultats obtenus dans deux itérations successives seront identiques à la précision du calcul près. Dans ce cas, on terminera la séquence itérative par une comparaison que l'on exploitera aussitôt par un branchement conditionnel qui relancera ou non la séquence itérative une nouvelle fois.

La structure d'une telle séquence sera celle du schéma ci-contre.

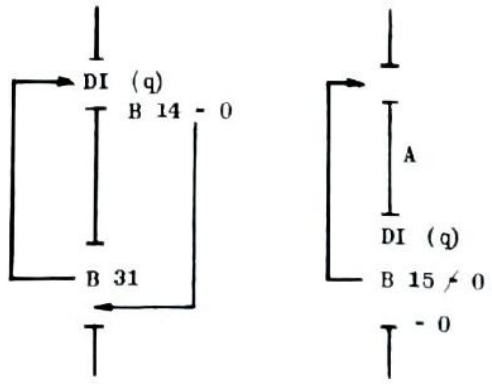
b) On connaît le nombre d'itérations. C'est un cas très fréquent en calcul matriciel. On a à recommencer la même séquence de calcul sur des données différentes, un nombre de fois bien déterminé qui dépend de l'ordre des matrices sur lesquelles on fait les calculs. Dans ce cas, le contrôle des itérations se fait au moyen d'un comptage. En fait, ce contrôle s'effectue par une instruction synthétique centrale, dite décomptage itération (D.I.) dont on exploite le résultat par un branchement comme il est expliqué plus loin.

Outre le catène qui la contient elle-même, cette instruction D I nécessite deux mémoires de manoeuvre associées d'adresses successives q et q + 1 que l'on peut choisir n'importe où en Mémoire Rapide. En début d'opération, ces deux mémoires contiennent dans leurs 15 premières positions le nombre N d'itérations à effectuer (exprimé en binaire bien entendu), les 9 premières positions étant à 0. L'adresse à q est indiquée (éventuellement par l'intermédiaire d'une F.A.) dans la partie X de l'instruction D.I.

Chaque fois que le Distributeur de Programme appelle l'instruction D.I. :

1) Le contenu de la mémoire d'adresse q est transféré dans le registre B.
2) Ce contenu n est comparé à 0 et :
   si n - 0 a) la position 14 du registre A est forcée à 1.
   b) le contenu du catène q + 1 soit N est envoyé dans le catène q qui est ainsi remis en état.
   si n ≠ 0 a) la position 15 du registre A est forcée à 1.
   b) le contenu du catène q est diminué d'une unité.

L'exploitation de l'instruction D.I. pourra donc se faire par des branchements 14 ou 15 suivant que D.I. aura été placée en tête ou en queue de séquence conformément au retenu suivant :



REMARQUE : a) si la séquence A est à faire N fois, les catènes associés q et q + 1, devront être chargés de N si D.I. est en tête, de N - 1, si D.I. est en queue. En effet dans ce dernier cas, la séquence A aura déjà été exécutée une fois lorsque le D.P. appellera le D.I.

b) L'instruction D.I. n'est valable que si le nombre contenu dans le

catène q, la première fois que le distributeur de programme appelle cette D I est différent de 0.

Aussi si ce nombre est le résultat d'un calcul et s'il n'est pas exclu qu'il puisse être nul, il faudra faire précéder la D I d'une séquence de comparaison de ce nombre à 0.

c) Le registre A est modifié en totalité par une D I et non pas seulement les positions 14 et 15. Ainsi on ne pourra exploiter après une D I, le résultat d'une comparaison faite avant.

4.2.- Progression de Compteur (P.C.)

Cette instruction permet de majorer de 1 la quantité contenue dans les 15 premières positions d'une mémoire d'adresse quelconque q.

Cette adresse q est indiquée en partie X de l'instruction (éventuellement avec F.A.).

L'opération se fait en deux temps :

1°) Le contenu des 24 positions du catène q, est envoyé dans le registre B.
2°) Le contenu des 15 premières positions est majoré de 1 et renvoyé à l'adresse q, les 9 dernières positions de ce catène n'étant pas modifiées. Le registre B n'est pas touché par cette majoration et contient après l'opération l'ancien contenu du catène q.

4.3.- Substitution d'instruction (S.I.)

Cette instruction a pour but de faire exécuter immédiatement par le Distributeur de Programme, une instruction dont l'adresse en Mémoire Rapide se trouve à ce moment dans les 15 premières positions du registre B. Cette instruction ne comporte donc ni partie X, ni F.A. et, de ce point de vue, peut être rapprochée, de l'instruction S.Q.

Si l'instruction exécutée grâce à S.I. n'est pas un branchement, le programme reprendra au numéro d'instruction qui suit la S.I. Au contraire si cette instruction est un branchement effectif, le programme continuera sur la branche extraordinaire du branchement.

IV - REMARQUES GENERALES SUR LA SUBSTITUTION IMPLICITE ET EXEMPLES

On a vu deux sortes d'instructions chargeant les registres de l'Unité Centrale :

. Transfert
. Garnissage.

Il importe d'insister sur leurs différences.

. Les Transferts portent sur 24 positions et sont à substitution implicite, c'est-à-dire qu'en l'absence de F.A., X indique non pas la quantité dont on charge le registre, mais l'adresse où l'on trouvera la quantité dont on charge le registre. Il s'agit véritablement du transfert d'un catène de Mémoire Rapide dans un registre de l'Unité Centrale.

. Les Garnissages sont tout à fait analogues aux garnissages des R.A.C. Ils ne portent que sur 15 positions. Mais surtout il n'y a pas de substitution implicite. En l'absence de F.A., X désigne directement la quantité dont on garnit le registre. Il en est de même d'ailleurs pour les instructions AB, SB et CB.

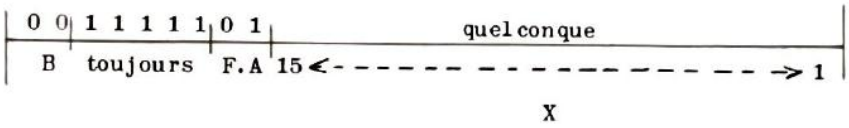
Pour mettre en lumière les différences entre ces sortes d'instructions, nous allons programmer le petit problème suivant :

En adresse 1.025 de la Mémoire Rapide, on a une instruction quelconque avec variable libre ; pour plus de précision, supposons qu'il s'agisse d'un branchement toujours renvoyant à une adresse contenue dans une mémoire donnée de la Mémoire Rapide (ce qui signifie que l'on a S dans la F.A.). Mais on ne connaît pas l'adresse de cette mémoire et avant chaque utilisation de cette instruction, il faut calculer cette adresse pour la mettre dans la partie X de cette instruction. Ce calcul résulte de l'addition à une quantité fixe (64 par exemple) du contenu d'une mémoire (d'adresse 100 par exemple).

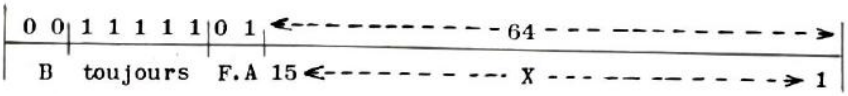
Le programme pour calculer cette instruction est le suivant :

```
T.B.    1.025
G.B.      64
A.B.     100 S
R.B.    1.025
```

Le T.B. a pour effet d'amener dans le registre B le contenu intégral (c'est-à-dire sur 24 positions) de la mémoire 1.025 (substitution implicite). Le contenu de B sera alors :

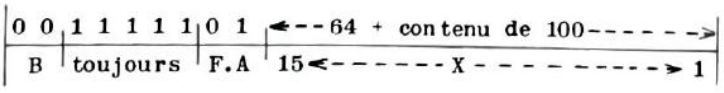


Le G.B. enverra dans les 15 premières positions de B la quantité 64 (pas de substitution implicite) en effaçant ce qui pouvait s'y trouver, mais en maintenant intactes les 9 autres positions. En B, on aura :



Le A.B. ajoutera, dans ces 15 positions, le contenu de l'adresse 100 à 64 (noter qu'il a fallu mettre l'indice S puisque A.B. n'a pas non plus de substitution implicite).

En B, on aura alors :



qui est l'instruction que l'on voulait calculer. Par le R.B. 1.025 (qui s'effectue sur 24 positions), on la range alors à l'adresse voulue.

On aurait pu également programmer :

```
T.B.    1.025
G.B.     100 S
A.B.      64
R.B.    1.025
```

puisque l'addition est une opération commutative.

V - TABLEAU DE CODE DES DIRECTIVES CENTRALES

	T.O.		C.C.	X	F.A.	
Rangement n° instruction	R	I		Adresse		
Garnissage du R.N.E.	G	E		Quantité (1)		
Rangement n° Elément	R	E		Adresse		
Transfert dans A	T	A				
Rangement de A	R	A				
Transfert dans B	T	B				
Rangement de B	R	B				
Garnissage de B (sur 15 posit.)	G	B		Quantité (1)	rien R ou M	rien ou S
Addition dans B (sur 15 posit.)	A	B				
Soustraction dans B (sur 15 posit.)	S	B				
Comparaison dans B (sur 15 posit.)	C	B				
Substitution de quantité dans B	S	Q				
Substitution d'instruction	S	I				
Forçage d'un B à "jamais"	F	J		Adresse	rien R ou M	rien ou S
Forçage d'un B à "toujours"	F	T				
Forçage inversion "Jamais-Toujours"	F	I				
Forçage à zéro d'un catène	F	Z				
Forçage à "un" d'un catène	F	U				
Décomptage itération	D	I				
Progression de compteur	P	C				

(1) ou Adresse si S en F.A.

LES ÉLÉMENTS CENTRAUX DE TRAITEMENT

1°) LE CALCULATEUR ARITHMÉTIQUE

I - STRUCTURE DES NOMBRES

Le Calculateur Arithmétique est l'élément qui effectue les 4 opérations arithmétiques plus un certain nombre d'opérations de servitude que nous donnerons en détail au § II. Toutefois, avant de parler de son fonctionnement, il faut décrire la structure des nombres sur lesquels il travaille. On a dit, au début de ce volume, que le GAMMA 60 faisait les calculs directement sur les nombres exprimés en décimal codé binaire. Il nous faut maintenant préciser, ne serait-ce que pour parler du signe puisqu'on veut travailler sur des nombres algébriques.

1. Les nombres traités par le Calculateur Arithmétique sont *de longueur constantes, deux catènes.*
2. Afin de pouvoir, dans ces deux catènes, représenter des nombres d'ordres de grandeur très différents, on utilise la notation *semi-logarithmique.*

Rappelons sur un exemple ce dont il s'agit. Soit le nombre + 4.832. On peut encore l'écrire sous les formes suivantes :

$$+ 0{,}4832 \times 10^{4}$$
ou bien $+ 0{,}004832 \times 10^{+6}$
ou bien $+ 0{,}0004832 \times 10^{+7}$ etc ...

De même, le nombre - 0,0000542 peut très bien s'écrire encore :

$$- 0{,}000000542 \times 10^{+2}$$
ou $- 0{,}542 \times 10\text{-}4$
ou encore $- 0{,}0054 \times 10^{-2}$.

On voit donc que tout nombre peut toujours s'exprimer sous la forme du produit d'un nombre plus petit que 1 (en valeur absolue) par une puissance de 10 positive ou négative, que ce nombre soit lui-même plus petit ou plus grand que 1, entier ou non :

$$0{,}m \times 10^{P}.$$

Pour qu'un nombre soit parfaitement déterminé, il faut donc connaître :

- son signe
- la quantité m située juste après le "0" dans la notation précédente, quantité que l'on appelle *mantisse.*
- la puissance P de 10 par laquelle il faut multiplier 0,m pour avoir la valeur absolue du nombre.

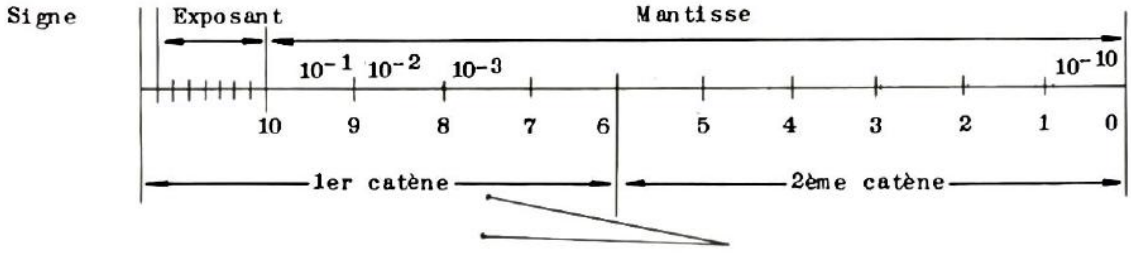
Représenter ainsi un nombre par sa mantisse et son exposant, c'est utiliser la représentation semi-logarithmique. Les exemples donnés montrent d'ailleurs que, pour un même nombre, il peut y avoir plusieurs représentations semi-logarithmiques possibles selon que, après le "0", on tolère ou non d'avoir un certain nombre de "0" précédant les chiffres significatifs de la mantisse. Toujours sur ces mêmes exemples, on peut vérifier la règle suivante :

Si, dans la mantisse, on décale les chiffres significatifs de n positions vers la droite, la puissance de 10 est augmentée de n.

Si, au contraire, on les décale vers la gauche, la puissance de 10 est diminuée de n.

Pour la représentation dans le GAMMA 60, on majore systématiquement de 40 la puissance de 10, on évite ainsi les exposants négatifs sous réserve de se limiter à des nombres supérieurs à 10^{-40}, ce qui est bien suffisant.

Nous avons dès lors tous les éléments permettant de définir la structure des deux catènes qui contiennent un nombre prêt à être utilisé par le Calculateur Arithmétique.

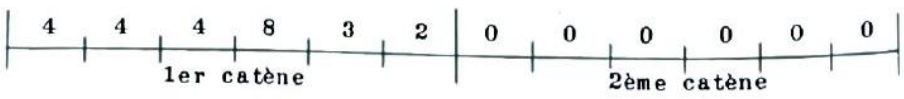


Le premier catène contient le signe, l'exposant et les 4 premiers chiffres (poids forts) de la mantisse, le deuxième catène les 6 derniers chiffres de la mantisse. Chacun des chiffres de la mantisse occupe une tétrade conformément à la notation "décimal codé binaire".

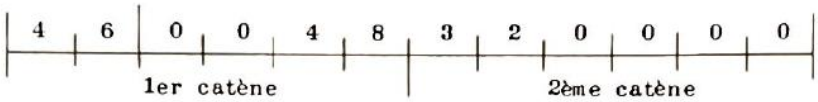
Les deux premières tétrades "poids forts" du premier catène sont donc disponibles pour recevoir le signe et l'exposant. La première monade contient le signe (0 pour + et 1 pour -), les 3 autres monades de la 1ère tétrade contiennent le premier chiffre de l'exposant (qui est donc au maximum égal à 7), la 2ème tétrade contient le 2ème chiffre de l'exposant.

En conséquence, dans ce système, l'exposant est au plus égal à 79, et un nombre a, au plus, 10 chiffres significatifs. Cette limitation à 10 chiffres significatifs ne signifie pas qu'il soit impossible dans le GAMMA 60 de traiter des nombres de plus de 10 chiffres. Cela signifie simplement que les opérations synthétiques faites par le Calculateur Arithmétique, au reçu d'une directive, portent simplement sur ces opérandes de 10 chiffres. Si l'on veut traiter des opérandes de plus de 10 chiffres, on fait appel à un sous-programme dit de *"double précision"* plus simple bien sûr, mais de même nature que celui auquel on doit faire appel pour, par exemple, extraire une racine carrée ou calculer un sinus, tous ces sous-programmes n'utilisant que les opérations que le Calculateur Arithmétique sait effectuer au reçu d'une simple directive.

Le nombre 4.832 sera donc représenté, abstraction faite du signe, par :

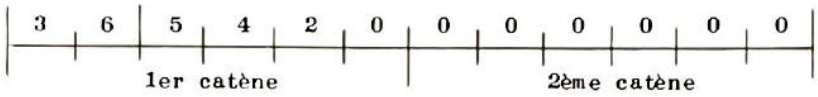


ou bien par :

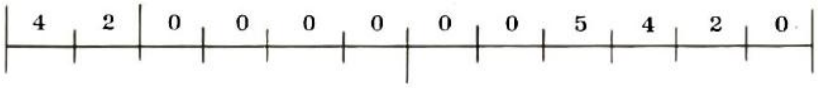


etc ...

et le nombre 0,0000542 par :



ou par :



etc ...

On peut vérifier, sur ces exemples, les règles données précédemment au sujet des décalages.

N.B. - C'est le traducteur qui permet de mettre les nombres sous la forme semi-logarithmique. Voir chapitre sur le Traducteur.

II. VIRGULE NORMALISEE, VIRGULE PROGRAMMEE

Les résultats élaborés par le Calculateur Arithmétique sont aussi des nombres en semi-logarithmiques. Comme on le verra par la suite, on peut les obtenir soit en "virgule normalisée", soit en "virgule programmée". Qu'entend-on par là ?

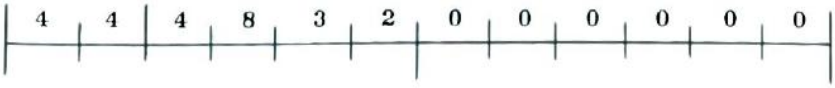
Parmi les nombreuses façons d'écrire un nombre en semi-logarithmique, il n'y en a qu'une qui permette, avec un nombre de chiffres donnés, d'avoir le maximum de précision, c'est-à-dire de chiffres significatifs : c'est celle qui consiste à choisir la puissance de 10 de sorte que le premier chiffre à gauche de la mantisse soit significatif, c'est-à-dire que :

$$0,1 \leqslant 0,m.$$

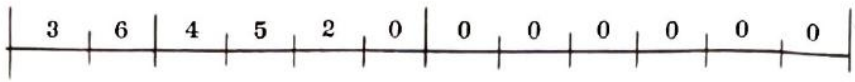
Programmer une opération en "virgule normalisée", c'est la programmer de sorte que le *résultat* apparaisse sous cette forme; c'est s'assurer que le résultat sera obtenu avec le maximum de précision : si un report s'effectue sur le dernier chiffre vers la gauche, automatiquement un décalage de 1 chiffre s'effectue vers la droite tandis que l'exposant se trouve majoré de 1; à ce moment là, si le chiffre tout à fait à droite était significatif, il est perdu mais c'est le chiffre de poids le plus faible et ceci n'a pas d'importance. Par contre, on est sûr de ne jamais perdre de chiffres de poids forts.

Les opérandes peuvent être cadrés n'importe comment du moment qu'ils sont en semi-logarithmique. Le Calculateur fait automatiquement les cadrages nécessaires après les avoir alimentés et avant de faire les calculs proprement dits.

Si 4.832 est le résultat de n'importe quelle opération programmée en "virgule normalisée", il apparaitra sous la forme :



0,0000452 apparaitrait sous la forme :



L'inconvénient de la "virgule normalisée" est que lorsque l'on élabore une série de résultats dont les ordres de grandeur sont différents, les chiffres correspondants à un même rang d'unités ne sont pas toujours dans les mêmes positions des catènes (à ce moment là, les exposants sont différents). Lorsque l'on fait du calcul scientifique, cela n'a pas d'importance mais lorsque l'on fait du calcul comptable, on désire souvent imprimer une telle liste de résultats et, dans ce cas, on veut que tous les chiffres correspondant au même rang d'unité soient les uns en dessous des autres.

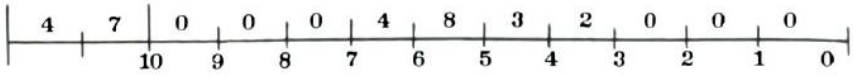
Sans que l'on ait encore parlé du Traducteur, on conçoit qu'en virgule normalisée, le programme d'édition et de traduction serait très compliqué : il faudrait. pour chaque nombre, analyser l'exposant afin de savoir où l'on doit placer les chiffres que l'on va traduire.

C'est pourquoi l'on a prévu également de pouvoir commander les opérations de l'unité arithmétique en *"virgule programmée"*. Dans ce cas, le *résultat* apparaît de sorte que les chiffres correspondant à un rang donné d'unités (unités, dizaines, centaines, etc...) soient toujours dans la même tétrade du nombre (comme en virgule normalisée, le cadrage des opérandes est sans importance). Ceci revient à positionner la virgule à un endroit bien précis. Dans le paragraphe précédent, on a, sur le schéma donnant la structure du nombre, numéroté de 0 à 10 les bornes limitant les tétrades où se trouve la mantisse. La borne 0 est celle se trouvant à l'extrême droite, la borne 10 est celle qui sépare la mantisse de l'exposant.

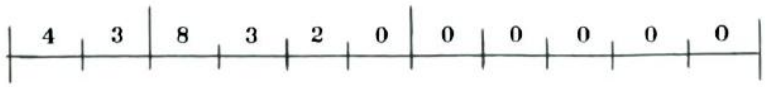
Lorsque l'on voudra faire une opération en virgule programmée, on devra donc indiquer la position n (de 0 à 10) de la virgule. Du même coup, on fixera la valeur de l'exposant à 50 - n.

Ainsi, lorsque l'on aura élaboré toute une série de résultats en virgule programmée à n, la traduction et l'addition seront très simplifiées. On n'aura plus besoin de consulter l'exposant : on sait que le nombre contenu dans telle tétrade devra toujours se placer dans telle colonne de l'imprimé.

Par exemple, si 4.832 est le résultat de n'importe quelle opération effectuée en virgule programmée à 3, il apparaîtra sous la forme :



En contrepartie de cet avantage, on voit tout de suite apparaître un certain inconvénient à la virgule programmée : on ne pourra jamais travailler sur des nombres $\geqslant 10^{10}$ en virgule programmée à 0 et de façon générale sur des nombres $\geqslant 10^{10-n}$ en virgule programmée à n. En effet, dans ce cas, les poids forts seraient perdus. Par exemple, si l'on devait obtenir 4.832 en virgule programmée à 7, on aurait en fait :



Dans ce cas, le Calculateur Arithmétique se met en incident.

Cela signifie qu'il faut se soucier du nombre maximum de chiffres du nombre; mais comme, de toute façon, il faut le faire pour prévoir le nombre de colonnes d'imprimantes, l'inconvénient est minime.

Il faut noter qu'en virgule programmée, l'exposant est obligatoirement compris entre 40 et 50 et qu'en conséquence, on ne peut travailler que sur des

nombres dont l'ordre de grandeur est compris entre :

$0,m$ et 10^{10}

avec $0,m \geqslant 10^{-10}$.

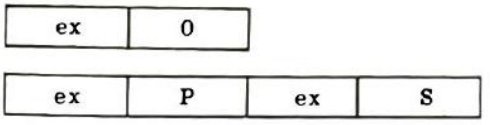
III. DESCRIPTION ET FONCTIONNEMENT DU CALCULATEUR ARITHMETIQUE

1. Le Calculateur Arithmétique comporte :
   -un opérateur sur signe,
   -un opérateur sur mantisse,
   -un opérateur sur exposant.

A chaque opération, ces 3 opérateurs sont mis en oeuvre sans que le programmeur ait à se soucier séparément de l'un ou de l'autre.

Comme pour tout élément, au reçu d'une directive, le Calculateur Arithmétique commence par alimenter ses opérandes dans certains registres internes, effectue l'opération à la fin de laquelle le résultat se trouve aussi dans un registre interne avant d'être éventuellement renvoyé en Mémoire Rapide.

Ces registres internes du Calculateur Arithmétique sont des registres de 2 *catènes* (1 nombre en semi-logarithmique). Il y en a 3 :



a) Le Registre P (principal)

C'est le registre qui reçoit le 1er opérande (celui qui est désigné le premier dans la directive - cf.§ IV).

En fin d'opération, il contient les chiffres poids forts du résultat. En effet, il se peut que le résultat d'une opération sur 2 nombres de moins de 10 chiffres soit un nombre de plus de 10 chiffres.

Dans ce cas, en virgule normalisée, il s'est produit le nombre nécessaire de décalages à droite, de sorte que le Registre P ne contient que les poids forts, les poids faibles ayant été transférés dans :

b) Le Registre S (secondaire)

Ce registre, qui contient donc éventuellement les poids faibles d'un résultat, est aussi un registre de deux catènes qui contient un nombre, signe, exposant et mantisse. Le signe est le même que celui du registre P, son exposant est celui de P diminué de 10. L'ensemble des registres P et S s'appelle *l'accumulateur*.

c) Le Registre 0 (opérande)

est celui qui, en début d'opération, reçoit de la Mémoire Rapide, le deuxième opérande (c'est-à-dire celui cité le second dans la directive).

2. A la fin de chaque opération, le résultat *est comparé à 0*. Le résultat apparaît dans les positions 7, 8 et 9 du catène qualitatif :

7 ——►- 0
8 ——►< 0
9 ——►> 0

Ceci est extrêmement intéressant. En effet, il arrive souvent au cours d'un calcul que, selon le signe d'un résultat partiel, on doive évoluer dans tel ou tel

sens. Dans le programme, on écrira alors un branchement conditionnel 7, 8 ou 9 aussitôt après la directive commandant l'opération dont le signe du résultat doit déterminer la suite du programme sans avoir, au préalable, à commander une comparaison à 0 qui nécessiterait d'ailleurs l'utilisation d'une mémoire de manoeuvre contenant 0.

Ceci est vrai non seulement pour les opérations arithmétiques mais encore pour les opérations de servitude et spécialement celle de transfert (cf. § 4) après laquelle il peut souvent être utile de connaître le signe du nombre que l'on vient de transférer.

3. Le Calculateur Arithmétique à 4 R.A.C., numérotés de 0 à 3, tous banalisés, c'est-à-dire que n'importe lequel peut, soit contenir l'adresse d'un opérande, soit l'adresse où l'on doit ranger le résultat. A priori, il pourrait sembler que 3 auraient été suffisants puisque les opérations se font avec 2 opérandes et 1 résultat.

C'est vrai, mais le fait d'en avoir 4, joint à la possibilité de les faire progresser ou non comme on va le voir tout de suite, permet, en combinant astucieusement l'emplacement des données et des résultats en Mémoire Rapide, l'utilisation de tel ou tel R.A.C., et leur progression ou non progression, de pouvoir programmer de très longues séquences de programme en n'utilisant que des directives sans avoir besoin de les faire précéder d'instructions d'adresses. En tout état de cause, ceci permet au moins de diminuer considérablement le nombre de ces instructions d'adresses, d'où une réduction de la longueur des programmes et de leur temps d'exploitation.

Bien que les nombres occupent deux catènes, leur transfert entre Mémoire Rapide et Calculateur Arithmétique se fait catène par catène, le R.A.C. correspondant progressant de 1 à chaque transfert. Ainsi, normalement, après chaque opération, les R.A.C. mis en cause ont progressé de 2. Mais, comme on le verra au § suivant, on a aussi la possibilité de programmer les opérations de sorte que certains des R.A.C. mis en cause (ou même tous si l'on veut) soient, après le transfert de l'opérande correspondant, ramenés à leur valeur initiale : on dit alors que l'opération s'est effectuée "sans progression" de ces R.A.C. (cette expression est un peu impropre puisqu'il est bien évident que les R.A.C. ont progressé au cours des transferts et que si au total l'opération s'est effectuée sans progression, c'est parce que, après les transferts, on les a ramenés à leurs valeurs initiales).

4. Le contrôle du transfert des opérandes dans le Calculateur Arithmétique se fait comme tout transfert, catène par catène, au moyen de la clef par 7 qui accompagne chaque catène. Une fois les opérandes transférés dans le Calculateur Arithmétique, ce contrôle est relayé par un contrôle local qui est, lui aussi, un système de clef par 7, mais décimal. Pour chaque opérande (nombre décimal de deux catènes), on forme une clef qui est le reste de sa division (dans le système décimal bien entendu) par 7. On fait alors l'opération indiquée sur les nombres eux-mêmes ainsi que sur leurs clefs et on vérifie que la clef du résultat est identique au résultat obtenu en faisant l'opération sur les clefs des opérandes.

Ce système de "preuve par 7" est tout à fait analogue à la preuve par 9 qu'effectue l'écolier. Toutefois, il est beaucoup plus sûr que la preuve par 9, en effet, celle-ci ne décèle pas une interversion de chiffres dans le résultat; 2.548 et 8.452 par exemple ont la même clef par 9. Au contraire, la clef par 7 ne laisse pas passer ce genre d'erreur : les restes des divisions par 7 des deux nombres cités sont, en effet, 0 et 3.

5. Les temps opératoires du Calculateur Arithmétique sont les suivants :

Addition-Soustraction	: 100	microsecondes
Multiplication	: 300	"
Division	: 600	"

Il s'agit ici de temps globaux qui comprennent l'alimentation du programme, des opérandes et le temps opératoire proprement dit. Il est bien évident qu'il s'agit de temps moyens, le temps réel pouvant varier autour de cette moyenne selon que l'on a plus ou moins d'instructions d'adresses dans une instruction complète, plus ou moins de transferts d'opérandes et aussi selon l'importance des opérandes pour la multiplication et la division.

IV. LES OPERATIONS ET LES DIRECTIVES CORRESPONDANTES

1. Tableau de code

	T.O.			C.C.		X				F.A.	
						V	0_1	0_2	R		
Addition Soustraction Multiplication	A S M	D O U	D U L	P N ou A	D ou rien	0 à 10 ou rien					
Division normale Division reste	D D	I I	V R	P ou N	D ou rien	0 à 10 ou rien					
Transfert	T	E	R	P ou N	D ou rien	0 à 10 ou rien					
Multiplication) par une)puissance Division) de 10	M D	P P	D D		D ou rien	0 à 15					
Inversion de signe Prise de valeur absolue	I P	S V	I A		D ou rien						
Comparaison en Valeur Absolue Comparaison Algébrique	C C	V A	A L								
Arrondi	A	R	R								

Dans les cases blanches laissées sous 0_1 et R, on peut écrire :
0, 1, 2 ou 3 (R.A.C. avec progression, cf. § 2)
Z, U, D ou T (R.A.C. sans progression, cf. § 2)
P (Registre P).

Dans celles qui sont sous 0_2, on peut trouver la même chose sauf P.

2. *Généralités*

Lorsqu'on commence une séquence élémentaire sur le Calculateur Arithmétique, il y a évidemment lieu de commencer par garnir tous les R.A.C. que l'on utilisera au cours du calcul. Par la suite, on aura encore des instructions d'adresse, des Branchements des Directives Centrales mais on aura surtout des Directives de l'élément qui déclencheront les opérations élémentaires du calcul.

Le tableau de code ci-dessus donne la liste de toutes ces opérations élémentaires et le code B des directives correspondantes. Comme il apparaît aussitôt, ces directives sont des directives sans variable libre : il n'y a pas de F.A. On voit qu'en code B, la zone X est divisée en 4 zones, la première de 2 caractères, les autres de 1. En code A, le T.K. occupe une tétrade comme pour toute directive élément, les 5 autres tétrades suivantes formant la partie caractéristique de l'instruction contenant, en code directement interprétable par le Calculateur Arithmétique, tous les renseignements portés dans le code B que nous allons décrire.

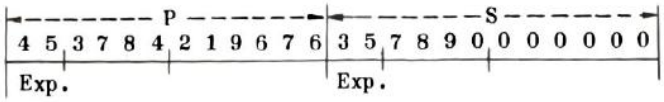
a) Dans la zone T.O., on trouve évidemment un code mnémotechnique à 3 lettres désignant la nature de l'opération que l'on veut faire.

b) Dans la première colonne de C.C., on indique sous quelle forme on veut obtenir le résultat :

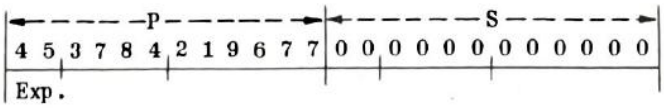
P pour "virgule programmée"
N pour "virgule normalisée"
A pour "virgule normalisée avec arrondi"

L'arrondi est l'arrondi au chiffre le plus proche.

Cette fonction est intéressante dans les opérations où le résultat peut apparaître en partie dans le registre S. Elle consiste à ajouter 5 au chiffre de ce registre qui vient immédiatement après l'exposant, ce qui a pour effet de provoquer éventuellement un report sur la dernière position de P. Ainsi l'arrondi sur :



donnera :



Comme on le voit, cette fonction n'existe, synthétisée dans la directive, que dans le cas de la virgule normalisée; ceci est dû à des contingences de code A. Pour le cas de la virgule programmée, il existe la directive ARR qui permet de le faire.

La directive MUL.A équivaut à MUL.N suivi de ARR. En virgule programmée, on n'a pas le choix, il faut écrire MUL.P suivi de ARR.

Une dernière remarque : l'arrondi porte toujours sur la dernière position du registre principal qu'il s'obtienne par le code complémentaire A ou par la directive ARR. En virgule normalisée, obtenu par le C.C. "A", il porte donc sur le 10ème chiffre significatif. Si l'on voulait faire porter le 5ème ou le 6ème par exemple, il faudrait commander l'opération en virgule programmée de sorte que le 5ème ou 6ème chiffre apparaisse dans la dernière position de P. On obtiendrait ensuite l'arrondi par ARR.

Bien entendu, il existe des opérations où ce premier code complémentaire serait sans signification.

c) Le deuxième code supplémentaire indique si l'on veut extraire le résultat en simple (rien) ou double (D) capacité. *"Double Capacité"* signifie que l'on désire ranger en Mémoire Rapide après opération le contenu de l'accumulateur complet (P et S soit 4 catènes). Il est évident que ceci n'a de sens que dans les opérations où il est possible que S reçoive les poids faibles du résultat.

Il ne faut pas confondre ceci avec les opérations en *"double précision"* où ce sont les opérandes qui sont des nombres de 4 catènes, opérations qui se font par sous-programme. Ici, ce n'est que le résultat qui est en 4 catènes, les opérandes étant en deux catènes.

Lorsqu'on range le résultat en Mémoire Rapide en double capacité, le R.A.C. correspondant progressera évidemment de 4 au cours du transfert mais il reviendra ensuite automatiquement à sa valeur initiale. D'où la règle de programmation :

Dans les opérations "double capacité", le R.A.C.
Résultat sera obligatoirement "sans progression".

b) Dans les deux premières colonnes de X (zone V), on indiquera un nombre de 0 à 10 dans le cas d'une opération en virgule programmée. Ce nombre indique la position de la virgule du résultat *dans le registre P*, les positions 0 à 10 étant celles des bornes entre chiffres définies dans la structure du nombre.

Il faut bien noter que ceci entraîne l'impossibilité de placer la virgule du résultat dans S.

Dans les opérations en virgule normalisée, cette zone reste vierge.

Il existe enfin deux opérations que nous verrons plus en détail où l'on y indique un nombre de 0 à 15 qui est une puissance de 10.

e) Les 3 colonnes 0_1, 0_2 et R servent à indiquer les R.A.C. contenant les adresses des opérandes.

0_1 et 0_2 : opérandes 1 et 2
R : résultat.

Dans 0_1 et R, on peut trouver les mêmes caractères

. 0, 1, 2 ou 3 indiquent les n° de R.A.C. lorsqu'on désire les laisser progresser.

. Z, U, D ou T (initiales de zéro, un, deux, trois) indiquent les n° de R.A.C lorsqu'on désire qu'ils se maintiennent à leur valeur (R.A.C. sans progression).

. P lorsqu'on désire prendre pour 1er opérande le résultat qui se trouve dans l'accumulateur, ou lorsque l'on désire laisser le résultat dans l'accumulateur.

Noter que, dans une opération en double capacité, on ne peut trouver dans R que Z, U, D ou T.

Dans la colonne 0_2, on peut trouver 0, 1, 2, 3, Z, U, D ou T avec les mêmes significations que dans 0_1 et R. Mais on ne peut y trouver P. On a dit, en effet, dans le § III, que le deuxième opérande était obligatoirement pris en Mémoire Rapide et qu'il était envoyé dans le Registre 0.

3°/ Description des différentes opérations

a) Addition, Soustraction, Multiplication. Ces 3 opérations à 2 opérandes sont groupées dans le tableau de code parce que, sauf pour le T.O., elles se programment de la même façon. Tout ce qu'on a dit dans les généralités s'y applique intégralement et il n'y a aucune difficulté spéciale : virgule programmée, normalisée ou normalisée avec arrondi, extraction du résultat en simple ou double capacité, nécessité de prendre obligatoirement 0_2 en Mémoire Rapide. Dans la soustraction, c'est 0_2 que l'on retranchera de 0_1.

Une remarque essentielle toutefois : *avant chacune de ces opérations, le contenu de S est effacé, même si l'on indique P dans* 0_1. Cette remarque est importante : en effet, supposons par exemple que nous désirions faire la somme d'une série de nombres situés en Mémoire Rapide.

Il vient aussitôt à l'esprit de faire un cumul progressif. On fera la somme des deux premiers nombres en indiquant P dans la colonne R de façon à laisser ce résultat partiel dans l'accumulateur. On ajoutera à ce résultat partiel le 3è nombre en programmant une addition avec P dans 0_1 et R et on recommencera (éventuellement avec un processus itératif utilisant le Décomptage Itération si les nombres en Mémoire Rapide sont rangés selon une loi simple) jusqu'au dernier terme à additionner. Pour la dernière opération, il faudra évidemment indiquer en R le R.A.C. contenant l'adresse où l'on veut ranger le résultat. Cette méthode est parfaitement normale, mais il faudra se garder de programmer les opérations en double capacité car ceci n'aurait aucun sens. En effet, avant chaque opération, le contenu de S est effacé : donc le rangement de S en Mémoire Rapide à la fin du calcul ne donnerait pas les poids faibles de la somme, mais simplement les poids faibles (éventuellement) de son dernier terme, ce qui est tout à fait différent. Si, dans un tel cumul progressif, on veut avoir le résultat sur 4 catènes, il faudra employer une méthode de double précision. La double capacité n'est valable que pour une opération simple dont on range aussitôt le résultat en Mémoire Rapide.

b) *Division*. La division est une opération à deux résultats, le quotient et le reste. Or, d'après la structure générale de la programmation de l'unité arithmétique, on ne peut extraire qu'un seul résultat dans une opération.

D'où les deux divisions :
- Division normale (DIV) : c'est le quotient qu'on extrait
- Division reste (DIR) : c'est le reste qu'on extrait.

Dans les deux opérations, de toute façon, le quotient apparaît dans S, le reste dans P. Dans la DIV, le contenu de S est ensuite transféré dans P, puis à l'adresse désignée dans R (si c'est P qui est indiqué dans R, le quotient reste simplement en P). Dans la DIR, le quotient reste dans S et c'est le reste alors contenu dans P qui est transféré.

L'indication P ou N du 1er code complémentaire porte toujours sur le quotient en DIV ou en DIR. Par exemple, programmer la virgule à 0 veut dire que l'on arrête la division dès que l'on obtient des décimales. On peut alors avoir dans S seulement 2 ou 3 chiffres significatifs précédés de 0. Le reste de cette division incomplète apparaît en P.

Il en résulte que "double capacité" n'a de sens que pour la DIR. Noter d'ailleurs que, la programmation de la virgule portant sur le quotient, on ne sait où se trouve la virgule du reste.

Le contenu de S se trouve effacé avant toute division même si l'on a indiqué P dans 0_1.

Si l'on veut extraire à la fois le reste et le quotient, on peut évidemment programmer une DIV et une DIR avec les mêmes opérandes (ce qui revient à faire deux fois l'opération). Mais il est plus rapide de faire une DIR, en indiquant double capacité en code complémentaire ; le reste et le quotient sont alors transférés tous les deux en mémoire rapide dans des adresses successives.

c) *Le Transfert* (TER). C'est l'opération de servitude du Calculateur arithmétique qui permet de faire d'une façon très analytique tout ce qu'on ne peut pas faire de façon synthétique dans les autres opérations.

Elle sert, comme son nom l'indique, à transférer un *nombre* d'une mémoire à une autre. C'est donc une opération à 1 opérande : en 0_1 on indique la mémoire origine, en R la mémoire destinatrice. Le transfert se fait toujours par l'intermédiaire du registre P et le nombre transféré, comme le résultat de toute opération, est ainsi automatiquement comparé à 0.

On pourra profiter de cette opération pour faire passer un nombre dont on ne sait pas s'il est en normalisé ou en programmé (et avec la virgule en quelle position ?) sous la forme que l'on veut : c'est ainsi que l'on pourra fixer la représentation du reste d'une division. Par cette opération, on pourra aussi éventuellement faire de simples décalages en indiquant la même chose en O_1 et R en la faisant, par exemple, avec virgule programmée à 3 pour un nombre dont on sait qu'il est en virgule programmée à 5. Ceci pourra amener la perte des poids faibles ou des poids forts (dans ce dernier cas, le Calculateur Arithmétique est mis en incident) si on fait l'opération en simple capacité. Si on fait l'opération en double capacité, le contenu de S est alors aussi extrait lui aussi et on est garanti contre la perte des poids faibles, mais pas contre celle de la perte des poids forts.

Le contenu de S est effacé avant cette opération, sauf si l'on indique P dans O_1 (double capacité possible).

Il faut bien noter qu'il est *interdit de se servir de cette opération pour transférer d'une adresse de Mémoire Rapide à une autre adresse de Mémoire Rapide deux catènes qui contiendraient autre chose qu'un nombre en semi-logarithmique :* 2 groupes de 4 lettres ou un indicatif numérique par exemple.

En effet, d'une façon générale, il y a peu de chances pour que le Calculateur Arithmétique puisse interpréter ces catènes comme des nombres car si l'on prend un catène contenant 4 lettres et si on le divise en six tétrades, il y a fort peu de chances pour qu'on ne trouve pas, parmi elles, des tétrades qui ne fassent pas partie du code décimal codé binaire (10 possibilités sur 16 en effet sont seulement utilisées dans ce code). Le calculateur arithmétique se mettra alors en incident. Dans les cas où cette interprétation serait possible, du fait de la programmation de la virgule, on pourrait effectuer des décalages et ranger en Mémoire Rapide des caractères n'ayant rien à voir avec ceux que l'on y avait pris.

d) Multiplication ou division par une puissance de 10 (M.P.D. et D.P.D.). Cette opération ne porte que sur les exposants auxquels on ajoute ou retranche le nombre de 0 à 15 écrit dans la zone V.

Si l'on indique P dans O_1, on peut la programmer en double capacité, mais il n'y a pas de décalages possibles dans cette opération : ni P, ni N en 1ère colonne de C.C.

On va voir comment on peut utiliser cette opération pour arrondir un nombre à la centaine la plus proche : l'arrondi portant sur la dernière position de P, il faudrait pour cela pouvoir programmer la virgule dans S avant l'arrondi. Or cela est impossible. On agira donc de la façon suivante : on programmera avec virgule à 0 et résultat laissé en P, la dernière opération donnant le nombre à arrondir. On fera ensuite une D.P.D.2 qui fera passer son exposant de 50 à 48. Une TER avec virgule à 0 fera le décalage de 2 à droite et remise de l'exposant à 50, le dernier chiffre de P contenant alors le chiffre des centaines. Une ARR fera alors l'arrondi sur ce chiffre, mais le nombre que l'on aura ainsi extrait en Mémoire Rapide sera exprimé en centaines. Pour l'avoir en unités, il faudra faire suivre l'arrondi d'une M.P.D.2.

e) Inversion de signe et prise de valeur absolue (I.S.I. et P.V.A.). Les noms de ces deux opérations suffisent à les définir. Rien à indiquer ni en 1er code complémentaire, ni en V, ni en O_2, ce qui montre les possibilités réduites de l'opération qui est ramenée à un transfert de nombre accompagné d'inversion de signe ou de prise de valeur absolue.

Le registre S est effacé avant l'opération sauf si l'on indique P dans O_1. Ce n'est d'ailleurs que dans ce dernier cas que la double capacité a un sens.

f) Comparaison algébrique ou en valeur absolue (C.V.A. et C.A.L.). Ces opérations se font sur 2 nombres de 2 catènes ; la double capacité n'a aucun sens. Les nombres n'ont pas besoin, pas plus que dans n'importe quelle opération, d'être préalablement cadrés au moyen d'une virgule programmée au même endroit.

Il n'y a que les colonnes O_1 et O_2 à programmer. Rien à indiquer en R puisque le résultat apparaît dans le catène qualitatif : mêmes positions que pour la comparaison sur 1 catène en Unité Centrale.

1	2	3	4	5	6
$>$	$<$	$=$	$\geqslant$	$\leqslant$	$\neq$

g) Arrondi (ARR). On a déjà expliqué le fonctionnement de cette opération. Elle vient aussitôt après une opération en virgule programmée. L'opérande est obligatoirement le contenu de P et S. Il n'y a, outre le T.O., qu'à indiquer en R où l'on désire ranger le résultat.

A la fin de l'opération, le registre S est effacé.

2°) LE COMPARATEUR GÉNÉRAL

1. FONCTIONS DU COMPARATEUR GENERAL

1°/ La fonction essentielle du Comparateur Général est évidemment la comparaison des quantités, soit numériques, soit alphanumériques, en vue de faire des classements, des interclassements, des tris ...

La comparaison s'effectue sur ces quantités catène par catène, le Comparateur considérant ces catènes comme des nombres binaires de 24 chiffres binaires. Le résultat de cette comparaison est valable également pour le contenu codé de ces catènes, qu'il s'agisse de lettres (le code interne est tel que le nombre binaire de 6 chiffres, représenté par la combinaison correspondant à la lettre, est plus grand pour Z que pour A) ou de chiffres décimaux (indicatifs numériques). Mais on *exclut de comparer des nombres en semi-logarithmique*, opération qui ne se fait que dans le Calculateur Arithmétique : une première raison réside dans le fait que ces nombres sont algébriques et que, par conséquent, s'il s'agit de nombres négatifs, le plus grand est celui qui a la plus petite valeur absolue ; le Comparateur Général ne sait pas faire cette distinction. De plus, même s'il s'agit de nombres positifs, il y a encore un autre inconvénient qui apparaît sur l'exemple suivant : soit les nombres

5	0	0 0 0 0	0 0 0 1 8 2
Exp.			

et

4	8	0 0 1 8	5 0 0 0 0 0
Exp.			

Le Comparateur Général considérerait ces nombres comme des indicatifs de 12 chiffres et dirait que le premier est plus grand que le second. Or, c'est l'inverse puisque le premier est 182 et le second 185.000. Le résultat ne peut être exact que si ces nombres sont *positifs et soit en virgule normalisée, soit en virgule programmée à la même valeur*. A ce moment, si un nombre a un exposant plus grand qu'un autre, il est lui-même plus grand.

D'ailleurs, en virgule normalisée, les deux nombres ci-dessus s'écrivent :

4	3	1 8 2 0	0 0 0 0 0 0
Exp.			

et

4	6	1 8 5 0	0 0 0 0 0 0
Exp.			

et le plus grand apparaît aussitôt.

2°/ Il y a deux sortes de comparaisons :

. la comparaison simple qui consiste à comparer deux quantités entre elles.

. la comparaison à deux bornes qui consiste à situer une quantité variable par rapport à un intervalle défini par deux quantités fixes qui en sont les bornes. Cette fonction pourrait évidemment se faire par une succession de comparaisons simples ; il ne s'agit là que d'une opération plus synthétiques.

3°/ En plus de cette fonction principale de comparaison, on a attribué au Comparateur Général deux fonctions secondaires :

a) servir d'intermédiaire pour les transferts Mémoire Rapide - Mémoire Rapide de longueur quelconque. On sait, en effet, qu'il n'est pas possible de transférer directement un catène d'une adresse de Mémoire Rapide à une autre, mais

qu'il faut passer par un organe intermédiaire. Jusqu'ici, nous avons vu deux possibilités :

. par l'intermédiaire de l'Unité Centrale grâce à un transfert dans les registres A ou B (TA ou TB) suivi d'un rangement des registres A ou B (RA ou RB). Dans ce cas, l'unité de transfert est d'un catène.

. par l'intermédiaire du Calculateur Arithmétique grâce à l'opération TER. Dans ce cas, l'unité de transfert est de 2 catènes, mais on a vu, à propos du Calculateur Arithmétique, que l'on ne pouvait transférer ainsi que des nombres en semi-logarithmique et qu'il était interdit de transférer ainsi des catènes contenant des informations quelconques.
Le Comparateur Général peut, lui, servir d'intermédiaire pour transférer des informations de nature et de longueur quelconque.

b) effacer toute une zone de Mémoire Rapide. On a vu qu'il existe une directive centrale qui permet également de le faire, mais il faut alors une directive par catène. Ici une directive suffit pour toute une zone de Mémoire Rapide.

II. DESCRIPTION ET FONCTIONNEMENT GENERAL

1°/ Les R.A.C.

Contrairement aux R.A.C. du Calculateur Arithmétique, ceux du Comparateur Général sont spécialisés. Ils sont au nombre de 3 référencés 0, 1 et 2.

Avant les directives, on les charge des adresses des quantités à traiter. Pendant les opérations, ils progressent automatiquement de 1 à chaque transfert pour préparer le suivant.

Il s'ensuit qu'à la fin d'une directive de transfert Mémoire Rapide - Mémoire Rapide ou d'effacement de zone, ils contiennent l'adresse majorée de 1 du dernier catène transféré ou effacé : contenu final - contenu initial + longueur de l'opération.

En revanche, dans les opérations de comparaison qui s'arrêtent dès que le résultat est acquis (cf. § II.2), le contenu final des R.A.C. peut être plus petit que le contenu initial majoré de la longueur de l'opération donnée dans la directive.

Dans la comparaison simple, 2 R.A.C. suffisent et ils contiennent les adresses des quantités à comparer. L'un est obligatoirement le R.A.C. 0, le deuxième est à volonté soit le R.A.C. 1, soit le R.A.C. 2. La comparaison est faite dans le sens quantité indiquée par le R.A.C. 0 par rapport à la quantité indiquée par le R.A.C. 1 ou 2.

Dans la comparaison à deux bornes, les 3 R.A.C. sont nécessaires. Obligatoirement, le R.A.C. 0 reçoit l'adresse de la quantité à situer dans l'intervalle défini par les deux bornes. Les R.A.C. 1 et 2 reçoivent les adresses des quantités constituant les bornes de cet intervalle.

Dans le transfert Mémoire Rapide - Mémoire Rapide, c'est le R.A.C. 0 qui contient l'adresse de la mémoire émettrice, le R.A.C. 1 ou 2 l'adresse de la mémoire réceptrice.

Dans l'effacement, c'est le R.A.C. 0 qui contient obligatoirement l'adresse de la zone à effacer.

2°/ Les Registres et les Organes de comparaison

Outre le registre de liaison et le registre catène qualitatif, le comparateur général comprend :

a) *les registres opérandes* (capacité 1 catène) qui sont au nombre de deux et reçoivent depuis la Mémoire Rapide les catènes désignées dans les R.A.C.

Le premier reçoit l'opérande correspondant au R.A.C.O.

Le second reçoit l'opérande correspondant au R.A.C. 1 ou 2 dans la comparaison simple et, alternativement, les opérandes correspondant aux R.A.C. 1 et 2 dans la comparaison à deux bornes.

b) *le registre longueur* destiné à recevoir la longueur en catènes desopérandes longueur indiquée par la directive elle-même. La capacité de ce registre étant de 8 positions binaires, la longueur maximum qu'il puisse contenir est de 255. On n'a donc pas le droit d'indiquer des longueurs supérieures dans les directives. En cours d'opération, ce registre se décompte de 1 à chaque catène et c'est son passage par 0 qui détermine la fin de l'opération sauf dans les comparaisons où l'arrêt peut se produire avant.

c) *les organes de comparaison* qui sont de quatre types :

- comparaison en égalité
- comparaison en supériorité
- comparaison en égalité à 0
- comparaison en égalité à 1.

Quand on parle d'égalité à 0 ou à 1, on entend, par là, reconnaissance d'un catène vierge ou d'un catène dont toutes les monades sont des 1.

Dès que l'organe de comparaison en supériorité signale une inégalité entre 2 catènes, les transferts et la comparaison s'arrêtent. En effet, les comparaisons se faisant poids forts en tête, la première inégalité détermine le sens de la comparaison quels qu'auraient pu être les résultats de comparaisons sur les catènes suivants.

Toutefois, si, à ce moment-là, la comparaison à 0 ou à 1 d'un des opérandes a toujours donné une réponse positive, la comparaison se poursuit jusqu'à ce que l'on trouve un catène différent de 0 ou de 1 et éventuellement jusqu'à épuisement de la longueur indiquée.

Les réponses conjuguées des organes de comparaison en égalité et supériorité fournissent alors les six cas possibles de comparaison simple :

$$\ll \; = \; \neq \; \gg$$

La conjugaison des différentes réponses est effectuée dans des mémoires d'analyse dont les conclusions sont matérialisées par le positionnement de plusieurs monades du catène qualitatif du Comparateur Général. La détection d'un opérande nul ou entièrement composé de catènes bourrés de 1 est également signalée dans le catène qualitatif.

III. LES DIRECTIVES

1°/ Généralités et tableau de code

Outre le T.O. de 3 lettres, chaque directive comporte, dans sa partie X, la longueur (en nombre de catènes) sur laquelle porte l'opération. Cette longueur est au maximum de 255. Comme toujours, cette partie X peut être modifiée par la F.A. Il convient de remarquer qu'une longueur nulle ne provoque pas d'exécution de la directive. Le catène qualitatif reste vierge.

Le R.A.C. 0 est toujours implicite. Les R.A.C. 1 et 2 le sont également lorsqu'il s'agit d'opérations où ils sont tous deux utilisés ou d'opérations où aucun d'eux ne l'est. Dans celles où seulement l'un d'entre eux est utilisé, il faut indiquer son numéro dans le premier code complémentaire.

	T.O.			C.C	X	F.A.	
Comparaison simple	C	M	P	1 ou 2	longueur (maxi 255)	Rien, M ou R	Rien, ou S
Comparaison à deux bornes	C	D	B				
Effacement avec longueur	E	F	L				
Transfert Mémoire Rapide-Mémoire Rapide	T	R	R	1 ou 2			

2°/ Comparaison simple (C.M.P.)

Le premier opérande est celui dont l'adresse est contenue dans le R.A.C. 0. Le deuxième opérande est celui dont l'adresse est contenue dans le R.A.C. 1 ou 2 dont on doit indiquer le numéro dans la 1ère colonne de codes complémentaires.

Si l'on mesure les temps en cycles Mémoire Rapide, le déroulement de la comparaison est le suivant à partir de la réception de la directive :

1er cycle : Réception Directive et décodification.

2ème cycle : Demande de Transfert de Quantité (D.T.Q.) pour le 1er catène du premier opérande.

3ème cycle : D.T.Q. pour le 1er catène du 2ème opérande.

4ème cycle : Décomptage longueur.

5ème cycle : Réception du 1er catène du 1er opérande, et simultanément :
- D.T.Q. pour le 2ème catène du 1er opérande.

6ème cycle : Réception du 1er catène du 2ème opérande, et simultanément :
- D.T.Q. pour le 2ème catène du 2ème opérande.

7ème cycle : Début des opérations de comparaison sur les premiers catènes, et simultanément :
- Décomptage longueur.

8ème cycle : Fin des opérations de comparaison sur les premiers catènes, et simultanément :
- Réception du 2ème catène du 1er opérande
- D.T.Q. pour le 3ème catène du 1er opérande

9ème cycle : Réception du 2ème catène du 2ème opérande, et simultanément :
- D.T.Q. pour le 3ème catène du 2ème opérande

10ème cycle : Début des opérations de comparaison sur les 2èmes catènes, et simultanément :
- Décomptage longueur.

etc ...

Le travail s'arrête soit par la détection d'une inégalité à la fin de deux cycles de comparaison (8ème, 11ème, 14ème, etc ...), soit par manque d'opérandes à la suite de l'arrêt de leur alimentation, arrêt qui se produit par le passage du décomptage à 0. Le diagramme des temps qui vient d'être donné montre qu'alors le R.A.C. 0 contient l'adresse majorée de 3 du dernier catène sur lequel a porté la comparaison, à moins qu'il ne se soit produit en cours de route un arrêt des D.T.Q. par passage à 0 du décomptage longueur. Le R.A.C. 1 (ou 2) contient l'adresse majorée seulement de 2 (sous les mêmes réserves) du catène correspondant du 2ème opérande.

Il s'ensuit que *l'on ne peut jamais prévoir le contenu final des R.A.C. sauf*

dans les comparaisons sur une longueur de 1 ou 2 catènes où il est alors égal en tout état de cause au contenu initial majoré de la longueur.

Le résultat apparaît dans le catène qualitatif par le positionnement à 1 de certaines monades 1 à 10 avec la signification suivante :

1	2	3	4	5	6	7	8	9	10
$>$	$<$	-	$\geqslant$	$\leqslant$	$\neq$	RAC.0 - 0	RAC.0 - 1	RAC.1 ou 2 - 0	RAC.1 ou 2 - 1

3°/ Comparaison à deux bornes (C.D.B.)

Dans cette directive, les trois R.A.C. sont implicites avec la spécialisation indiquée précédemment.

Le processus opératoire est le suivant :

1er cycle : Réception de la directive et décodification.

2ème cycle : D.T.Q. pour le 1er catène de la variable x (RAC. 0).

3ème cycle : D.T.Q. pour le 1er catène de la borne p (RAC.1).

4ème cycle : D.T.Q. pour le 1er catène de la borne q (RAC.2) et décomptage longueur.

5ème cycle : Réception 1er catène de x.

6ème cycle : Réception 1er catène de p.

7ème cycle : Début comparaison sur premiers catènes de x et p, et simultanément :
- Réception 1er catène de q
- Emission D.T.Q. pour le 2ème catène de x.

8ème cycle : Fin de comparaison sur premiers catènes de x et p, et simultanément :
- Emission D.T.Q. pour le 2ème catène de p.

9ème cycle : Début comparaison sur premiers catènes de x et q, et simultanément :
- Emission D.T.Q. pour le 2ème catène de q et décomptage longueur.

10ème cycle : Fin comparaison sur premiers catènes de x et q, et simultanément :
- Réception 2ème catène de x.

11ème cycle : Réception 2ème catène de p.

12ème cycle : Début comparaison sur seconds catènes de x et p, et simultanément :
- Réception 2ème catène de q
- Emission D.T.Q. pour le 3ème catène de x

etc ...

Comme pour la comparaison simple, les opérations s'arrêtent à la fin des 4 cycles de comparaison (10ème, 15ème, 20ème, etc ...) soit par détection d'une inégalité, soit par manque d'opérandes à la suite de l'arrêt de leur alimentation, arrêt qui se produit par le passage du décomptage à 0. Le diagramme des temps qui vient d'être donné montre qu'alors les 3 R.A.C. contiennent les adresses majorées de 2 des derniers catènes sur lesquels a porté la comparaison, à moins qu'il ne se soit produit en cours de route un arrêt des D.T.Q. par passage à 0 du décomptage longueur.

Il s'ensuit que, dans cette opération également, *on ne peut jamais prévoir le contenu final des R.A.C. sauf dans les comparaisons sur une longueur de 1 ou 2 catènes où il est alors égal en tout état de cause au contenu initial majoré de la longueur*.

Dans cette opération, seule la variable x est comparée à 1 et 0. Le résultat apparaît dans le catène qualitatif par le positionnement à 1 de certaines des

monades 1 à 8 avec la signification suivante :

1. x > p et q
2. x < p et q
3. x compris dans l'intervalle p - q, bornes incluses
4. x = p
5. x = q
6. x compris dans l'intervalle p - q, bornes exclues
7. x nul
8. x composé de catènes bourrés de 1

4°/ Effacement avec longueur (EFL)

C'est le R.A.C. 0 qui contient au début l'adresse initiale de la zone à effacer. En fin d'opération, il a progressé de la longueur de cette zone.

5°/ Transfert Mémoire Rapide - Mémoire Rapide (TRR)

C'est le R.A.C. 0 qui contient l'adresse initiale de la zone émettrice et le R.A.C. 1 ou 2, l'adresse initiale de la zone réceptrice. En fin d'opération, les 2 R.A.C. ont progressé de la longueur du transfert.

IV. TEMPS OPERATOIRES

Les temps opératoires sont, mesurés en nombre de cycles Mémoire Rapide :

CMP : 5 + 3 × longueur effective de la comparaison

CDB : 5 + 5 × longueur effective de la comparaison

FZL : 2 + 1 cycle par catène effacé

TRR : 2 + 3 × longueur de la zone transférée.

V. INCIDENTS

Outre les résultats, le catène qualitatif signale les incidents au cours du travail de la façon suivante :

pos. 23 : dépassement de capacité de longueur (dépassement de 255) à la réception de la directive.

pos. 22 : décodification directive erronée.

pos. 20 et 21 : incident transfert sur canal distributeur.

pos. 18 et 19 : incident sur comparaison.

pos. 16 et 17 : incident décomptage longueur.

Tous ces incidents sont de plus regroupés sur la position 24 comme dans tous les éléments.

3°) LE TRADUCTEUR

1. FONCTIONS DU TRADUCTEUR

Lorsqu'on entre pour la première fois des informations dans le Gamma 60, on le fait à partir de supports externes qui sont soit la carte, soit la bande perforée. Sur ces supports, les informations sont codées en un code propre au support, code qui est différent du code interne sur lequel travaille la machine. Il se pose donc un problème de traduction du code externe en code interne. Inversement, lorsque l'on sort des résultats finals sur carte perforée, bande perforée ou imprimante, il se pose un problème de traduction de code interne en code externe.

Enfin, lorsque l'information est en code interne, on peut désirer qu'elle se présente soit en un état directement apte au *traitement* (nombres en semi-logarithmique, mots numériques nettement séparés des mots alphanumériques, etc...), soit sous l'état le plus apte à la *mémorisation* (c'est-à-dire le plus tassé et sans tenir compte du découpage en catènes). C'est encore le rôle du Traducteur de permettre à l'information de passer de l'état *traitement* à l'état *mémorisation* ou l'inverse. Ceci sera explicité davantage par la suite.

1. *Les codes*

Du côté codes internes, on a :

.le code numérique : 1 caractère - 1 chiffre - 4 monades
.le code alphanumérique : 1 caractère - 6 monades
.le code binaire pur : 1 caractère - 1 monade

Du côté codes externes, on a :

.les codes "carte perforée" : 1 caractère - 1 colonne - 12 monades
.les codes "bande perforée" : 1 caractère - de 5 à 8 monades selon le type de bande utilisé
.le code imprimante identique au code interne alphanumérique à 6 monades par caractère

Les éléments d'entrée envoient en Mémoire Rapide une image fidèle des informations telles qu'elles sont codées sur le support correspondant et c'est cette image que le Traducteur doit interpréter en code interne. Le fait que la carte perforée ne soit pas lue caractère par caractère, c'est-à-dire colonne par colonne, mais ligne par ligne, pose un problème particulier de transposition lignes-colonnes qui est également résolu par le Traducteur (cf. § II.3. Organisation fonctionnelle pour l'introduction des cartes).

Pour la bande perforée où l'information est lue caractère par caractère, l'information est transmise en Mémoire Rapide à raison de 3 caractères par catène quel que soit le type de bande utilisé; chaque caractère est complété à 8 monades par des blancs dans le cas d'introduction de bande perforée de moins de 8 canaux.

A la sortie, le rôle du Traducteur est de composer en Mémoire Rapide une image des informations qui puisse être directement comprise par l'élément de sortie correspondant :

.une "photographie" de la carte si l'on veut sortir par un perforateur de cartes,

.3 caractères par catène pour la bande perforée,

.enfin, pour la sortie sur imprimante, il faut "composer" en Mémoire Rapide la ligne à imprimer à raison de 4 caractères par catène (code interne 6 monades), ce qui représente 30 catènes pour une ligne de 120 caractères.

2. *Les "cas d'emploi"*

Ce que l'on vient de voir, du rôle du Traducteur et des différents codes mis en jeu, montre que son fonctionnement sera quelque peu différent selon qu'il s'agira de traduire, en code interne, les informations d'une carte perforée en vue de leur "mémorisation", ou, par exemple, de traduire, en vue de leur impression, des informations qui se trouvent en code interne mais à l'état "traitement", en code interne entièrement alphanumérique 6 positions.

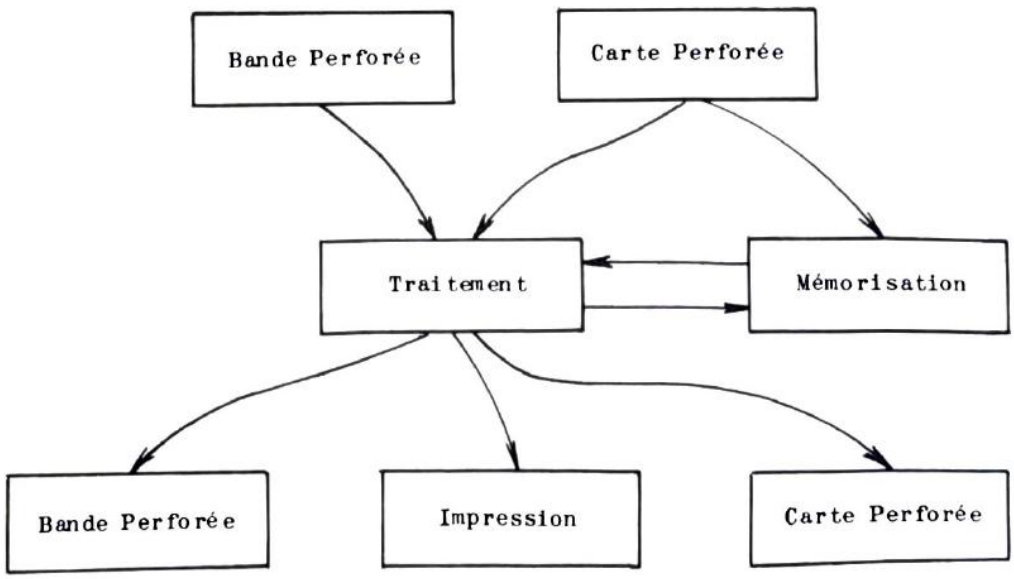
Il apparait donc qu'il est nécessaire d'envisager différents "cas d'emploi" du Traducteur.

On peut avoir :

.à l'entrée : la bande perforée et la carte perforée,
.à la sortie : la bande perforée, la carte perforée, l'état imprimé,
.à l'intérieur du Gamma 60 : le code interne à l'état *"traitement"* ou à l'état *"mémorisation"*.

On pourrait penser qu'outre le cas d'emploi qui consiste à passer de *"traitement"* à *"mémorisation"* ou l'inverse, les autres cas d'emploi du Traducteur couvrent toutes les possibilités de passage de n'importe quel état d'entrée vers n'importe quel état de code interne (traitement ou mémorisation) et de n'importe quel état de code interne vers n'importe quel état de sortie.

En fait, il est apparu que pratiquement, tous les besoins étaient couverts en ne retenant qu'un certain nombre de ces cas d'emploi, comme indiqué par le schéma ci-dessous.



Les flèches font apparaitre les différents cas d'emploi suivants :

.Bande vers Traitement
.Carte vers Traitement
.Carte vers Mémorisation
.Traitement vers Bande
.Traitement vers Carte
.Traitement vers Impression
.Traitement vers Mémorisation
.Mémorisation vers Traitement

auxquels il convient d'ajouter un 9ème cas :

. Mémorisation vers Mémorisation, dans lequel on fait "défiler" l'information à travers le Traducteur sans modifier son état de mémorisation mais en sachant toujours où l'on en est pour pouvoir passer au cas d'emploi "Mémorisation vers Traitement" dès qu'il en sera besoin.

3. Les formes de l'information

Si l'on appelle "mot" un ensemble d'informations qui constitue un tout vis-à-vis de n'importe quelle sorte de traitement qu'il pourra subir au cours de ses passages en Gamma 60, on distingue 3 sortes de mots :

. les mots alphanumériques (essentiellement des libellés comme des noms, des adresses, etc ...) qui, en code interne, utilisent 6 monades par caractère,

. les mots numériques (indicatifs numériques par exemple) qui, en code interne, utilisent 4 monades par chiffre,

. les nombres qui doivent faire l'objet de calculs.

Tout mot est, en outre, caractérisé par sa longueur. Ici, il faut distinguer 3 notions :

. *la longueur théorique :* c'est le nombre de caractères maximum nécessaire pour représenter un mot : elle est définie par l'énoncé même du problème. C'est le travail de l'analyste de définir la longueur théorique.

. *la longueur réelle :* c'est le nombre de caractères effectivement nécessaire pour représenter le mot : elle est obligatoirement plus petite ou égale à la longueur théorique.

. *la longueur pratique :* c'est la capacité réservée sur un support déterminé pour contenir le mot. Pour un même mot, la longueur pratique dépend du support utilisé : sur la carte perforée par exemple, elle est égale à la longueur théorique, sur la bande perforée, à la longueur réelle ; en code interne, et à l'état traitement, elle s'exprime en un nombre entier de catènes ; c'est la longueur théorique arrondie au catène supérieur pour les mots alphanumériques et numériques, c'est 2 catènes, toujours, pour les nombres.

Pour faciliter la programmation du Traducteur, on a prévu des formes "standard" de représentation de l'information. Dans ces cas, le Traducteur peut fonctionner de façon semi-automatique. On distingue la forme "standard programmée" et la forme "standard normalisée". Cette distinction n'est effective que pour les nombres mais on la maintient de façon purement formelle pour les mots numeriques et alphanumériques ; cela signifie que si l'on traduit un article comprenant une succession de mots alphanumériques, numériques et de nombres en "standard programmé", on positionnera une fois pour toutes le Traducteur en "standard programmé" grâce à un code complémentaire dans la directive préparatoire qui définit le cas d'emploi (cf. § III) ; cependant que, si l'on traduit un article se composant de la même succession de mots alphanumériques, numériques et de nombres, cette fois-ci en "standard normalisé", on positionnera une fois pour toutes le Traducteur en "standard normalisé" alors que la représentation des mots numériques et alphanumériques sera rigoureusement la même que dans le cas précédent.

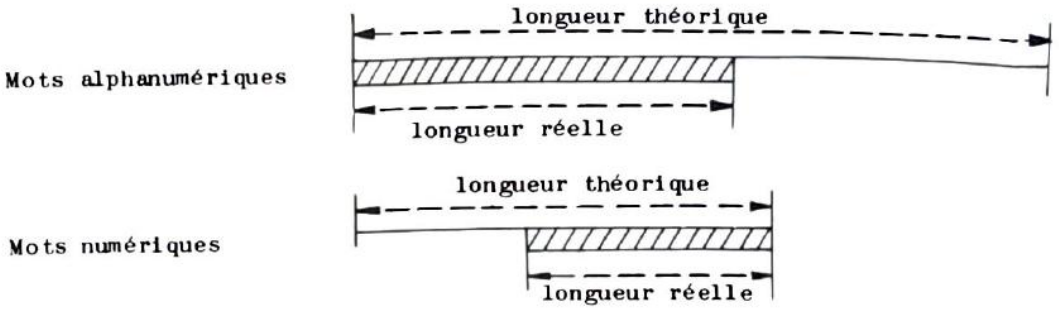
Nous allons examiner ces formes de représentation pour les 3 sortes de mots (alphanumériques, numériques, nombres) en fonction de la nature du support externe ou de l'état interne, traitement ou mémorisation.

3.1.- Mots numériques et alphanumériques. Il n'y a pas de distinction à faire entre forme standard programmée et standard normalisée.

a) sur carte perforée ou imprimé

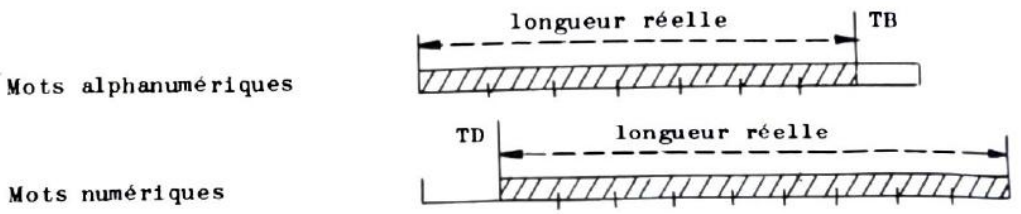
Un dessin de carte ou d'imprimé étant caractérisé par la rigidité de sa

structure, ces mots sont représentés avec leur longueur théorique, cadres à gauche s'il s'agit de mots alphanumériques et à droite s'il s'agit de mots numériques. Leur longueur pratique est la longueur théorique.



b) sur bande perforée

Ici, la souplesse du support permet de représenter ces mots avec leur longueur réelle, suivis du code tabulation pour les mots alphanumériques ou précédés d'une tabulation décimale pour les mots numériques.



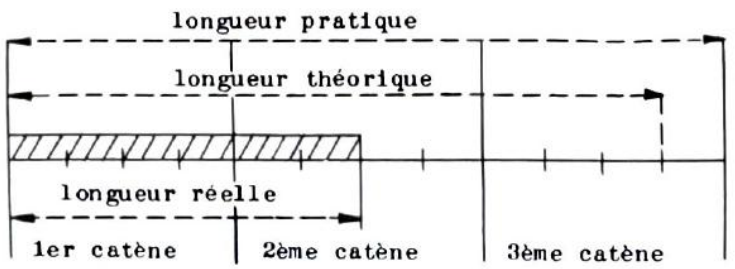
c) à l'état interne traitement

Le mot, étant comme on l'a défini plus haut, un ensemble d'informations constituant un tout vis-à-vis de n'importe quelle sorte de traitement, et le catène étant organiquement l'unité de transfert d'information entre Mémoire Rapide et éléments, il s'ensuit qu'à l'état traitement la longueur pratique d'un mot se mesurera en un nombre entier de catènes.

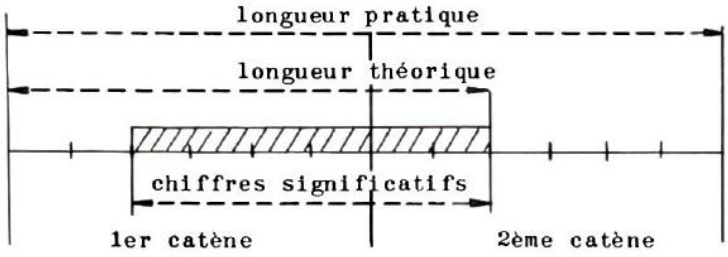
Pour un mot alphanumérique, cette longueur pratique sera égale au quotient, arrondi à l'entier supérieur, de la longueur théorique (en caractères) par quatre. Pour un mot numérique, ce sera le quotient par 6. Le Traducteur effectue automatiquement l'arrondi au catène à la fin de chaque mot. On voit donc qu'à l'état traitement il ne saurait y avoir de mélange code 6 positions, code 4 positions dans un même catène.

Dans cette longueur pratique, les mots sont cadrés à gauche.

Exemple : Mot alphanumérique de longueur théorique 11 lettres et longueur réelle 6 lettres :



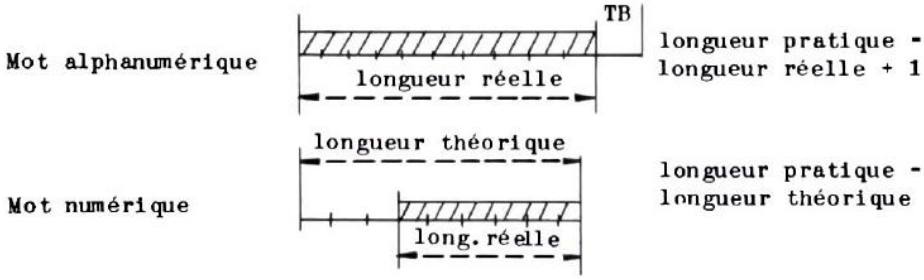
Mot numérique de 8 chiffres de longueur théorique :



Remarquons que pour un mot numérique (il s'agit en général d'un indicatif), il est vain de distinguer entre longueur théorique et longueur réelle : les chiffres significatifs sont cadrés à droite dans la longueur théorique, les zéros à gauche se codant comme les espaces en code 4 positions.

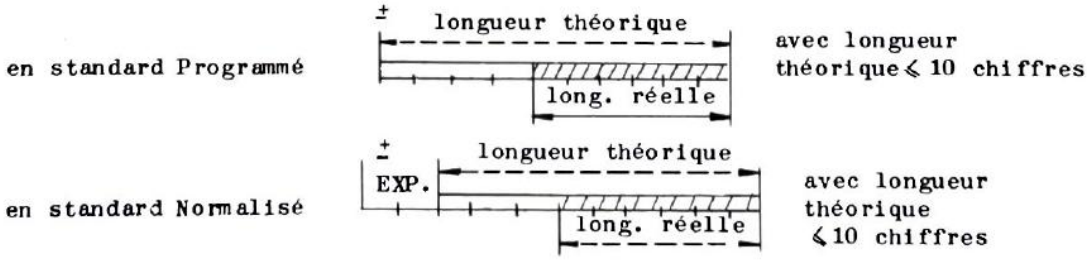
d) à l'état interne Mémorisation

Les mots alphanumériques sont représentés en longueur réelle suivis du code tabulation (code 31 en 6 monades). Les mots numériques sont en longueur théorique et cadrés à droite. Il n'y a aucun inconvénient à avoir un même catène à cheval sur 2 mots de nature différente ; le mélange des différents codes internes à l'intérieur d'un même catène peut donc se trouver à l'état mémorisation.



3.2.- Les nombres algébriques. C'est ici qu'il convient de distinguer entre forme standard Programmée et standard Normalisée :

a) *sur Carte Perforée*



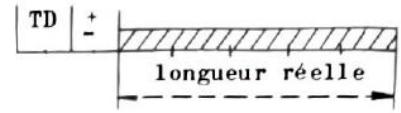
Le signe est perforé en superposition sur le premier chiffre du nombre :

12 pour +
11 pour -

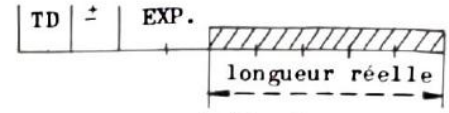
Toutefois, il est possible que le signe + ne soit pas représenté en standard programmé. Cela est alors indiqué en code complémentaire dans la directive cas d'emploi (cf. § III).

b) *sur Bande Perforée*

en standard Programmé



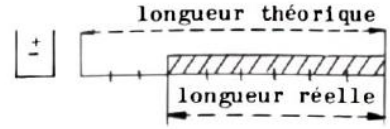
en standard Normalisé



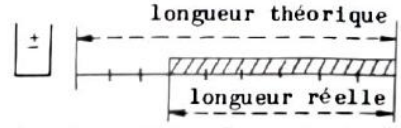
Ils sont représentés en longueur réelle, précédés d'une tabulation décimale et du signe (qui est alors un caractère comme les autres).

c) *sur Imprimé*

en standard Programmé



en standard Normalisé



Les signes sont des caractères à imprimer comme les autres. Au moment de l'édition, il faudra leur consacrer des caractères 6 monades comme pour les chiffres (code 47 pour +, code 46 pour -). Noter ici que seul l'espace entre le signe et l'exposant est introduit automatiquement par le Traducteur, les autres espaces sont à programmer.

d) *à l'état interne Traitement*

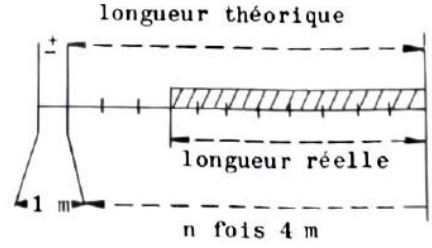
Les nombres sont représentés sous leur forme semi-logarithmique sur 2 catènes, en virgule programmée ou normalisée, prêts à être traités par le Calculateur Arithmétique. Tout ceci a été expliqué à propos de cet élément.

e) *à l'état interne Mémorisation*

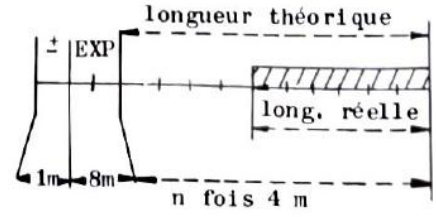
Le signe est représenté par 1 monade (0 pour + et 1 pour -). En standard programmé, le nombre est ensuite représenté sur sa longueur théorique en code 4 monades.

En standard normalisé, le nombre est ensuite représenté par son exposant (2 chiffres soit 8 monades) et sa mantisse en longueur théorique (4 monades par chiffre).

Standard programmé



Standard normalisé



Il faut bien noter ici la différence avec l'état Traitement : il y avait là 8 monades pour le signe et l'exposant, ici il y en a 9, mais si la mantisse n'a que 4 chiffres, elle n'occupera que 16 monades, ce qui fait au total 25 monades au lieu de 48 (2 catènes) à l'état Traitement.

3.3.- En *conclusion*, il faut bien noter que ces formes standard ne sont pas exclusives d'autres formes de représentation. Elles signifient simplement qu'avec ces formes, le Traducteur peut travailler de façon semi-automatique tandis qu'avec d'autres formes, la programmation sera totalement analytique.

3.4.- *En résumé* : dans les formes standard normalisées, l'exposant est toujours perforé ou imprimé, en carte, bande ou imprimante. Dans les formes standard programmées, c'est le Traducteur qui injecte l'exposant dans l'état interne Traitement, l'exposant n'est pas représenté dans les formes externes.

Remarque : Nous n'avons pas parlé ici des informations en binaire pur ; ce sont soit des nombres en binaire, soit des mots dont le "profil" a une certaine signification logique, la géographie d'un article par exemple. En tout état de cause, ces mots binaires n'existent qu'en code interne et sont toujours représentés en longueur théorique. Cette longueur théorique se mesure en monades et, à l'état Traitement, les mots binaires sont comme les autres arrondis au catène.

II. STRUCTURE ET FONCTIONNEMENT DU TRADUCTEUR

1. Généralités

De façon générale, les éléments reçoivent les instructions qui leur disent comment travailler sous forme de "Directives". C'est un peu plus compliqué pour le Traducteur : en effet, d'après ce qu'on a dit de ses différents cas d'emploi et des différentes formes de l'information, on voit que le Traducteur est un organe très souple qui permet de faire à peu près tout ce que l'on veut en dehors des calculs; il en résulte que les instructions qu'il doit recevoir sont obligatoirement très analytiques (même si les formes standard permettent un fonctionnement plus automatique) et, en conséquence, très nombreuses. Si toutes ces instructions étaient données sous forme de directives, il en résulterait que, puisque les directives passent par le Distributeur de Programme, le Traducteur serait un élément qui occuperait le Distributeur de Programme beaucoup plus que les autres et qu'en conséquence, il se pourrait que ces autres éléments soient souvent en attente du Distributeur de Programme lorsque le Traducteur fonctionnera.

C'est pourquoi l'on a divisé les instructions données au Traducteur en deux sortes :

. *Les Directives*, analogues aux directives des autres éléments mais qui ne lui donnent que les renseignements très généraux.

Comme on le verra au moment de leur description, elles se bornent essentiellement à préparer des positionnements, à déclencher la traduction ou à l'arrêter.

. *Le Cliché* : c'est une série d'informations qui définissent de façon précise le travail à accomplir sur les informations entrant dans le Traducteur. Un cliché de traduction sera ainsi composé d'un certain nombre de codes-cliché ou plus simplement de clichés. Chaque code-cliché sera un code de 6 positions binaires qui, théoriquement, définira le travail à effectuer sur un caractère de l'information à traiter. On devrait donc donner autant de clichés que de caractères à traiter, mais on a souvent à effectuer un travail de même nature sur un certain nombre de caractères consécutifs : on condense donc l'écriture du cliché en indiquant le nombre de fois qu'il doit être répété. Ceci se fait de la façon suivante en code A :

Les 5 premières monades de l'hexade indiquent la nature du cliché (32 possibilités), la sixième indique si l'hexade suivante doit être considérée comme un nouveau cliché ou le nombre de fois qu'il faut appliquer le cliché que l'on vient de donner.

Par exemple :

| 1 0 1 1 0 0 | 1 1 0 0 1 1 |

signifie que le cliché codé 12 doit s'appliquer 51 fois.

Par contre :

| 0 1 1 1 0 0 | 0 0 1 1 1 1 |

signifie que l'on a affaire à deux clichés ne s'appliquant qu'une fois chacun.

Si la longueur que l'on désire indiquer dépasse 63, on répète le cliché avec longueur, suivi de la longueur 63 autant de fois qu'il est nécessaire et on termine avec la longueur nécessaire pour compléter à la valeur exacte. Par exemple :

| 1 0 1 1 0 0 | 1 1 1 1 1 1 | 1 0 1 1 0 0 | 0 0 1 1 1 1 |

Le cliché code 12 doit être appliqué : 63 + 15 - 78 fois.

Ainsi, au moment d'une traduction, on aura en Mémoire Rapide :

. d'une part, le *Programme* avec ses instructions d'adresse, ses branchements et ses directives,

. d'autre part, le *cliché* de traduction qui sera dans une autre zone de Mémoire Rapide à raison de 4 codes ou longueurs par catène.

Ce cliché est un complément du programme mais, contrairement aux directives, il n'est pas alimenté dans le Traducteur par l'intermédiaire du Distributeur de Programme. Il l'est directement à la demande du Traducteur, exactement comme les quantités à traduire, par l'intermédiaire du Distributeur de Transfert.

C'est pourquoi le Traducteur a un RAC spécialisé chargé de suivre catène par catène l'alimentation du cliché, exactement comme d'autres RAC suivant l'alimentation des opérandes.

Dès maintenant, il convient de noter que tant que le Traducteur fonctionne sur son cliché de traduction, il fonctionne de façon autonome. Il en est ainsi jusqu'à ce qu'il rencontre un cliché spécial RP (retour au programme) ou CQ (catène qualitatif) qui a pour effet de faire que le Traducteur fait une demande d'instruction au Distributeur de Programme afin de reprendre le programme à l'adresse contenue dans son R.A.I.

2. *Principaux Organes*

Ce paragraphe vise simplement à énumérer les principaux organes du Traducteur en les décrivant et donnant un bref aperçu de leur rôle ; ce dernier point de vue sera complété par la suite.

2.1.- Registres. Outre le registre de liaison, le Traducteur comprend :

a) le Registre d'Entrée qui reçoit les informations à traiter catène par catène. C'est dans ce registre qu'elles sont alors prises caractère par caractère pour être traitées. Dans le cas d'introduction de cartes, ce registre n'est pas utilisé.

b) le Registre de Sortie qui reçoit les informations traitées caractère par caractère et renvoie catène par catène ces informations en Mémoire Rapide en passant par le registre de liaison. Il n'est pas utilisé dans le cas de sortie sur cartes.

c) le Registre Cliché de capacité également 1 catène, qui reçoit le cliché de traduction catène par catène et le fournit au programme local code-cliché par code-cliché.

d) le Registre Directive et Catène Qualitatif qui reçoit du Distributeur de Programme les directives et dans lequel se forme le catène qualitatif au cours du travail autonome du Traducteur.

2.2.- *Les Matrices et organes de transcodage*

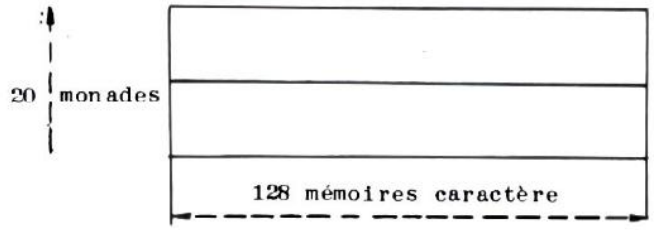
a) la Matrice de transposition - C'est une matrice plane de 12 x 24 - 288 positions binaires. Elle permet de regrouper par caractères de 12 monades les informations lues dans 24 colonnes de cartes et contenues dans 12 catènes, réalisant ainsi la transposition lignes-colonnes.

Elle remplace le registre d'entrée à l'introduction des cartes et le registre de sortie pour la sortie des résultats sur cartes.

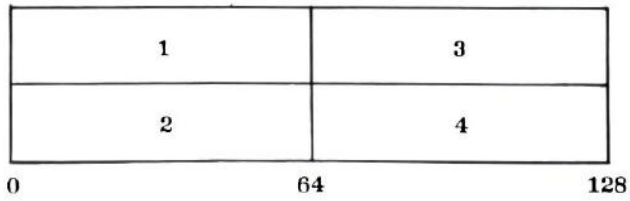
b) la Matrice d'Interprétation des Codes (M.I.C.)

C'est une matrice plane de 128 mémoires dites "mémoires caractère" ayant chacune une capacité de 20 monades. C'est l'organe essentiel de la traduction. *Le code à traduire sélectionne l'une de ces mémoires, mémoire dont le contenu représente le code traduit.* Ce système est très souple car on peut garnir la MIC (grâce à une directive spéciale) avec n'importe quel code; il suffit d'en constituer une table en Mémoire Rapide (1 catène par mémoire caractère) et d'en garnir la MIC.

Chaque mémoire caractère peut contenir en fait 2 codes, y compris, pour chacun, sa clef de parité et la clef de parité de son adresse dans la MIC. Bien entendu, seul l'un de ces 2 codes est sélectionné chaque fois que l'on fait appel à une mémoire caractère donnée. Ceci se fait grâce à un positionnement spécial indiqué par la directive cas d'emploi (cf. § III). Ceci veut donc déjà dire que la MIC est splittable en longueur, c'est-à-dire au milieu des mémoires caractères.



De plus, le code interne étant à 6 monades et comportant 64 possibilités, il s'ensuit que, d'une façon générale, il suffirait de 64 mémoires caractère et non de 128. C'est vrai dans la plupart des cas d'utilisation (pour les exceptions cf. II.5) et l'on n'utilise que les mémoires 0 à 63 ou 64 à 127. Ceci veut dire que la MIC est également splittable en largeur et que, finalement, dans la plupart des cas simples d'utilisation, on n'en utilise que le quart, l'une des parties numérotée de 1 à 4 sur le schéma ci-dessous.



On peut donc envisager facilement la possibilité de faire des séquences de traduction successives concernant des codes différents sans avoir besoin de garnir la MIC avant chaque séquence. Il suffit pour chaque séquence d'utiliser une partie différente de la MIC que l'on aura garni préalablement une fois pour toutes. Comme on le verra, cette possibilité a ses limites, puisque la capacité de la MIC est

limitée. En II.5, on expliquera les différents splittages possibles et il sera facile d'en tirer les conséquences à ce sujet.

Mais, d'après le principe même du fonctionnement de la MIC, le code à traduire représente, dans les cas où seulement un quart de MIC est utilisé, l'adresse (0 à 63) du code traduit dans la partie de MIC employée. C'est aussi le cas lorsque l'on traduit de la carte perforée; et, cependant, le code 12 positions de la carte perforée ne correspond pas à un système d'adresses de 0 à 63 ; il est donc nécessaire d'avoir un organe qui passe du code 12 positions de la carte à un code intermédiaire 6 positions représentant l'adresse du caractère traduit dans la MIC. C'est le rôle de :

c) le Prétranscodeur

C'est un organe entièrement câble, non soumis au programme, qui permet de passer du code de la carte 12 positions à un code intermédiaire 6 positions représentant l'adresse dans la MIC du code interne correspondant. Puisque c'est un organe entièrement câblé, il y a un prétranscodeur pour la lecture des cartes BULL et un prétranscodeur pour la lecture des cartes IBM.

d) Le Post-transcodeur

Inversement, pour la sortie sur carte, le code interne ne peut sélectionner dans une demi-mémoire-caractère de la MIC, directement un code 12 positions. C'est un code intermédiaire 7 positions qui est sélectionné et qui, par l'intermédiaire du post-transcodeur, fournit le code carte 12 positions (7 positions pour le code intermédiaire ne coûtent pas plus cher pour la MIC et fournissent la possibilité d'un post-transcodeur meilleur marché). Il n'existe qu'un post-transcodeur pour sortir en cartes BULL. Le post-transcodeur alimente la matrice de transposition pour reconstituer la carte.

2.3.- Les RAC

Le Traducteur possède 4 RAC numérotés de 0 à 3 et tous spécialisés :

. *Le RAC 0* contient l'adresse des informations à traduire (à alimenter dans le registre d'entrée ou la matrice de transposition selon les cas).

. *Le RAC 1* contient l'adresse où doivent être rangées les informations traduites (en provenance du Registre Sortie ou de la matrice de transposition selon les cas).

. *Le RAC 2* sert à deux fins :

a) lorsque, avant une traduction, on veut garnir la MIC d'une table de codes se trouvant en Mémoire Rapide, c'est le RAC 2 qui doit contenir l'adresse de cette table.

b) au cours de la traduction même, il doit contenir l'adresse du cliché.

. *Le RAC 3* est utilisé dans l'organisation dite de "datum de reprise" (cf. § IV).

3°/ Organisation fonctionnelle du Traducteur

Nous allons décrire l'organisation fonctionnelle du Traducteur dans un cas particulier qui utilise tous ses organes, la traduction de cartes perforées. La généralisation sera faite ensuite.

On sait que les cartes sont lues ligne par ligne. Ainsi, le lecteur de cartes prend simultanément connaissance de 80 informations élémentaires qu'il range dans :

$\frac{80}{24}$ - 4 catènes,

les informations des 24 premières colonnes dans un catène, celles des colonnes 25 à 48 dans un 2ème, 49 à 72 dans un 3ème, le 4ème ne contenant que les 8 dernières colonnes.

Mais, pour chaque ligne, les catènes sont envoyés dans des mémoires telles qu'après la lecture des 12 lignes, l'image des 24 premières colonnes est contenue par 12 catènes consécutifs. Ainsi, par exemple, pour la ligne des 9 :

.les 24 premières colonnes seront envoyées dans le catène 1001
.les colonnes 25 à 48 " " " 1013
.les colonnes 49 à 72 " " " 1025
.les colonnes 73 à 80 " " " 1037

La ligne des 8 sera ensuite mémorisée de la façon suivante :

.les 24 premières colonnes dans le catène 1002
.les colonnes 25 à 48 " " 1014
.les colonnes 49 à 72 " " 1026
.les colonnes 73 à 80 " " 1038

et ainsi de suite jusqu'à la ligne des 12 qui sera mémorisée :

.les 24 premières colonnes dans le catène 1012
.les colonnes 25 à 48 " " 1024
.les colonnes 49 à 72 " " 1036
.les colonnes 73 à 80 " " 1048

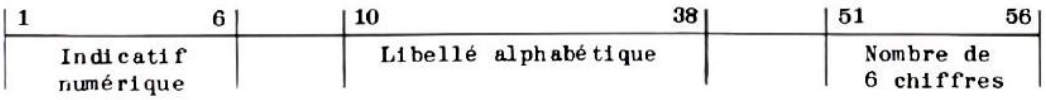
On voit donc qu'ainsi l'image d'une carte non traduite sera contenue dans 48 catènes, les 12 premiers contenant les 24 premières colonnes, les 12 suivants les 24 colonnes suivantes, etc...

A ce moment, le Traducteur interviendra. On aura dû au préalable :

.lui indiquer son cas d'emploi Carte vers mémorisation par exemple,
.garnir son RAC 0 de l'adresse de l'image de la carte (ici 1001), son RAC 1 de l'adresse où l'on veut ranger le résultat de la traduction et son RAC 2 de l'adresse du cliché.

Dès lors, le Traducteur pourra fonctionner de façon autonome. Pour suivre ce fonctionnement, on pourra se reporter au schéma se trouvant en fin de paragraphe.

Pour préciser les idées, donnons le dessin de la carte à traduire :



Le nombre est représenté en standard programmé, c'est-à-dire avec signe superposé sur le premier chiffre de la mantisse.

Le Traducteur devra, avant la traduction proprement dite, réaliser la transposition lignes colonnes qui aura été préparée suivant la façon dont la carte aura été enregistrée en Mémoire Rapide. Pour cela, le Traducteur devra faire 12 demandes de quantités successives de façon à emmagasiner dans la matrice de transposition les 12 catènes 1001 à 1012 qui contiennent intégralement les 24 premières colonnes de la carte. Ce remplissage de la MT se fait simplement par l'intermédiaire du Registre de liaison.

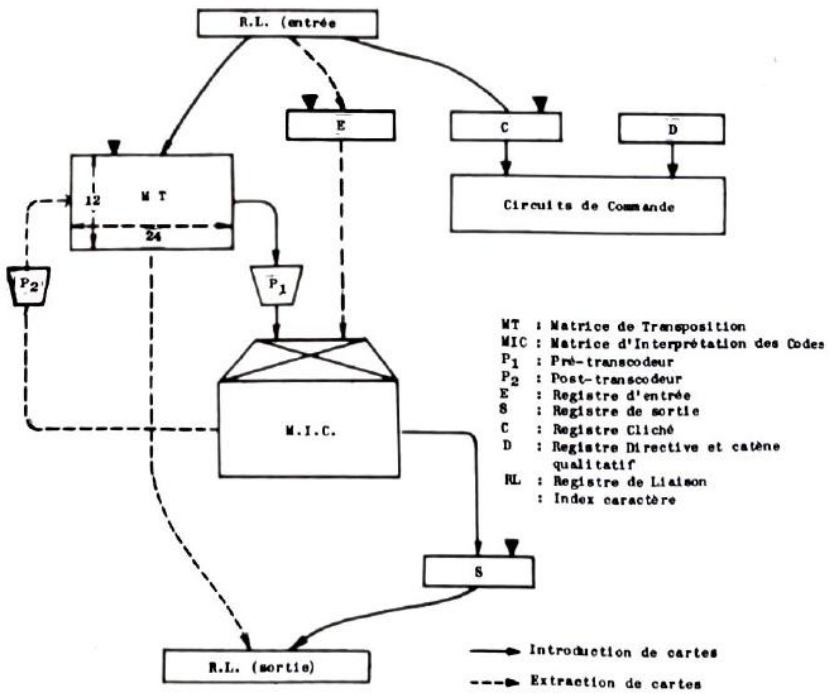
Le Traducteur alimente alors dans le registre C le premier catène de codes-cliché et commencera son travail. Il trouvera d'abord des codes-cliché lui disant que le premier mot est un mot numérique de 6 caractères, qu'il faut donc traduire en code 4 positions les 6 premières colonnes de carte, c'est-à-dire les 6 premiers caractères se trouvant dans la matrice de transposition. Pour cela, chaque caractère

passe dans le prétranscodeur, d'où il ressort en code intermédiaire 6 positions pour aller sélectionner dans la MIC le code interne correspondant. A la sortie de la MIC les 6 chiffres de l'indicatif iront s'accumuler dans le registre de sortie S dont le contenu sera alors envoyé en Mémoire Rapide à l'adresse désignée par le RAC 1.

Le Traducteur trouvera alors des clichés indiquant qu'il faut sauter les 3 colonnes suivantes, puis traduire, en code alphanumérique 6 positions, les 29 qui viennent ensuite numérotées sur la carte 10 à 38. Durant ce travail, chaque fois que le registre C sera épuisé, le Traducteur alimentera un nouveau catène de codes-cliché et chaque fois que le registre sortie sera rempli, il sera envoyé en Mémoire Rapide. Lorsque durant la traduction de la zone alphabétique, on sera arrivé à la fin de la matrice de transposition, on la remplira à nouveau des catènes 1013 à 1024 contenant les colonnes 25 à 48 de la carte.

On voit donc que le travail du Traducteur sera suivi pas-à-pas *par trois index*, l'un évoluant sur les 24 colonnes de la MT, l'autre sur les 4 clichés du registre C, le troisième sur les caractères du registre S.

Dans le cas des colonnes 10 à 38 de la carte où il y a un libellé alphabétique, il s'agit par définition de longueur variable, de longueur maximum 29 caractères. Puisque l'on s'est mis dans le cas d'emploi, Carte vers Mémorisation, il sera représenté en code interne en longueur réelle suivi de code tabulation. Supposons que cette longueur réelle soit de 13 lettres ; avec le code tabulation, cela fait 14 caractères, soit 3 catènes 1/2, ce qui signifie que l'index du registre S sera arrêté au milieu. On trouvera alors dans le cliché des codes disant de sauter les 12 colonnes 39 à 50, puis de traduire le nombre de 6 chiffres. Comme, grâce à un code complémentaire dans la directive donnant le cas d'emploi, on s'est placé en "standard programmé", le Traducteur interprétera automatiquement le signe superposé sur le premier chiffre et injectera 1 monade (1 ou 0) dans le registre S aussitôt après le code tabulation (à l'état mémorisation on peut mélanger les différentes sortes de codes à l'intérieur d'un même catène), puis les 6 chiffres seront traduits en code 4 positions et viendront se mettre directement à la suite du signe (revoir la forme standard programmée en mémorisation).



SCHEMA FONCTIONNEL DU TRADUCTEUR A L'INTRODUCTION ET A L'EXTRACTION DE CARTES

L'index du registre S se retrouvera finalement sur la 13ème monade et le Traducteur continuera de même jusqu'à épuisement de la carte.

Dans le sens extraction sur carte, le travail du Traducteur s'effectue en sens inverse (flèches en trait interrompu de schéma) ; les circuits du traducteur sont positionnés ainsi par la directive cas d'emploi "Traitement vers Carte". Ici le registre entrée est mis en oeuvre et la matrice de transposition est utilisée à la place du registre de sortie.

Lorsque le support externe est la bande perforée, la MT n'est pas mise en jeu et tout se passe entre les registres d'entrée, de sortie et la MIC. Enfin, lorsqu'il s'agit d'une utilisation entre les 2 états interne Traitement et Mémorisation, la MIC est hors de circuit.

4°/ Le Contrôle Interne du Traducteur

Les informations à traduire comme le cliché arrivent dans le Traducteur catène par catène accompagnées de leur clef par 7. A ce moment, on prend en considération la nature des informations pour associer à chaque caractère une clef de parité et le contrôle interne du Traducteur est fondé sur cette clef de parité. On a vu qu'à l'intérieur des mémoires caractère de la MIC, chaque caractère était accompagné de sa clef de parité et de la clef de parité de son adresse dans la MIC, c'est-à-dire la clef de parité du caractère à traduire. On voit alors comment on peut vérifier le bon fonctionnement de la sélection MIC en vérifiant l'identité de la clef du caractère entrant et de la clef d'adresse du caractère sélectionné.

A la sortie du Traducteur, un générateur de clef par 7 forme la clef de chaque catène avant de l'envoyer en Mémoire Rapide.

5°/ Les Types d'Opérations Complémentaires (T.O.C.) et l'utilisation de la MIC

Lorsque l'on traduit de la bande perforée (cas d'emploi Bande vers Traitement), on rencontre des codes de service (tabulation normale ou décimale, codes à maintien par exemple) qui ne correspondent pas à un caractère de l'information, lettre ou chiffre, qui puisse être traduit en code interne. Ils correspondent à certaines fonctions de repérage de la nature de l'information perforée, mais à leur entrée dans le Traducteur, celle-ci ne sait pas les repérer et comme n'importe quel code externe, ils vont donc aller sélectionner une mémoire caractère de la MIC.

Dans la mémoire ainsi sélectionnée, on trouvera, non pas un caractère en code interne, mais une sorte d'instruction (un type d'Opération Complémentaire), reconnaissable et compréhensible par le Traducteur, qui déclenche la fonction qui lui est attribuée.

5.1.- Pour comprendre le fonctionnement de ces T.O.C., il faut ouvrir une parenthèse sur la *structure des Mémoires Caractère de la MIC*. On sait qu'en fait c'est seulement la moitié de ces mémoires qui est utilisée à un instant donné ; il faut donc décrire l'utilisation des 10 positions binaires ainsi définies.

Les 6 ou 7 premières positions contiennent le caractère en code interne, ou le code intermédiaire avant le post-transcodeur : 6 seulement pour le code interne dans le cas de traduction Carte vers Traitement ou Mémorisation ; 7 pour le code intermédiaire avant le post-transcodeur dans le cas d'emploi Traitement vers carte.

Les 8ème et 9ème positions contiennent respectivement les clefs de parité du contenu de ces 6 ou 7 positions et la clef de parité de l'adresse dans la MIC de la mémoire caractère considérée.

La 10ème position contient 0 lorsqu'il s'agit vraiment d'un caractère, et 1 lorsqu'il s'agit d'un TOC. Le Traducteur décelant ce 1 interprète alors le profil

des 9 premières positions comme une instruction définissant le TOC et l'exécute.

5.2.- Les différents TOC que l'on peut aussi rencontrer sont :

. *La Tabulation Normale (TBN)*

On la rencontre à la fin d'un mot alphanumérique de 6 lettres par exemple alors que le cliché de traduction indiquant une longueur théorique de 11. Le TOC correspondant injecte alors 5 caractères blancs (c'est-à-dire avance la sortie jusqu'à épuisement de la longueur théorique). Il injecte un 1 en position 15 du catène qualitatif.

. *La Tabulation avec Retour du Chariot (TRC)*

C'est le code qui est perforé dans la bande quand la dactylo va à la ligne lorsqu'elle frappe un état. Le TOC correspondant joue d'abord le même rôle que la tabulation normale en complétant le mot à la longueur théorique.

Mais de plus, il injecte un 1 en position 16 du catène qualitatif. En effet, en supposant que la ligne de l'état comporte 5 mots, il se peut que certaines lignes ne comportent pas les 1, 2, 3 ou 4 derniers mots. On ne peut alors obliger la dactylo à frapper autant de codes tabulation que de mots absents en fin de ligne, comme elle le fait s'il s'agit de mots absents en cours de ligne. Il ne faut pas non plus que le Traducteur perde le synchronisme cliché-information. Pour cela, il faudra, dans ce cas, faire suivre le cliché de traduction de chaque mot de la ligne d'un retour au programme (RP) pour pouvoir tester la position 16 du catène qualitatif du Traducteur afin de savoir si la tabulation ne s'est pas accompagnée d'un retour chariot et, dans l'affirmative, reprendre le cliché de traduction à l'endroit qui correspond au début de la ligne. Notons d'ailleurs que ce mode d'exploitation de la bande n'est donné qu'à titre d'exemple : il existe des quantités de mode d'exploitation de la bande perforée et l'on peut très bien aussi donner à la dactylo la consigne de frapper toutes les tabulations de mots absents en fin de ligne. Ainsi l'exploitation du TOC "Retour de Chariot" peut donner lieu à des variantes fonction des consignes données à la dactylo qui a perforé la bande.

. *La Tabulation Décimale (TBD)*

Sur la bande perforée, le code tabulation décimale précédant un mot numérique indique le nombre de chiffres de ce mot. Si ce mot suit un mot alphanumérique, il joue également vis-à-vis de ce dernier le rôle d'une tabulation normale. Le TOC sélectionné alors dans la MIC jouera alors lui aussi 2 rôles.

a) Il provoquera l'injection d'autant de caractères alphanumériques blancs qu'il le faut pour compléter la longueur théorique du mot alphanumérique.

b) Il provoquera ensuite l'appel par le Traducteur du cliché de Traduction correspondant au mot numérique et, avant de permettre à la traduction elle-même de s'exécuter, il injectera le nombre de blancs nécessaires pour compléter la longueur réelle indiquée par le TOC à la longueur théorique indiquée par le cliché de traduction.

. *L'Annulation de Caractère Entrée (ANC)*

C'est le code perforé effacement (7 trous) que l'on superpose à un caractère erroné ; il ne lui correspond aucun code interne ; le TOC sélectionné correspondant l'annule purement et simplement, ce qui correspond bien à sa fonction.

. *Le Code Cliché imprévu (contrôle de synchro information cliché).*

Outre les codes d'information et les codes spéciaux qui déclenchent les TOC déjà vus, on peut trouver sur la bande perforée d'autres codes dits codes-clichés. Leur existence correspond à une organisation de la bande perforée où la reconnaissance de l'information ne tient pas seulement à la connaissance du mouvement du chariot de la machine à écrire au cours de sa création, mais aussi au fait que, de place en place, on perfore des codes-clichés qui définissent la nature de l'information qui suit. Ces codes-clichés se trouvent donc normalement en des points prévisibles dans le cliché de traduction. Leur exploitation normale consiste à les envoyer,

sans passer par la MIC, dans le catène qualitatif, à les tester, et, en fonction de ce test, à continuer le programme où il faut.

Mais il se peut qu'au cours de la création de la bande, la dactylo ait commis des erreurs dont elle ne s'est pas aperçu et qu'à la place d'un code-cliché prévu, il se trouve un code caractère normal que l'on envoie donc dans la catène qualitatif.

On s'en apercevra par le fait que les codes-clichés et eux seuls possèdent une perforation en voie Z. Le problème consiste alors à rattraper le synchronisme information cliché de traduction ; pour cela, la séquence de programme à laquelle on doit alors se renvoyer, doit envoyer dans le catène qualitatif tous les codes suivants et les tester jusqu'à reconnaissance d'un code-cliché dont la nature permet alors de savoir où on en est et donc de reprendre la traduction de la suite ; tandis que l'on range à part tout ce que l'on a traduit depuis le code-cliché précédent afin de détecter ultérieurement la nature de l'erreur.

Il se peut aussi qu'au cours de la traduction elle-même, on tombe sur un code-cliché alors que l'on ne s'y attendait pas. La mémoire-caractère alors sélectionnée contient un TOC, le TOC "Code Cliché Imprévu" qui a pour effet de positionner la position 24 du catène qualitatif, la position 17 (code cliché imprévu) et envoie dans les positions 1 à 8 le code cliché rencontré lui-même, ce qui en permet l'analyse et l'exploitation. Il faut ici considérer 2 cas :

a) le code cliché imprévu est un code cliché que l'on aurait dû rencontrer un peu plus loin et sa rencontre anticipée est due à une erreur dont la dactylo ne s'est pas aperçue et qui a provoqué la rupture de synchronisme information cliché. Il faut donc rattraper le synchronisme comme précédemment en mettant à part ce que l'on a traduit depuis le code cliché précédent.

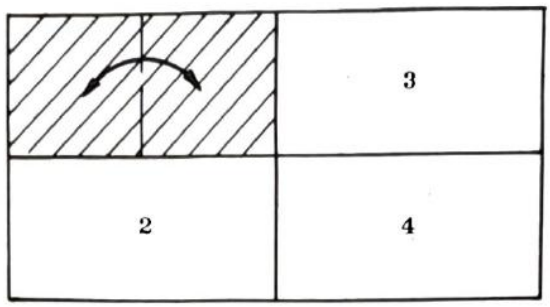
b) le code cliché imprévu est le code erreur que la dactylo a perforé lorsqu'elle s'est aperçue qu'elle a fait une erreur plus importante que le dernier caractère frappé (dans ce cas, on aurait simplement le code effacement). Ce code signifie que l'on annule tout ce qui précède depuis un endroit donné et que l'on recommence à partir de cet endroit selon les consignes reçues. Il faut reprendre le cliché de traduction aussi à cet endroit en annulant la traduction précédente.

Dans certains cas, les consignes de perforation prévoient de perforer après le code erreur, le code cliché correspondant à l'endroit où l'on reprend la perforation. Le programme de traduction doit alors l'analyser pour savoir où reprendre.

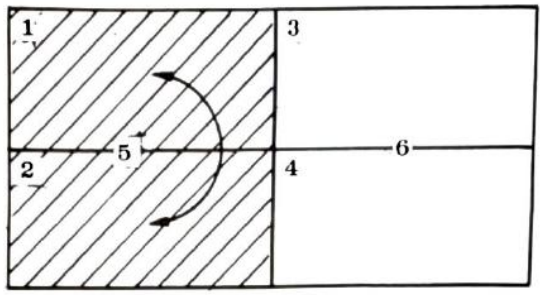
5.5.- D'autres TOC tiennent à l'existence de *codes à maintien*. Ils provoquent ce que l'on appelle des *basculements de la MIC*.

5.3.1.- Basculements MIC provoqués par le code externe. En effet, sur la bande perforée 5 canaux, un même code perforé signifie un chiffre ou une lettre selon qu'il fait partie d'un groupe précédé du code à maintien chiffre ou du code à maintien lettre. Comme avec 5 canaux, on ne peut obtenir que 32 combinaisons différentes, la sélection dans la MIC ne pourra se faire que sur 32 mémoires-caractères. C'est ce qui se fait tant qu'on se trouve à l'intérieur d'un groupe de lettres par exemple. Les mémoires sélectionnées contiennent les codes internes des lettres correspondantes. Mais lorsque l'on rencontre le code à maintien "chiffres", on sélectionne un TOC qui fait "basculer" la MIC sur 32 autres mémoires caractères, ce qui veut dire que les codes alors rencontrés sélectionneront à l'intérieur de ces 32 autres mémoires, des mémoires contenant des chiffres en code interne et ceci jusqu'à ce que l'on rencontre le code à maintien "lettre" qui fera à nouveau basculer la MIC dans l'autre sens.

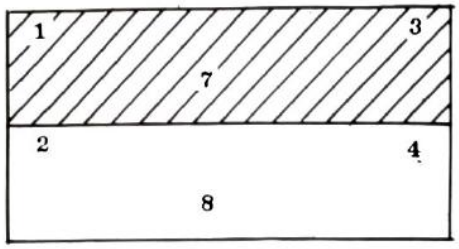
Ainsi, la traduction de bande 5 canaux utilisera l'une des zones 1, 2, 3 ou 4 de la MIC avec basculement entre l'une et l'autre moitié de cette zone.



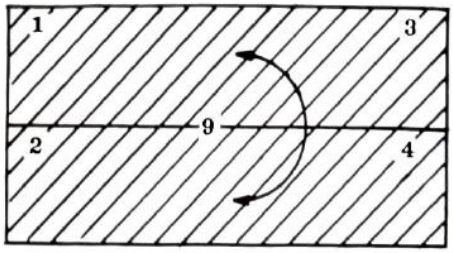
De même, avec la bande 6 canaux avec code à maintien, il y aura un basculement de la MIC entre les zones 1 et 2 ou 3 et 4. Ce cas d'utilisation de la MIC correspond au code 5 ou 6 placé dans la zone OD-MIC de la directive "cas d'emploi" (cf. § III). Le TOC qui produit cette bascule est, bien entendu, différent de celui qui produit la bascule dans le cas de la bande 5 canaux.



Dans le cas de la bande 8 trous sans code à maintien, la sélection se fait sur les 128 mémoires de la MIC (le canal de parité n'entre pas dans la sélection). On utilise alors soit le haut, soit le bas de la MIC, ce qui correspond aux codes 7 et 8 dans l'OD-MIC de la directive cas d'emploi.



Enfin la bande 8 trous avec code à maintien (majuscule, minuscule) utilise un basculement de la MIC entre ces zones 7 et 8. Ce cas est codé 9 dans l'OD-MIC.



5.3.2.- Basculements MIC provoqués par le code interne. Tout ceci concerne les codes à maintien que l'on peut trouver sur le support externe qu'est la bande perforée. En code interne, où l'on a 64 possibilités, les lettres et les chiffres sont codés différemment mais il n'y a aucune distinction entre majuscules

et minuscules. Si l'on veut garder cette distinction entre code interne (pour ressortir plus tard sur bande perforée), il est nécessaire d'avoir aussi en code interne des codes à maintien, ce sont les codes 29 et 30 que l'on appelle encore 64 et $\overline{64}$ parce qu'ils correspondraient au poids 64 ou à son absence dans un code à 7 positions.

Dans ce cas, la rencontre d'un code 64 ou $\overline{64}$ dans les cas d'emploi "Traitement vers Carte" ou "Traitement vers Imprimante" est purement et simplement non prise en considération puisqu'il n'y a pas de distinction majuscule-minuscule sur les supports externes correspondants. En revanche, si l'on sort sur une bande perforée où la distinction existe, ce code à maintien 64 ou $\overline{64}$ provoque le basculement de la MIC.

Pour la sortie sur bande perforée 5 trous où il y a une distinction chiffres-lettres grâce à des codes à maintien, on n'utilise qu'un quart de la MIC sans basculement puisque, en code interne, on a les 64 codes correspondants aux 64 mémoires-caractères. Cependant, lorsque l'on passe de la catégorie chiffres à la catégorie lettres, il faut injecter sur la bande le code à maintien correspondant. C'est le Traducteur qui en prend l'initiative de la façon suivante : dans les mémoires caractères, la 7ème position (avant les positions 8 et 9 des clefs de parité) sert à repérer s'il s'agit d'un code de la catégorie lettres ou de la catégorie chiffres; le Traducteur teste cette position et chaque fois qu'il constate un changement de catégorie, il injecte à la sortie le code à maintien correspondant.

5.4.- Résumé sur les codes à maintien

.Différents codes existants

I) en externe (exclusivement sur bande perforée) :

.Bande perforée 5 canaux, codes à maintien lettres ou chiffres

.Bande perforée 8 canaux, codes à maintien majuscules-minuscules

II) en interne :

Uniquement code à maintien Majuscules-chiffres ou code à maintien minuscules + certains signes particuliers. Ces codes (6 positions) sont respectivement les codes 29 et 30 que l'on appelle encore 64 et $\overline{64}$ parce qu'ils correspondraient à l'absence ou à la présence d'un poids 64 si le code interne Gamma 60 était à 7 positions.

.Fonctionnement

I) en code externe :

.à l'entrée (cas d'emploi bande vers traitement) : ils sont interprétés par les TOC correspondants et provoquent le basculement de la MIC comme expliqué ci-dessus.

.à la sortie (cas d'emploi traitement vers bande) : les codes à maintien sont créés sur la bande à partir de la reconnaissance de la 7ème position de la mémoire-caractère qui spécifie la catégorie ("lettre ou chiffre", "majuscule ou minuscule") à laquelle appartient le code. Chaque fois que l'on change de catégorie, le Traducteur injecte le code à maintien correspondant (qui se trouve dans les adresses 29 ou 30 de la MIC).

II) en code interne :

.à l'entrée (du Traducteur) : le code 64 ou $\overline{64}$ est détecté automatiquement et provoque le basculement de la MIC (comme avec un TOC) dans le cas d'emploi "Traitement vers bande". Il est purement et simplement supprimé dans le cas d'emploi "Traitement vers Carte" et "Traitement vers Imprimante".

.à la sortie (du Traducteur, dans le cas d'emploi Bande vers Traitement). C'est encore la reconnaissance de la 7ème position de la mémoire-caractère sélectionnée qui permet au Traducteur d'injecter le code 64 ou $\overline{64}$ lorsqu'il reconnait que l'on passe de la catégorie majuscule ou minuscule et inversement.

. .il convient d'ajouter qu'en code interne, le basculement reprend systématiquement en $\overline{64}$ à la fin de chaque mot. Ceci veut dire que si deux mots se suivent en "minuscules" tous deux, ils sont tous deux précédés du code 64 alors que seul le premier l'est sur la bande perforée.

III. LES DIRECTIVES DU TRADUCTEUR

		TO			CC	X				FA
						V	0_1	0_2	R	
	Garnissage MIC	G	M	I	G D					
	Cartes BULL vers Traitement	C	A	T	P	S ou N ou –	(0 D– MIC) 1 à 9			
	Cartes IBM vers Traitement	C	E	T						
	Cartes BULL vers Mémorisation	C	A	M						
	Cartes IBM vers Mémorisation	C	E	M						
	Traitement vers cartes BULL	T	C	A						
	Bande perforée vers Traitement	B	P	T						
	Traitement vers Bande Perforée	T	B	P						
	Traitement vers Imprimante	T	I	P						
	Traitement vers Mémorisation	T	V	M						
	Mémorisation vers Traitement	M	V	T						
	Mémorisation vers Mémorisation	M	V	M						
	Renvoi au cliché	R	C	L	G –	E S –				
	Fin de Traduction	F	I	N	R					

1. Structure des directives

On distingue 3 sortes de directives :

a) Le garnissage MIC : GMI

Cette directive permet de garnir la moitié gauche ou droite de la MIC (mémoires caractères 0 à 63 ou 64 à 127) d'une table de transcodage préalablement constituée en Mémoire Rapide sur 64 catènes.

On indique G ou D en première colonne de code complémentaire selon que l'on veut garnir la partie gauche (parties 1 et 2) ou droite (parties 3 et 4) de la MIC, On doit regarnir la MIC chaque fois que l'on veut changer son contenu, mais non avant chaque utilisation du Traducteur. En effet, elle est rémanente, même à une libération du Traducteur.

b) Les directives cas d'emploi

Ce sont des directives préparatoires qui ont pour effet d'effectuer un certain nombre de positionnements dans le Traducteur. On a suffisamment expliqué ce que l'on entend par cas d'emploi, pour ne pas avoir besoin de décrire en détail chacune d'elles :

.Le TO désigne le cas d'emploi mais, de plus, précise, dans les cas où la carte est en jeu, s'il s'agit de carte Bull ou IBM.

.La première colonne de code complémentaire indique sous quelle forme l'information est traitée :

P : standard programmée
N : standard normalisée
Rien : forme non standard. Aucun automatisme, le Traducteur doit être programmé de façon entièrement analytique.

.La seconde colonne de code complémentaire n'est utilisée que dans les cas d'emploi où la carte perforée est en jeu et où les nombres sont en forme standard. Dans ces cas, le signe est superposé (11 ou 12) au premier chiffre; on veut laisser la latitude à l'utilisateur de perforer ou non le signe +. S dans cette colonne signifie "signe + perforé", rien signifie "signe + non perforé".

.Enfin, dans la zone V, on précise par un chiffre de 1 à 9 la zone de MIC que l'on désire utiliser. La signification de ces différents codes a été donnée au § II.5.

c) *Les directives d'exécution*

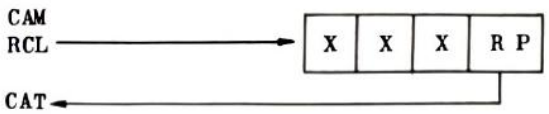
Les directives cas d'emploi étant des directives préparatoires, elles ne renvoient pas directement au cliché. A cet effet, elles doivent donc être suivies d'une directive d'exécution RCL qui permet au Traducteur de fonctionner de façon autonome sur le cliché. Il n'y a rien à indiquer dans les codes complémentaires de cette directive, sauf si l'on utilise le système de "datum de reprise" (cf.V) : G en premier code complémentaire signifie alors "avec gestion du datum de reprise" et E ou S en 2ème code complémentaire précise alors si ce système est utilisé à l'entrée ou à la sortie du Traducteur.

A la fin d'une séquence de traduction, on doit écrire une directive de Fin de Traduction (FIN) avant la coupure appelant l'élément suivant. Les raisons de cette directive seront données au § III.2. En premier code complémentaire, on doit indiquer R (Rangement du Datum de reprise) si l'on a fonctionné en utilisant le Datum de Reprise.

2. Remarques sur l'utilisation des directives

a) Le passage d'une directive cas d'emploi à une autre provoque l'arrondi au catène du registre de sortie et son éjection en Mémoire Rapide, lorsque la forme traitement se trouve dans la deuxième.

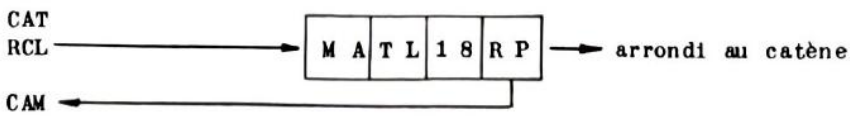
Exemple :



provoque l'arrondi au catène.

Cependant, si la forme traitement se trouve à la sortie dans la première, c'est le cliché de retour au programme lui-même qui provoque l'arrondi.

Exemple :



Ceci permet de dire d'une façon générale que le *passage d'une directive cas*

d'emploi à une autre, lorsque l'on sort du Traducteur en code interne *s'accompagne d'un arrondi au catène* du registre de sortie (voir exception en d).

Lorsque la forme traitement est à l'entrée, la question ne se pose pas pour le registre d'entrée puisque chaque mot est naturellement arrondi au catène.

b) De la remarque précédente, il découle la règle suivante : *les instructions d'adresses doivent être placées entre la directive cas d'emploi et la* RCL. En effet, supposons la séquence :

```
M V M
    G 1     1000
R C L
    G 1     2000
M V T
```

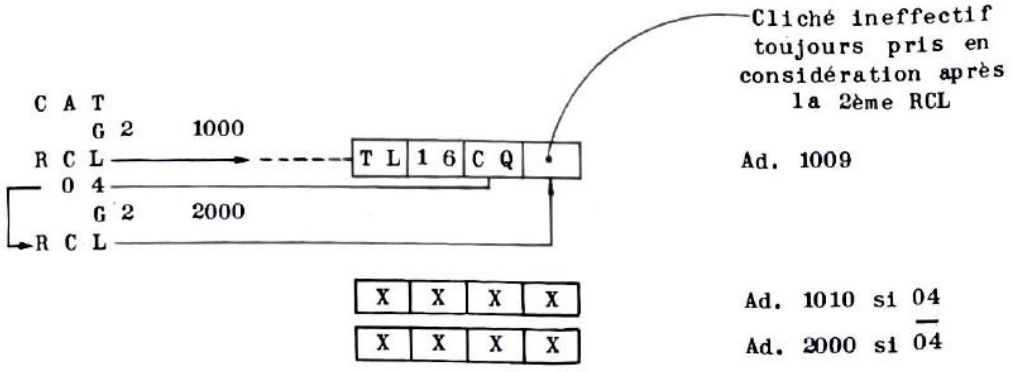
et supposons que l'on revienne au programme sur la 2ème instruction G1 lorsque le registre sortie est à moitié rempli, et que le dernier catène a été rangé à l'adresse 1011.

Puisque l'on se trouvait en Mémorisation à la sortie, le cliché RP ne provoque pas l'arrondi au catène. C'est seulement la directive MVT qui le fait alors que l'on a déjà garni le RAC 1 de l'adresse 2000, ce qui fait que le résidu de la traduction effectuée au cours de la MVM précédente est rangé en 2000 et non en 1012 comme on voulait le faire. Il faut donc programmer :

```
M V M
    G 1     1000
R C L
M V T
    G 1     2000
```

c) La directive RCL renvoie dans le registre cliché, au code cliché désigné par l'index. Ce n'est pas le fait de revenir au programme par un cliché RP ou CQ (ce dernier envoyant en outre le caractère entrée dans le catène qualitatif) qui remet à zéro le registre cliché. Donc, si l'on revient au programme pour modifier un RAC d'entrée ou de sortie, ou pour changer de cas d'emploi, on peut écrire les codes clichés de la suite de la traduction directement après le cliché de retour au programme même si ce dernier n'est pas le dernier d'un catène. La RCL de retour au cliché après ces modifications reprendra toujours le cliché aussitôt après le code de retour au programme.

Il en résulte que si le retour au programme a pour but d'effectuer une variante sur le cliché par jeu d'adresse sur le RAC 2, le catène cliché contenant le cliché de retour au programme doit être complété par des clichés ineffectifs :



d) La FIN joue plusieurs rôles

.elle éjecte le registre sortie (ou la totalité de la matrice de transposition

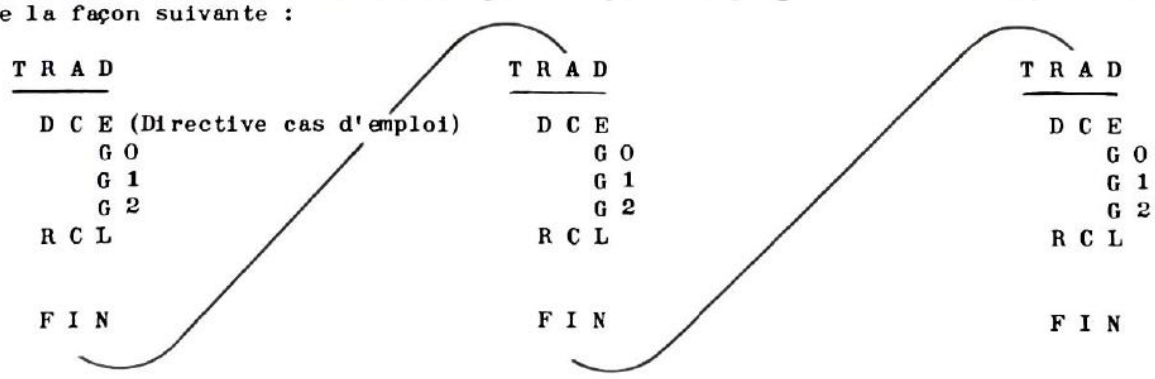
en TCA) lorsque la forme traitement n'est pas en sortie.

.elle efface toutes les rémanences du Traducteur sauf la MIC et les RAC. Cet effacement des rémanences concerne notamment la remise à zéro des index des registres entrée, sortie, cliché et de la matrice de transposition.

Cette directive ne libère pas le Traducteur, ce rôle étant dévolu, comme pour tous les éléments, à la coupure qui suit pour appeler un autre élément. On aurait pu concevoir qu'au reçu de cette coupure, le Distributeur de Programme envoie à l'élément demandeur (le Traducteur) un message qui aurait eu le même rôle que la directive FIN; cette organisation aurait été valable si ce rôle s'était borné à remettre les index à zéro. Mais il s'agit aussi de renvoyer en Mémoire Rapide le résidu de la Traduction qui se trouve dans le registre de sortie ou la matrice de transposition. Pour cela, le Traducteur doit faire des demandes de transfert de quantité à l'unité centrale mais l'élément demandé par la coupure étant alors au travail, il risquerait de se trouver en compétition avec ce dernier sur les circuits de priorité et surtout d'être minoritaire par rapport à lui. L'élément demandé pourrait alors alimenter comme opérandes le contenu d'adresses de Mémoire Rapide qui, normalement, devraient être les résultats de la traduction précédente, mais qui ne le seraient pas puisque le Traducteur ne les y aurait pas encore rangées. C'est pour cette raison qu'il est nécessaire d'avoir une directive effectuant ce rangement avant la mise au travail de l'élément suivant par la coupure.

Ce problème ne se pose que pour le Traducteur : dans le calculateur arithmétique ou le calculateur logique ou le comparateur général, le résultat est envoyé en entier en Mémoire Rapide à la fin de l'exécution de la directive et avant la demande d'instruction qui provoquera l'appel de la coupure sur un autre élément.

On doit donc toujours mettre une FIN à la fin d'une séquence de traduction avant la coupure sur l'élément suivant. Mais il y a plus : il peut même être nécessaire d'en placer au cours d'une séquence de traduction : c'est lorsque cette séquence élémentaire, relative à l'élément traducteur, se compose *fonctionnellement* en fait de plusieurs "traductions" que l'on pourrait programmer chacune séparément de la façon suivante :

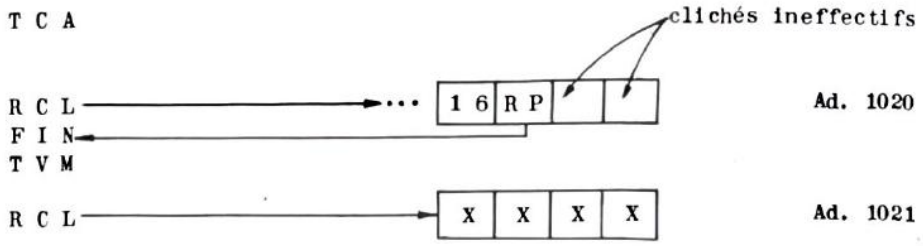


Si, fonctionnellement, on désire faire ces 3 traductions l'une après l'autre au cours d'une même séquence de programme, on écrira simplement ces 3 séquences l'une après l'autre comme indiqué par les flèches ci-dessus. Si, de plus, on ne désire pas que le Traducteur puisse se libérer en cours de route pour prendre éventuellement un autre travail sur une autre séquence simultanée à celle-ci, on supprimera même les deux dernières instructions de coupure réalisant ainsi une séquence élémentaire au cours de laquelle on aura deux directives FIN en plus de celle qui la termine. Ces deux directives sont-elles utiles ? Généralement oui, puisque s'agissant de trois "traductions" *fonctionnellement* différentes quoique regroupées en une même séquence élémentaire, il faut, au départ de chacune d'elles, effacer les rémanences du Traducteur. Toutefois si, par le jeu normal du fonctionnement de celui-ci, il se trouve que seul le registre de sortie ne soit pas remis à zéro à la fin d'une "traduction", la directive cas d'emploi qui commence nécessairement la "traduction suivante" y suffit généralement (cf. § "a"). Dans ce cas, on peut, au

cours de la séquence élémentaire, supprimer également les FIN. Mais, lorsque l'on est dans un cas d'exception à la règle du § "a" ou que l'on a à effacer d'autres rémanences que le registre de sortie (matrice de transposition par exemple), la directive FIN est nécessaire.

Un exemple qui constitue une exception à la règle du § "a" illustrera ce qui vient d'être dit : supposons que l'on dispose d'un article à l'état Traitement et que l'on veuille en sortir une partie sur Carte, Bande ou Imprimante et stocker le reste en Mémorisation. A la fin de la traduction de la partie que l'on veut sortir sur support externe, il se peut que l'on ne soit pas arrivé à la fin du registre de sortie ou de la matrice de transposition. Ce n'est pas le cliché de retour au programme (pour changer de cas d'emploi) qui l'éjectera en Mémoire Rapide puisqu'on n'est pas en forme traitement à la sortie du Traducteur. Ce n'est pas non plus la TVM que l'on trouvera qui jouera ce rôle (et pour la même raison). Sortie sur support externe et mise à l'état Mémorisation constituent bien deux "traductions" différentes; on se trouve de plus dans une exception à la règle énoncée en fin du § "a"; le changement de cas d'emploi ne s'accompagne pas de l'éjection de la sortie. Pour jouer ce rôle, il faut placer avant la TVM une FIN. Il ne faut pas oublier qu'alors le registre cliché est lui aussi remis à zéro et, en conséquence, écrire le cliché à exécuter dans le cadre de la TVM dans un nouveau catène même s'il restait de la place à la suite du RP qui renvoyait à la FIN.

La programmation sera :



IV. LES CODES CLICHES

Clichés			Fonctions
Positionnement	Principal	M A	Mot Alphanumérique (6 monades en code interne)
		M N	Mot Numérique (4 monades en code interne)
		N A	Nombre Arithmétique (semi-log., avec Exp. mais sans signe)
		N S	Nombre avec Signe (semi-log. pour Traitement en C.A.)
		B N	Binaire (1 monade)
	Secondaire	C E	Numérique avec Certification
		T Z	Numérique avec codage de tous les zéros
Execution		T U	Transfert d'un caractère
		T L	Transfert avec longueur (de 2 à 63 caractères)
		E U	Annulation d'un caractère à l'entrée
		E L	Annulation de l'entrée avec longueur (de 2 à 63 caractères)
		S U	Injection d'un espace à la sortie
		S L	Injection d'espaces à la sortie avec longueur (2 à 63)
		K U	Injection d'une constante
		K L	Injection de constantes avec longueur (de 2 à 63 constantes)
		R U	Injection d'une constante sans avancer la sortie
		R L	Injection de n constantes (2 à 63) sans avancer la sortie
		R P	Retour au Programme
		C Q	Envoi d'un caractère dans le catène qualitatif

1. Les codes clichés en code B

On a vu qu'en code A, chaque catène contenait 4 codes-clichés ou longueurs. En code B, la nature de chaque code-cliché est indiqué par 2 lettres (cf. tableau ci-dessus), les longueurs sont indiquées en numérique décimal par 2 chiffres. Par exemple :

T	L	5	1	E	L	2	4

sera finalement contenu dans un catène en code A.

On ne peut écrire qu'une longueur au plus égale à 63. Si l'on a besoin d'indiquer une longueur plus grande, il faut faire comme en code A, c'est-à-dire répéter le cliché avec des longueurs telles que leur somme soit égale à la longueur totale désirée. Par exemple, pour la longueur 133, on écrira :

T	L	6	3	T	L	6	3
T	L	0	7	X		X	

2. Les clichés de Positionnement

Les directives cas d'emploi effectuent déjà un certain nombre de positionnements, mais c'est insuffisant. Il faut encore préciser la nature des mots à traduire. Par exemple, en CAT, il est fort possible de traduire une zone de carte ne contenant que des chiffres en code 4 positions ou en code 6 positions, selon que l'on précise dans le cliché le positionnement MN ou MA. On distingue des clichés de positionnement principal et de positionnement secondaire.

a) Les clichés de positionnement principal

.Ils désignent la nature du mot à traduire (cf. § I) : Alphanumérique, Numérique, Nombres (arithmétique ou avec signe), Binaire.

Les positionnements NA et NS permettent une programmation semi-automatique grâce aux formes standard (programmée ou normalisée).

.Ils sont valables jusqu'au prochain cliché de positionnement rencontré.

.Ils effectuent l'arrondi au catène sur le code interne du positionnement précédent (à condition que l'on soit à l'état traitement). Si ce dernier est NA ou NS, l'arrondi s'effectue éventuellement à 2 catènes, de façon à se cadrer toujours sur les deux catènes du nombre. Ces arrondis rendent inutiles (dans ce cas) les annulations d'entrée ou de sortie.

b) Les clichés de positionnement secondaire

.Ils ne peuvent être employés qu'après un positionnement principal MN, NA ou NS préalable.

.CE : le code certification est injecté devant le premier chiffre significatif sauf si la longueur réelle est égale à la longueur théorique. Il doit précéder immédiatement le cliché d'exécution TL et n'est valable que pour la longueur correspondante.

.TZ : le Traducteur supprime normalement les zéros à gauche, et ne code que les zéros à droite pour l'impression; ce cliché élimine cette suppression, tous les zéros sont codés. Il est valable pour tous les clichés d'exécution qui suivent et ne retombe qu'au prochain positionnement principal. Sans lui, la mémoire de codification des zéros à droite se maintient jusqu'au prochain positionnement principal, ce qui permet par exemple de pouvoir injecter des blancs ou des points entre tranches de 3 chiffres en ne codant que les zéros à droite par une suite de clichés TL - 03 - KU - . - TL - 03 - KU - . - etc ...

3. Les clichés d'exécution

Par eux-mêmes, les clichés de positionnement ne déclenchent aucun travail.

Ils doivent être suivis de clichés d'exécution en général avec longueur.

.TU ou TL permettent d'exécuter la traduction selon les positionnements prévus dans la directive cas d'emploi et les clichés de positionnement. Par exemple, après une CAT :

M A	T L	1 8	M N

provoquera la traduction de 18 colonnes de carte en code interne 6 positions et sur la longueur théorique 18. MN effectuera l'arrondi au catène.

Mais, après une directive CAM, l'index d'entrée avancera toujours de 18 colonnes, tandis qu'à la sortie, le mot sera en longueur réelle (en code 6 positions) et suivi du code tabulation (code 31). MN ne provoquera pas l'arrondi au catène, le mot suivant viendra se placer immédiatement à la suite.

.EU – EL : Annulation de 1 ou plusieurs caractères à l'entrée. Selon les cas d'emploi et les positionnements, permet de sauter à l'entrée une zone de carte ou des caractères en code interne, et ceci sans avancer la sortie.

.SU – SL : Injectent un ou plusieurs espaces à la sortie. Provoquent ainsi un avancement de la sortie sans avancement de l'entrée.

.KU – KL : Injection de constantes.

Après KU, on désigne immédiatement la constante à injecter. Après KL, on indique d'abord le nombre de constantes à injecter, puis les constantes elles-mêmes (lettres, chiffres ou signes spéciaux), à raison d'une par code cliché. Les constantes sont injectées dans le code défini par la directive cas d'emploi et le cliché de positionnement. Par exemple : injection de la constante numérique 219 (3 caractères) :

M N	K L	0 3	2
1	9		

En positionnement binaire, seule la longueur figure, la constante injectée étant systématiquement 1. Par exemple, pour injecter 19 en binaire, soit 1 0 0 1 1, on écrira :

B N	K U	S L	0 2
K L	0 2	x	x

On remarquera que les longueurs inférieures à 10 sont toujours représentées par 01, 02 ..... 09 pour respecter la règle de programmation en code B qui veut que les codes-clichés et les longueurs soient représentés par 2 caractères (lettres ou chiffres) afin de les distinguer des constantes à injecter toujours représentées par 1 caractère (lettre, chiffre ou signe retenu pour l'imprimante).

.RU et RL : permettent la superposition d'un ou plusieurs poids binaires dans un caractère de sortie.

RU est particulièrement employé pour injecter le signe dans la première monade d'un nombre lorsque l'on est en programmation analytique non standard. On programme RU - 8 pour le signe -. On injecte ensuite l'exposant par KL 02 suivi de sa valeur sur 2 clichés.

RL est employé pour injecter dans une même colonne de carte une série de perforations dont la combinaison ne correspond à aucun caractère. Soit par exemple à injecter une perforation en lignes 9, 7, 4 et 12 d'une même colonne de carte ; on écrira :

R L	0 3	9	7
4	K U	•	x

On injecte les 3 premières constantes 9,7 et 4 par RL 03 et la dernière par un KU pour faire avancer la sortie. L'ordre d'injection des constantes importe d'ailleurs peu ; on aurait pu écrire aussi bien :

R L	0 3	*	9
7	K U	4	x

On remarquera que l'on écrit * et non pas 12. En effet, si une perforation en ligne 9 signifie bien 9, en ligne 7 signifie 7, une perforation en ligne 12 signifie * et il faut l'écrire ainsi en code B. De ce fait, est respectée, la règle qui veut que toute constante soit désignée par un seul caractère : 12 aurait signifié une longueur et, après KU, cela n'aurait eu aucun sens.

De même qu'une perforation en ligne 12 correspond à * , une perforation en ligne 11 correspond à • .Ainsi les signes utilisés pour injecter des perforations dans les différentes lignes d'une même colonne de carte sont :

9, 8, 7, 6, 5, 4, 3, 2, 1, 0, ., *

correspondant aux lignes :

9, 8, 7, 6, 5, 4, 3, 2, 1, 0, 11, 12.

On remarque que tous ces caractères correspondent chacun à une seule perforation. En effet, RU et RL, qui permettent l'injection en superposition de caractère, ne peuvent injecter qu'une monade à la fois.

. RP : retour au programme. Outre sa fonction de retour au programme, il joue le rôle d'un cliché d'exécution en effectuant l'arrondi au catène du registre de sortie (capacité pratique du mot), si l'on était en "Traitement" à la sortie (c'est même l'arrondi à 2 catènes si le positionnement précédent était NA ou NS).

. CQ : envoi du caractère entrée tel quel dans le catène qualitatif et renvoi au programme automatiquement. L'entrée est avancée automatiquement sauf en CAT ou CAM pour permettre l'exploitation d'un code éventuellement superposé au premier caractère d'un mot. Comme RP, il effectue l'arrondi au catène du côté sortie en code interne. De ce point de vue, il joue aussi le rôle d'un cliché de positionnement.

V – COMPLEMENTS SUR LA TRADUCTION DES NOMBRES

1. Lorsque l'on est dans une *forme standard*, le fonctionnement du Traducteur est automatique dès lors que l'on a positionné en NA ou NS. Signe et Exposant sont transférés et transcodés automatiquement selon les formes standard prévues au § 1. Il suffit de programmer le transfert de la mantisse par un TL + longueur.

Toutefois, lorsque l'on passe d'une forme standard programmée en code externe ou en mémorisation (où l'exposant ne figure pas) vers la même forme programmée mais à l'état traitement (où l'exposant doit figurer), il faut injecter l'exposant : ceci se fait en écrivant purement et simplement sa valeur sur 2 clichés entre le cliché NA ou NS et le TL + lg de la mantisse. Par exemple :

N S	4		8	T L
0 7	M A		X	X

l'arrondi se fait automatiquement à 2 catènes même si la mantisse avait une longueur inférieure ou égale à 4. Il faut noter que l'exposant à injecter est l'exposant réel augmenté de 40 pour avoir directement la représentation utilisée par le C.A.

2. Lorsque l'on est en forme quelconque (non standard), le fonctionnement du Traducteur n'est nullement automatique. Il faut programmer la traduction des nombres de façon entièrement analytique.

Par exemple, si l'on a sur une carte un nombre de 3 chiffres dont on sait qu'il est négatif mais que le signe n'est pas perforé et que la virgule se trouve après le 2ème chiffre, on devra, après la directive CAT (rien en code complémentaire), se placer en positionnement MN, injecter le signe - par RU - 8, injecter l'exposant 42 par KL - 02 - 4 - 2, puis transférer la mantisse par TL 03, et si l'on veut que le positionnement suivant fasse l'arrondi sur le 2ème catène, faire avancer la sortie d'au moins 2 et au plus 7, pour se trouver à l'intérieur du 2ème catène, puisque, n'étant pas en standard, l'arrondi se fait simplement sur le catène :

M N	R U	8	K L
0 2	4	2	T L
0 3	S L	0 2	X

Cette méthode, pour correcte qu'elle soit, est lourde. On peut remarquer, en effet, que si ce nombre est toujours négatif, on peut le traduire comme un nombre positif et ultérieurement, au cours des calculs dont il constitue un opérande, transformer une ADD en SOU ou l'inverse. On le traduira alors par le cliché qui bénéficie de l'automatisme dû aux formes standard :

N A	4	2	T L
0 3	X	X	X

Grâce à NA, pas besoin de KL avant l'exposant ni de SL à la fin, l'arrondi se faisant automatiquement à 2 catènes.

Enfin, si sur une carte un signe est perforé sur une colonne par la combinaison 9-8-2 pour - ou 9.8.3 pour + suivi des chiffres sur les colonnes suivantes, il n'y a aucune difficulté à considérer ce nombre comme un mot alphanumérique et à le traduire comme tel en code 6 positions, mais ceci suppose que l'on n'aura aucun calcul à faire ensuite avec lui. Si l'on veut le traduire sous la forme Traitement, comme on n'est pas en standard, il faut procéder de façon analytique et pour ce faire :

.envoyer la colonne signe dans le catène qualitatif par CQ,

.tester s'il s'agit d'un signe - (présence de la perforation 2) par un branchement 07,

.dans l'affirmative, injecter le signe par RU - 8 précédé de MN,

.dans tous les cas, injecter l'exposant par KL 02 + EXP précédé de MN,

.enfin, après avoir avancé l'entrée par SU (CQ n'avance pas l'entrée sur une carte), transférer la mantisse.

Normalement, les règles qui viennent d'être données suffisent, mais il a semblé préférable de résumer tous les cas que l'on peut rencontrer dans le tableau ci-après.

Cas d'emploi	CC	Positionnement	Traitement du signe			Traitement l'exposant				Transfert Mantisse	Arrondi à 2 catènes entr. ou sort.	
			Injection Programmée	Transfert automatique	Annulation automatique	Injection Programmée	Transfert automatique	Annulation Programmée	Annulation automatique		A Programmer	Automatique
CAT	-	MN	RU-8			KL-02 + EXP				TL + lg	SL + lg (1)	
BPT	P	NA			X	EXP				TL + lg		X
		NS		X		EXP				TL + lg		X
MVT	N	NS		X			X			TL + lg		X
CAM MVM	-	MN								TL + lg		
	P	NS		X						TL + lg		
	N	NS		X			X			TL + lg		
TCA TBP	-	MN						EL-02		TL + lg	EL + lg (1)	
		NA			X				X	TL + lg		X
TIP (2)	P	NA			X				X	TL + lg		X
		NS		X					X	TL + lg		X
TVM	N	NS		X			X			TL + lg		X

(1) La raison de SL + lg, a déjà été expliquée; de même pour arrondir à 2 catènes l'entrée traitement, il faut en MN quelconque être sûr de s'être déjà avancé dans le 2ème catène; d'où EL + lg.

(2) En TIP, forme N, le Traducteur injecte automatiquement un blanc entre le signe et l'exposant, mais si l'on en veut un entre l'exposant et la mantisse ou, en forme P, entre le signe et la mantisse, il faut le programmer par un SU que l'on écrit immédiatement après NS :

N S	S U	T L	0 5		

VI. LE DATUM DE REPRISE

Le datum de reprise a été créé pour permettre une utilisation pratique de la bande perforée. Son emploi peut être étendu à d'autres utilisations comme, par exemple, le remplissage des blocs rubans.

1. Enoncé du problème

Chaque fois qu'on lit de la bande perforée, on alimente en Mémoire Rapide un certain nombre de catènes, nombre qui est indiqué par la directive de lecture. Chacun de ces catènes contient 3 caractères. Ceci signifie que l'on lit la bande par blocs d'un nombre constant de caractères. Du fait que les informations sont enregistrées sur la bande en longueur réelle, on ne peut faire aucune hypothèse quant à leur configuration à l'intérieur de chaque bloc lu en Mémoire Rapide. On ne sait pas à priori où commencent et où finissent les articles. Il se peut entre autres que la lecture de la bande s'arrête au milieu d'un mot.

Ce dernier fait est particulièrement gênant dans le Traducteur car on ne peut arrêter le cliché au milieu d'un mot. On doit donc adopter une certaine méthode de programmation qui permette d'éviter cet ennui mais impose, comme on le verra plus loin, de mémoriser l'état de certains organes du Traducteur pour pouvoir le libérer pendant la lecture du bloc suivant. Ces organes pourront être ainsi remis en état dès que l'on reprend la traduction après alimentation en Mémoire Rapide du bloc de lecture suivant.

2. Méthode de programmation

.On connait la capacité maximum de l'article théorique, soit par exemple 30 catènes.

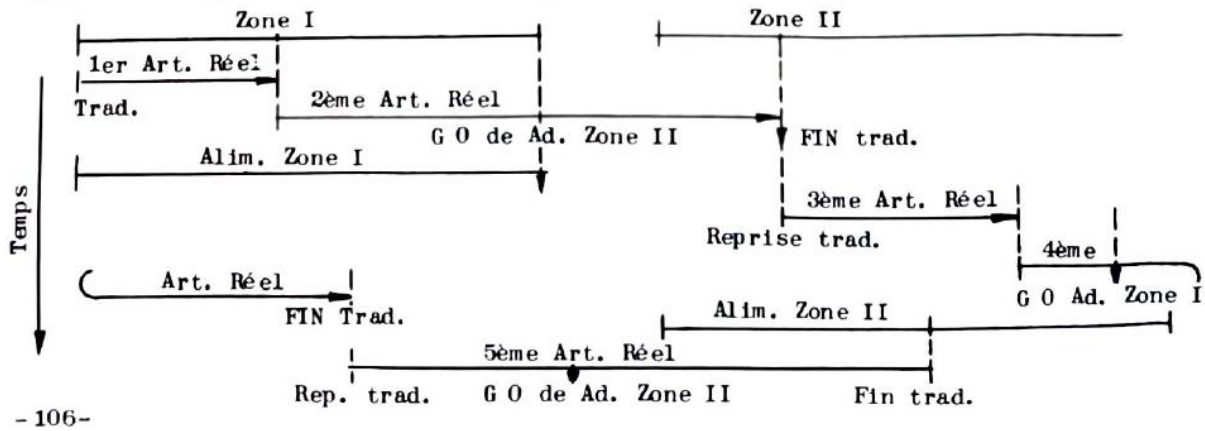
.On réserve en Mémoire Rapide deux zones de trente catènes si l'on ne veut pas de simultanéité entre l'alimentation et la traduction, trois zones dans le cas contraire. Le temps de traduction étant beaucoup plus faible que celui de lecture (de l'ordre de 100 fois moins), on n'a pratiquement aucun intérêt à créer une simultanéité. Nous nous bornerons donc au cas de deux zones.

.On commence la traduction lorsque l'on a alimenté les deux zones et on relance la traduction d'un article tant que l'on n'est pas passé de la zone I à la zone II au cours de la traduction d'un article.

.Dès que ce passage a eu lieu, on n'est plus sûr qu'il reste encore un article dans la zone II. Il faut donc réalimenter la zone I avant de relancer la traduction. Mais la lecture de la bande étant très lente, on ne peut pas immobilier le Traducteur qui doit se mettre au service d'autres séquences. On le libère donc, mais en mémorisant les différents organes nécessaires à la reprise ultérieure de la traduction (ses registres d'entrée et de sortie, ainsi que la position de leurs index, mais non le registre cliché puisque l'on est en fin d'article et que le cliché de traduction d'un article est obligatoirement arrondi au catène).

.Cette méthode exige également que lorsque, au cours de la traduction d'un article, on passe d'une zone à l'autre, on puisse effectuer le changement d'adresse sur le RAC 0.

.L'ensemble des moyens qui permettent la réalisation de ces différentes fonctions s'appelle le "Datum de Reprise".



3. Description organique

On associe au programme de traduction un groupe de 7 mémoires consécutives et constituant à proprement parler le datum de reprise. Ces mémoires peuvent être des mémoires de manoeuvre du sous-programme; nous les repérerons de 1 m à 7 m. Leur contenu est le suivant :

1 m : Longueur de chaque zone (30 cat. dans l'exemple, maximum 255 catènes)
2 m : 0
3 m : 0
4 m : 0
5 m : Instruction de garnissage du RAC 0 avec substitution sur mémoire Y
6 m : Longueur de chaque zone (maximum 255 catènes)
7 m : Instruction de soustraction de 7 sur RAC 3.

On associe, de plus, au programme de traduction 3 autres mémoires, consécutives ou non aux précédentes, et consécutives ou non entre elles. Ce sont :
.la mémoire Y contenant l'adresse de la zone II,
.la mémoire Y' contenant l'adresse de la zone I,
.Une mémoire vierge où l'on rangera à un moment donné le contenu du RAC 0

a) Démarrage de la traduction (cf. organigramme en fin de paragraphe) - On alimente successivement les zones I et II et on démarre la traduction de la zone I par les instructions suivantes :

```
T R A D
  B P T
      G - 0 de l'adresse de la zone I
      G - 1 de l'adresse où l'on veut ranger l'article traduit
      G - 2 de l'adresse du cliché
      G - 3 de l'adresse 1 m (du datum de Reprise)
  R C L - G - E
```

Le fait d'avoir précisé les codes complémentaires G et E dans la directive RCL fait que le Traducteur appelle d'abord les 4 premières mémoires du datum de reprise.

Il garnit alors une mémoire de décomptage interne du contenu de 1 m et les contenus de 2 m, 3 m et 4 m (soit 0) sont respectivement envoyés dans les registres d'entrée et de sortie et les mémoires de positionnement de leurs index, ce qui aurait pour effet de remettre à zéro toutes les rémanences du Traducteur si cela n'avait pas déjà été fait par une FIN. Le RAC 3 contient alors 5 m.

La traduction des articles de sa zone peut alors commencer normalement. Au cours de cette traduction, la mémoire de décomptage est diminuée de 1 à chaque alimentation de catène à l'entrée. Il faut bien noter que si, au cours de la traduction d'un article, on a des RCL, ce sont des RCL normales dans codes complémentaires.

b) Fin de zone - Il arrive un moment où le décompteur passe par 0 au cours de la traduction d'un article, et même peut être au milieu d'un mot. Le Traducteur se fige alors dans sa fonction de traduction. Le RAC 3 contenant 5 m, il alimente le contenu de cette mémoire qui est une instruction de garnissage du RAC 0 du contenu de Y, c'est-à-dire l'adresse de la zone suivante qui, au départ, est la zone II. Il l'envoie alors au Distributeur de Programme et la lui fait exécuter pour son compte (c'est pourquoi on appelle une telle instruction une instruction dictée).

Le Traducteur alimente ensuite le contenu de 6 m dont il regarnit la mémoire de décomptage, puis le contenu de 7 m qui est encore une instruction dictée qui a pour effet de ramener le contenu du RAC 3 à 1 m.

Il peut alors reprendre la traduction jusqu'à la fin de l'article.

c) *Réalimentation de zone* - Lorsque l'on arrive à la fin de l'article, il faut réalimenter la zone suivante car l'on n'est pas sûr que l'article suivant tiendra tout entier dans la zone où l'on est. Or, au moment du passage du décompteur à 0, un test a été placé en position 18 du catène qualitatif.

Toute traduction d'article doit donc se terminer par un branchement sur cette position. S'il n'y a aucun test, on relance la traduction de l'article suivant. S'il y a un test, on doit réalimenter une zone et pendant ce temps libérer le Traducteur après avoir mémorisé ses rémanences.

Pour cela, on devra :

1) effectuer une bascule sur les mémoires Y et Y'. Y contiendra alors l'adresse où alimenter la zone que l'on devra traduire après celle où l'on s'est arrêté en fin d'article.

2) ranger le contenu du RAC 0 dans la mémoire prévue à cet effet.

3) programmer une FIN avec R en code complémentaire. C'est cette directive qui mémorisera les registres du Traducteur dans les mémoires du datum de reprise prévues à cet effet :
.la longueur résiduelle de la zone contenue dans le décompteur, dans la mémoire 1 m (le RAC 3 contient en effet 1 m à ce moment),
.le registre d'entrée en 2 m,
.Le registre de sortie en 3 m,
.les positions des index de ces 2 registres en 4 m.

On peut alors alimenter une autre zone pendant que le Traducteur libéré est disponible pour travailler éventuellement pour le compte d'autres séquences. A la fin de l'alimentation, on pourra reprendre la traduction après avoir garni le RAC 0 de l'adresse mise précédemment en réserve et le RAC 3 de l'adresse 1 m. La directive RCL-G-E aura pour effet d'alimenter d'abord les 4 premiers catènes du datum de reprise comme au moment du démarrage. Le registre de décomptage sera garni du contenu de 1 m qui est, cette fois, la longueur résiduelle à traduire dans la zone où l'on s'était arrêté. Le registre d'entrée sera garni du contenu de 2 m (ou son dernier état avait été mémorisé), celui de sortie du contenu de 3 m et leurs index positionnés comme indiqué en 4 m.

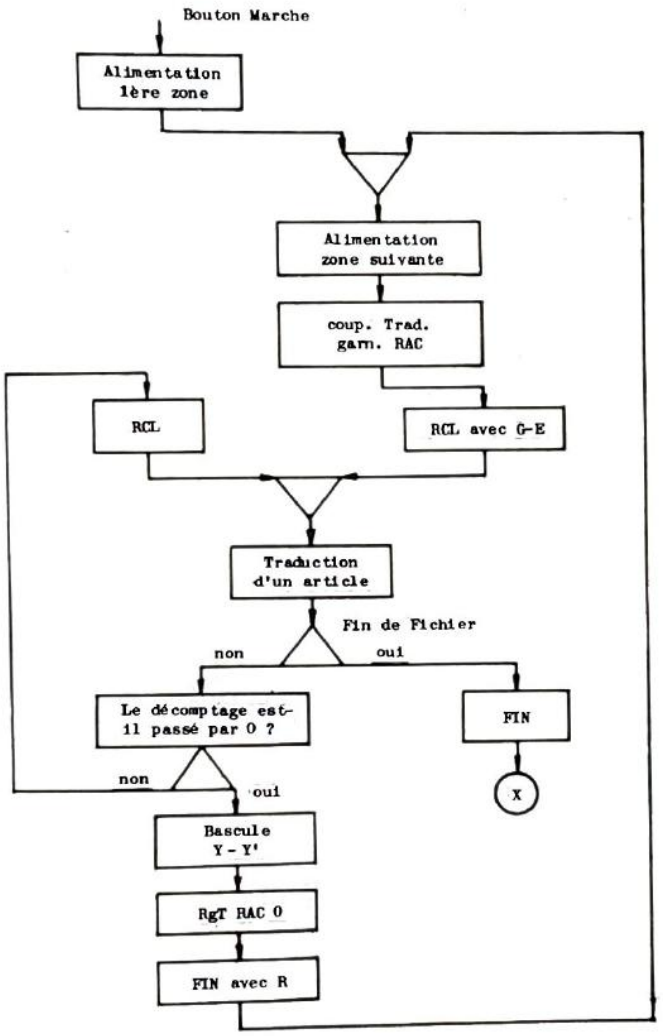
En cours de fonctionnement, la structure du datum de reprise est la suivante :

1 m : Longueur résiduelle
2 m : Contenu du registre d'entrée
3 m : Contenu du registre de sortie
4 m : Index registre d'entrée / Index registre de sortie
5 m : Garnissage RAC 0 du contenu de Y
6 m : Longueur totale de la zone (maximum 255 catènes)
7 m : Soustraction de 7 sur RAC 3.

Remarque : Le "datum de reprise" peut être utilisé pour gérer la sortie du Traducteur lorsque l'on est en TBP pour sortir sur perforateur de bande. A ce moment, l'instruction dictée 5 m porte sur le RAC 1 et c'est ce RAC 1 que l'on doit ranger en fin de traduction d'article après passage d'une zone à l'autre. La directive de départ est alors RCL-G-S (gestion du "datum de reprise" à la sortie).

Le datum de reprise peut, de façon analogue, être utilisé pour gérer l'exploitation des blocs ruban magnétique.

4. Organigramme de Programmation



VII. PROFIL DU CATENE QUALITATIF

Position 1 : Caractère binaire rangé par CQ
1 à 4 compris : Caractère numérique rangé par CQ
1 à 6 compris : Caractère alphanumérique rangé par CQ
1 à 8 compris : Caractère Bande Perforée rangé par CQ
1 à 12 compris: Colonne de carte rangée par CQ
13 : Zone absente; traduction nulle entre deux directives "cas d'emploi" ou entre une "cas d'emploi" et FIN
14 : Mot absent; traduction nulle entre 2 clichés de positionnement principal
15 : Tabulation normale
16 : Tabulation avec retour du chariot
17 : Code cliché imprévu (un test est alors placé en position 24 et le code cliché est envoyé en position 1 à 8)
18 : Passage par 0 du registre de décomptage en utilisation du datum de reprise.

4°) LE CALCULATEUR LOGIQUE

I. FONCTIONS DU CALCULATEUR LOGIQUE

Le Calculateur Logique a deux fonctions principales :

1. Effectuer les 4 opérations arithmétiques sur des nombres binaires purs de 24 chiffres binaires au maximum (1 catène).

2. Effectuer des opérations logiques de découpage et de filtrage d'informations. Ainsi, grâce au Calculateur Logique, on peut manipuler l'information au stade élémentaire de la monade, c'est-à-dire au stade le plus analytique.

Grâce au Calculateur Logique, il est possible de faire des calculs d'adresses compliqués et accomplir toutes les fonctions que nous pouvons imaginer et qui n'ont pas été synthétisées dans des directives centrales soit parce que trop compliquées, soit parce que trop rares. Nous verrons d'ailleurs, à titre d'exemple, comment on peut exécuter la fonction logique de forçage d'un branchement à "toujours" (directive centrale FT).

C'est encore le Calculateur Logique qui est largement mis à contribution dans le programme de "transposition" du code B en code A ou dans le programme d'"Assemblage" de sous-programmes qui permet de réaliser le programme complet d'un problème à partir des sous-programmes rédigés séparément.

II. STRUCTURE DU CALCULATEUR LOGIQUE

Le Calculateur Logique est constitué essentiellement outre le registre de liaison et le registre catène qualitatif de :

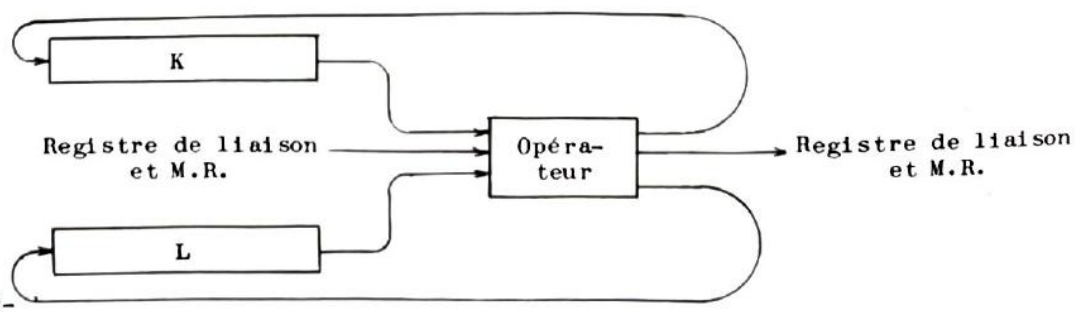
.deux registres à décalage, à accès série et 24 positions binaires chacun. On les dénomme K et L.

.un opérateur.

.4 RAC.

L'opérateur travaille position par position. Il comprend un additionneur-soustracteur binaire, un comparateur et un dispositif de filtrage. C'est à travers lui que sont bouclés les registres K et L et c'est par son intermédiaire que ces registres ont accès (à l'entrée comme à la sortie) au registre de liaison et, par là, aux canaux et à la Mémoire Rapide.

Le schéma fonctionnel du Calculateur Logique est donc le suivant :



Les 4 RAC, numérotés 0, 1, 2 et 3 sont entièrement banalisés et peuvent être avec ou sans progression comme dans le Calculateur Arithmétique.

De plus, de façon analogue à ce qui se passe pour le registre P du Calculateur Arithmétique, le registre L reçoit, en fin d'opération, le résultat et ce résultat peut :

. soit être maintenu dans L,

. soit être envoyé dans une adresse quelconque de la Mémoire Rapide, en indiquant dans la colonne "R" de la directive :

- soit L

- soit le N° du RAC contenant l'adresse M.R. où l'on désire envoyer le résultat (dans ce dernier cas toutefois, il est bien entendu que, comme toujours, c'est un "double" du résultat que l'on envoie en Mémoire Rapide, "l'original" restant dans L et pouvant être éventuellement réutilisé directement comme opérande dans l'opération suivante).

III. DIRECTIVES DU CALCULATEUR LOGIQUE

	TO	CC	X				FA
			V	O_1	O_2	R	
Transfert cadré dans K	TCK		(1)			(2)	
Transfert cadré dans L	TCL					0	
Transfert de K dans L	TKL					1	
Maintien cadré de L	MCL					2	
Insertion de K dans L	IKL		0 à 24			3	
Forçage à zéro dans L	FZL					Z	
Forçage à un dans L	FUL					U	
Addition binaire	ADD					D	
Soustraction binaire	SOU		"1"			T	
Multiplication binaire	MUL					ou	
Division binaire	DIV					L	
Forçage au complément dans L	FCL						
Intersection logique	INT						
Réunion logique	REU						
Comparaison binaire et logique	CBL						

(1) décalage, ou longueur, ou les deux selon les opérations
(2) L n'a aucun sens pour les 2 premières opérations

1. Structure des directives

Un rapide examen de ce tableau des directives fait apparaître leur structure:

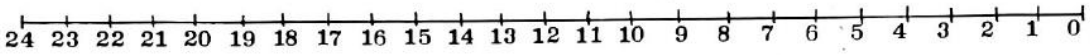
a) Ce sont des directives sans variable libre, donc sans F.A.

b) Ce sont des directives à une adresse. Contrairement à celles du Calculateur Arithmétique dans lesquelles on pouvait désigner, par l'intermédiaire des RAC, à la fois l'adresse des 2 opérandes données (colonnes O_1 et O_2) et l'adresse où ranger le résultat (colonne R), on ne dispose ici que d'une seule colonne (R) pour

désigner l'adresse où l'on veut ranger le résultat de l'opération. Il en découle la nécessité d'avoir des directives de servitude (les deux premières du tableau TCK et TCL) pour amener dans les registres K et L les données de l'opération que l'on désire faire; ces données sont également indiquées au moyen d'un n° de RAC dans la colonne R qui est ainsi, d'une façon tout à fait générale, une colonne opérande (donnée ou résultat). Dans cette colonne, l'adresse de l'opérande sera désignée par le n° de RAC la contenant :

. 0, 1, 2 ou 3, si l'on veut que le RAC progresse
. Z, U, D ou T, si l'on veut que le RAC ne progresse pas.
. si l'on désire laisser le résultat dans L sans le transférer en Mémoire Rapide, on indique simplement L dans cette colonne.

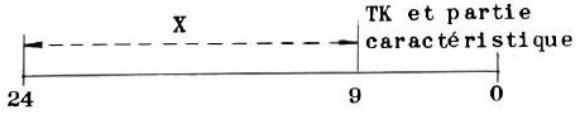
c) En revanche, toute directive comporte, outre la fonction principale par laquelle elle est désignée, des possibilités de filtrage ou de décalage monade par monade. Cette possibilité apparaît dans les 2 colonnes "V" de la partie X où l'on inscrit un nombre "1" de 00 à 24 ; ce nombre désigne soit une longueur (en monades), soit l'intervalle entre deux monades d'un catène, intervalle repéré selon le schéma ci-dessous :



2. *Description des différentes directives et leur utilisation*

a) Transfert cadré dans K (ou dans L) TCK - TCL. Par ces instructions, on alimente dans le registre désigné (K ou L) le catène dont l'adresse se trouve dans le RAC cité. Ceci s'accompagne d'un décalage vers la droite d'un nombre de positions égal à 1, avec maintien du contenu sur tout le registre.

Par exemple, si une telle instruction, avec "15" en 1, porte sur un catène contenant une instruction canonique à variable libre, le profil du contenu final du registre est :



Le code L en colonne R est sans signification dans ces deux directives.

b) Transfert de K dans L (TKL). Le résultat final obtenu est le suivant :

. Transfert de K dans L à partir de la position 0, d'un nombre de positions égal à "1" qui indique donc le nombre de positions de l'opérande.

. Maintien de K à ses cadrages et contenu initiaux.

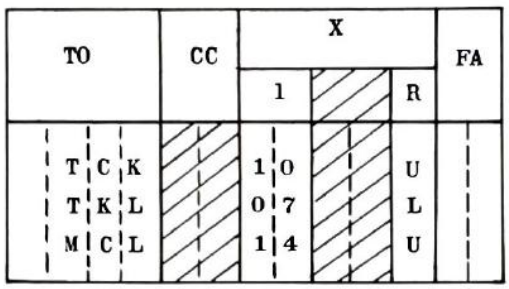
. Effacement de L dans la zone 24 - l et cadrage à droite de la zone l transférée.

De plus, *dans cette opération, comme dans toutes les suivantes sauf la dernière, on peut déclencher le rangement* du contenu final de L en Mémoire Rapide, à l'adresse indiquée par le RAC cité (avec ou sans progression) en colonne R ; le code L dans cette colonne indique qu'il n'y a pas de rangement.

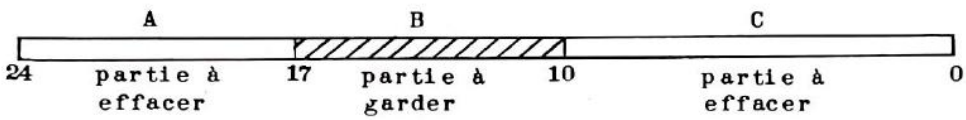
c) Maintien cadré de L (MCL). Le contenu de L est décalé de 1 positions vers la droite, l'ensemble étant maintenu grâce au bouclage de L sur lui-même au cours du décalage.

A titre d'exemple d'utilisation de ces premières instructions, voyons comment, étant donné un catène quelconque en Mémoire Rapide, on peut le maintenir tel quel dans un intervalle donné, de 10 à 17 par exemple, et effacer tout ce qui se trouve en dehors de cet intervalle. Nous supposons que le RAC n° 1 par exemple a été préalablement garni de l'adresse de ce catène.

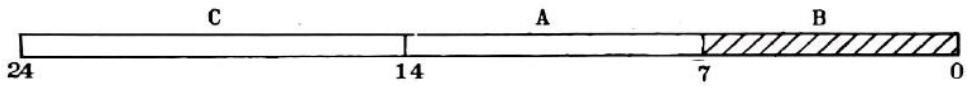
Ceci pourra se faire grâce aux 3 instructions :



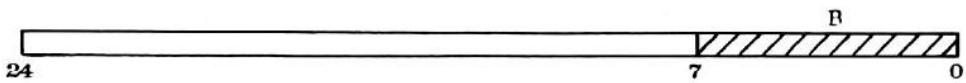
Etat initial du catène en Mémoire Rapide :



Après la TCK, on a dans K :

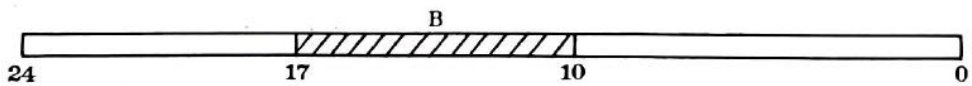


Après la TKL où "1" indique la longueur à filtrer, on a dans L :



Seule la partie B a été transférée dans L, position pour position, le reste de L ayant été effacé (bien noter que l'ancien contenu de L dans les positions 7 à 24 n'était pas A + C). Ce résultat n'est pas transféré en Mémoire Rapide puisque l'on a affiché L en colonne R.

Après la MCL, on a dans L :



Il a fallu afficher 14 en "1" pour effectuer ce cadrage parce que le décalage se faisant vers la droite, il faut pour décaler vers la gauche de "n" positions, effectuer un décalage bouclé sur lui-même de "24 - n" positions vers la droite. Enfin, puisque l'on a indiqué le RAC 1 en colonne R, ce résultat est transféré tel quel à l'adresse qu'il contient, c'est-à-dire toujours celle dont il avait été initialement garni puisque c'est U qui avait été affiché en colonne R du TCK.

On peut remarquer que :

. la TCK prépare le filtrage, son "1" représentant le début de la zone à filtrer.

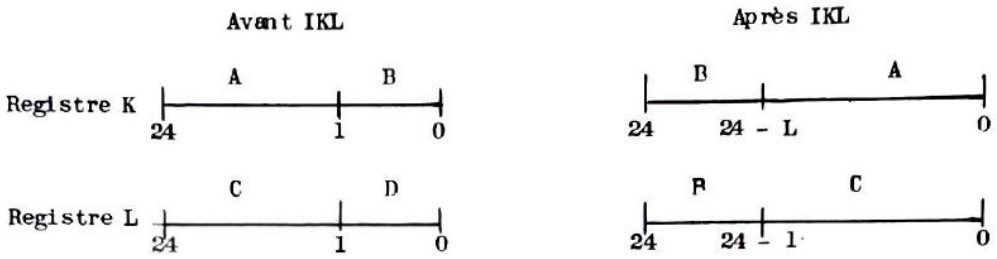
. la TKL effectue le filtrage, son "1" représentant la longueur filtrée.

Ainsi, chaque fois que l'on voudra filtrer une partie de catène, il faudra faire une TCK suivie d'une TKL. Il y aura ensuite lieu de recadrer éventuellement le résultat, si besoin est, par une MCL.

d) Insertion de K dans L (IKL). Insertion de K dans L ; cette opération affecte, à partir de la position 0, un nombre de positions égal à "1". K est maintenu à son contenu initial.

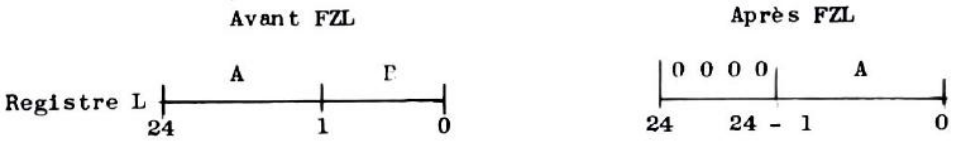
La zone 24 - 1 de L est également maintenue à son contenu initial.

Mais les deux registres K et L se retrouvent en fin d'opération décalés de "1" positions vers la droite par rapport au cadrage initial.



Cette opération permet de composer entièrement un catène dans le registre L à partir de morceaux de catènes différents. Ceci se fait par une succession de TCK suivis chacun d'une IKL. La TCK alimente le catène "donneur" et le cadre de façon à amener à droite la partie à insérer, la IKL fait l'insertion en laissant le cadrage tel que le registre L soit prêt à recevoir par une nouvelle insertion le morceau de catène qui doit venir à la suite de celui qui vient d'être inséré. On termine éventuellement le tout par une MCL qui fait le cadrage définitif.

e) Forçage à zéro (ou à un) dans L (FZL et FUL). Les 1 premières positions du registre L sont forcées à zéro (ou à un). Le contenu de la zone 24 - 1 est maintenu et, comme dans l'opération précédente, l'ensemble du registre L se retrouve décalé de 1 positions vers la droite par rapport au cadrage initial.

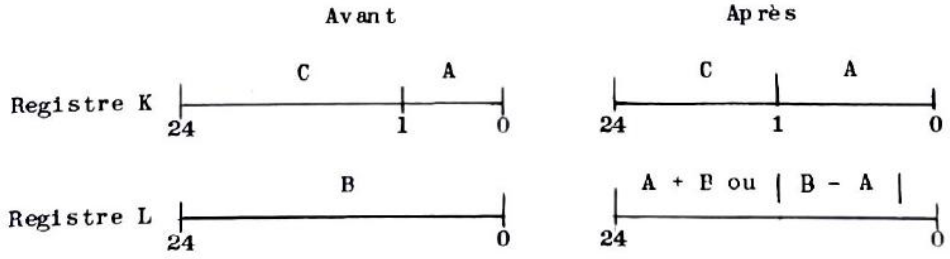


Ceci permet, par une succession de FZL et de FUL, de constituer dans L un catène de profil quelconque.

f) Addition ou soustraction binaire (ADD - SOU). Addition ou soustraction binaire des 1 premières positions du registre K à la totalité du registre L.

Maintien de K à ses contenu et cadrage initiaux.

En soustraction, le résultat négatif est automatiquement redressé ; c'est |B - A|



Comme on le voit, *le premier opérande doit être seul dans le registre L.* Si, à l'origine, il se trouve dans un catène Mémoire Rapide accompagné d'autres informations, il doit être filtré ; il n'en est pas ainsi pour le 2ème opérande dans le registre K.

En conséquence, préalablement à la directive ADD ou SOU, il faudra alimenter les opérandes dans K et L de la façon suivante :

. Si l'un au moins des deux opérandes est seul dans son catène, on l'alimentera et on le cadrera dans L par une TCL. Le second opérande, qu'il soit seul ou non dans son catène, sera alimenté et cadré dans K par une TCK.

Dans ces opérations d'ailleurs, il n'y a pas de distinction à faire entre premier et deuxième opérande :

$$A + B = B + A \text{ et } |A - B| = |B - A| .$$

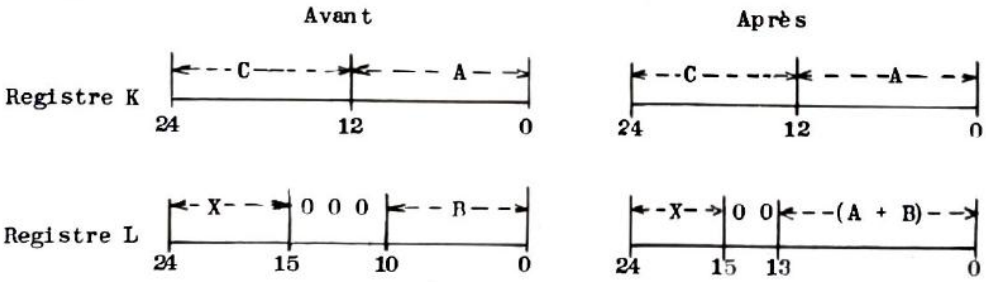
. Si aucun des deux opérandes n'est seul dans son catène, il faudra filtrer celui que l'on alimentera dans L. Ceci pourra se faire comme il a été expliqué précédemment par une TCK suivie d'une TKL. On alimentera et cadrera ensuite l'autre opérande dans K par une TCK.

Remarque : La règle qui vient d'être donnée n'est pas absolue ; elle doit être quelque peu tempérée : il est des cas où l'opérande B peut ne pas être seul dans L et cette possibilité peut même parfois être très intéressante à exploiter.

Supposons que le catène contenant B (qui en occupe par exemple les 10 premières positions) contienne également autre chose dans ses 9 dernières positions, les positions 10 à 15 étant des "zéros" ; supposons d'autre part que A soit contenu dans les 12 premières positions d'un second catène, les 12 dernières positions étant quelconques.

On peut alimenter A dans K et B dans L sans filtrage, puis commander l'addition. D'après ce qui vient d'être dit sur le fonctionnement de cette opération, les 9 dernières positions (16 à 24) de L seront inchangées tandis que A + B apparaîtra dans les 13 premières positions de L (s'il y a eu un report sur la dernière position à droite ; 12 premières dans le cas contraire).

ADD 12



D'une façon générale, pour pouvoir opérer de cette façon, il faut et il suffit que l'opération A + B ne provoque pas, sur la dernière position à droite, un report qui empiéterait sur la zone X, auquel cas celui-ci pourrait se transmettre plus loin et modifier totalement la zone X. Il suffit alors de savoir que cette zone ne fait pas partie du résultat qui se trouve dans le reste du catène.

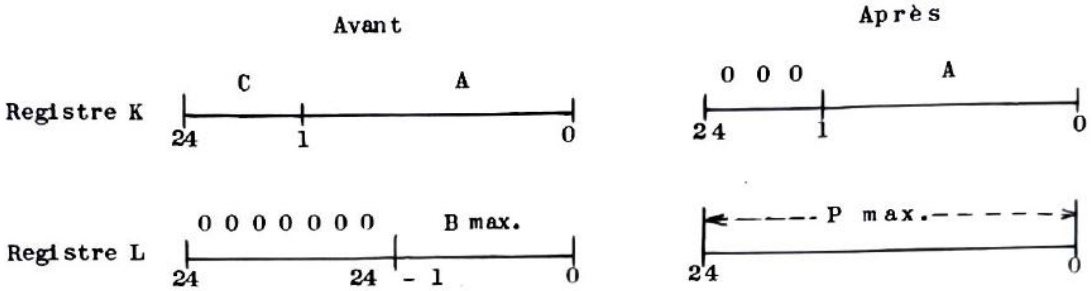
En revanche, pour pouvoir, dans les mêmes conditions, faire une soustraction, il faut être sûr que B soit plus grand que A, c'est-à-dire que l'opération ne donne pas lieu à un redressement du résultat. Sinon, c'est le catène tout entier (zone X et zone B) qui sera considéré comme opérande B et le résultat qui apparaîtra finalement dans L n'aura rien à voir avec B - A.

g) Multiplication binaire (MUL). Le Multiplicande se trouve en K dans les l premières positions, la zone 24 - l ayant un contenu quelconque.

Le Multiplicateur se trouve en L seul et cadré à droite.

La somme des capacités multiplicateur et multiplicande est au plus égale à 24.

En fin d'opération, le produit se trouve dans L cadré à droite ; dans K, on retrouve le multiplicande à son cadrage initial, mais la zone 24 - l est effacée.



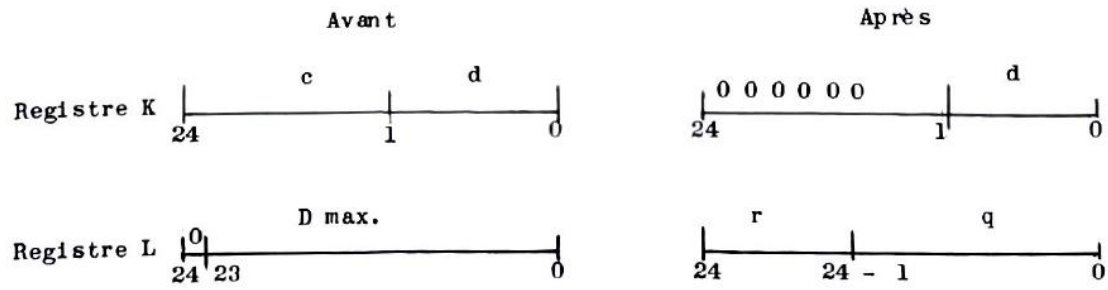
La multiplication étant une opération commutative comme les précédentes, on pourrait redire, à propos de l'alimentation de ses opérandes, exactement ce qui a été dit pour ADD et SOU mais, en considérant cette fois-ci comme absolue la règle qui veut que le multiplicateur B soit seul dans L ; c'est de cette façon que l'on obtient le temps minimum pour l'alimentation des opérandes. Mais le temps total d'une opération se compose, d'une part, du temps d'alimentation des opérandes, d'autre part, du temps opératoire proprement dit ; or, dans la multiplication, ce temps opératoire est, pratiquement (pour plus de précisions cf. le dernier § temps opératoires) directement proportionnel au nombre de chiffres significatifs (nombre de 1) du multiplicateur qui se trouve à l'origine dans L. Il faut donc, d'une manière générale, prendre comme multiplicateur le facteur le plus court même s'il n'est pas seul dans son catène et que l'autre le soit ; ce faisant, on allonge légèrement le temps d'alimentation des opérandes (3 directives au lieu de 2), mais on réduit le temps opératoire et, au total, on y gagne presque toujours. Dans le dernier § (temps opératoires), on précisera ceci par un calcul.

h) Division binaire (DIV). Le Diviseur est en K sur les l premières positions, le contenu des positions 24 - l étant quelconque.

Le Dividende est en L, seul et cadré à droite, son nombre de chiffres étant au maximum égal à 23.

En fin d'opération, le quotient apparaît en L, sur les (24 - l) positions de droite, le reste sur les l positions de gauche. Dans chacune de ces zones, quotient et reste sont cadrés à droite.

Le registre K est maintenu à son cadrage initial, mais il ne contient plus que le diviseur, les positions 24 - l ayant été effacées.



Cette opération n'étant pas commutative, aucune variante n'est possible pour l'alimentation des opérandes. Deux cas se présentent :

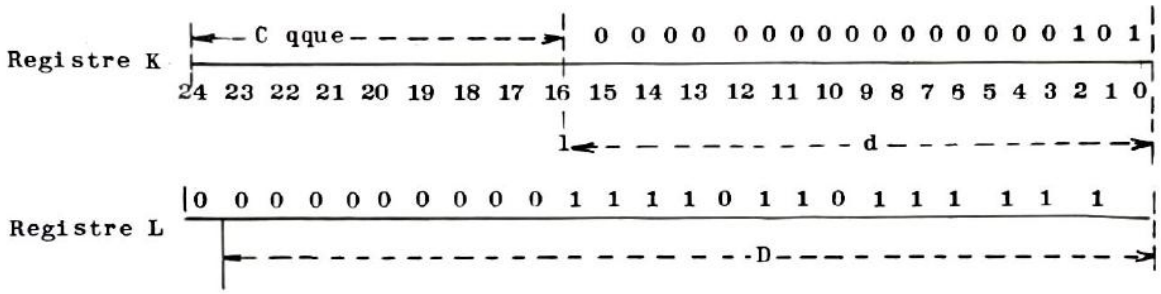
- le dividende est seul dans son catène : on alimente diviseur et dividende respectivement en K et en L par une TCK et une TCL.
- le dividende n'est pas seul dans son catène : on l'alimente d'abord en K par une TCK, puis en L en le filtrant par une TKL. On alimente ensuite le diviseur en K par une TCK.

Remarques importantes

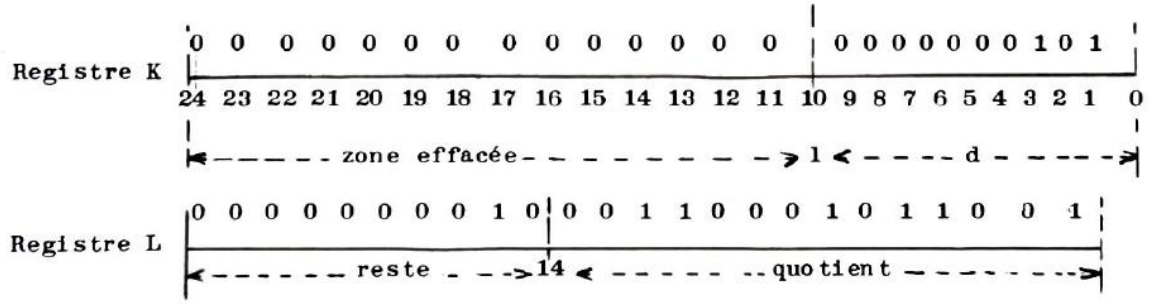
1. En définissant en colonne "V" la longueur théorique "l" du diviseur, on définit du même coup la longueur maximum du quotient (24 - l)
2. Le poids faible du quotient correspond toujours au rang des unités.

De ces deux remarques, il découle que, s'il se présente dans une opération de division un cas numérique tel que le quotient dépasse les dimensions fixées dans l'instruction (24 - l), on obtient un résultat aberrant et non pas les poids forts de ce quotient ; le calculateur logique se met en incident.

Supposons par exemple que l'on ait :

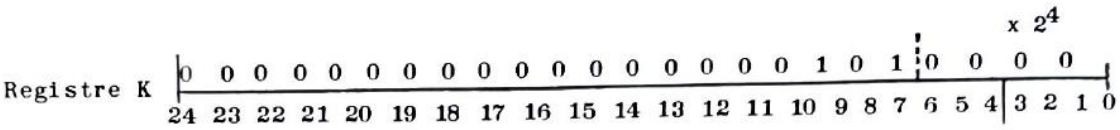


Si l'on programme alors une DIV avec 16 en colonne V, le Calculateur se mettra en incident ; en effet, ce mode de programmation donne une capacité de 24 - 16 - 8 pour le quotient alors qu'il est en réalité : 110001011001 de capacité 12, le reste étant 10. Par contre, si l'on programme une DIV avec 10 en colonne V, on aura en fin d'opérations :

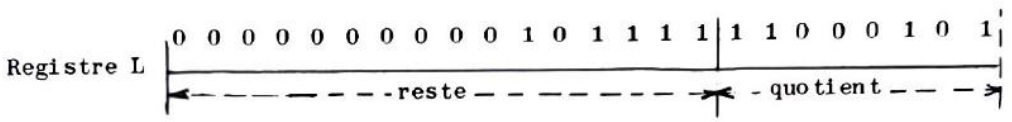


Il est enfin des cas où l'on peut désirer obtenir seulement les poids forts du quotient et, simultanément, le reste correspondant à ce rapport. La solution consiste alors à multiplier au préalable le diviseur par la puissance de 2 correspondant au rang du poids le plus fort que l'on désire abandonner dans le quotient ; soit "n" ce rang. Ceci s'obtient de façon très simple si le diviseur est seul dans son catène ; il suffit, au moment de son alimentation par une TCK, de le cadrer convenablement en laissant n zéros à sa droite. Sinon, il faut le filtrer, généralement par la séquence déjà vue TCK, TKL, MCL avec retour en Mémoire Rapide, puis le réalimenter dans K avec le cadrage convenable.

Pour l'exemple proposé précédemment, si l'on ne veut obtenir que les 8 premiers chiffres du quotient, c'est-à-dire laisser tomber les 4 derniers, il faut d'abord alimenter d dans K de la façon suivante :



On pourra alors écrire une DIV avec 16 en colonne V. Le registre L contiendra en fin d'opération :



i) Forçage au complément dans L (FCL). Les l premières positions de L sont forcées au complément, les autres étant maintenues telles quelles. Le cadrage n'est pas modifié.

Par exemple, une FCL 6 sur

donnera :

j) Intersection ou Réunion Logique (INT ou REU). Intersection ou réunion logique des l premières positions de K avec les l premières positions de L.

Les positions 24 - l de K et de L peuvent avoir un contenu quelconque.

Le résultat apparaît dans les l premières positions de L, les positions 24 - l n'étant pas modifiées.

Le registre K est maintenu à ses contenu et cadrage initiaux.

Rappelons que l'intersection logique fait apparaître un 1 dans toutes les positions où il y en a dans les 2 opérandes à la fois. Par exemple :

0 1 1 0 0 0 0
et 1 0 1 0 1 0 0

donnent 0 0 1 0 0 0 0

La réunion logique fait apparaître un 1 dans toutes les positions où il y en a dans l'un au moins des deux opérandes. Les deux opérandes précédents donnent :

1 1 1 0 1 0 0

k) Comparaison binaire et logique (CBL). L'opération porte sur les l premières positions de K et les l premières positions de L, les positions 24 - l des deux registres ayant des contenus quelconques. En fin d'opération, K et L sont maintenus à leurs contenu et cadrage initiaux, le résultat étant enregistré dans le catène qualitatif.

Il est à noter que, dans cette directive, il n'y a rien à afficher en colonne R puisqu'il n'y a pas de résultat quantitatif à transférer en Mémoire Rapide.

Les significations des diverses positions du catène qualitatif sont les suivantes :

1. $L > K$
2. $L < K$
3. $L = K$
4. $L \geqslant K$
5. $L \leqslant K$
6. $L \neq K$
7. L inclus dans K (ce qui veut dire que le résultat de la réunion de L et K serait identique à K ou que le résultat de l'intersection serait identique à L). Par exemple CBL 4

avec K : 1011) (Réunion 1011 $\equiv$ K
et L : 1001) (Intersection 1001 $\equiv$ L

8. L non inclus dans K (la réunion de K et L est différente de K ou l'intersection est différente de L). Par exemple CBL 5

avec K : 10101
L : 11100

9. K inclus dans L (Réunion de K et L identique à L).
10. K non inclus dans L (Réunion de K et L différent de L)
11. K exclu de L (le résultat de l'intersection serait nul ; on peut dire alors aussi que L est exclu de K. La complémentarité en est un cas particulier). Par exemple avec CBL 6 :

K : 101001)
L : 000110) Intersection 000000

12. Empiétement de K sur L (ou de L sur K. Le résultat de l'intersection serait différent de 0). Par exemple avec CBL 6 :

K : 101001)
L : 011110) Intersection 001000

13. Résultat de réunion est "tout à un" (la complémentarité est un cas particulier. Deux quantités sont complémentaires si 11 et 13 sont vérifiés à la fois). Par exemple avec CBL 5 :

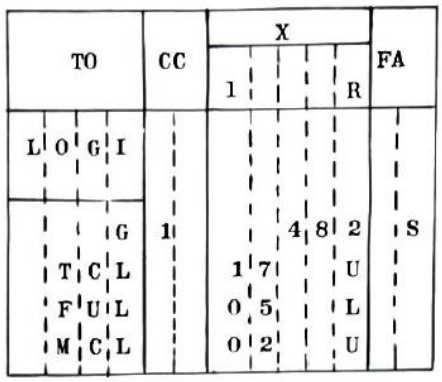
K : 10101)
L : 11010) Réunion 11111

14. Résultat de réunion comporte au moins un "0". Par exemple avec CBL 5 :

K : 10101)
L : 10011) Réunion 10111

Exemple : Nous allons voir maintenant comment on peut, à l'aide du Calculateur Logique, réaliser la directive centrale F.T.

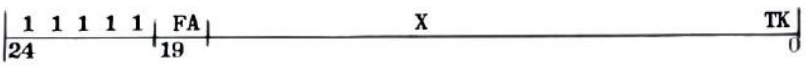
Le programme sera :



On connaît le profil d'un "Branchement"

Après la coupure et le garnissage de RAC, la TCL alimente l'instruction en L et la cadre de façon à préparer le forçage à un dans ses positions 17 à 22. On a alors dans L :

Dans R, on a écrit U pour que le RAC 1 ne progresse pas et puisse resservir pour ranger l'instruction modifiée à la même adresse. Après la FUL 05, on a dans L :



Il faut alors recadrer le catène par un décalage de 2 positions à droite et le ranger en Mémoire Rapide. C'est ce que fait la MCL 02 avec U en colonne R.

Finalement, en 6 lignes de programme, nous avons effectué le même travail que dans l'instruction F.T. Ceci montre bien l'intérêt des directives centrales lorsqu'elles correspondent à des opérations logiques fréquentes.

IV - TEMPS OPERATOIRES

Tous les temps donnés dans ce § le sont en nombre de cycles Mémoire Rapide.

Le temps d'exécution d'une directive se décompose en deux parties :

1°- *Le temps d'alimentation de l'instruction :* 6 cycles.

Ces six cycles comprennent les délais des circuits de priorité qui reçoivent les demandes d'instructions des éléments et le temps de l'alimentation proprement dite, c'est-à-dire le temps qui s'écoule entre le moment où la directive est envoyée au programme local du Calculateur Logique et le moment où elle est exploitable.

2°-*Le temps opératoire proprement dit.*

On distingue de ce point de vue 3 directives :

a) les directives d'alimentation et de cadrage des opérandes (TCK et TCL) : temps opératoire - 6 + 1/2 cycles.

b) la comparaison binaire et logique (CBL) qui ne comporte jamais de rangement de résultat quantitatif en Mémoire Rapide : temps opératoire : 12 cycles.

c) toutes les autres directives dont le travail peut se terminer par le rangement du résultat (contenu de L) en Mémoire Rapide. Dans ce dernier cas, les temps opératoires donnés ci-après doivent être augmentés de 3 cycles qui représentent le temps de rangement de ce résultat. Temps opératoires :

TKL	: 12 cycles	SOU	: 24 cycles
MCL	: 1/2 cycles	MUL	: 36 + 12 n - $\frac{1}{2}$
IKL	: 1/2 cycles	DIV	: 24 + 12 (24 $\frac{1}{2}$ 1)
FZL-FUL	: 1/2 cycles	FCL	: 12 cycles
ADD	: 12 cycles	INT-REU	: 12 cycles

Remarques

a) dans la MUL, "n" désigne le nombre de 1 du multiplicateur et "l" la capacité du multiplicande (n 24 - 1).

b) Les 4 opérations arithmétiques mises à part, les temps opératoires donnés par le tableau ci-dessus s'obtiennent facilement en décomposant chaque opération de la façon suivante :

. alimentation de l'opérande (éventuellement)	6 cycles
. décalages	1/2 cycles
. opérations avec filtrage sans décalage	12 cycles
. rangement du résultat en Mémoire Rapide (éventuellement)	3 cycles

Exemple : A titre d'illustration de ces temps, nous allons calculer le temps nécessaire pour effectuer la multiplication :

A × B - C

A est seul dans son catène et cadré à droite, sa capacité étant de 12 chiffres.

B a une capacité de 6 chiffres, il se trouve entre les intervalles 10 et 16 de son catène où il n'est pas seul.

C doit être rangé en Mémoire Rapide en fin d'opération.

On sait que le multiplicateur doit être seul en L. On peut agir de 2 façons différentes :

1) On prend A comme multiplicateur et B comme multiplicande. Ces opérandes seront alimentés en 2 instructions :

TCK 10 pour B
TCL 00 pour A

La multiplication se fera ensuite par une MUL 06.

Le temps total se décomposera ainsi, en supposant que tous les chiffres du multiplicateur A (au nombre de 12) sont des 1 :

. alimentation des 3 directives 3 × 6	18 cycles
. alimentation des opérandes : pour B : 6 + 10/2 -	11 cycles
pour A : 6 + 0/2 -	6 cycles
. temps de MUL 06 : 36 + 12 × 12 - 6/2 -	177 cycles
auquel il convient d'ajouter 3 cycles pour le rangement du résultat	3 cycles
Total	215 cycles

2) On prend B comme multiplicateur mais il faut alors le filtrer par une TCK 10, suivie d'une TKL 06. On alimente alors A par une TCK 00, puis on commande MUL 12.

Le temps se décomposera en :

. alimentation des 4 directives : 4 × 6		24 cycles
. alimentation des opérandes :	TCK 10 : 6 + 10/2	11 cycles
	TKL 06 :	12 cycles
	TCK 00 : 6 + 0/2 -	6 cycles
. temps de MUL 12 :	36 + 12 × 6 - 12/2 -	102 cycles
	rangement du résultat	3 cycles
	Total	158 cycles

On voit donc que cette méthode, bien que faisant appel à 4 directives, est préférable.

LES ÉLÉMENTS PÉRIPHÉRIQUES

ORGANES D'ENTRÉE-SORTIE

LE LECTEUR - PERFORATEUR DE CARTES

I - FONCTIONS DU LECTEUR-PERFORATEUR

1. Le lecteur-perforateur comprend normalement une piste qui permet d'effectuer soit la lecture de cartes perforées, soit la perforation de cartes vierges à 300 c/m. Mais il est impossible de lire et de perforer la même carte dans des zones différentes à cette cadence.

A cette piste principale, il peut être ajouté un dispositif supplémentaire consistant en une piste secondaire d'alimentation de cartes vierges à perforer. Ceci permet, au cours de la lecture d'un fichier principal, de l'interclasser avec un autre fichier que l'on perfore sur des cartes vierges alimentées à partir de la piste secondaire. Il faut bien remarquer que, là encore, lecture et perforation s'effectuent sur des cartes différentes.

Dans ce qui suit, nous décrirons l'organisation du lecteur-perforateur le plus complet, laissant au lecteur le soin de faire les restrictions nécessaires à l'utilisation d'un lecteur-perforateur à une seule piste.

2. Les fonctions du Lecteur-perforateur sont les suivantes :

a) Utilisation en lecteur :
- . lecture carte par carte
- . contrôle de la lecture carte par carte
- . sélection des cartes pour cause de mauvaise lecture ou autre.

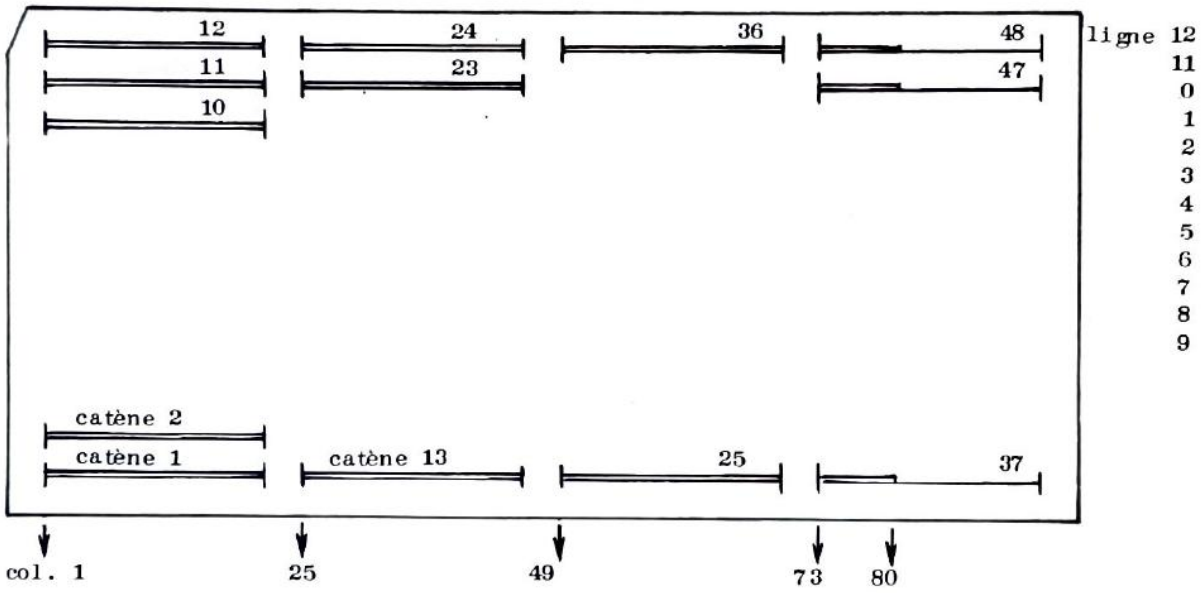
b) Utilisation en perforateur :
- . perforation carte par carte
- . contrôle de la perforation
- . sélection des cartes mal perforées.

c) Utilisation simultanée en lecteur-perforateur :
- . lecture de cartes sur la piste principale et contrôle
- . perforation de cartes alimentées de la piste secondaire et contrôle
- . interclassement des cartes lues et perforées.

3. Les cartes à lire ou à perforer sont des cartes 80 colonnes, à 12 positions par colonne, comportant ou non une perforation.

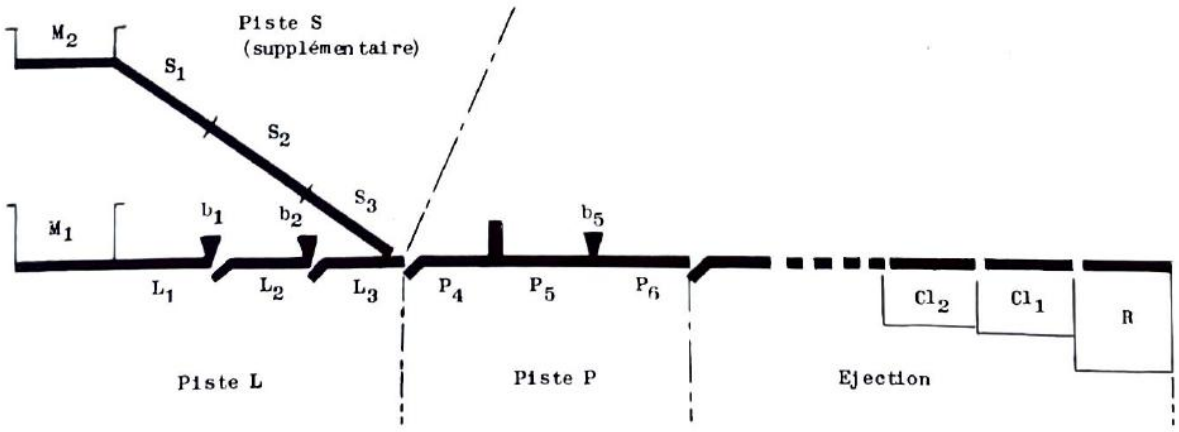
En lecture, le Lecteur-Perforateur constitue en Mémoire Rapide une "photographie" de la carte perforée, chaque position de celle-ci correspondant à 1 monade, 0 si position non perforée, 1 si position perforée. Réciproquement, en perforateur, le Lecteur-Perforateur perfore, sur la carte, son image précédemment constituée en Mémoire Rapide grâce au Traducteur.

L'ensemble est rangé sur 48 catènes de la façon suivante :



II – CONSTITUTION ET FONCTIONNEMENT

1. Constitution



a) Conformément au schéma ci-dessus, la piste principale du Lecteur-Perforateur comprend :

. un magasin d'alimentation M1 pouvant contenir 3.000 cartes perforées à lire, ou 3.000 cartes vierges à perforer.

. une piste de lecture L comportant 2 brosses consécutives b_1 et b_2 pour la lecture et le contrôle,

. une piste de perforation P faisant suite à la piste L précédente et comprenant un bloc de perforation ligne par ligne suivi d'une brosse de lecture b_5 pour contrôle,

. un mouvement d'éjection E, assurant l'acheminement des cartes sortant de P vers une case de réception principale R de 2.700 cartes ou deux cases de réception secondaires de 700 cartes chacune et référencées clapet 1 et clapet 2.

b) La piste secondaire d'insertion de cartes vierges à perforer comprend :

. un magasin d'alimentation M_2 destiné à contenir les cartes vierges et de capacité 800 cartes,

. une piste S permettant d'insérer les cartes vierges dans la piste principale entre la partie lecture et la partie perforation.

c) Sur le schéma ci-dessus, on a numéroté L_1, L_2, L_3, P_4, P_5, P_6, les positions successives, à la fin de chaque cycle machine, d'une carte parcourant la piste principale. S_1, S_2, S_3 repèrent les positions successives d'une carte parcourant la piste S.

2. *Fonctionnement en marche normale*

a) *avancement des cartes*

. Suivant l'utilisation désirée, lecture ou perforation depuis M1, perforation depuis M_2, la piste du Lecteur-Perforateur est composée de S, P et E ou de L, P et E.

. Dans les deux cas, le Lecteur-Perforateur est lancé cycle par cycle ; il fonctionne en Lecteur dans le cycle qui suit une directive de lecture, et en perforateur dans le cycle qui suit une directive de perforation.

. En marche normale, les cartes progressent d'une position dans la piste par directive entraînant un avancement. Il y a exception pour la partie éjection, en mouvement permanent, qui achemine immédiatement toute carte qui lui parvient vers les cases réception. Chaque cycle d'avancement provoque une nouvelle alimentation depuis l'un des magasins, ce qui entraîne la présence de plusieurs cartes dans des positions différentes. Nous allons voir la façon dont ces cartes sont traitées simultanément.

b) Fonctionnement en Lecteur

. A la fin d'un cycle de lecture, la carte passée sous b_1 se trouve en L_2, et son image est enregistrée en Mémoire Rapide, comme il a été expliqué précédemment. Au cycle de lecture suivant, la même carte passe sous b_2 pour une 2ème lecture de contrôle. Mais les quantités lues alors ne sont pas envoyées en Mémoire Rapide. Au contraire, chaque fois qu'une ligne est lue, les 4 catènes qui lui correspondent dans la lecture sous b_1 sont rappelés de la mémoire rapide pour être comparés localement dans l'élément au résultat de la nouvelle lecture. Une différence dans la comparaison est signalée en position 7 du catène qualitatif.

. La présence de plusieurs cartes consécutives dans la piste entraîne donc, par cycle, une lecture sous b_1 avec introduction en Mémoire Rapide, et une lecture simultanée sous b_2, pour contrôler la carte qui a été lue sous b_1 au cycle précédent.

c) Fonctionnement en Perforateur

. Avant le lancement d'un cycle de perforation du Lecteur-Perforateur, les informations à extraire doivent être enregistrées en Mémoire Rapide. Après ce

cycle, la carte perforée, se trouve en P_5. Au cycle suivant, elle passe sous b_5, elle est lue et comparée localement à l'image à perforer que l'on a conservée en Mémoire Rapide, par un mécanisme identique à celui du contrôle de la lecture. Une différence dans la comparaison est signalée en position 7 du catène qualitatif.

. Le fonctionnement est analogue à celui de la lecture ; chaque cycle comporte la perforation d'une carte et une lecture de contrôle pour la carte perforée au cycle précédent.

3. *Splittage de la piste*

. La piste du Lecteur-Perforateur est splittable, ce qui veut dire que, pour un cycle déterminé, l'avancement des cartes peut être validé dans une partie restreinte de la piste. Les différents splittages sont indiqués par des codes complémentaires dans les directives. Ce sont :

- Splittage après b_1 : avancement des cartes situées en L_2, L_3, P_4, P_5, P_6, éjection.
- Splittage après b_2: avancement des cartes situées en L_3, P_4, P_5, P_6, éjection.
- Splittage entre les parties L et P ou S et P : avancement des cartes dans les parties perforation et éjection.
- Splittage permanent après P_6 en cas de non-avancement de la piste ; ceci est produit par le mouvement permanent de la partie Ejection.

. Le splittage est rendu nécessaire par des opérations tout à fait particulières et peu fréquentes, intervenant généralement en cas d'erreur révélée par contrôle, ou dans le passage de la lecture à la perforation, ou de la perforation à la lecture. Il peut entraîner une discontinuité dans l'alimentation des cartes dont il faut éventuellement tenir compte dans le retour en marche normale du Lecteur-Perforateur.

4. *Sélection des cases clapet*

. Après passage sous b_2 ou sous b_5 et avant validation d'un nouveau cycle machine, une carte peut être sélectionnée pour être envoyée en case clapet 1 ou 2.

. Les sélections de clapet sont le fait de directives préparatoires que l'on appelle soit à la suite d'une détection de mauvaise lecture ou perforation, soit pour toute autre raison.

Dans les deux cas, l'effet de la directive "sélection de clapet" est de valider l'éjection de la carte présente à un poste donné dans une case clapet. Cette éjection n'aura lieu que lorsque cette carte aura atteint la partie éjection de la piste par le jeu des directives d'exécution qui provoquent son avancement.

Il convient de noter que la sélection peut être perdue si on demande ensuite un avancement splitté de la piste ne provoquant pas l'avancement de la carte sélectionnée. Le problème ne se pose que dans le cas d'une carte sélectionnée après b_2 ; dans ce cas, la directive sélection de clapet doit être suivie d'une directive provoquant l'avancement de la carte de L_3 en P_4.

5. *Les R.A.C.*

Le Lecteur-Perforateur comprend 2 R.A.C. spécialisés références 0 et 1.

. En lecture : le R.A.C. 0 indique l'adresse où l'on doit ranger en Mémoire Rapide le résultat de la lecture sous b_1. Le R.A.C. 1 indique l'adresse où l'on doit prendre en Mémoire Rapide les quantités à comparer au résultat de la lecture sous b_2.

. En perforation : le R.A.C. 0 indique l'adresse où se trouve l'image de la carte à perforer. Le R.A.C. 1 indique l'adresse où l'on doit prendre en Mémoire Rapide les quantités à comparer au résultat de la lecture sous b_5 (image de la carte perforée au cycle précédent).

Les R.A.C. ne sont pas utilisés pour un cycle de simple avancement de piste et, d'autre part, en cas de non-passage de carte sous un poste, le R.A.C. qui lui est affecté n'évolue pas.

III - LES DIRECTIVES DU LECTEUR-PERFORATEUR

Le Lecteur-Perforateur, comme tout élément, a un programme local qui décodifie les directives et sait donc parfaitement faire la distinction entre les instructions de lecture et celles de perforation. La coupure qui le met au travail est donc, en code A, la même, qu'on l'utilise en Lecteur seul, en Perforateur seul, ou les deux avec le concours de la piste secondaire. En code B, on écrira à volonté LECT ou PECT suivi du numéro de Lecteur-Perforateur en X ; mais, bien entendu, ces deux T.O. correspondent à la même coupure en code A ; il sera naturel, en code B, d'écrire LECT si on ne l'utilise qu'en lecture et PECT si on ne l'utilise qu'en perforation.

1. Directives préparatoires

Ce sont les directives de sélection de clapet :

. SCL : sélection de clapet pour la carte située après b_2.

. SCP : sélection de clapet pour la carte située en piste P après b_5.

. Pour ces deux directives, le numéro de clapet sélectionné est indiqué en deuxième colonne de code complémentaire. Ces directives doivent être suivies de directives d'exécution.

2. Directives d'exécution

Il existe 3 sortes de directives d'exécution qui déclenchent les opérations. Ce sont :

LEC : Lecture
PER : Perforation
API : Avancement mécanique de la piste sans lecture ni perforation.

Les 3 directives provoquent toujours un avancement de la piste, mais les fonctions de lecture, de perforation, de contrôle, n'ont lieu que si une carte passe effectivement sous le poste considéré et ce passage peut être vérifié par programme en exploitant le catène qualitatif.

Dans les 3 cas, le mode de splittage de la piste est précisé en codes complémentaires par les indications suivantes :

M_1 : avancement normal depuis le magasin 1 (mouvement de toute la piste principale L, P et E).

M_2 : avancement depuis le magasin 2 (mouvement de S, P et E ; n'a de sens qu'avec PER ou API)

B_1 : avancement de la partie de piste située après b_1 (n'a de sens qu'avec LEC)

B_2 : avancement de la partie de piste située après b_2 (n'a de sens qu'avec API pour vider la piste après lecture).

P : avancement des parties P et E de la piste seulement (n'a de sens qu'avec PER et API).

3. Directives de service

Les interventions importantes de l'opérateur sur le Lecteur de cartes sont prévues dans le programme lui-même par des directives de service : DMI (demande de manipulation incident) et DMF (demande de manipulation fin de fichier).

. Ces deux directives ont toutes deux un premier effet identique : elles provoquent un blocage du programme local de l'élément sur la directive d'exécution suivante jusqu'à intervention manuelle de l'opérateur. D'une façon plus détaillée, elles provoquent une demande d'instruction de la part de l'élément auquel le Distributeur de Programme envoie alors une nouvelle directive d'exécution. L'effet de la directive de service précédente est de bloquer cette directive d'exécution de sorte que le Lecteur-Perforateur s'arrête à la fin de son cycle mécanique. La directive d'exécution ne devient effective que lorsque l'opérateur est intervenu.

. Elles signalent à l'opérateur qu'il doit intervenir : la DMI en provoquant l'allumage d'une lampe rouge sur le Lecteur-Perforateur, la DMF l'allumage d'une lampe verte.

a) Circonstances d'utilisation de la DMI

Cette instruction est appelée par le programme à la suite de la détection d'un incident tel que bourrage de piste, magasin vide ou case de réception pleine. Le Lecteur-Perforateur s'arrête, la lampe rouge s'allume, l'opérateur intervient et, à la fin de son intervention, appuie sur 2 boutons : Déverrouillage et Marche, ce qui a pour effet de valider la directive d'exécution que la DMI avait bloquée.

Le cas de magasin vide peut se produire dans deux circonstances :

. l'opérateur a oublié de le regarnir en temps utile. Il répare son oubli et appuie sur les boutons Déverrouillage et Marche.

. on est arrivé en fin de lecture du fichier, et l'opérateur, en mettant le dernier paquet de cartes du fichier, a oublié de le signaler en appuyant sur le bouton Fin de fichier. Il répare son oubli en appuyant sur ce bouton, puis sur les boutons Déverrouillage et Marche, ce qui a pour effet de permettre la lecture et l'éjection des dernières cartes du fichier comme indiqué ci-dessous.

Il faut bien noter qu'en cas de magasin vide, l'arrêt se produit aussitôt, une carte étant arrêtée devant b_1 de sorte qu'il n'y ait aucune discontinuité dans l'alimentation des cartes une fois l'incident réparé.

b) Circonstance d'utilisation de la DMF

. En lecture de cartes :

Normalement, lorsque l'on met dans le magasin de lecture le dernier paquet de cartes du fichier, on appuie sur le bouton Fin de Fichier. Dans ce cas, lorsque le magasin est vide, le lecteur ne s'arrête pas ; la dernière carte passe sous b_1, puis sous b_2 ; à ce moment-là, il n'y a plus de carte passant sous b_1 et un test apparaît en position 6 du catène qualitatif. Il permet d'aiguiller le programme sur une séquence et de vider la piste qui se termine par une DMF, laquelle provoque l'allumage de la lampe verte et bloque la directive d'exécution suivante. L'opérateur sait alors, d'après sa fiche de travail, qu'il doit vider la case réception, alimenter un autre fichier dans le magasin et remettre le Lecteur-Perforateur en marche par appui sur Déverrouillage et Marche.

. En perforation :

La fin de fichier est décelée par le code interne qui aiguille le programme sur une séquence permettant de contrôler et d'éjecter la dernière carte perforée, séquence qui, comme la précédente, se termine par la DMF invitant l'opérateur à agir.

4. Tableau des directives

La combinaison des différents T.O. et splittages prévus permet de composer

toutes les instructions désirables, quelquefois même de plusieurs façons. Par exemple, LEC B.2 et API B.2 entraînent toutes deux le vidage du Lecteur-Perforateur à partir de b_2. Nous nous bornerons, dans le tableau suivant, à indiquer la combinaison correspondant à l'utilisation la plus rationnelle du Lecteur-Perforateur.

	T.O			C.C		X	F.A
Sélection clapet 1 après b2	S	C	L		1		
Sélection clapet 2 après b2	S	C	L		2		
Sélection clapet 1 après b5	S	C	P		1		
Sélection clapet 2 après b5	S	C	P		2		
Lecture normale	L	E	C	M	1		
Lecture avec splittage après b_1	L	E	C	B	1		
Perforation normale depuis M_1	P	E	R	M	1		
Perforation normale depuis M_2	P	E	R	M	2		
Perforation sans alimentation	P	E	R	P			
Avancement de piste :							
-pour vidage lecture	A	P	I	B	2		
-pour vidage depuis M_2	A	P	I	M	2		
-pour vidage piste P	A	P	I	P			
Demande de Manipulation Incident	D	M	I				
Demande de Manipulation Fin de Fichier	D	M	F				

5. Profil du catène qualitatif

1. Carte passée sous b_1
2. Carte passée sous b_2
3. Carte passée de L à P
4. Carte passée sous bloc de perforation
5. Carte passée sous b_5
6. Carte passée sous b_2 mais non sous b_1 (si appui préalable sur bouton Fin de Fichier)

24 + 23 + 22 : bourrage éjection
24 + 23 + 21 : bourrage piste opératoire
24 + 23 + (18, 19 ou 20) : incidents technologiques
24 : regroupement de tous les incidents (dont ceux non spécialement signalés tels que Magasin vide ou case de réception pleine).

IV. PERFORMANCES

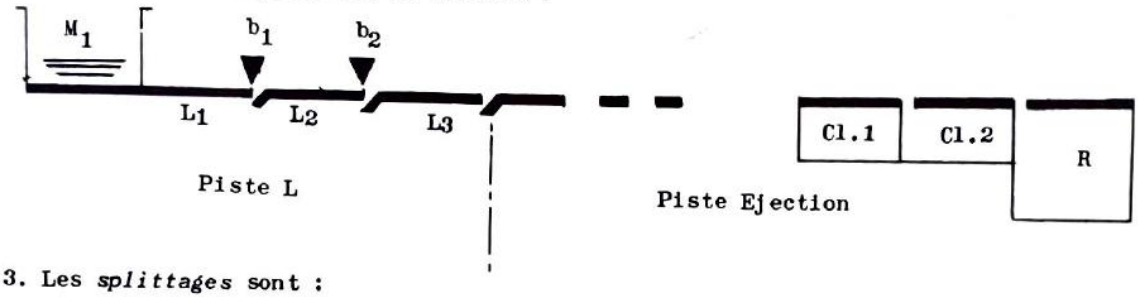
.Le temps disponible entre l'envoi du catène qualitatif après lecture, perforation ou contrôle, et le lancement du cycle suivant, correspond à environ 1.000 cycles mémoire centrale.

.En marche normale, le Lecteur-Perforateur fonctionne donc à 300 cartes par minute, en lecture ou en perforation, à la seule condition que la séquence d'exploitation du contrôle et de lancement du cycle suivant ait une durée inférieure à 1.000 cycles M.C.

NOTE COMPLEMENTAIRE SUR LE LECTEUR DE CARTES

Outre le Lecteur-Perforateur, il existe un autre élément, le Lecteur de cartes, dont les possibilités en lecture de cartes sont identiques à celles du Lecteur-Perforateur, mais qui ne possède qu'une piste de lecture.

1. La coupure qui le met au travail est LECT accompagnée de son numéro.
2. Le schéma de la piste est le suivant :



3. Les *splittages* sont :
   - .identiques à ceux du Lecteur-Perforateur après b_1 et b_2.
   - .systématiques après L_3 en cas de non-avancement de la partie L (mouvement continu de la partie E).

4. Il n'y a qu'une directive de *sélection de clapet* : S C L

   Elle sélectionne le clapet pour la carte située en L_3 après b_2. Le numéro de clapet sélectionné (1 ou 2) est indiqué en 2ème colonne de code complémentaire.

5. Les directives *d'exécution* sont :
   - .L E C . M 1 pour Lecture Normale
   - .L E C . B 1 pour Lecture de contrôle sous b_2 exclusivement (splittage après b_1).
   - .A P I . B 2 : Avancement de la piste avec splittage après b_2 pour vidage de la piste en fin de lecture.

6. Les directives de service sont encore DMI et DMF.

ORGANES D'ENTRÉE-SORTIE

L'IMPRIMANTE GAMMA 60

1. DESCRIPTION ET FONCTIONNEMENT

L'Imprimante permet l'impression en général d'un seul état et, sur demande, de deux états superposés ou juxtaposés. Nous décrirons l'Imprimante la plus complète, c'est-à-dire celle possédant deux entraînements de papier distincts, le fonctionnement de l'Imprimante à un entraînement s'en déduisant facilement.

1. Principe de fonctionnement

Ce sont des Imprimantes à 120 positions et 300 lignes/minute.

Il y a 60 caractères d'impression possibles (chiffres, lettres ou caractères spéciaux). Ces 60 caractères sont répétés 120 fois sur un cylindre d'impression de sorte que le long d'une même génératrice du cylindre, on trouve 120 fois le même caractère et que le long d'une même circonférence, on trouve les 60 caractères différents. Ce cylindre tourne constamment autour d'un axe horizontal.

Le papier entraîné par picots défile dans un plan parallèle à l'axe du cylindre. Une rangée de 120 petits marteaux se trouve de l'autre côté du papier et le projette contre le cylindre au moment où le caractère à imprimer passe devant le papier. Le déplacement mécanique de l'un des marteaux crée une impulsion électrique qui, combinée avec la position du tambour, permet de contrôler que le caractère imprimé est bien celui que l'on désirait imprimer.

2. Description du chemin de papier

L'Imprimante comprend (cf. schéma) :

a) un chemin de papier de 180 pas de large (1 pas - 1 caractère . 8 pas par pouce) divisé en 3 parties :

.120 pas correspondant au tambour d'impression,

.à gauche 20 pas et à droite 40 pas permettant un positionnement "débordant" du papier.

b) deux dispositifs d'entraînement de papier dits "Entraînement 1er feuillet, entraînement 2ème feuillet", absolument indépendants l'un de l'autre. L'entraînement est fait par picots situés sur deux crémaillères qui peuvent coulisser le long de leur support. Par suite, la distance entre 2 crémaillères est variable, ainsi que la distance de celles-ci par rapport aux bords du chemin de papier.

Le papier se déplace par interligne simple, double ou triple, ou par saut avec comptage d'interlignes.

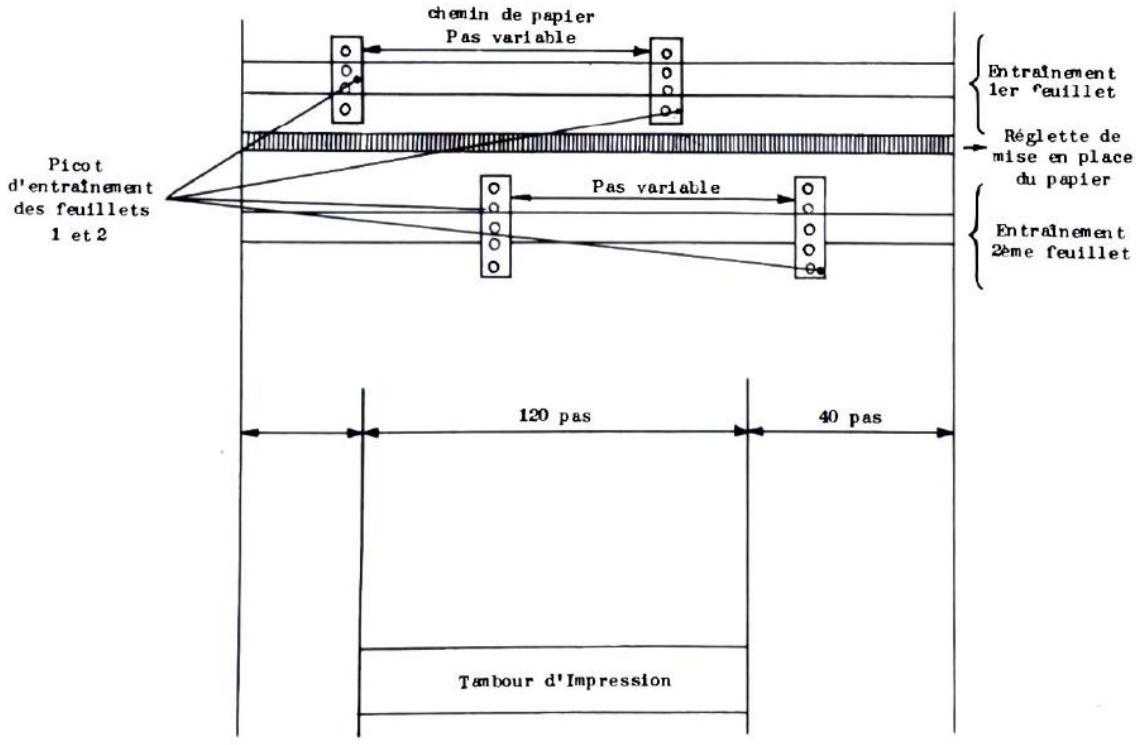
c) Deux entraînements de ruban. Les rubans peuvent avoir une largeur variable adaptée à celle de l'état. Ils se déplacent dans le même sens que le papier, c'est-à-dire *verticalement* et non horizontalement comme dans les machines à écrire ordinaires.

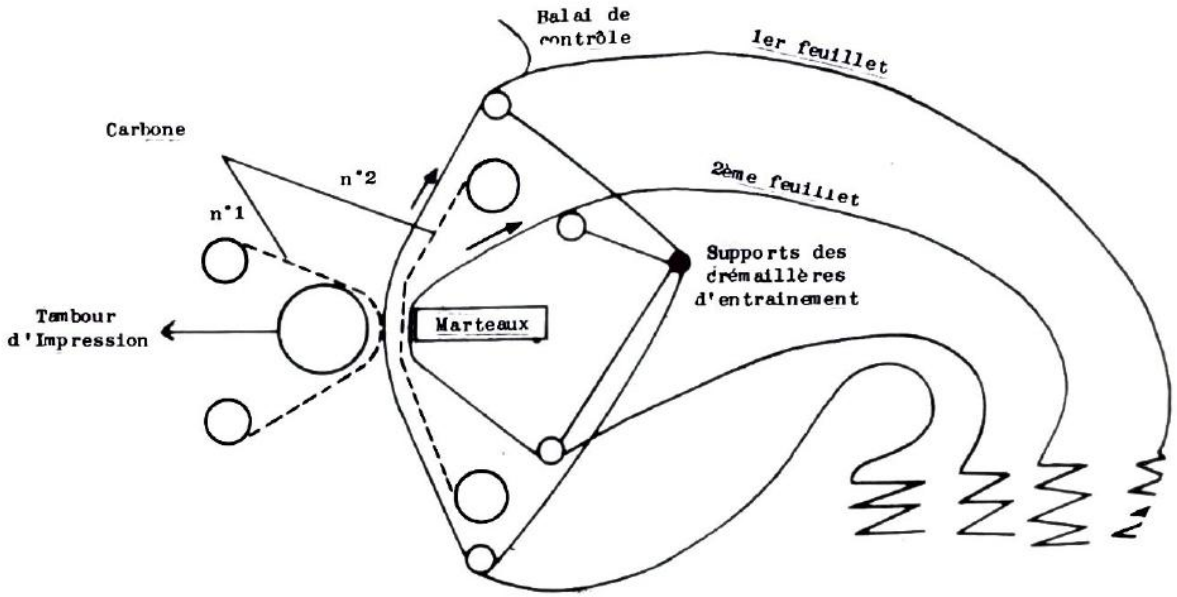
Le ruban n° 1 est en carbone, soie ou coton; les rubans soie ou coton sont utilisés alternativement dans les deux sens jusqu'à usure. Le ruban n° 2 est toujours en carbone, le débit est réglé de façon qu'il ne serve qu'une fois.

d) Une réglette graduée facilitant la mise en place du papier.

e) Des dispositifs divers permettant l'engagement automatique du papier, la recherche de repère déterminant la position de la 1ère ligne à imprimer (niveau d'impression), l'éjection du papier en fin de travail.

f) Des dispositifs permettant de détecter :
- .un désynchronisme dans le mouvement des feuillets 1 et 2
- .un saut normal
- .une rupture du papier, du carbone
- .l'approche fin de papier permettant le raboutement
- .la fin de papier, de carbone ...





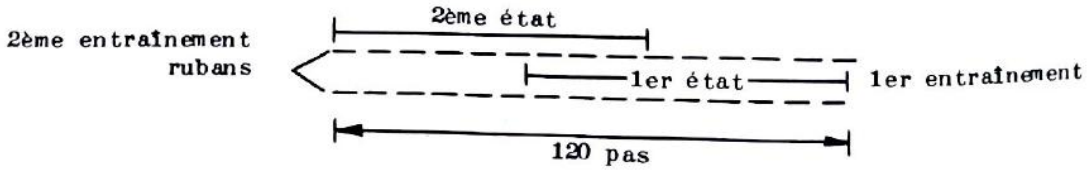
II. DIFFERENTS TYPES D'ETATS

L'Imprimante peut imprimer des états :

.en utilisant le même entraînement si les sauts sont les mêmes sur les 2 états (imprimé unique, imprimé en Y, etc...). Evidemment, on peut utiliser aussi le 2ème entraînement, mais celui-ci étant indépendant du 1er, il faut commander les sauts deux fois (voir plus loin les directives SDF et IMP).

.en utilisant les deux entraînements si les sauts sont différents sur les 2 états. Ceci permet les variantes suivantes :

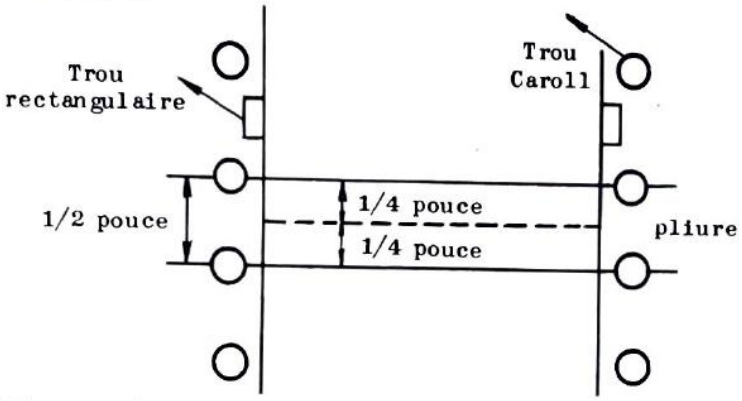
-Imprimés superposés (les rubans n° 1 et 2 sont nécessaires)
-Imprimés juxtaposés (le ruban n° 1 suffit)
-Imprimés mixtes



(les rubans n° 1 et 2 sont nécessaires).

Norme des états

.De part et d'autre du cliché d'impression, l'état comprend une bande marginale avec des trous Caroll d'entraînement. Ces trous ont 5/32 de pouce de diamètre et sont distants de 1/2 pouce. Pour les états "accordéon", la pliure est équidistante de 2 trous Caroll.



.Il existe, en plus, des trous rectangulaires situés sur le bord interne de la bande marginale. Ces trous servent :

-à la mise en place du papier lors de son alimentation.

-au contrôle du synchronisme en cours d'impression. Pour cela, un balai (il y en a un pour chaque entraînement) détecte la présence du trou rectangulaire en début d'impression d'un nouveau feuillet (le trou ne se trouve pas obligatoirement en début de feuillet. Il peut être à un niveau quelconque et c'est à ce niveau que s'effectue le contrôle : le papier étant arrêté pendant l'impression, le balai dans le trou). Ce balai envoie une monade dans le CQ (pos.1 et 2) qui peut être exploité par programme, pour le contrôle du synchronisme et permettre la correction par programme d'incident.

.Les états varient quant à :

-la longueur par demi-pouce
-la largeur par 1/8 pouce

1 pas - 3 mm 178 (8 caractères/pouce)
1 interligne - 4 mm 24 (6 interlignes/pouce)

Une ligne d'impression est toujours dans l'axe d'un trou Caroll ou à ± 4 mm 24 de cet axe.

Le nombre total d'états en liasse est de 6 - (1er et 2ème feuillets compris, soit 5 carbones en tout).

III. PROGRAMMATION

La programmation se fait ligne par ligne d'impression.

La ligne à imprimer est "éditée" en mémoire rapide dans le code interne à 6 positions. Cette fonction est réalisée par le Traducteur avec la directive cas d'emploi TIP.

Même si la ligne à imprimer est inférieure à 120 caractères, il faut réserver en MC 30 catènes, qui sont tous envoyés à l'Imprimante lorsqu'elle effectue sa demande de quantité. Par suite, pour une ligne de moins de 120 caractères, *il ne faut pas oublier :*

.soit d'effacer la zone de 30 catènes avant de faire l'édition,
.soit d'effacer le reste de cette zone à la fin de l'édition.

L'Imprimante possède un seul RAC n° 0 précisant l'adresse initiale en MC où se trouve la ligne à imprimer.

Elle est appelée par la coupure IMPR.

1. Pour imprimer une ligne, il faut :

a) Employer, *si l'on utilise le 2ème entraînement, la Directive préparatoire* SDF (sauf deuxième feuillet), c'est cette directive qui fait avancer le 2ème feuillet.

Dans la partie X, on indique la longueur du saut en interlignes quelle qu'elle soit. Si, dans la partie Y, on met 0, on imprime sur la même ligne; si on met 1, on saute de 1 interligne; si on met n, on saute de n interlignes.

Le saut maximum est de 255 interlignes ($n \leqslant 255$).

La partie FA permet d'utiliser des mémoires de manoeuvre pour indiquer la longueur du saut.

b) Utiliser la directive d'exécution IMP qui fait :

.sauter le 1er feuillet,

.imprimer la ligne en fin de saut (si la fin du saut du 2ème feuillet a eu lieu.

	TO				CC	X	FA
Saut deuxième feuillet		S	D	F		Longueur en interligne Maxi : 255	
Saut premier feuillet et Impression d'une ligne		I	M	P		id	
Demande de Manipulation Incident		D	M	I			
Demande de Manipulation Fin de Fichier		D	M	F			

La partie X donne la longueur du saut en interlignes exactement comme pour la SDF (elle est aussi inférieure à 255 interlignes).

Note : La directive SDF précède obligatoirement la directive IMP.

2. Mécanisme du Saut

On a déjà dit que les sauts des 2 feuillets sont absolument indépendants. Pour cela, il existe 2 compteurs d'interlignes qui sont chargés par les directives SDF et IMP et décomptés en cours de saut.

.Si les variables X de la SDF *et* de l'IMP sont inférieures à 10, le saut a lieu dans l'intercycle, c'est-à-dire sans cycle supplémentaire.

.Si *l'une des deux* variables X est supérieure à 10, l'Imprimante fait automatiquement un cycle à vide pour permettre au saut de s'effectuer. Un cycle à vide permet de sauter 50 interlignes (21,2 cm).

.Par suite, "Pour tout saut supérieur à 10 interlignes," soit 4,24 cm, il faut compter un cycle supplémentaire par 50 interlignes.

3. Les Directives de Service DMI, DMF

Comme pour le Lecteur de cartes, les interventions importantes de l'opérateur sont prévues dans le programme par les directives :

DMI : Demande Manipulation Incident
DMF : Demande Manipulation Fin de Fichier

.Ces 2 directives ont le même effet : elles provoquent un blocage du programme local de l'élément sur la directive d'exécution suivante jusqu'à intervention manuelle de l'opérateur. D'une façon plus détaillée, elles provoquent une demande d'instruction de la part de l'élément auquel le Distributeur de Programme envoie alors une nouvelle directive d'exécution. L'effet de la directive de service précédente est de bloquer cette directive d'exécution de sorte que l'Imprimante s'arrête à la fin de son cycle mécanique. La directive d'exécution ne devient effective qu'après intervention de l'opérateur.

.Elles signalent à l'opérateur qu'il doit intervenir : la DMI en provoquant l'allumage d'une lampe rouge, la DMF l'allumage d'une lampe verte.

a) Circonstances d'utilisation de la DMI – Cette instruction est appelée par le programme à la suite de la détection d'un incident tel que : déchirure de papier, de ruban, désynchronisme dans le mouvement des feuillets, erreur d'impression, etc ... L'Imprimante s'arrête, la lampe rouge s'allume. Après allumage de la lampe rouge, l'opérateur agit suivant des indications très strictes distribuées par lampes (simple incident) ou transcrites au pupitre (incident grave, reprise de feuillet). Puis, il appuie sur le bouton "Marche", ce qui a pour effet de valider la directive d'exécution bloquée par la DMI.

b) Circonstances d'utilisation de la DMF – La DMF est utilisée pour signaler à l'opérateur la fin du travail en cours. Il doit agir suivant sa feuille de travail. Ex.: changement de papier, mise en place du travail à suivre, ou se réfère au chef d'atelier.

L'extinction de la lampe verte est provoquée par l'appui sur le bouton "Marche", qui débloque la directive d'exécution du travail suivant, s'il y a lieu.

IV. PROFIL DU CATENE QUALITATIF

CQ

23	+ 1	Balai de contrôle entraînement 1
	2	Balai de contrôle entraînement 2
	3	Approche FP 1
	4	Approche FP 2
	5	Fin ruban 1
	6	Fin ruban 2
	7	Incident. Regroupe 3, 4, 5, 6
24	+ 22	Fusible. Commande locale positionnée
24	+ 16	Erreur d'impression
	17	Transfert insatisfait
	18	Contrôle de transfert directive
	19	Fin de papier ne permettant plus le raboutement
	21	Sécurité saut

MÉMOIRES AUXILIAIRES

LE RUBAN MAGNÉTIQUE

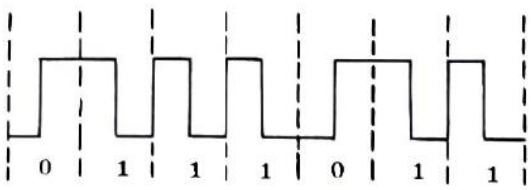
1. STRUCTURE DU RUBAN MAGNETIQUE

1. Le Ruban Magnétique est le support idéal pour stocker les informations sous un faible volume et pour un faible prix. Il est constitué d'un ruban en matière plastique (mylar) de 12,7 mm de large et recouvert d'oxyde magnétique. Les informations y sont enregistrées par catène telles qu'elles sont en Mémoire Rapide, c'est-à-dire la plupart du temps en code interne; toutefois, il n'est pas interdit de penser qu'elles puissent y être enregistrées en code externe, par exemple à raison de 3 caractères par catène telles qu'elles sortent d'un Lecteur de bande perforée avant traduction en code interne.

2. Le Ruban Magnétique comprend *8 pistes d'enregistrement* en parallèle, réparties sur toute sa largeur. Il n'y a aucune piste de service, le principe de fonctionnement des dérouleurs (cf. § II) étant tel qu'il n'exige aucune préparation préalable du ruban (prémarquage ou signalisation des zones défectueuses).

3. Le principe de l'enregistrement est l'*enregistrement en modulation de phase*, c'est-à-dire que chaque monade est représentée par la succession de deux points magnétisés en sens différents : c'est le sens du changement de magnétisation qui précise la nature de la monade 0 ou 1.

Exemple :



Ce mode d'enregistrement a l'avantage de rendre moins sensibles les défauts d'homogénéité de la couche d'oxyde et de permettre, à la lecture, une autosynchronisation par piste (cf. § II) qui dispense d'utiliser une piste de synchronisation ou un code spécial ruban qui devrait assurer cette synchronisation.

4. Si l'on appelle "colonne" l'ensemble des 8 monades situées l'une à côté de l'autre sur les différentes pistes, chaque *catène* est enregistré sur les 8 pistes sur 4 colonnes consécutives de la façon suivante :

.Les 24 monades du catène proprement dit sont enregistrées sur 6 pistes (4x6),

.Une 7ème piste reçoit les 3 monades de la clef par 7 du catène (la 4ème monade disponible n'étant pas utilisée),

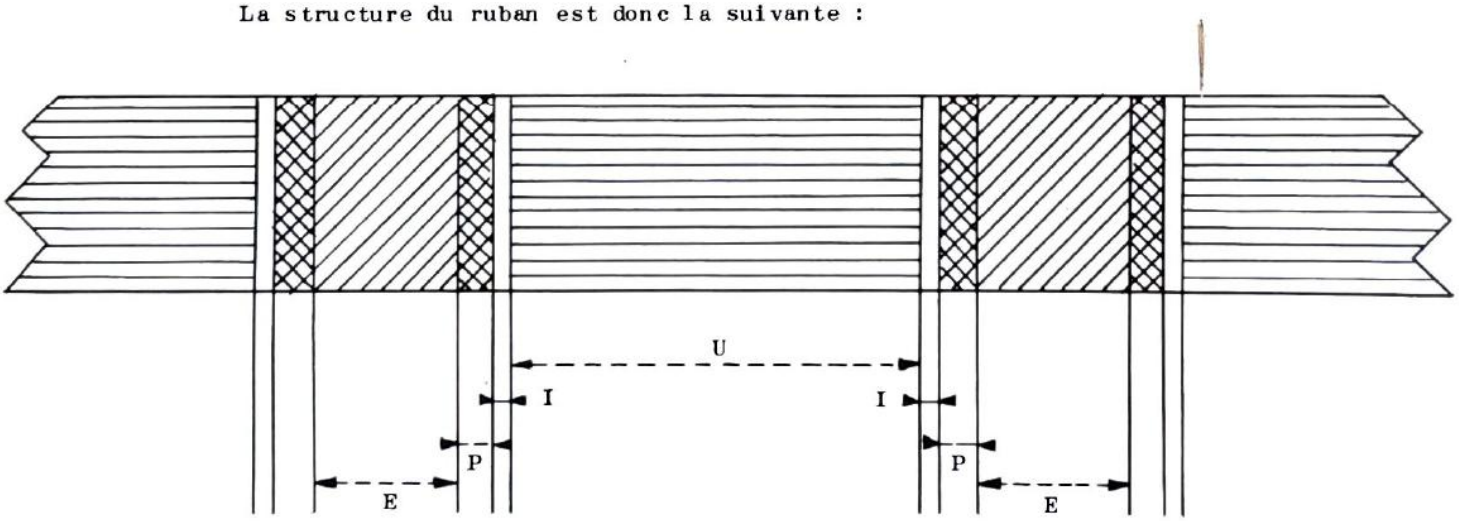
.Enfin, le ruban étant un support relativement plus fragile que les autres (tambour par exemple), on a renforcé le contrôle de la clef par 7 en utilisant la 8ème piste pour recevoir, par colonne, la clef de parité des 6 monades se trouvant sur les 6 pistes d'enregistrement du catène proprement dit.

5. La densité d'enregistrement de chaque piste est de 79 monades au cm, ce qui signifie que le ruban contient sensiblement *20 catènes au cm.*

La lecture comme l'enregistrement du ruban ne peut se faire correctement que lorsqu'il défile à sa vitesse normale. Il s'ensuit que l'on ne peut s'arrêter entre deux catènes enregistrés avec la densité indiquée ci-dessus : malgré l'efficacité du freinage et de l'accélération, le ruban ne peut s'arrêter et reprendre sa vitesse normale sur une longueur de l'ordre du dixième de millimètre. Il est donc nécessaire de grouper les catènes en *blocs* à l'intérieur desquels la densité est de 20 catènes au cm et qui sont séparés par des zones non enregistrées dites espaces d'arrêt-marche. Le bloc est ainsi le quantum de lecture ou d'enregistrement : on ne peut pas s'arrêter à l'intérieur des blocs, mais seulement entre eux.

6. *Les blocs sont de longueur variable par catène de 16* (minimum) *à 4.095* (maximum) catènes. Il s'ensuit que chaque bloc doit être accompagné de sa longueur enregistrée sur le ruban lui-même. De plus, chaque bloc doit être précédé d'une zone pilote pour déclencher l'auto-synchronisation en lecture.

La structure du ruban est donc la suivante :



a) U : zone utile de longueur variable par catène de 16 à 4.095 catènes ; contient l'information à exploiter.

b) l : longueur de la zone utile mesurée en catènes et exprimée en binaire ; occupe 1 catène au début et à la fin de chaque bloc pour permettre la lecture dans les deux sens.

c) P : zone pilote ; prend l'emplacement de deux catènes au début et à la fin de chaque bloc et sert à déclencher l'auto-synchronisation en lecture (cf § II). Sur chaque piste, on trouve 0 1 0 1 0 1 0 0.

d) E : espace d'arrêt-marche magnétisé en continu sur chaque piste. Sa longueur est de 2 cm.

7. Outre ces blocs normaux d'enregistrement, on peut trouver sur le ruban des blocs spéciaux dits *"blocs-marque"*. Ces blocs, de longueur 16 catènes, sont composés uniquement de "zéros" enregistrés sur toutes les pistes ; en dehors de cette zone utile de 16 catènes, ils comprennent comme les autres blocs une zone pilote au début et à la fin, mais ne comportent pas de catènes longueur.

En lecture, ils sont reconnus par le fait qu'après les 2 colonnes de zéros qui terminent la zone pilote, on trouve encore des colonnes de zéros. Il n'y a que dans les blocs marque qu'il puisse en être ainsi : en effet, dans les blocs ordinaires, la clef de parité de chaque colonne étant en fait une clef d'imparité, il s'ensuit que, même dans le cas où les 6 pistes significatives d'une colonne ne contiennent que des zéros, la piste de clef de parité contient un 1. En principe, la rencontre d'une seule colonne de zéros suffit à reconnaître un bloc marque, mais par mesure de sécurité, on attend d'en avoir rencontré 3 pour déclencher le processus afférent à cette reconnaissance : placement d'un test en position 19 du catène qualitatif tandis que le bloc lui-même n'est pas transmis en Mémoire Rapide. La longueur d'un bloc marque est de 16 catènes parce que c'est la longueur minimum d'un bloc.

En écriture, les blocs marque sont enregistrés à l'aide d'une directive spéciale sans qu'il soit nécessaire de les constituer au préalable en Mémoire Rapide. On verra au § suivant les principes de leur utilisation, mais on peut dès maintenant dire qu'ils servent essentiellement à repérer sur un même ruban les séparations entre différents fichiers, parties de fichiers ou groupes d'articles.

8. Les rubans sont fournis en bobines de longueurs standard 730 ou 1.100 mètres. Si l'on estime que la longueur moyenne des blocs est de 250 catènes environ, et d'après ce que l'on a dit de la densité de l'enregistrement et de la longueur des espaces arrêt-marche, la capacité correspondant à chacune de ces deux longueurs est donnée par le tableau suivant :

	730 m	1.100 m
en catènes	1.200.000	1.800.000
en lettres	4.800.000	7.200.000
en chiffres	7.200.000	10.800.000

II - LE DEROULEUR DE RUBAN : FONCTIONNEMENT ET PERFORMANCES

L'on sait que tout "élément" Gamma 60 se compose logiquement de deux parties:

. le programme local qui constitue l'élément à proprement parler du point de vue de l'organisation logique ; c'est lui qui fait les demandes d'instructions ou de quantités à l'Unité Centrale et qui en reçoit les directives.

. les circuits de position qui effectuent le travail sous le contrôle du programme local. Les circuits de positions constituent *l'appareil* qui travaille.

Cette distinction qui a été faite dans les généralités sur l'organisation logique du Gamma 60 n'a pas eu l'occasion d'être davantage mise en lumière dans la description des différents éléments.

Nous l'expliciterons ici en disant que l'élément qui permet l'exploitation du ruban magnétique se compose d'un programme local que nous appellerons *"unisélecteur"* et d'un appareil qui est le *"dérouleur de ruban"* lui-même avec son mécanisme de déroulement du ruban et ses têtes de lecture et d'écriture.

Les raisons de cette distinction comme celles de la dénomination "unisélecteur" apparaîtront clairement aux § III et IV. Pour le présent §, nous considérerons simplement de façon approximative que l'unisélecteur est le programme local du dérouleur.

1°/ Généralités et Performances

1.1.- Outre les mécanismes de déroulement, le dérouleur de ruban comprend une tête de lecture et une tête d'enregistrement par piste. Toutes les têtes d'enregistrement sont groupées sur une même transversale au ruban de façon à écrire simultanément toutes les monades d'une même colonne ; de même pour les têtes de lecture.

On peut lire dans les deux sens mais on ne peut écrire que dans le sens avant. Le contrôle de l'écriture se fait automatiquement par relecture immédiate par la tête de lecture sous laquelle passe le ruban après la tête d'écriture.

De plus, une directive spéciale permet de "lire" un bloc sans transférer les informations en Mémoire Rapide (cf. exploitation des blocs marque).

1.2.- Que ce soit en lecture ou en écriture, la vitesse de défilement du ruban est sensiblement de 1,90 m par seconde, ce qui correspond à une fréquence d'exploitation de 15 kilocycles par piste ; ceci signifie *qu'un catène est lu ou écrit en 266 microsecondes.*

Le ruban peut se lire ou s'écrire en s'arrêtant à chaque bloc. Dans ce cas, le temps de décélération pour obtenir l'arrêt est de 5 millisecondes ainsi que le temps d'accélération pour obtenir la vitesse normale nécessaire à la lecture ou l'écriture du bloc suivant.

Si le temps de traitement proprement dit des informations contenues dans chaque bloc est inférieur à leur temps de lecture, le ruban peut fonctionner en marche continue sans s'arrêter entre les blocs. Dans ce cas, les espaces d'arrêt-marche défilent à la vitesse normale ; si l'on intégre leur temps de passage dans celui de lecture des catènes du bloc utile et si l'on se place toujours dans l'hypothèse d'une longueur moyenne de bloc de 250 catènes environ, on peut dire que *temps moyen de lecture ou d'écriture d'un catène sur ruban est de 315 microsecondes.* Le temps d'exploitation d'un ruban de 730 mètres en marche continue est ainsi de *6 minutes 20 secondes* (9 minutes 30 secondes pour un ruban de 1.100 m).

1.3.- *Le rebobinage* des rubans qui est une opération purement mécanique se fait sur le dérouleur fonctionnant de façon autonome sans contrôle de l'unisélecteur. La vitesse de rebobinage est alors de 3 mètres/seconde en moyenne avec une vitesse de pointe de 3,8 m. Il faut *4 minutes* pour rebobiner un ruban de 730 m (6 minutes pour un ruban de 1.100 m).

Le ruban ne possédant aucun prémarquage magnétique, il en résulte, entre autres, que rien n'indique, sur le ruban lui-même, que l'on approche de sa fin et que l'on doit faire évoluer le programme en conséquence. Cette détection d'approche de fin de ruban est faite par une cellule photoélectrique recevant un faisceau lumineux tangentiel à la bobine lorsque celle-ci ne contient plus que 5m de ruban environ. Cet incident normal d'exploitation est signalé dans le catène qualitatif (position 20).

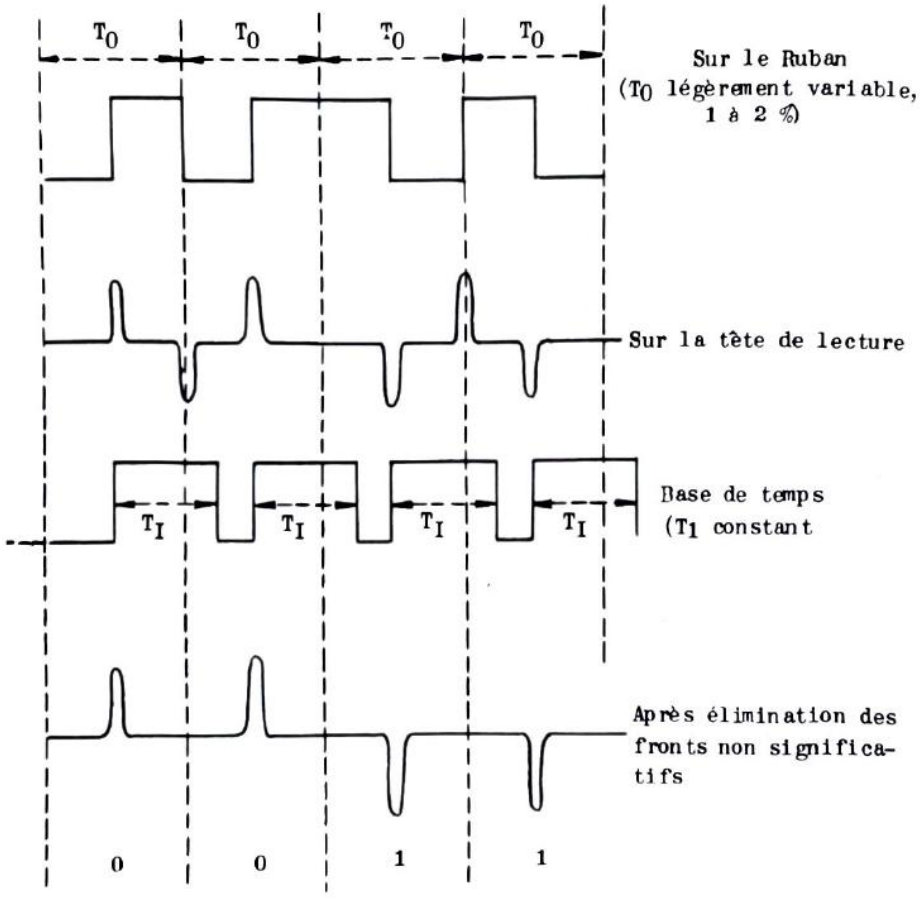
2°/ Fonctionnement en lecture

2.1.- La façon dont les informations sont représentées sur le ruban (cf. § I.3) montre qu'il existe 1 ou 2 "fronts" par monade selon qu'il s'agit de monades consécutives de sens inverse (0,1 ou 1,0) ou de même sens (0,0 ou 1,1).

Dans le cas de 2 fronts, seul l'un d'entre eux est significatif, celui qui est au centre de la monade ; et cependant le passage de chaque front sous la tête de lecture déclenche une impulsion dans les circuits de lecture : il faut donc éliminer celles correspondant aux fronts non significatifs. Ceci se fait de la façon suivante :

Chaque front significatif déclenche une impulsion dont la durée est comprise

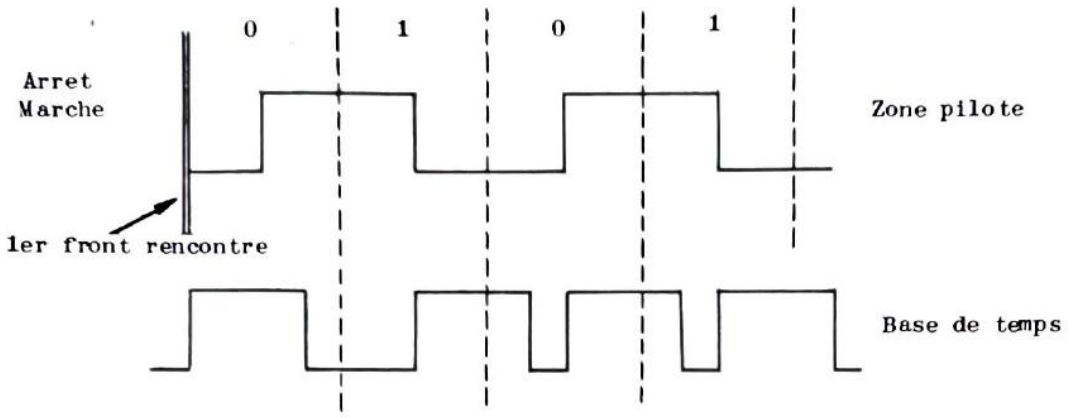
entre le temps de passage d'une monade entière et la moitié de ce temps (environ les 3/4) ; pendant la durée de cette impulsion les fronts qui peuvent se présenter ne sont pas pris en considération. C'est ce mécanisme qui permet de réaliser une autosynchronisation par piste, absorbant ainsi du même coup les faibles variations relatives de vitesse (de l'ordre de 1 à 2%) qui peuvent se produire dans le déroulement linéaire du ruban au cours de la lecture et la différence possible entre la vitesse de lecture et d'enregistrement (1 à 2% également). Le schéma ci-dessous explicite ce mécanisme :



Mais, pour que ce mécanisme fonctionne, il faut qu'au départ, l'on se soit bien synchronisé sur un front sûrement significatif. C'est le rôle de la zone pilote 0 1 0 1 0 1 0 0. Dans les six premières monades, il n'y a que des fronts significatifs puisque c'est une succession de monades de sens contraires ; c'est ce qui permet la synchronisation au départ.

Supposons en effet qu'à la frontière de l'espace arrêt-marche (magnétisé en continu) et de la zone pilote, on décèle un front ; il déclenchera labase de temps qui empêchera la validation du front significatif du zéro ; le premier front que l'on retrouvera sera, lui, obligatoirement un front significatif (celui du 1), déclenchera la base de temps et ainsi de suite .... la synchronisation sera prise.

On voit qu'en toute rigueur, la succession 0 1 suffirait mais, à cause de l'imprécision possible de frontière entre zone arrêt-marche et zone pilote, on a mis une succession de trois fois 0 1 pour que la synchronisation se fasse à coup sûr sur les monades suivantes, si elle a été ratée sur les 2 premières. Le fait



de rencontrer ensuite une succession de deux 0 pour terminer indique que le catène que l'on va lire maintenant est la longueur du bloc.

2.2.- Ces préliminaires étant exposés, le fonctionnement du dérouleur en lecture est le suivant :

a) prise de synchronisme sur la zone pilote.

b) lecture du catène longueur "l". Ce catène n'est pas transmis en Mémoire Rapide, mais simplement dans un registre spécial de l'unisélecteur qui sait alors qu'il doit contrôler le travail du dérouleur pour la lecture d'un bloc de "l" catènes (cette longueur ne peut être indiquée dans la directive, puisque les blocs qui se succèdent peuvent être de longueurs différentes et inconnues en général au moment de la rédaction du programme).

c) lecture des "l" catènes suivants avec transfert en Mémoire Rapide à l'adresse indiquée dans le RAC (RAC 0) de l'unisélecteur.

Le fait d'avoir une zone pilote et une zone longueur en début et en fin de bloc rend possible la lecture arrière comme la lecture avant.

2.3.- *En lecture avant*, le RAC de l'unisélecteur doit être garni de l'adresse où l'on veut transmettre le premier catène du bloc utile. Il progresse de 1 à chaque catène transféré.

En lecture arrière, le RAC doit être garni de l'adresse où l'on veut transférer le dernier catène du bloc (le premier que l'on lit). Il regresse de 1 à chaque catène transféré. Ainsi, *l'image du bloc en Mémoire Rapide est la même, qu'il ait été lu en marche avant ou en marche arrière.*

2.4.- *La lecture est contrôlée* en vérifiant que la clef du catène lu est la même que la clef lue et que la clef de parité de chaque colonne lue est identique à la clef de parité lue. En cas de mauvaise lecture, la lecture se poursuit jusqu'à la fin du bloc et l'incident est signalé en position 18 du catène qualitatif. On l'exploite dans le programme d'incident en commandant une lecture en sens inverse, puis une lecture en sens normal afin d'éliminer la possibilité d'une mauvaise lecture fugitive, avant d'arrêter éventuellement le dérouleur pour intervention manuelle.

3°/ Fonctionnement en écriture

3.1.- En écriture, le fait de contrôler ce que l'on vient d'écrire par relecture immédiate sur la tête de lecture placée en aval interdit l'écriture dans les

deux sens. Ceci exige également qu'à l'arrêt, les deux têtes se trouvent dans l'espace d'arrêt-marche : cet espace est donc la somme de l'espace d'arrêt-marche proprement dit et la distance entre têtes soit 2 cm.

Le bloc à enregistrer doit être composé en Mémoire Rapide accompagné de ses deux catènes longueur. L'adresse dont doit être garni le RAC est celle du premier catène longueur. Ceci étant l'écriture d'un bloc se passe de la façon suivante :

a) le dérouleur enregistre les 2 catènes de la zone pilote sur commande directe de l'unisélecteur, c'est-à-dire sans que ces deux catènes aient besoin de figurer en Mémoire Rapide et d'être alimentés depuis elle.

b) le dérouleur appelle le 1er catène dont l'adresse est désignée par le RAC : le catène longueur. Il l'enregistre sur le ruban cependant que l'unisélecteur en garde mémoire dans une mémoire de décomptage pour la suite du travail. Le dérouleur appelle alors autant de catènes plus 1 que l'indique le contenu de ce catène longueur (le dernier étant le 2ème catène longueur).

c) le dérouleur enregistre la 2ème zone pilote comme la première sur contrôle direct de l'unisélecteur.

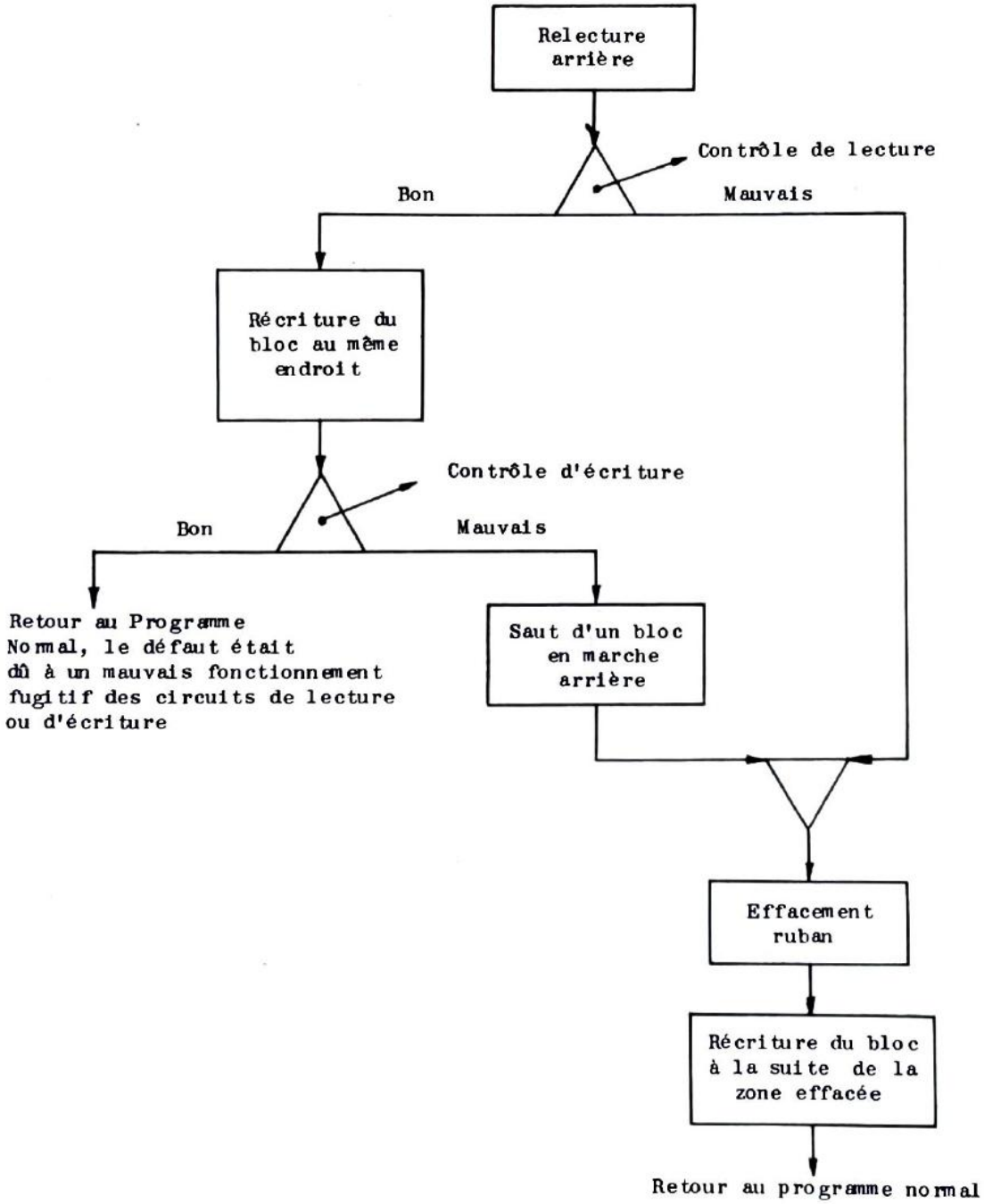
3.2.- Les rythmes utilisés pour l'écriture sont extérieurs et correspondent à une fréquence fixe de 15 kilocycles par canal. Dans les conditions normales, compte tenu des variations de vitesse linéaire de déroulement du ruban, un bloc de même capacité en catènes occupera des longueurs différentes suivant ces variations de vitesses. *Ce fait interdit donc la remise à jour partielle d'un ruban*, même pour le remplacement d'un bloc par un autre de même longueur en catènes. Par ailleurs, il faut remarquer que la remise à jour partielle est incompatible avec un contrôle organisé qui impose de conserver les rubans origine de l'information traitée.

3.3.- Le *contrôle de l'enregistrement* se fait, comme nous l'avons dit, par relecture immédiate en vérifiant la conformité des contrôles de clé et de parité. La détection de la non-conformité de ces contrôles est signalée en position 18 du catène qualitatif comme pour le contrôle de la lecture simple. Elle n'empêche pas le bloc de s'écrire entièrement.

Etant donné la nature de ce contrôle, la première chose à faire en cas de détection de mauvais enregistrement est de vérifier que ce mauvais enregistrement est bien dû à un défaut du ruban et non à un simple défaut fugitif dans les circuits d'écriture ou de relecture. Pour cela, on relira en marche arrière le bloc "mal enregistré" et on exploitera cette relecture comme indiqué sur l'organigramme ci-contre.

Dans le cas où le contrôle de relecture arrière ou de récriture au même endroit est mauvais, le défaut est bien à imputer au ruban. On lance alors, par une directive spéciale, un effacement du ruban sur une longueur d'une douzaine de centimètres (correspondant à la longueur du bloc moyen de 250 catènes) et on reprend l'écriture à la suite. Cet effacement correspond à une magnétisation continue sur toutes les pistes et revient donc à augmenter l'espace d'arrêt-marche entre deux blocs. On peut espérer que l'on a ainsi couvert le défaut et que l'enregistrement suivant sera bon. Dans le cas contraire, le processus recommence ; cela peut se produire si le bloc enregistré était de plus de 250 catènes et que le défaut se soit produit en fin de bloc.

Il convient de remarquer que le cas où le contrôle de relecture arrière a été trouvé bon et le contrôle de récriture à nouveau mauvais peut paraître à priori peu vraisemblable. Il n'en est rien : en effet, la relecture qui suit immédiatement l'écriture est faite dans des conditions "marginales" pour déceler la moindre faiblesse de l'enregistrement, tandis que la lecture arrière normale est faite dans des conditions normales ; ce cas signifie donc une faiblesse de



la couche d'oxyde qui donne un enregistrement d'une qualité marginale qu'une lecture normale accepte, mais non le contrôle d'écriture. On remarquera d'ailleurs qu'il faut alors revenir à l'origine du bloc avant de lancer l'effacement. Ceci se fait par une directive spéciale (cf. § II 1.1) qui permet de sauter un bloc sans rien transférer en Mémoire Rapide.

3.4.- Il est nécessaire d'assurer la protection de certains rubans (programmes, fichiers de base) contre certaines erreurs de programmation. Cette protection se traduit par une interdiction d'écriture sur les dérouleurs qui les supportent : pour cela, il y a, sur chaque dérouleur, un commutateur qui permet d'interdire toute écriture. Cette interdiction est rappelée par un test en position 16 du catène qualitatif.

4°/ Exploitation des blocs-marque

On a vu au § 1 ce qu'étaient les blocs-marque, qu'à la lecture leur passage était signalé en position 19 du catène qualitatif sans qu'ils soient eux-mêmes transférés en Mémoire Rapide et qu'on les enregistrait par une directive spéciale sans avoir besoin de les constituer au préalable en Mémoire Rapide.

Ces blocs ne contiennent donc aucune information si ce n'est qu'ils constituent précisément une marque sur le ruban, marque qui devra être exploitée par le programme selon le sens qu'on lui aura donné.

Les blocs marque ont été créés pour indiquer les débuts et fins de ruban. Mais en pratique leur utilisation s'est généralisée à bien d'autres fins. Les blocs marque indiquent généralement que le bloc suivant a une structure particulière ou qu'il y a dans l'information contenue par le ruban, passage d'un groupe à un autre.

Ils serviront par exemple à séparer les différents fichiers que l'on pourra trouver sur un même ruban, ou encore les différents sous-fichiers, ou groupes d'articles d'un même fichier. Certains seront exploités dans la façon même de lire le ruban, d'autres uniquement dans le traitement de l'information lue. Certains jalonneront simplement les blocs normaux signalant que l'on passe d'un groupe d'information à un autre groupe de même nature ; d'autres seront suivis de blocs normaux contenant une information de structure spéciale différente de celle des autres blocs normaux : les blocs d'identification. On trouvera ainsi un bloc d'identification au début de chaque fichier, mais on pourra encore en trouver à l'intérieur d'un même fichier pour faciliter la recherche des différentes parties de ce fichier : ces différentes parties de fichiers seront alors séparées par des blocs marque et encadrées au début et à la fin de 2 blocs d'identification identiques (pour la recherche dans les deux sens).

La recherche s'opérera alors de la façon suivante : on fera défiler le ruban sans transférer les informations lues en Mémoire Rapide, exception faite pour les blocs qui suivent les blocs marque et qui sont les blocs d'identification des différentes parties du fichier. C'est l'analyse de ce bloc d'identification qui dira s'il faut transférer en Mémoire Rapide les blocs suivants jusqu'au prochain bloc marque, parce qu'ils constituent effectivement la partie de fichier recherchée, ou bien s'il faut continuer simplement le défilement sans transfert en Mémoire Rapide jusqu'au prochain bloc marque.

Ce mode de fonctionnement qui évite les transferts inutiles sur les canaux n'est possible que grâce à l'existence des blocs marque et de la directive qui permet de "lire" un bloc sans le transférer en Mémoire Rapide.

III - PROGRAMMATION DU DEROULEUR EN MARCHE DISCONTINUE

1°/ L'Uniselecteur

Dans la plupart de leurs utilisations, les dérouleurs de rubans ne lisent ou enregistrent effectivement de l'information que pendant une fraction du temps de leur immobilisation par un fichier donné. Par exemple, cette fraction n'est que de quelques "pour cent" dans une conversion carte-ruban ou ruban-imprimante.

Si chaque dérouleur avait en propre des circuits de programme local comme les éléments de traitement ou d'entrée-sortie, ces circuits ne seraient également utilisés que pendant seulement une fraction du temps.

Pour des raisons d'économie, on a donc décidé de mettre dans le Gamma 60 moins de circuits de programme local et de circuits de transfert que de dérouleurs. Ces circuits de programme local doivent, dans la programmation, être explicitement distingués des dérouleurs comme on le verra par la suite ; ceci justifie de les distinguer aussi dans le langage courant par un nom spécial : Unisélecteurs.

Chaque unisélecteur dessert 12 dérouleurs de rubans, mais pour conserver à ces derniers toute leur souplesse d'exploitation, chacun d'eux peut être connecté à 3 unisélecteurs différents.

Le rôle de l'unisélecteur est donc double :

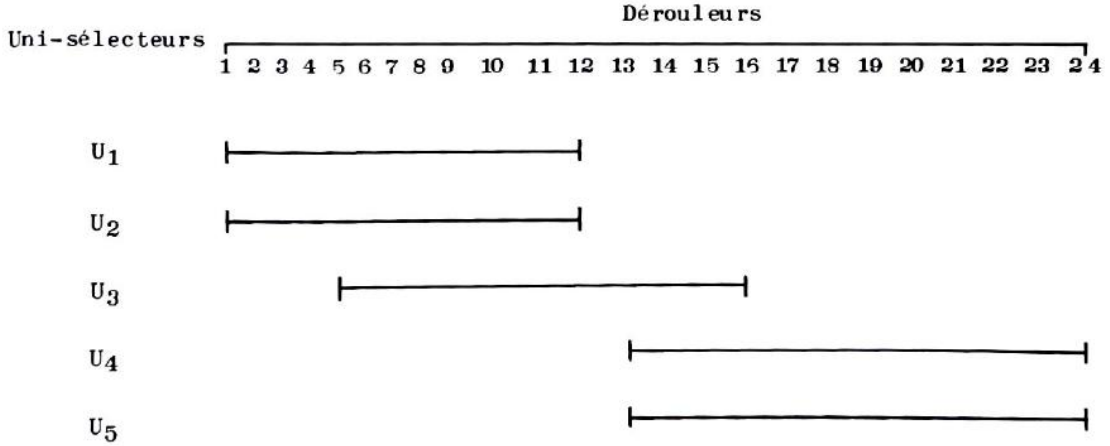
. sélectionner l'un des 12 dérouleurs auquel il est relié et établir la liaison entre ce dérouleur et l'unité centrale (ce qui justifie son nom).

. servir ensuite de programme local au dérouleur sélectionné.

2°/ Schéma des liaisons Unisélecteurs-Dérouleurs

Dans le cadre de ce qui a été dit au § précédent, l'organisation pratique suivante a été adoptée.

Le nombre maximum d'uniselecteurs connectables au Gamma 60 est de 10, répartis en deux groupes de 5. A chacun de ces groupes sont reliés un maximum de 24 dérouleurs (soit 48 au total numérotés de 1 à 48), chacun de ces dérouleurs étant lié à 2 ou 3 des uni-sélecteurs du groupe selon le schéma ci-dessous.



Comme on le voit, les dérouleurs 1 à 12 sont reliés aux unisélecteurs U_1 et U_2, les dérouleurs 13 à 24 aux unisélecteurs U_4 et U_5 et, de plus, les dérouleurs 5 à 16 le sont à l'unisélecteur U_3. La structure des deux groupes est identique ; il convient de remarquer qu'un dérouleur ne peut être relié qu'à des unisélecteurs appartenant au même groupe.

Dans le cas où un groupe n'est pas équipé de tous ses unisélecteurs, les unisélecteurs présents sont obligatoirement ceux qui portent les premiers numéros. Les 3 combinaisons possibles, par groupe, sont :

1 unisélecteur : U_1
3 unisélecteurs : U_1, U_2 et U_3
5 unisélecteurs : U_1, U_2, U_3, U_4 et U_5

En revanche, dans le cas où les groupes ne sont pas équipés de tous leurs dérouleurs, on peut utiliser n'importe quels numéros pourvu que ces numéros soient connectés à des unisélecteurs existant effectivement : ainsi avec 3 unisélecteurs dans un groupe, on ne peut utiliser que les numéros de dérouleurs 1 à 16.

3°/ Programmation

Tout ce qui vient d'être dit montre que l'élément Gamma 60 est bien l'unisélecteur et non le dérouleur. La coupure se fait donc sur l'unisélecteur : elle est codée D E R U et sa variable libre indique le numéro de l'unisélecteur. De même, le RAC appartient à l'unisélecteur et non au dérouleur.

Lorsque plusieurs séquences utilisant des dérouleurs différents mais le même unisélecteur sont en cours simultanément, elles débutent toutes par une coupure sur le même unisélecteur et c'est l'exploitation de la chaîne de reprise de l'unisélecteur qui permet à celui-ci de fonctionner alternativement au service de l'un ou l'autre dérouleur.

Le tableau des directives qui permettent aux dérouleurs de fonctionner comme il a été expliqué au § II est le suivant :

		T O	C C	X	F A
Directives préparatoires	Ecriture	E C R		N° de dérouleur si $\overline{FA}$ ou adresse du n° de dérouleur si FA	
	Lecture Avant	L A V			
	Lecture Arrière	L A R			
Directives d'exécution	Exécution avec transfert effectif (selon la directive préparatoire)	T R A			
	Lecture sans transfert en Mémoire Rapide (saut)	S A U			
	Effacement	E F F			
	Ecriture d'un bloc marque	E C M			
Directives de Service	Rebobinage	R E B		N° de dérouleur si $\overline{FA}$ ou adresse de n° de dérouleur si $\overline{FA}$	
	Rebobinage et demande de manipulation normale	R E M			
	Demande de Manipulation Normale	M A N			
	Demande de Manipulation Incident	M A I			

Ce tableau montre qu'il existe 3 sortes de directives :

1) les directives préparatoires dans lesquelles on indique en variable libre le n° du dérouleur auquel elles s'adressent. Elles ont un double rôle :

a) elles établissent les liaisons entre le dérouleur désigné dans la directive et l'unisélecteur désigné par la coupure qui précède.

b) elles préparent les circuits de l'unisélecteur et du dérouleur à fonctionner selon le type de marche attendu : Ecriture, Lecture avant ou Lecture arrière. Mais elles ne lancent aucune opération immédiate dans le dérouleur. Le départ de l'opération est donné par l'arrivée de la directive d'exécution. Il convient enfin de remarquer que les effets de ces directives se maintiennent au delà des coupures.

2) les directives d'exécution. Elles sont sans variable libre, s'adressant au dérouleur désigné dans la directive préparatoire qui précède.

- TRA est la directive d'exécution normale qui met le dérouleur au travail selon le type de marche spécifié par la directive préparatoire. Elle est relative à un bloc dont la longueur est spécifiée aux deux extrémités du bloc lui-même. Pour la lecture ou l'écriture de plusieurs blocs, un programme itératif est donc nécessaire.
- SAU permet, en lecture, de sauter un bloc sans le transférer en Mémoire Rapide. Si l'on veut sauter plusieurs blocs, un programme itératif est nécessaire. Cette directive n'a pas de sens après ECR.
- EFF provoque l'effacement du ruban (par magnétisation continue de toutes les pistes) sur une longueur d'une douzaine de centimètres correspondant à la longueur du bloc moyen de 250 catènes.
- ECM : écriture d'un bloc marque (que l'on n'a pas besoin de composer au préalable en Mémoire Rapide).

3) les directives de service. Ce sont des directives avec variable libre qui indique le numéro du dérouleur auquel elles s'adressent. Les opérations auxquelles elles correspondent (rebobinage ou manipulations ou les deux) sont longues et ne nécessitent pas l'intervention des circuits de l'unisélecteur qui ne doit donc pas être immobilisé pendant le temps de leur exécution. Leur exploitation logique est donc différente de celle des autres directives, préparatoires ou d'exécution. Pour ces dernières, qui occupent l'unisélecteur pendant tout le temps de leur exécution, l'unisélecteur, comme tout élément, ne fait une nouvelle demande d'instruction au Distributeur de Programme qu'à la fin de leur exécution : ainsi, après une directive TRA, l'unisélecteur ne fait une D.I. qu'à la fin de la lecture ou de l'écriture du bloc.

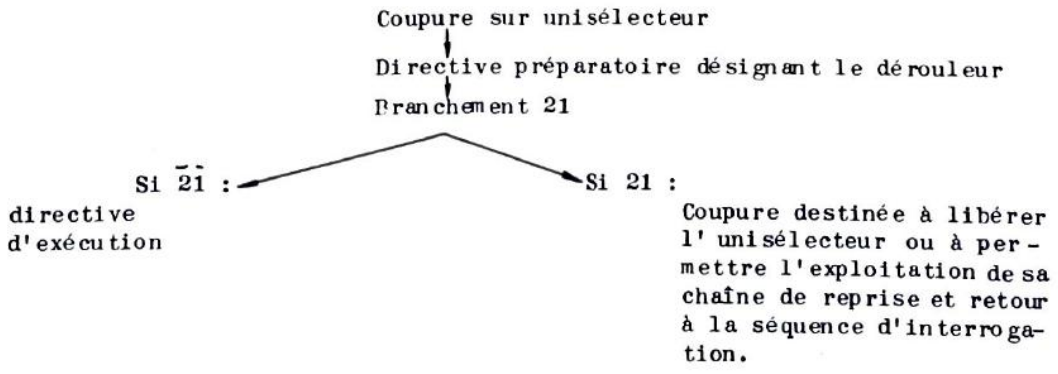
Au contraire, dans le cas des directives de service, l'unisélecteur refait une demande d'instruction dès réception de l'ordre par le dérouleur. Ceci permet de rendre l'unisélecteur disponible pour diriger le travail d'un autre dérouleur, tandis que le premier dérouleur fonctionne de façon autonome tout en émettant un signal d'occupation correspondant à un test dans le catène qualitatif (position 21).

Ce signal d'occupation est mis à profit pour assurer le retour au programme après exécution de la directive de service.

Il y a deux possibilités :

a) l'unisélecteur ne dessert que ce dérouleur ou des dérouleurs concourant à la même unité de traitement ; dans ce cas, le programme peut lancer dans l'unisélecteur une directive d'exécution qui demeurera sans effet tant que le signal d'occupation du dérouleur subsiste. Dans ce cas, l'unisélecteur, qui mémorise la directive d'exécution, est bloqué jusqu'à la fin du rebobinage ou de la manipulation, mais ceci est sans importance étant donné l'hypothèse de départ.

b) l'unisélecteur dessert plusieurs dérouleurs appartenant à des unités de traitement différentes ; il est alors inadmissible de le bloquer. Il faut alors intercaler au début des séquences qui peuvent faire appel au dérouleur en cours de manipulation, une séquence d'interrogation de disponibilité de ce dérouleur. Cette séquence est la suivante :



Les différentes directives de service sont :

. REB : rebobinage. Le signal d'occupation cesse en fin de rebobinage permettant ainsi une nouvelle exploitation du ruban sans intervention manuelle.

. REM : rebobinage suivi de manipulation. Cette directive provoque l'allumage d'une lampe verte sur le dérouleur, lampe qui invite l'opérateur à intervenir. Après intervention, le fait pour l'opérateur de refermer la porte du dérouleur provoque l'appui sur un bouton "Fin de Manipulation", ce qui entraîne l'extinction de la lampe verte et la retombée du signal d'occupation (catène qualitatif en $\bar{2}\bar{1}$).

. MAN : demande de manipulation normale - provoque l'allumage de la même lampe verte et l'émission du signal d'occupation. L'extinction de la lampe comme la retombée du signal d'occupation est provoquée par l'appui sur "Fin de Manipulation", c'est-à-dire par la fermeture de la porte.

. MAI : demande de manipulation incident - provoque l'allumage d'une lampe rouge et l'émission du signal d'occupation. Après réparation de l'incident, la fermeture de la porte du dérouleur provoque l'extinction de la lampe rouge et la retombée du signal d'occupation.

IV - PROGRAMMATION DU DEROULEUR EN MARCHE CONTINUE

1°/ Le type de programmation indiqué au § précédent ne permet pas un fonctionnement des dérouleurs en marche continue, ni en lecture ni en écriture. En effet, les directives d'exécution étant relatives à un seul bloc et l'unisélecteur ne faisant une nouvelle D.I. qu'à la fin de l'exécution de la directive, c'est-à-dire à la fin de la lecture ou de l'écriture du bloc, il s'ensuit que le dérouleur s'arrêtera systématiquement à la fin de chaque bloc.

Si le dérouleur travaille au voisinage de son débit maximum, ce mode de fonctionnement, d'une part, occasionne une fréquence d'arrêt marche incompatible avec le bon fonctionnement mécanique du dérouleur et, d'autre part, ralentit le débit à cause des temps de décélération et d'accélération.

2°/ On a donc prévu les circuits de l'unisélecteur de façon à permettre, si l'on veut, le fonctionnement d'un dérouleur *en marche continue sans arrêts entre blocs*. Le principe de cette réalisation est de doter l'unisélecteur, qui ne possède qu'un circuit d'échanges de quantités, de deux éléments de programme. L'unisélecteur est donc vu de l'unité centrale comme un élément double ayant deux numéros d'élément et possédant 2 RAI et 2 RAC (chaque RAC étant toutefois référencé 0 puisque chacun est relatif à un élément différent).

En marche continue, le programme s'adressera alternativement sur les 2 éléments. Pendant l'exécution de la directive (écriture ou lecture d'un bloc) alimentée par un élément, l'autre élément qui est libre permet d'alimenter la directive relative au bloc suivant de façon à n'avoir aucune discontinuité dans le fonctionnement.

Les directives qui permettent ce mode de fonctionnement sont les mêmes que celles qui permettent le fonctionnement en discontinu. Il suffit, pour avoir le fonctionnement continu, de lancer le travail simultané des deux éléments par une séquence un peu spéciale (incorporée d'ailleurs dans un sous-programme standard). Les deux numéros d'éléments d'un même unisélecteur portent des numéros consécutifs, l'un pair, l'autre impair ; pour fixer les idées, nous prendrons les n° 1 et 2 dans la séquence de lancement en continu dont la structure est la suivante :

1.	D E R U	1	établit la liaison unisélecteur-dérouleur
2.	L A V	1 0	
3.	S I M U	4	Mise en activité simultanée des deux éléments qui font tous deux des demandes d' instruction au Distributeur de Programme
4.	F I	5	
5.	3 1 / 0 0	8	
6.	D E R U	1	
7.	3 1	9	
8.	D E R U	2	
9.	S I M U	1 4	
10. à 13.	Bascule de zones M.R.		
14.	G O	du résultat de la précédente bascule	
15.	T R A		
16.	3 1	4	

On a volontairement omis, dans cette structure de séquence, les catènes vierges à la suite des coupures. On voit que la SIMU qui est en ligne 3 n'a d'autre but que de lancer 2 fois de suite la séquence commençant en 4 qui, grâce à la FI, lance successivement en activité les éléments 1 et 2 qui, l'un après l'autre, demandent ensuite la séquence commençant en 9. Ainsi, le G O figurant en 14 est relatif tantôt au RAC de l'élément 1, tantôt au RAC de l'élément 2. La SIMU qui est en 9 a pour but de faire la bascule de zones Mémoire Rapide pendant que se déroule la directive TRA (procédé dit de microsimultanéité, TRA se trouvant sur la branche extraordinaire exécutée la première). Le relancement sur l'un ou l'autre élément se fait grâce au B 31 sur FI.

Il convient de remarquer enfin que la directive préparatoire LAV étant remanente à toute coupure, il est inutile de la répéter après les coupures 6 et 8 ; ce serait même nuisible : en effet, elle s'exécuterait alors que les transferts sont en cours sur l'autre élément de programme et les circuits de transferts étant uniques pour les 2 éléments, il s'ensuivrait une perturbation dans ces transferts.

3°/ *En cas de mauvaise lecture ou enregistrement* en marche continue (position 18), il faut s'arrêter, comme en marche discontinue, dès la fin du bloc mal lu ou mal écrit. A cet effet, la directive qui est alors en attente dans l'autre élément de l'unisélecteur se trouve alors organiquement bloquée par l'apparition de l'incident 18 et ne s'exécute pas mais provoque une nouvelle demande d'instruction de la part de l'élément. Dans le programme d'incident, les demandes des deux éléments sont alors regroupées sur une coupure à 2 conditions attendues, qui commence la séquence de correction, identique à ce qui a été expliqué au § II.

4°/ *En marche discontinue*, lorsque l'unisélecteur dessert alternativement plusieurs dérouleurs, seul le premier des 2 éléments de l'unisélecteur est utilisé et c'est sur lui qu'a lieu la coupure.

5°/ Lorsqu'un unisélecteur dessert un dérouleur en marche continue, il faut protéger cette séquence de programme des autres séquences qui pourraient être demandées à ce moment là sur d'autres dérouleurs et le même unisélecteur. En effet, lorsqu'un unisélecteur fonctionne en marche continue, il ne peut se produire de chaîne de reprise sur lui.

Cette éventualité n'est pas utopique ; la marche continue peut ne pas être absolue ; soit, par exemple, un fichier permanent de gros volume dans lequel on désire interclasser des articles nouveaux en petit nombre.

Le fichier permanent non modifié sera sur un dérouleur, les modifications sur un autre, le fichier modifié s'enregistrera sur un 3ème. Ce 3ème dérouleur utilisera un unisélecteur pour fonctionner en marche continue mais, comme il enregistrera au total la somme des 2 fichiers contenus sur les deux premiers dérouleurs, il s'ensuit que ces deux dérouleurs ne pourront marcher toujours en continu : chaque fois qu'il y a un article nouveau à interclasser, le dérouleur supportant le fichier non modifié devra s'arrêter et il est normal de penser à utiliser son unisélecteur pour desservir alors le fichier mouvement. Dans ce cas, les séquences de programme des 2 dérouleurs devront être protégées l'une de l'autre par un blocage de l'une sur appel de l'autre, sans compter sur le mécanisme de chaîne de reprise.

On dit alors que l'on utilise l'unisélecteur en *système fermé* par opposition au *système ouvert* d'un unisélecteur desservant plusieurs dérouleurs en marche discontinue.

6°/ En marche continue, la rencontre d'un bloc marque fait apparaître le test 19, mais ne bloque pas l'autre directive en attente dans l'autre élément. Autrement dit, on continue à lire encore le bloc qui suit et l'apparition du test 19 est, en général, exploitée pour faire arrêter le ruban à la fin de ce 2ème bloc (qui peut être lui aussi un bloc marque).

V. LE CATENE QUALITATIF

Le catène qualitatif envoyé en unité centrale à chaque demande d'instruction est le catène qualitatif de l'unisélecteur. Toutefois, dans ce catène qualitatif, certaines positions sont spécifiques des circuits de l'unisélecteurs ; d'autres comme la position 21 (dérouleur occupé), sont relatives au dérouleur connecté à l'unisélecteur à l'instant considéré. Tous les incidents relatifs aux dérouleurs proprement dits sont donc mémorisés dans des mémoires propres aux dérouleurs. Chaque dérouleur a ainsi sa mémoire d'interdiction d'écriture (position 16) ou d'occupation (21). C'est au moment où le dérouleur est connecté à l'unisélecteur par la directive préparatoire que le contenu de ces mémoires est transféré dans l'unisélecteur pour former, avec le contenu des mémoires incident propres à l'unisélecteur, le catène qualitatif.

Ce § a pour but de récapituler les significations des différentes positions du catène qualitatif, l'exploitation de la plupart d'entre elles ayant déjà été expliquées aux § précédents.

Sur la position 24, ont été regroupés tous les incidents qui ne peuvent donner lieu à un simple processus programmé et qui nécessitent une intervention humaine.

- 24. $\overline{22}$. $\overline{17}$: Incident décelé dans les circuits de l'unisélecteur. Le programme d'incident exécutera plusieurs essais sur l'unisélecteur pour décider de la suite des opérations :

  Incident permanent ——→ Dépannage

  Incident fugitif ——→ Reprise du travail

- 24. $\overline{22}$. 17 : sécurité d'appel. L'unisélecteur demande un dérouleur qui ne lui est pas connecté ou appelle un dérouleur qui est déjà sollicité par un autre unisélecteur.

- 24. 22. $\overline{17}$: incident mécanique ou électrique sur le dérouleur ; demande l'intervention du mécanicien.

Sur la position 23, sont regroupés les incidents dont la réparation est théoriquement possible par programme.

- 23. 16 : interdiction d'écriture. Apparaît après la directive ECR demandant un dérouleur comportant l'interdiction.

- 23. 18 : mauvaise lecture ou enregistrement.

- 23. 19 : lecture d'un bloc marque.

- 23. 20 : approche de fin de ruban.

- 23. 21 : dérouleur occupé.

L'exploitation de tous ces tests a été expliquée au cours des § précédents

MÉMOIRES AUXILIAIRES

LE TAMBOUR MAGNÉTIQUE

I - DESCRIPTION DU TAMBOUR

1°/ Généralités

Le Tambour Magnétique Gamma 30 tourne autour d'un axe vertical. Les informations y sont enregistrées sur des pistes circulaires réparties tout le long de la surface latérale du Tambour. Il y a une tête de lecture et d'enregistrement par piste. On ne peut lire ou enregistrer que sur une piste à la fois. Le quantum d'enregistrement ou de lecture est le catène. Enfin, le Tambour est une mémoire adressable par catène.

2°/ Enregistrement des informations - Capacité

Chaque catène accompagné de sa clef (soit 27 monades au total) est enregistré en série sur une même piste. Chaque piste contient ainsi 200 catènes enregistrés les uns à la suite des autres. Un espace de garde de 2 monades est laissé entre les catènes. De plus, à la fin des 200 catènes d'une piste, il est laissé un espace inutilisé de 4 catènes. Cet espace est nécessité par le temps de commutation de pistes. En effet, lorsque l'on lit ou enregistre une série de catènes à cheval sur 2 pistes, il faut, en cours de lecture ou d'enregistrement, commuter la tête de lecture-écriture ; ceci demande un certain temps pendant lequel le tambour continue à tourner ; pour pouvoir lire ou enregistrer la suite au début de la piste suivante, il faut donc que son début soit décalé par rapport à la fin de la précédente. Si l'on développe la surface latérale du Tambour, elle se présente donc comme une surface enregistrée sur une largeur de 200 catènes plus une bande inutilisée d'une largeur de 4 catènes.

Le Tambour comporte 128 pistes d'enregistrement plus une piste de synchronisation.

Sa capacité est donc de :

128 x 200 - 25.600 catènes
- 102.400 lettres
- 153.600 chiffres.

3°/ Performances

La vitesse de rotation du Tambour est de 3.000 tours par minute, soit un tour en 20 millisecondes.

Le temps de lecture d'un catène est donc de 20.000/204, soit 98 microsecondes. Mais, pour un long transfert à cheval sur 2 pistes, on peut dire que le temps moyen de lecture d'un catène est, du fait de l'intégration de l'espace de commutation de piste : 20.000/200, soit 100 microsecondes.

Le temps d'accès moyen est celui d'un demi-tour Tambour, soit 10 millisecondes.

Le temps d'accès maximum est d'un tour, *soit 20 millisecondes.*

II - PROGRAMMATION

1°/ Généralités

Le Tambour possède un R.A.C. qui, préalablement à tout transfert (Mémoire Rapide - Tambour ou Tambour - Mémoire Rapide), doit être garni de l'adresse initiale de la zone Mémoire Rapide objet du transfert. Mais, de plus, il est nécessaire de lui indiquer l'adresse initiale en tambour de la zone Tambour objet du transfert. Cette adresse ne peut être le contenu d'un 2ème R.A.C. puisque les R.A.C. ne sont relatifs qu'aux adresses Mémoire Rapide. Elle sera contenue par un registre spécial au tambour, dit Registre Adresse dans l'Elément R.A.E. Au cours des transferts, R.A.C. et R.A.E. progressent automatiquement à chaque catène transféré de sorte qu'en fin de transfert, ils contiennent l'un et l'autre l'adresse initiale de la zone qui suit immédiatement la zone du transfert.

2°/ Directives du Tambour

Le R.A.E. ne peut être garni comme un R.A.C. au moyen d'une instruction d'adresse. On le fait grâce à une directive préparatoire, la D.A.O. (Désignation Adresse Origine) dont la variable libre sert précisément à désigner cette adresse.

A part cette directive préparatoire, il y a deux directives d'exécution indiquant le sens et la longueur du transfert en catènes (cette longueur étant exprimée en binaire sur 15 monades).

	T.O.	C.C	X	F.A
Désignation Adresse Origine	D A O		Adresse Tambour	M, R ou rien / rien ou S
Transfert M.R.- Tambour	T R T		longueur	
Transfert Tambour - M.R.	T T R			

Comme un R.A.C., le R.A.E. est rémanent à l'intérieur d'une séquence élémentaire, ce qui fait que, comme pour un R.A.C., on peut utiliser cette rémanence pour éviter de le recharger avant chaque directive.

A la fin d'une séquence élémentaire, on peut mémoriser en Mémoire Rapide le contenu d'un R.A.C. grâce à une instruction d'adresse "Rangement". Pour le R.A.E., on dispose d'une possibilité équivalente.

En effet, après l'exécution d'une directive de transfert, le catène qualitatif accompagnant la demande d'instruction porte, dans les 15 positions poids faibles, le contenu final du R.A.E. Il suffira donc d'une directive centrale R.A. pour le mémoriser où l'on voudra en Mémoire Rapide.

3°/ Adressage du Tambour

En code B, le Tambour est adressable de façon continue de 0 à 25.599, les adresses de la première piste allant de 0 à 199, celles de la 2ème de 200 à 399, etc... Dans les directives D.A.O., ce sont ces adresses qu'il faut indiquer. Mais le programme de transposition code B - code A ne se contente pas simplement de transformer en binaire cette adresse exprimée en décimal.

En effet, en code A, des adresses Tambour, si elles sont toujours exprimées en 15 monades, le sont de la façon suivante :

. les 7 monades poids forts indiquent le numéro de la piste (0 à 127)

. les 8 poids faibles indiquent l'adresse dans la piste (0 à 199 alors qu'avec 8 on pourrait aller jusqu'à 255).

Ainsi, tous les catènes se trouvant dans les différentes pistes le long d'une même génératrice du Tambour portent la même adresse dans la piste.

Si l'on considère ces 15 monades comme constituant un seul nombre binaire, cela veut dire que, en code A, les adresses de la première piste vont de 0 à 199, celle de la deuxième de 256 à 511, etc... On voit que l'on a un adressage discontinu en code A. Lorsqu'au cours d'un transfert, on passe d'une piste à une autre, le R.A.E., au lieu de progresser de 1, progresse de 57, c'est-à-dire que le numéro de piste progresse de 1 tandis que l'adresse dans la piste passe de 199 à 0.

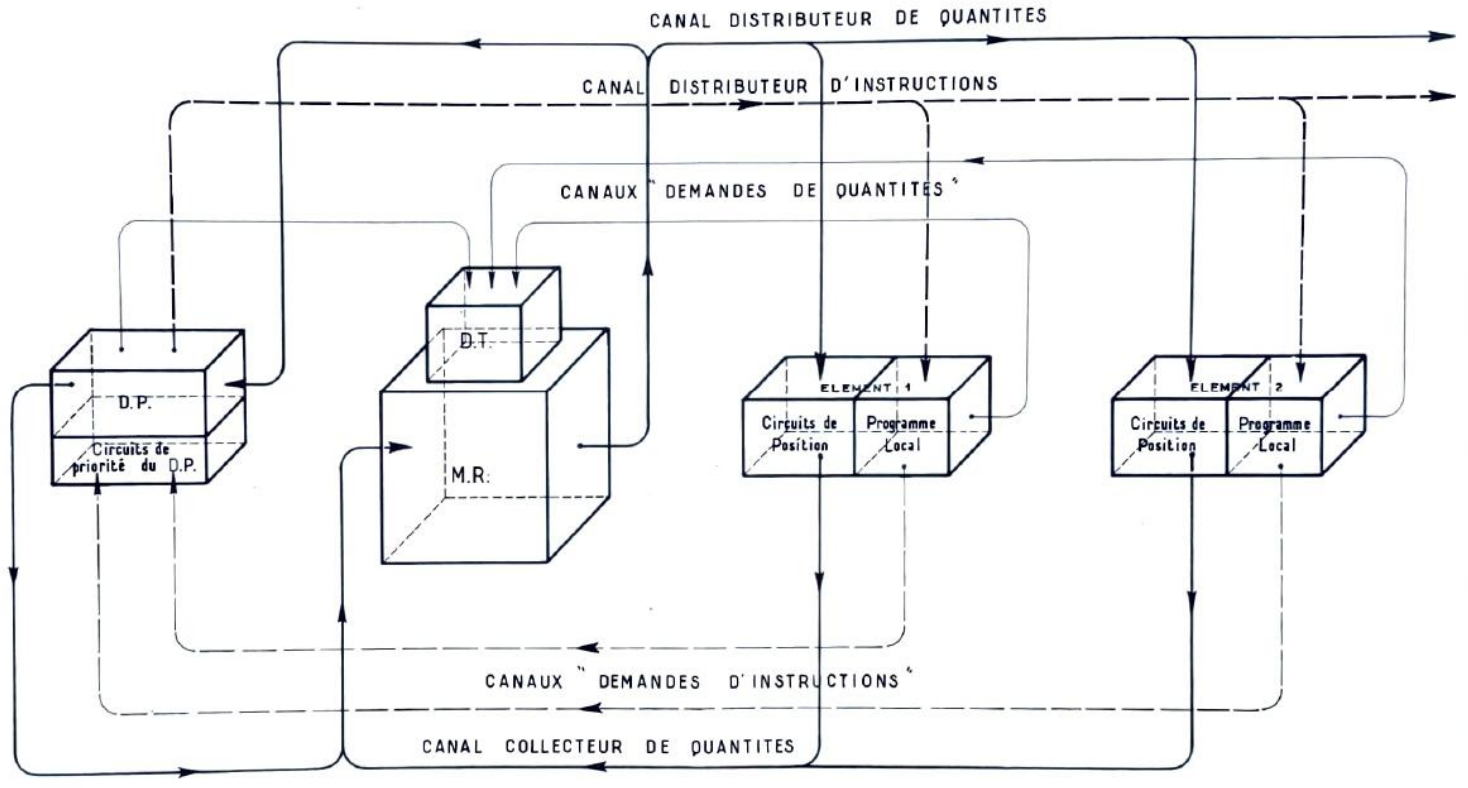
Si l'on n'a pas à tenir compte de cet adressage discontinu dans la programmation des D.A.O., il faut en tenir compte si l'on a à faire des calculs d'adresses sur les adresses Tambour. En particulier, il faut bien savoir que c'est l'adresse code A que l'on range en Mémoire Rapide par une R.A. à la suite d'une directive de transfert. On pourra la réutiliser sans précaution particulière par une D.A.O. avec substitution, mais si l'on a à faire préalablement un calcul à partir d'elle, il faut faire attention.

4°/ Dépassement de capacité

Lorsque l'on transfère en tambour une série d'articles de longueur variable, et si l'on ne prend pas de précautions spéciales, il se peut que l'on ne sache plus très bien à quelle adresse Tambour on en est arrivé. On peut donc lancer un transfert de 20 catènes alors que l'on se trouve à l'adresse 190 de la dernière piste. Dans ce cas, la fin du transfert ne se poursuivra pas sur le début de la première piste, mais le transfert sera bloqué, il y aura positionnement d'un test en position 23 du catène qualitatif et le Tambour fera une demande d'instruction qui permettra grâce à ce retour au programme, d'exploiter ce test comme on voudra par un branchement 23.

Ce dispositif permet donc de travailler en adresse origine variable en étant sûr de réserver les premières pistes Tambour.

ORGANISATION GENERALE DU GAMMA 60



MR = MEMOIRE RAPIDE DP = DISTRIBUTEUR DE PROGRAMME DT = DISTRIBUTEUR DE TRANSFERT

ORGANIGRAMME A TRAITER

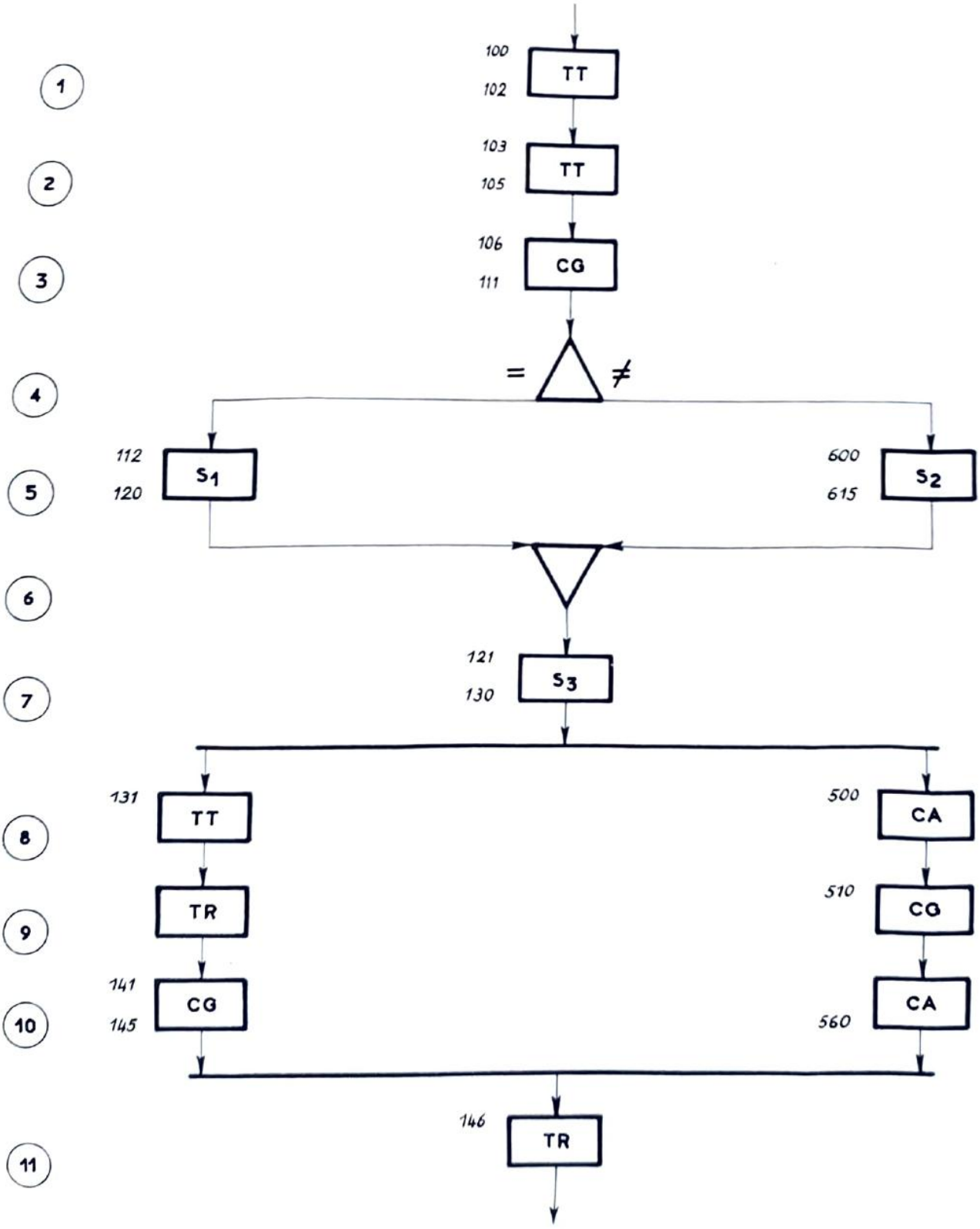
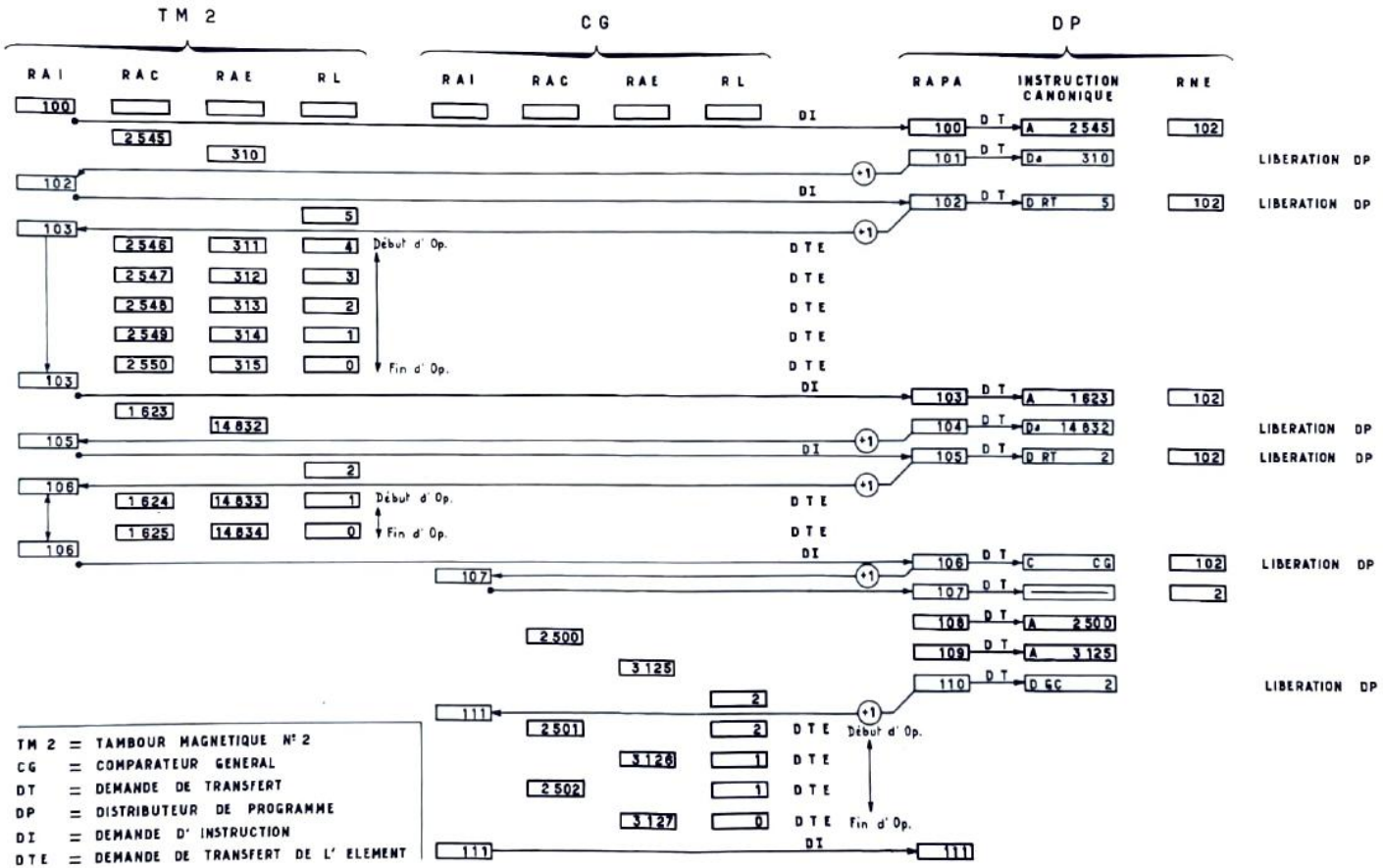


Planche N° 2

DIAGRAMME FONCTIONNEL GAMMA 60



CHAINE DE REPRISE GAMMA 60

Planche N° 3

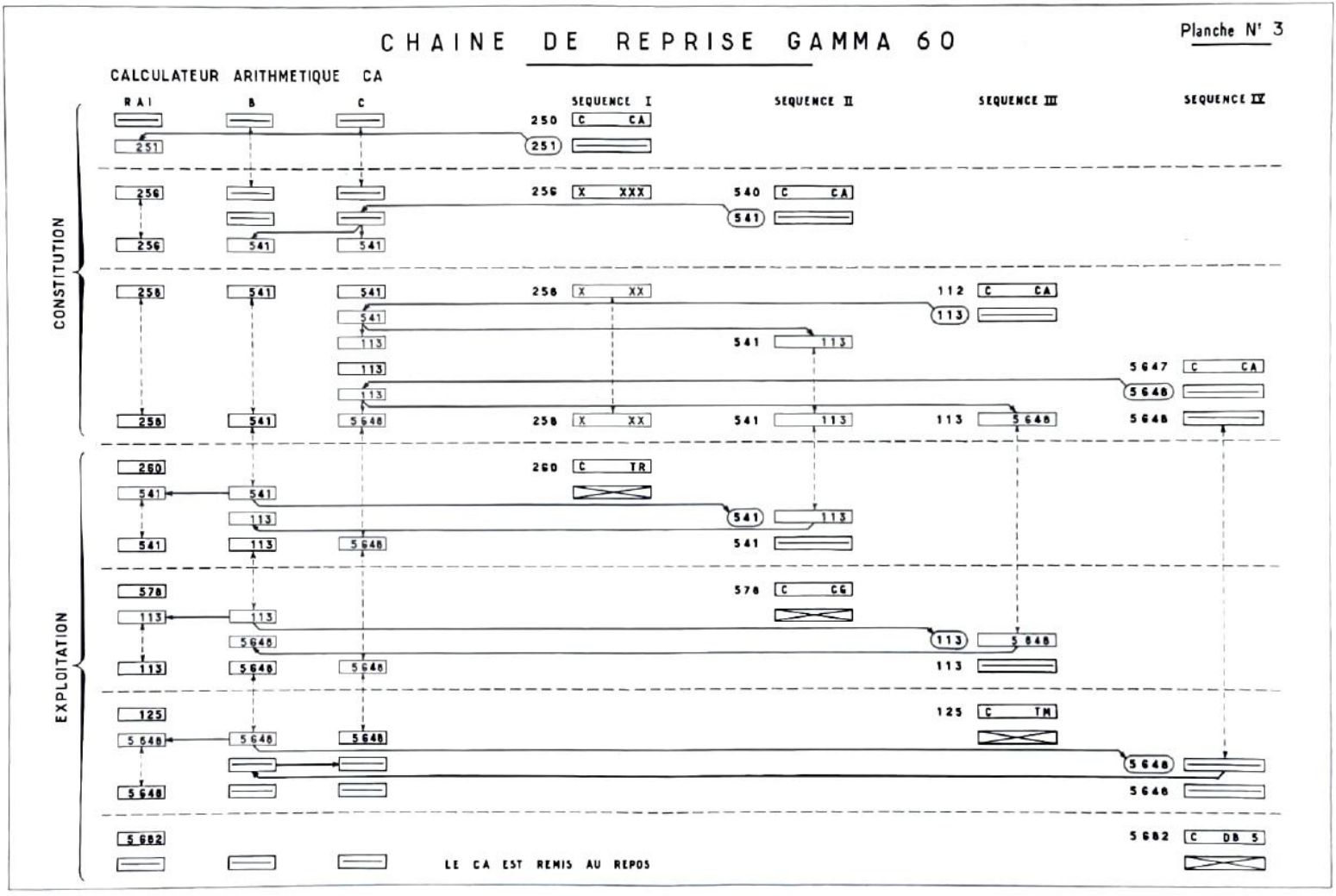


Planche N°. 4

CHAINE DE REPRISE SUR ELEMENT VIRTUEL

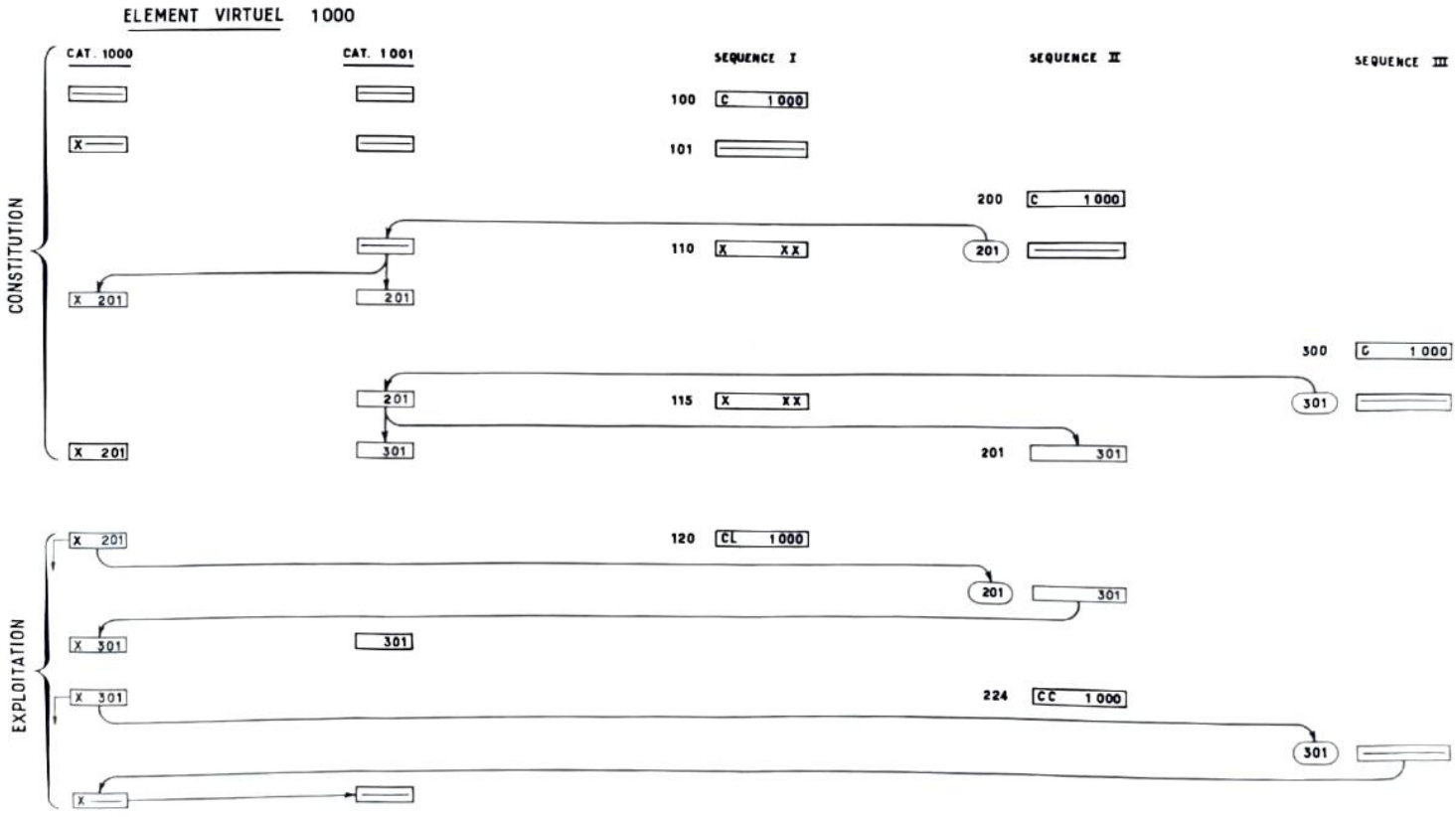
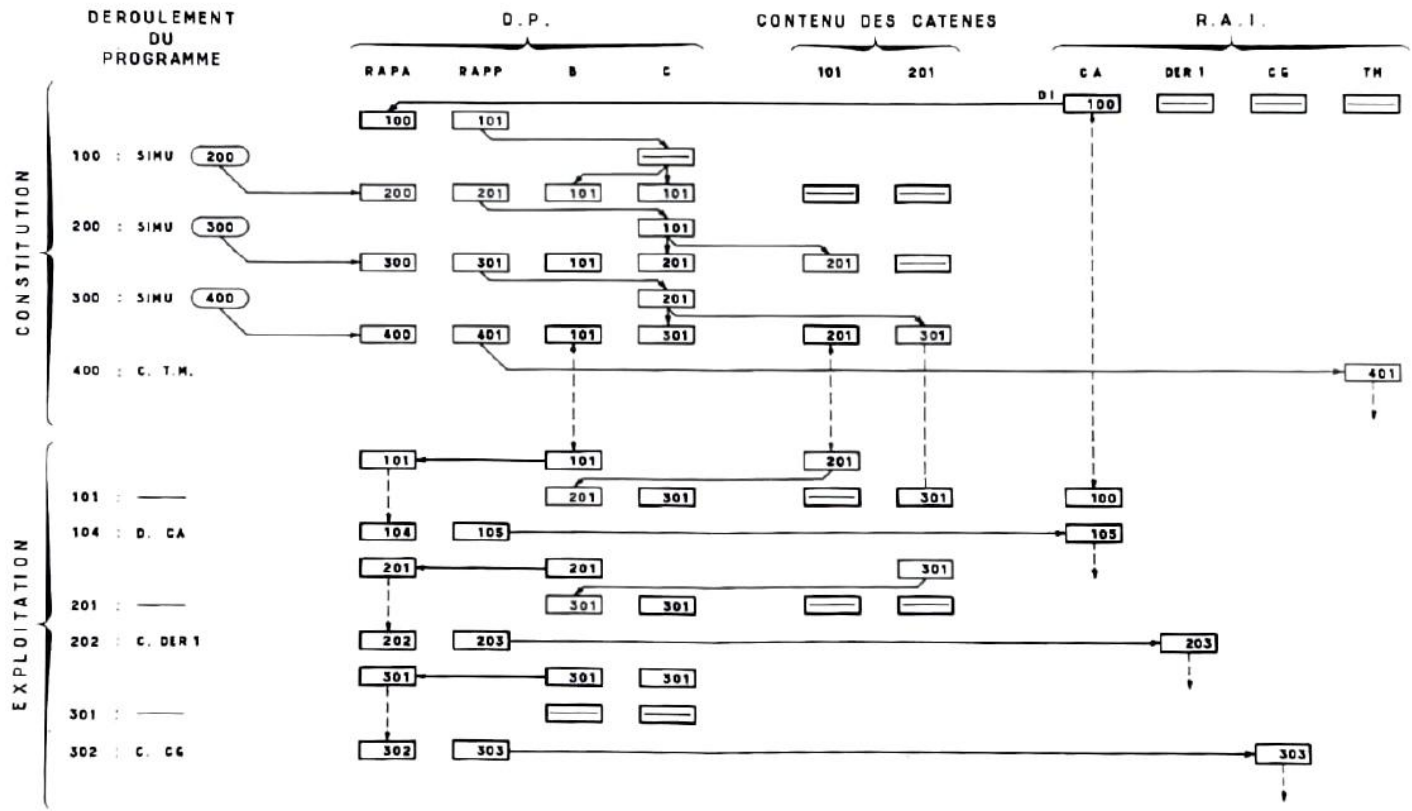


Planche N° 5

SIMU ET CHAINE DE REPRISE DU D.P.



Compagnie des Machines BULL — GAMMA 60 — Feuille N°: _____

PROBLEME : _____ PROG : _____

Traitement	Branch.	N° Ligne	TO	CC	X V	X 01	X 02	X R	FA	Observations
		98	TAMB					2		
		99								
		100	G	0		2	5	4	5	
		101	DAO				3	1	0	
		102	TRT						5	
		103	G	0		1	6	2	3	
		104	DAO		1	4	8	3	2	
		105	TRT						2	
		106	COMP							
		107								
		108								

Planche N°6

FEUILLE DE PROGRAMMATION

COURS DE PROGRAMMATION GAMMA 60

Errata

Page 8	ligne 37	lire : (R.L.N.) au lieu de : (R.L.H.)
Page 9	ligne 33	lire : les 15 premières positions au lieu de : dernières
Page 27	ligne 12	lire : dans la dernière monade au lieu de première
Page 31	ligne 35	lire : dans la dernière position au lieu de première
	ligne 41	lire : que la dernière position au lieu de première
	ligne 44	lire : (les 15 premières du catène 1.000) au lieu de : dernières
Page 35	ligne 45	lire : catène vierge suivant l'instruction de coupure au lieu de l'intention
Page 38	ligne 24	lire : provoquée soit par des instructions SIMU, soit par des instructions libérations d'éléments virtuels ; les adresses ..... au lieu de : provoquée, soit par des instructions libérations d'éléments virtuels, les adresses.....
Page 41	§ 2°/ b)	dessin du profil de catène : partie TK
		lire : 1110 au lieu de 1111
Page 45	ligne 10	lire : son complément à 32.768 soit 32.758 au lieu de : son complément : 32.768
Page 48	avant dernière ligne. lire : (indice R) au lieu de (indice B)	
Page 49	ligne 6	lire : qu'il numérote de 0 à m au lieu de 0 à n
Page 56	ligne 3	lire : $\geq$ $<$ = ... au lieu de $<$ $<$ = ...
Page 57	ligne 35	lire : conformément aux schémas suivants au lieu de au retenu ...
Page 61	ligne 24	lire : -0, 00542 au lieu de - 0, 0054
Page 64	ligne 28	lire : la traduction et l'édition au lieu de et l'addition
Page 68	ligne 43	lire : Si l'on voulait le faire porter sur le 5 ème au lieu de voulait faire porter le ....
Page 75	ligne 31	lire : $\leq$ $\leqslant$ = $\neq$ $\gg$ $\geq$ au lieu de $\leqslant$ $\leqslant$ = $\neq$ $\gg$ $\gg$
Page 104	ligne 34	lire : branchement 08 au lieu de branchement 07
Page 105	Dans le tableau, colonne "cas d'emploi" : supprimer le trait entre CAT et BPT	
Page 107	ligne 39	lire : RCL normales sans codes au lieu de dans codes....
Page 117	premier profil du registre K : supprimer 4 zéros dans la zone 0 à 16	
Page 121	ligne 8	lire : (n $\leqslant$ 24 - 1) au lieu de (n 24 - 1)
Page 133	ligne 26	lire : un pas = 3 mm 175 au lieu de 3 mm 178
Page 146	dernière ligne du tableau, colonne X : lire si FA au lieu de si $\overline{FA}$	